

PrimeSim™ EMIR Reference Manual

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	13
Related Products, Publications, and Trademarks	13
Conventions	14
Customer Support	15

1. Introducing PrimeSim EMIR	16
Electromigration Analysis	16
Nanometer Technology Trends	17
What Is Electromigration?	17
Failure Mechanisms	18
History of Studies into Electromigration	18
Electromigration Dependency on Physical Effects	19
Temperature	19
Wire Width	20
Wire Length	20
Impact of Physical Layout	21
EM Rules for Nanometer Processes	22
IR Drop Analysis	22
Integrating With Other Synopsys Tools	23

2. Getting Started	25
PrimeSim EMIR Analysis Flow	25
Setting Up PrimeSim EMIR	26
Running PrimeSim EMIR	27
Static Resistance Calculation	28
Dynamic Power Net EMIR Analysis	29
Signal Net EM Analysis	30
PrimeSim EMIR Post-Processing	30
What-if Analysis	30

3. Static Resistance Check	31
---	-----------

Contents

Introducing Static Resistance Check	31
Running Static Resistance Check	32
Calculating Layer Resistance Sensitivity and Layer Equivalent Resistance	32
Generating Output Reports	34
Post-Processing Static Resistance Calculation Results	35
Limitations	37
Loading Static Resistance Checking Results to Layout Editors	37
Viewing Violation Maps in Laker Layout Editor	38
Viewing Violation Maps in Custom Compiler Layout Editor	38
Viewing Violation Maps in Cadence Virtuoso Layout Editor	39
Commands for Static Resistance Check	39
addnetpin	41
addnetpinxy	41
addresistor	42
ccdb	42
ccdbcols	42
changeI2P	43
chkmissstrap	43
chkmissstrapboxsize	45
chkmissstraplayers	45
chkmissvia	46
chkmissviaboxsize	46
chkmissvialayers	47
file	47
gds	47
gdslabels	48
gdslayer0	48
gdslayers	48
gdsthresh	49
gdsviewer	49
ipin	50
itflayernames	50
layer0ohm	51
layerfactor	51
layer_res	51
layer_res_nworker	53
mergefinger	54

Contents

modifyresistor	55
mt	55
net	56
netdeletepad	56
netinclude	56
node2node	56
node2node_option	57
op2ipin	57
outfiltres	58
outlayers	58
png	58
report	59
rmax_thresh	60
routcols	61
rmin	61
rve	61
subnode	62
sortby	62
temp	63
tmpdir	63
4. Power Net EMIR Analysis	64
EMIR Flow Manager With PrimeSim Pro, PrimeSim SPICE, PrimeSim HSPICE and PrimeSim XA	64
Before You Start	65
Simulator Agnostic Solution for EMIR Analysis	65
EMIR Flow Manager Workflow	66
Flow Options in EMIR Flow Manager	67
Running EMIR analysis – library check flow	68
Running EMIR analysis – emir flow	70
Running EMIR analysis – fast_ir flow	71
Running EMIR analysis – static_ir flow	73
Specifying Nets for Power Net EMIR Analysis	74
Setting EMIR Nets	75
Defining Net Types	77
Handling Internal Power Nets	78
Selecting Time Windows for EMIR Analysis	79
Setting RATAU Values for Current Resolution in EMIR Analysis	79
Enabling EMIR File Splitting	79

Contents

Redirecting Temporary EMIR Files	80
Using Variable Time Steps	80
Viewing EMIR Analysis Results	81
Adding Design or Functional Resistors	81
Applying Electromigration Check on the Functional Resistors	82
EMIR Analysis With PrimeSim XA	83
Before You Start	83
Introducing the Power Net EMIR Analysis	84
Power Net EMIR Analysis Flow	85
Defining Net Pins by Specifying XY Coordinates	86
Performing Power Net EMIR Analysis - Simulation Stage	87
Enabling Power Net EMIR Flow	87
Preparing RATCL Files	88
Performing Power Net EMIR Analysis - Analysis Stage	89
Using User-Defined Current Waveforms for What-If Analysis	90
Enabling Multicore Processing	91
Advanced Electromigration and IR Drop Analysis	91
Power Net and Signal Net RASIM Reuse Flow	91
Command-Based What-If Analysis	94
Adding a Net Pin by XY Coordinates	98
EMIR Pipeline Processing	100
5. Signal Net EM Analysis	102
Introducing Signal EM Analysis	102
Running Phase I Simulation	103
Setting Up the Phase I Simulation in Signal EM Analysis	103
Running Pre-Phase I Net Filtering	104
Running Phase I and Phase II Simulations in the Signal EM Flow	106
Selecting Nets for Signal EM Analysis	106
Selecting a Time Window for Signal EM Analysis	109
Setting an ratau Value for Current Resolution in Signal EM Analysis	110
Enabling RASIM File Splitting	110
Redirecting Temporary EMIR Files	110
Using Variable Time Steps	111
Running Phase II Simulation	111
Defining Net Pins by Specifying XY Coordinates	113
Running Power Net EMIR and Signal Net EM Together	113
Examining Output Files	113

6. Static IR Drop Analysis	114
Overview of the Static IR Drop Analysis Flow	114
Running Static IR Drop Analysis	116
Specifying User-Defined Pin Waveforms	119
Viewing Static IR Drop Analysis Results	119

7. Running the EMIR Application	120
Getting Started	120
Setting Path	120
Opening the PrimeSim EMIR GUI	121
PrimeSim EMIR Home Page	123
Setting up PrimeSim EMIR Applications for Analysis	124
PrimeSim EMIR Setup GUI	125
Running EMIR Simulations	127
Library EMIR Setup	127
Design Setup Tab	127
Simulation Setup Tab	129
Analysis Setup Tab	130
Fast IR Setup	132
Design Setup Tab	132
Simulation Setup Tab	134
Analysis Setup Tab	135
Dynamic EMIR Setup	137
Design Setup Tab	137
Simulation Setup Tab	139
Analysis Setup Tab	140
Reporting Option Dialog Box	142
Static IR Setup	145
Design Setup Tab	145
Analysis Setup Tab	146
Static Res Check Setup	147
Design Setup Tab	147
Analysis Setup Tab	148
Running Extended Back-Annotation Flows	151
Parasitic Back Annotation Dialog Box	151
Viewing Analysis Results	152
Results Viewer	152

Contents

Pin Current/Voltage Table	156
Results Viewer Context-Sensitive Menu	156
Locating Small Shapes With High-Bin Violation Counts	157
Displaying the Violation Data in a Histogram	159
Redistributing Bin Values	162
Saving and Loading Results Viewer Configurations	162
Saving Results Viewer Configurations	162
Loading Results Viewer Configurations	163
Running EMIR What-If Analyses	163
About the EMIR What-If Analyses	164
Setting Up What-If Analyses	164
Saving What-If Analysis Setups	172
Running and Saving What-If Analysis Results	173
Loading and Viewing What-If Analysis Results	174
Exporting the What-If Resistor Data	174
EMIR What-If Analysis Dialog Box Reference	174
Resistor Table	176
Tracing Least Resistive Paths	177
About the Least Resistive Path Viewer	178
Workflow for Least Resistive Path Tracing	179
Preparing for Least Resistive Path Tracing	181
Software Requirements	181
NPF File	181
Display Setup	181
Tracing Paths From the PrimeSim EMIR GUI	181
Tracing Paths From the Layout Canvas	184
Before You Start	185
Tracing Paths From Violation Shapes	185
Tracing From Instance Pins	190
Tracing Using Shortcut Menu Command	190
Viewing Traced Path Details	192
Deleting Path Tracings	194
Exporting the Traced Path Results	194
Least Resistive Path Viewer Dialog Box Reference	195
Highlight Path	196
Options	197
Full Path Table	197
Path Segment Table	197
Buttons	197

Tcl Commands for Least Resistive Path Tracing	197
8. EMIR Output Reports and Maps	198
EMIR Output Files	198
Violation Map in ASCII Format	198
Display Resource File (DRF)	201
CCDB File	202
FSDB, OUT or WDF Waveform File	204
RADB Simulation Result File	204
RALOG Log File	204
Generating Violation Maps	205
Generating Violation Maps	206
Generating Multiple GDSII Files With One Command	207
Generating GDSII for All Analyses	207
User-Specified Layer Numbers	207
Layer Filtering	208
Names Inserted Into Geometry	208
Generating Layout Formats	208
Automatically Generating Violation Maps	208
9. EMIR Commands	209
Commands for EMIR Pre-processing	209
set_ra_functional_resistor	209
set_ra_net	211
set_ra_net_type	212
set_ra_option	214
set_ra_pwnet_driver	219
set_ra_pwnet_option	220
Commands for EMIR Analysis Post-processing	221
addnetpinxy	225
alterpad	225
analysisname	226
asciicols	226
asciicolsort	230
asciidigits	230
asciimapi	231
asciimapvmax	231

Contents

avgrelaxf	232
ccdbccols	232
ccdbmerge	235
coordunit	236
dtemp	236
emiroptions	237
emldalg	238
emldincdang	239
emthreshproc, emldlayers	239
emwidthflt	245
gdsdatatype	245
gdsdefrw	245
gdsfilechsymb	245
gdslabels	246
gdsmap	247
gdsmapi	247
gdsmapvmax	248
gdsoutmode	248
genspfview	249
gdsprops	249
itflayernames	251
jjmaxlog	252
layerea	252
layerh	252
layermap	252
netfrequency	253
nettype	253
outformat	253
outfiltres	254
outlayers	254
postnets	255
powernet	255
presimtime	257
printi	257
printipad	257
printipin	258
printpwrpinv	258
printv	258
printvmode	258

Contents

pwnetdriver	259
pwrtype	259
ra	259
raformat	260
ralayers	261
ralayout	261
rarve	263
raviewer	263
rawf	264
redi	264
redia	264
rediac	265
redimaxa, redirmsa, rediabsa, rediavg	266
redimaxw, redirmsw, rediabsw, rediavgw	266
rediw	267
redj	268
redv	269
redtrms	271
replace_waveform	272
reportirdrop	273
rvemapi	273
rvemapvmax	274
rveprops	274
signetascii	275
simnets	276
skiplayers	276
source	277
spresenable	277
swin	277
tau	279
temp	280
tmpdir	280
tstart	280
tstop	280
twin	281
usecursign	281
usew4siw	281
vref	282
vts	283

Contents

vtsopt	283
Commands for Signal Net EM Post-processing	284
avgrelaxf	284
dratio	285
iavmin	286
Current and Average Net Current	287
itflayernames	289
mt val	289
net_ccaps	290
outformat	290
rawf	290
selectsignets	291
signetascii	291
skipsignets	292
swin	292
Index	295

About The PrimeSim EMIR Reference Manual

The *PrimeSim™ EMIR Reference Manual* describes how to run the Synopsys PrimeSim electromigration and IR drop (EMIR) analysis. The following sections provide a guide to this manual, as well as to other documentation that accompanies this tool.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)

New in This Release

The PrimeSim EMIR tool is a part of the PrimeSim Reliability Analysis solution. Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in PrimeSim Reliability Analysis: What's New or the *PrimeSim Reliability Analysis Release Notes* on the [SolvNetPlus](#) site.

To see the *PrimeSim Reliability Analysis Release Notes*:

1. Go to the SolvNet Download Center located at the following address:
<https://solvnet.synopsys.com/DownloadCenter>
2. Select "PrimeSim Reliability Analysis," and then select a release in the list that appears.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the release notes site. If you are prompted to log in to SolvNetPlus upon clicking the link, log in, then click the link again to reach the article.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim EMIR tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related products:

- Cadence® Virtuoso®
- Custom Compiler™
- PrimeSim™ HSPICE®
- PrimeSim™ Continuum
- PrimeSim™ Pro and SPCIE
- PrimeSim™ XA
- PrimeWave™ Design Environment

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.

Convention	Description
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

1

Introducing PrimeSim EMIR

This chapter introduces the PrimeSim EMIR tool.

Advances in process technology and changing design styles are increasing the impact of electromigration (EM) and IR drop effects on the performance and reliability of analog, mixed-signal, memory and custom digital IP blocks in advanced technology nodes. The trend in nanometer technologies underscores the need for electromigration to be taken into account for all custom IP blocks. EM rules themselves are growing in complexity with additional dependencies on length and width.

Synopsys' PrimeSim EMIR tool provides the flexibility to support these complex rules, as well as to conduct EM analysis with RMS, average and peak currents. The static resistance check capability is provided for designers to run a quick sanity check to identify any mistakes in layout due to missing vias or narrow wires.

The PrimeSim EMIR tool is part of the Synopsys Reliability Analysis solution that unifies production-proven and foundry-certified reliability analysis technologies covering electromigration and IR drop analysis, high sigma Monte Carlo, MOS Aging, analog fault simulation, and circuit checks (ERC) to enable full-lifecycle reliability verification.

This chapter contains the following sections:

- [Electromigration Analysis](#)
- [IR Drop Analysis](#)
- [Integrating With Other Synopsys Tools](#)

Electromigration Analysis

This section discusses the various trends exacerbating EM effects as well as design and reliability analysis (EMIR) techniques to avoid them, as described in the following topics:

- [Nanometer Technology Trends](#)
- [What Is Electromigration?](#)
- [Electromigration Dependency on Physical Effects](#)

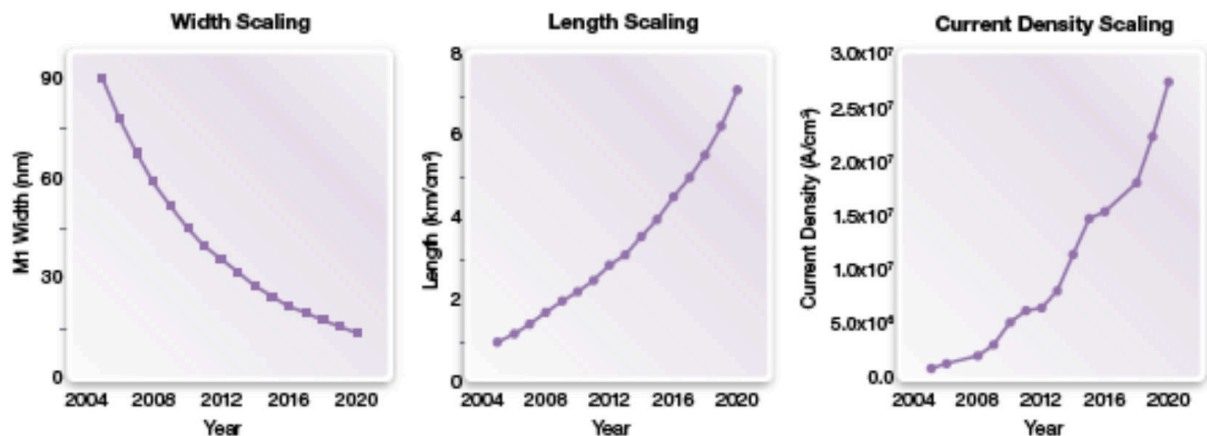
- [Impact of Physical Layout](#)
- [EM Rules for Nanometer Processes](#)

Nanometer Technology Trends

What general technology trends are contributing to the significance of EM and IR drop effects in modern IC designs? [Figure 1](#) provides examples of three distinct trends: width scaling, length scaling and current density scaling.

Driven by Moore's law, metal interconnect widths are decreasing exponentially, resulting in the shrinking of the overall cross-sectional area of interconnect. Then, with increasing integration of functionality and passive devices, the total interconnect length is exploding as well, which means that there are more wires on a die that are susceptible to EM effects. Finally, currents are not scaling proportionally to shrinking wire widths and, therefore, modern ICs have extremely high current densities.

Figure 1 Nanometer Technology Trends



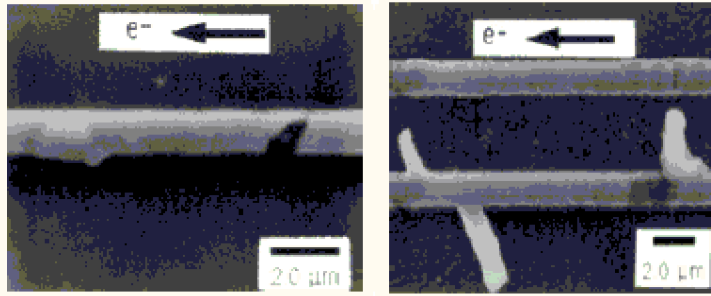
What Is Electromigration?

Electromigration is the gradual displacement of metal atoms in a semiconductor. It occurs when the current density is high enough to cause the drift of metal ions in the direction of the electron flow, and is characterized by the ion flux density. This density depends on the magnitude of forces that tend to hold the ions in place, i.e., the nature of the conductor, crystal size, interface and grain-boundary chemistry, and the magnitude of forces that tend to dislodge them, including the current density, temperature and mechanical stresses.

Failure Mechanisms

There are two different EM failure mechanisms that occur due to asymmetry in the ion flow. The first example in [Figure 2](#) shows a void where the outgoing ion flux exceeds the incoming ion flux, resulting in an open circuit. The second example shows a hillock where the incoming ion flux exceeds the outgoing ion flux, resulting in a short circuit.

Figure 2 *Void (Open Circuit) and Hillock (Short Circuit)*



History of Studies into Electromigration

The EM phenomenon was originally discovered by a lesser-known French scientist named Gerardin over 100 years ago. It only became of practical interest in 1966 with the introduction of the first ICs. A number of research activities were undertaken in the 1960s, the most significant of which was one undertaken by Jim Black at Motorola in 1969. A result of Black's work was an equation that is used to determine the mean time to failure (MTTF) of a metal wire when subjected to EM effects, as shown below:

$$MTTF = \frac{A}{J_N} \times \exp\left(\frac{E_a}{kT}\right)$$

Where the parameters of the equation are defined as:

A	Cross-section area-dependent constant'
J	Current density
N	Scaling factor, usually set to 2
E _a	Activation energy for electromigration
k	Boltzmann constant
T	Temperature

Electromigration Dependency on Physical Effects

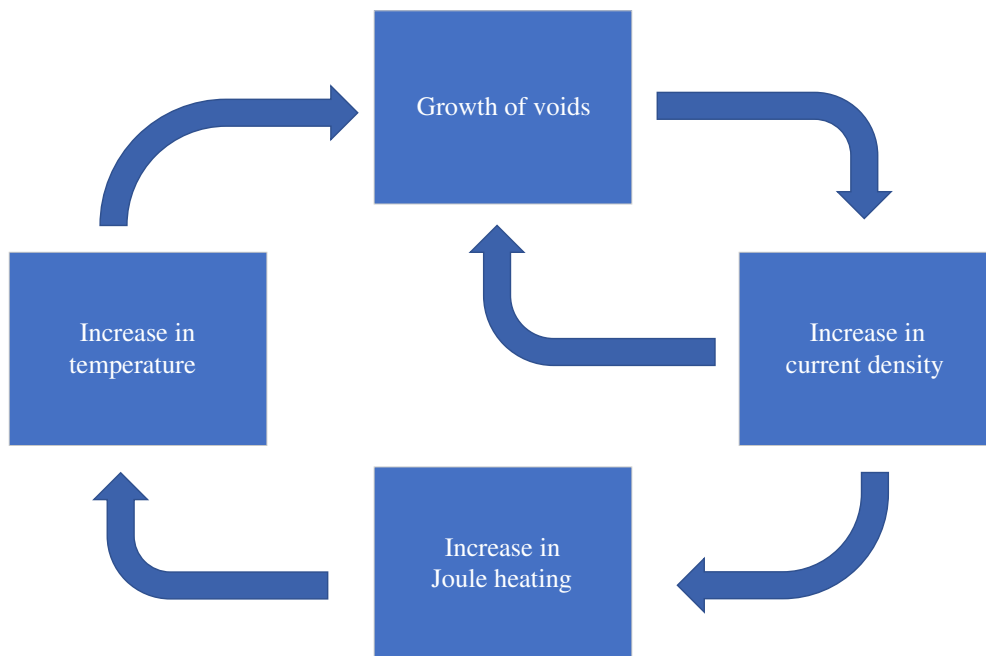
This section contains the following topics:

- [Temperature](#)
- [Wire Width](#)
- [Wire Length](#)

Temperature

From Black's equation, it is clear that failure due to electromigration is dependent on temperature; however, there is a more sinister dependence on temperature that accelerates failures due to voids. [Figure 3](#) highlights a cyclical positive feedback loop that ultimately ends in failure.

Figure 3 Electromigration Dependency on Temperature



Once a void begins to develop in a metal wire, the wire itself becomes narrower at that point. Due to the reduction in width, the current density increases and, therefore, the interconnect temperature increases due to Joule heating. Joule heating is a result of root-mean square (RMS) current. As the temperature of the wire increases, the growth of the void accelerates, and eventually an open circuit occurs. This is why it is critical to also take RMS current into account when performing EM analysis.

Wire Width

Current density is the primary factor influencing electromigration. By increasing the wire width, current density is reduced and susceptibility to EM is reduced. There is one exception to this rule and that is when the wire width falls below the average grain size of the interconnect material. This apparent contradiction is caused by the position of the grain boundaries, which in such narrow wires lie perpendicular to the width of the whole wire. Because the grain boundaries in these so-called “bamboo structures” are at right angles to the current, the boundary diffusion factor is excluded, and material transport is correspondingly reduced.

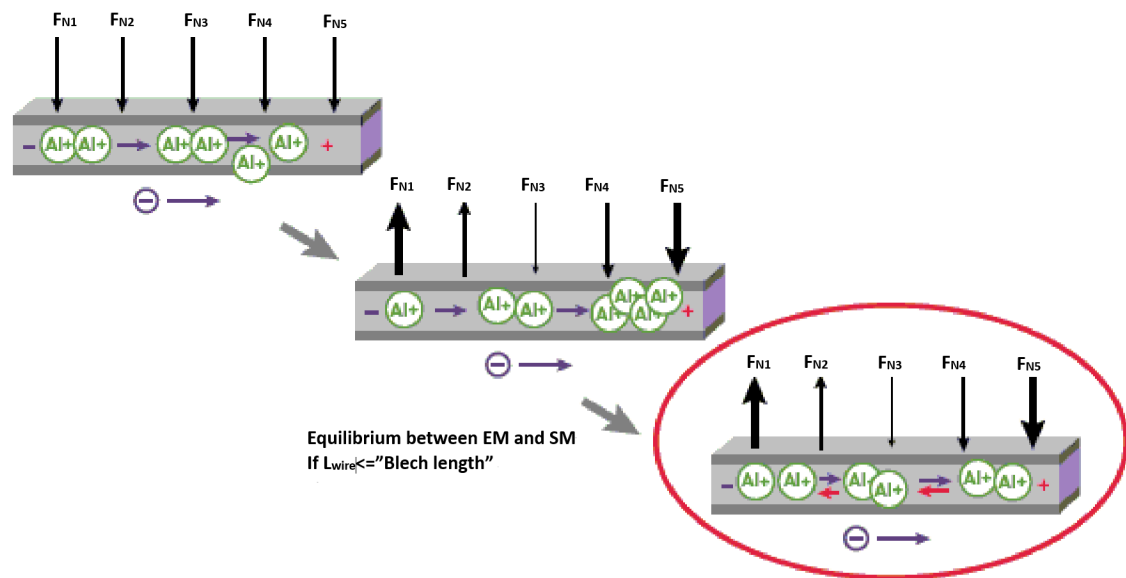
However, the maximum wire width possible for a bamboo structure is usually too narrow for signal lines of large-magnitude currents in analog circuits or for power supply lines. In these circumstances, slotted wires are often used, whereby rectangular holes are carved in the wires. Here, the widths of the individual metal structures in between the slots lie within the area of a bamboo structure, while the resulting total width of all the metal structures meets power requirements.

Wire Length

There is a lower limit for the length of interconnect that will be subjected to the effects of electromigration. It is known as “Blech length”, and any wire that has a length below this limit will not fail by electromigration. Here, a mechanical stress buildup causes a reversed migration process which reduces or even compensates the effective material flow towards the anode.

[Figure 4](#) depicts these so-called “immortal” wires.

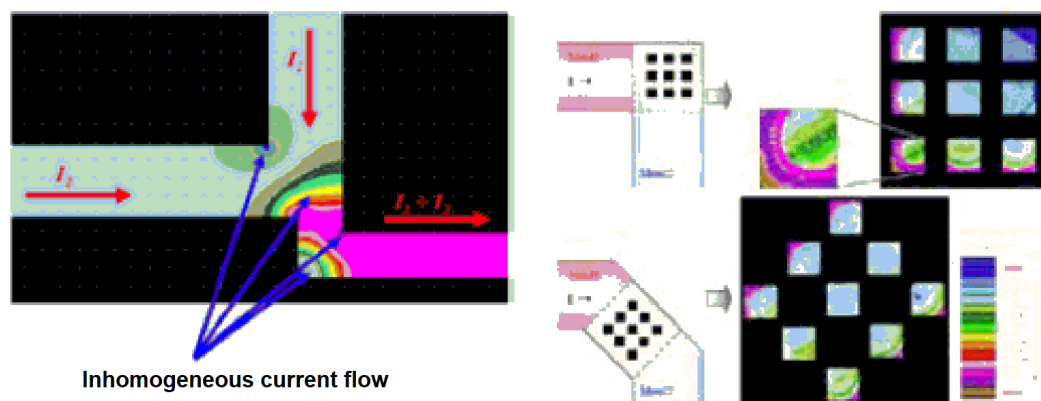
Figure 4 Illustration of “Blech Length”



Impact of Physical Layout

Interconnect layout has an impact on electromigration and current density. Aside from wire width, there are particular layout strategies one can use to minimize EM effects. The objective behind these strategies is to achieve homogenous current flow. Figure 5 shows two different layout strategies to achieve homogenous current flow. Firstly, 90° corners and rapid wire width reduction should be avoided. The current crowding and rapid increase in current density exacerbate EM. The right-hand image shows how critical via arrangement is to ensure homogenous current flow.

Figure 5 Strategies to Minimize EM



EM Rules for Nanometer Processes

Modern nanometer process technologies have very complex rules for current density limits that are not only dependent on wire widths but the length of wire, as well as the dimensions of the wire layers above and below a via layer.

[Table 1](#) and [Table 2](#) demonstrate the complexity of metal and via EM rules.

Table 1 Examples of Metal Layer EM Rules

Layer	Metal Length (μm)	Metal Width (μm)	I _{max} (mA)
Mx	L ≤ A	Any width	4*(em_coef)*width
	L > A	W ≥ B	2*(em_coef)*width
	L > A	W < B	1*(em_coef)*width

Table 2 Example of Via Layer EM Rules

Layer	Metal Length (μm)	Metal Width (μm)	i _{max} (mA)
Mx	L ≤ A; any width	L ≤ A; any width	4*(VIA_em_coef) per via
	L > A; W ≥ B	L ≤ A; any width	2*(VIA_em_coef) per via
	L ≤ A; any width	L > A; W ≥ B	2*(VIA_em_coef) per via
	L > A; W ≥ B	L > A; W ≥ B	2*(VIA_em_coef) per via
	Others	Others	1*(VIA_em_coef) per via

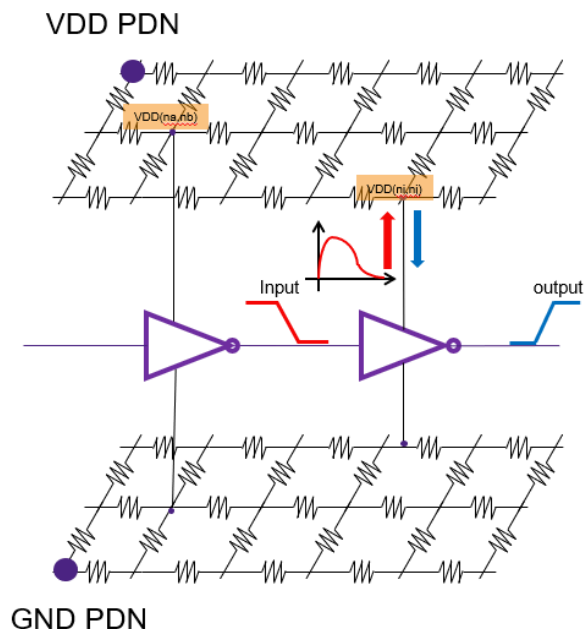
Synopsys PrimeSim EMIR solution provides support for flexible Tcl-based rule creation that enables designers to capture these complex nanometer EM rules. The Tcl interface enables the creation of procedures to calculate RMS as well as average and peak current values that can then also be used to verify the individual rules.

See Also

- [Preparing RATCL Files](#)

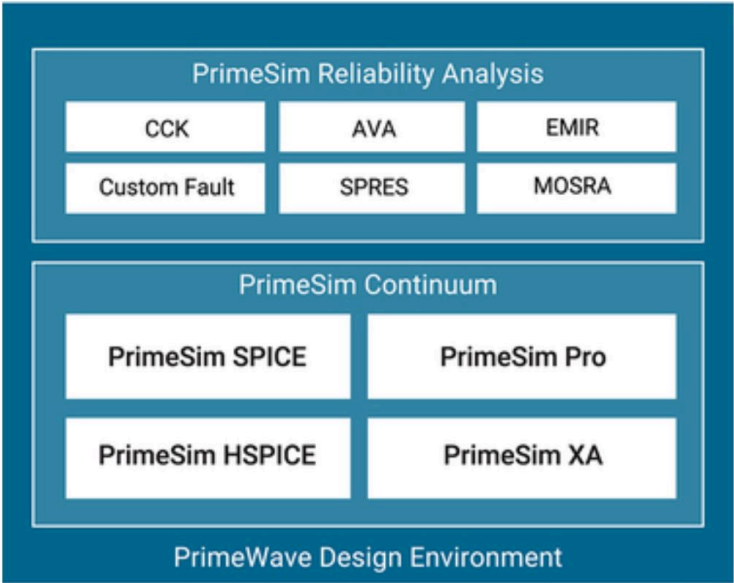
IR Drop Analysis

IR drop is the voltage drop in the metal wires constituting the power grid before it reaches the power pins of internal cells.



reliability verification experience across all PrimeSim Continuum engines and PrimeSim Reliability Analysis technologies with unified setup and results post-processing.

Figure 6 Tools in PrimeWave Design Environment



2

Getting Started

This chapter provides information on how you get started with the PrimeSim EMIR tool.

The PrimeSim EMIR tool supports the PrimeSim XA, PrimeSim Pro and SPICE sign-off simulators for FastSPICE circuit simulation and EMIR analysis. PrimeSim EMIR runs a SPICE simulation and compares the calculated current with violation thresholds to ensure the electromigration rules are respected. The violation thresholds are described in a Tcl file, called RATCL (.ratcl).

The RATCL file provides the Tcl interface to create EM rules for calculating RMS, as well as average and peak current threshold values that can then be used to verify the individual rules. Usually one average, one RMS and one peak current rule is defined per metal layer and one average current rule per via layer. Layer mapping is performed at the beginning of the RATCL file.

The PrimeSim EMIR tool uses internal variables to access the temperature, metal length, metal width and via area from the DSPF netlist.

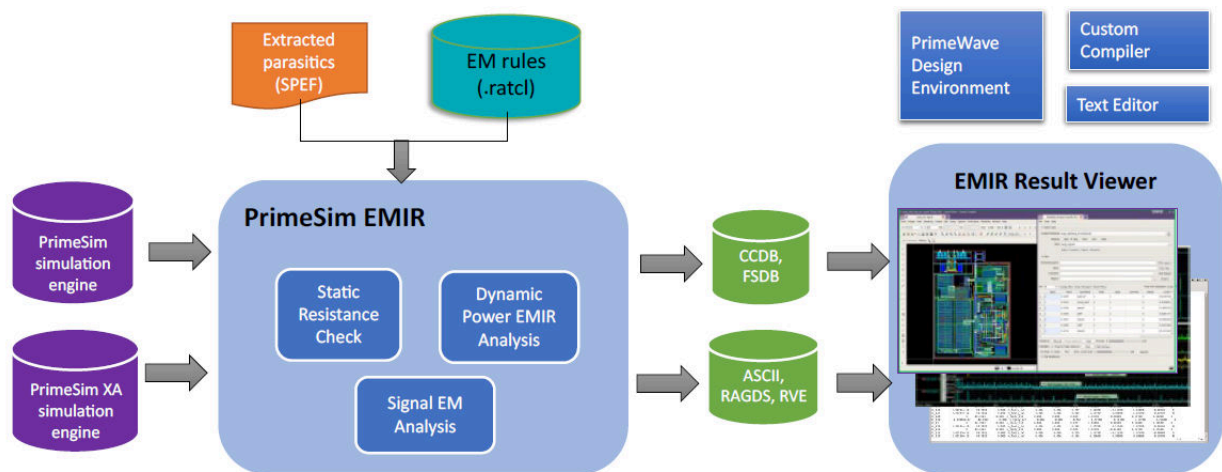
The chapter contains the following topics:

- [PrimeSim EMIR Analysis Flow](#)
- [Setting Up PrimeSim EMIR](#)
- [Running PrimeSim EMIR](#)
- [Static Resistance Calculation](#)
- [Dynamic Power Net EMIR Analysis](#)
- [Signal Net EM Analysis](#)
- [PrimeSim EMIR Post-Processing](#)
- [What-if Analysis](#)

PrimeSim EMIR Analysis Flow

The following figure illustrates the PrimeSim EMIR flow.

Figure 7 PrimeSim EMIR Flow



Setting Up PrimeSim EMIR

Before running the PrimeSim EMIR tool, you must complete the following setup:

- Configuring PrimeSim EMIR License
- Setting the Path to the Executable

Configuring PrimeSim EMIR License

You must set up one of the following environment variables to point to the license paths. When specifying the pointer to the license paths, use the full path of the license file, the `port@host` syntax or a combination of the two syntax types separated by colon.

`SNPSLMD_LICENSE_FILE`

A pointer to the Synopsys license paths

`LM_LICENSE_FILE`

A pointer to all license paths. If you specify both environment variables, `SNPSLMD_LICENSE_FILE` takes precedence over `LM_LICENSE_FILE`.

In C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

In Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

Setting Up the Path to the Executable

The PrimeSim EMIR executable is called by the launch script path. The launch script is in the PRIMESIM_EMIR_INSTALL_DIR/bin directory. You should set up the shell path to include the path to the PrimeSim EMIR launch script.

For example:

C-Shell:

```
set path = (PRIMESIM_EMIR_INSTALL_DIR/bin $path)
```

Bourne, Korn, or Bash shells:

```
export PATH=PRIMESIM_EMIR_INSTALL_DIR/bin:$path
```

For example:

```
set path = ( /tools/synopsys/primesim_emir_2023.12/bin $path )
```

or

```
export PATH=/tools/synopsys/primesim_emir_2023.12/bin:$PATH
```

Sets the path to the launch script at the /tools/synopsys/primesim_emir_2023.12/bin directory.

Running PrimeSim EMIR

To start the PrimeSim EMIR tool, enter the following command at the UNIX command prompt:

```
primesim_emir -flow [flow_options] -c emir_config -o [outpath/]outfile
```

Specify the flow options to analyze and the name of config file in which top level netlist and emir options are provided. For a detailed description about emir config formats and command line options, see *EMIR Flow Manager With PrimeSim Pro*, *PrimeSim SPICE*, *PrimeSim HSPICE* and *PrimeSim XA* sections and *Using Command Line Options* sections.

Using Command Line Options

The following tables describe the command line options and their usage:

Command Line Options	Description
<code>-flow [libchk emir fast_ir static_ir]</code>	Specifies the flow for analysis. libchk: logic library emir analysis emir: analog/memory block emir analysis fast_ir: dram/flash ir analysis static_ir: static analysis
<code>-c [emir_config]</code>	Specifies the file name of emir config file
<code>-dp [number of processes]</code>	Specifies the number of processes to run for power or signal net EMIR analysis in parallel in phase II. For details, see the EMIR Pipeline Processing section in the PrimeSimEMIR Reference Manual.
<code>-dpconfig [CDPL_config_file]</code>	Specifies the Common Distributed Processing Library (CDPL) host configuration file for power or signal net analysis in phase II. For details, see the EMIR Pipeline Processing section in the PrimeSimEMIR Reference Manual. See Common DPLibrary User Guide for the <code>cdpl_config_file</code> syntax.
<code>-o [outpath/]outfile</code>	Specifies the name of the directory and prefix name of the output files. <code>outpath</code> is the name to the output directory path and <code>outfile</code> is the prefix name of the output files. If <code>outpath</code> is not specified, the default is the current run directory. If <code>outfile</code> is not specified, the default name is <code>primesim_emir</code>
<code>-webhelp</code>	Opens a full online help collection in a web browser. Click "How to Use Help" in the lower left corner of the Help window for details about how to use Help features such as the Help icons, Search, and Bookmarks.

Static Resistance Calculation

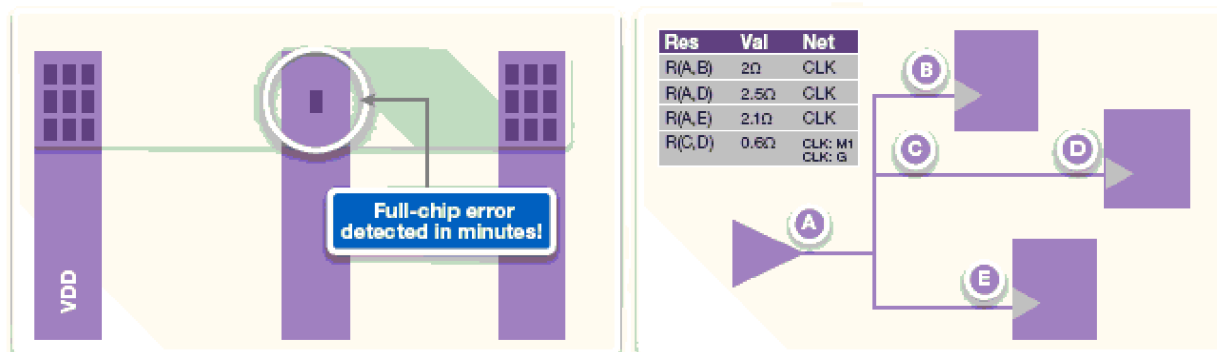
Before performing a complete dynamic simulation for measurement of IR drop effects, a quick check on power and ground bus resistance can be used to catch gross errors, such as missing vias and undersized metalization. PrimeSim determines the resistance from each external power pad connection to the power pins of internal blocks, and generates text reports and resistance maps for graphical visualization of power and ground bus resistance through all metal and via layers. The resistance map is available in GDSII,

CCDB, and other formats. The analysis results can be visualized in the Custom Compiler interface and all resistance paths can be cross-probed and debugged.

The static resistance calculation capability is available only when you run the EMIR analysis with the PrimeSim XA simulation engine.

For a detailed description about running static resistance check with the PrimeSim EMIR tool, see the [Static Resistance Check](#) chapter.

Figure 8 Static Resistance Calculation



Dynamic Power Net EMIR Analysis

The PrimeSim EMIR tool has the capability to analyze IR drop and electromigration in power and ground buses, while simultaneously accounting for the effects of interconnect resistance on dynamic circuit performance. The unique direct-coupled methodology provides highly accurate measurement of power bus currents by performing full-circuit simulation in the presence of back-annotated parasitics. PrimeSim EMIR overcomes the limitations of simulation with millions of extracted parasitic resistors, by incorporating built-in compression and reduction algorithms to maintain accuracy, capacity and performance.

Using this direct-coupled methodology, PrimeSim EMIR tool acquires accurate transient current waveforms during dynamic simulation and uses them for detailed measurement of IR drop and current density in the complete, unreduced power and ground nets. An alternative approach taken by the PrimeSim EMIR tool is to estimate IR drop by characterizing cells before they are placed into a circuit layout. Graphical outputs in GDSII, CCDB and other formats allow designers to overlay results on their layout design for analysis and debug.

For more information, see the [Power Net EMIR Analysis](#) chapter.

Signal Net EM Analysis

The PrimeSim EMIR tool calculates current densities in narrow signal nets to determine their susceptibility to electromigration. Bidirectional current flow is correctly considered, including calculation of the RMS currents required for monitoring Joule heating within the design. These results are presented in GDSII, CCDB or other formats for physical visualization. PrimeSim EMIR can include all of the design's extracted coupling capacitors in addition to the grounded capacitors. This provides a high level of precision in determining the current entering or leaving segments of signal net interconnect, thus enhancing the quality of results.

For more information, see the [Signal Net EM Analysis](#) topic.

PrimeSim EMIR Post-Processing

The PrimeSim EMIR tool provides the post-processing feature to process the simulation results for calculating electromigration and voltage drop violations. By default, the post-processing feature is automatically invoked in the analysis stage of the EMIR analysis flow. In the two-phase EMIR analysis with PrimeSim Pro or PrimeSim SPICE, you can manually invoke the post-processing function. This is useful when you want to run multiple EMIR analysis runs on the same design or to generate output files in different formats (or contents) for various purposes.

For more information, see the post-processing topics in [EMIR Analysis With PrimeSim XA](#) and . For a detailed description about how to manually invoke post-processing in two-phase EMIR flow, see the [topic](#).

What-if Analysis

The PrimeSim EMIR tool provides the *what-if* analysis feature to analyze and make changes to an existing SPF file before doing actual changes in the layout. In what-if analysis flow, you first specify which resistors to modify at the time of reading the SPF file for back-annotation at phase I of EMIR flow. The PrimeSim EMIR tool reads in the modified SPF file and uses it as if it were extracted from the modified original layout. This allows you to make changes without going through the full layout/extraction/simulation flow, thus shortening the turnaround time in the entire post-layout verification flow.

For more information, see [Command-Based What-If Analysis](#).

3

Static Resistance Check

This chapter provides information on how to perform static resistance calculation with the PrimeSim XA simulation engine.

This chapter contains the following sections:

- [Introducing Static Resistance Check](#)
- [Running Static Resistance Check](#)
- [Generating Output Reports](#)
- [Post-Processing Static Resistance Calculation Results](#)
- [Loading Static Resistance Checking Results to Layout Editors](#)
- [Commands for Static Resistance Check](#)

Introducing Static Resistance Check

During the layout design of complex nanometer ICs, it can be valuable to quickly assess the parasitic effects of power net wiring, without performing a complete simulation. The PrimeSim EMIR tool provides the static resistance check capability to perform the complex calculation of all pad-to-pin or pad-to-internal instance node resistances.

More specifically, for each instance pin (*|I) in the DSPF file, an effective resistance is calculated to all pads (*|P) connected together. To perform such calculations, no prelayout netlist is necessary and only the net description in DSPF format is needed. The result of the calculation is generated as a list of instance pins and their resistances (in the order from largest to smallest values). In addition to the text output, a graphical display of the resistance distribution on the layout can be generated.

Note:

PrimeSim static resistance calculation only considers the parasitic resistors within a NET section in the DSPF file for the calculation of pad-to-pin or pad-to-internal node resistance. It does not take into account the functional resistors extracted in the Instance section.

Running Static Resistance Check

To run static resistance checking, use the command:

```
xa -r <resistance_command_file> [-o <output_prefix>]
```

The *resistance_command_file* contains setup commands for resistance checking. See the [Commands for Static Resistance Check](#) section for a complete list of the commands.

Examples

The following example runs static resistance check and sets the prefix of the output files as *res*.

```
xa -r r.tcl -o res
```

The *r.tcl* command file contains the following information to specify the DSPF file name and net name for calculation.

```
file extracted.spf
net gnd
rmin 0.05
```

When the static resistance calculation is complete, the tool generates the following files:

- *res-gnd.rlog*: Log file
- *res-gnd.rout*: Resistances from pads to pins, sorted in decreasing order, pin coordinates (if available) and names
- *res-gnd.minr100nr1000.report*: A subset of the *res-gnd.rout* file
- *res-gnd.png*: A color map of resistances (red for maximum resistance, green for zero) for quick viewing
- *res-gnd.sprtcl* file

Calculating Layer Resistance Sensitivity and Layer Equivalent Resistance

The static resistance check tool calculates effective resistance from the top-level port of the power network to each device pin. When checking contribution of different interconnect layers to pad-to-pin resistance, you can use the [layer0ohm](#) command to zero the layer resistivity to exclude contribution of the specified layers to the total pad-to-pin resistivity.

To calculate layer-wise contribution to the effective resistance, use the `layer_res` and `layer_res_nworker` commands by adding the commands to your command file used for static resistance check. For example,

```
layer_res -sens m1 m2 m3
layer_res_nworker 1
```

where

- m1, m2 and m3 are the layer names whose contribution to effective resistance is required. Any valid layer name from the DSPF file can be chosen, including via and contact layer names.

The tool creates an additional report (*.sens) in the output folder. This *.sens report file describes the contributions of the specified layers to the total effective resistance. Taking the following report as an example, for #1, the total effective resistance is 47.85 Ohms, where 10.50 comes from layer m1, 0.78 from m2, and 0.44 from m3. The percentage contribution of each layer to the total effective resistance is also reported.

Layer resistance sensitivity (Ohm). dR(lvl) = R_orig - R_layer0ohm (dR/R_orig %)

Layer mapping:
lvl layer_name
139 m1
141 m2
144 m3

Count	R_orig	dR(139)	dR(141)	dR(144)	node_name
#1	119.09	10.52 (8.83%)	0.789993 (0.66%)	0.449997 (0.38%)	vsse:15346
#2	119.06	10.49 (8.81%)	0.790001 (0.66%)	0.449997 (0.38%)	vsse:15335
#3	118.84	10.53 (8.86%)	0.789993 (0.66%)	0.449997 (0.38%)	vsse:15344
#4	118.8	10.49 (8.83%)	0.790001 (0.66%)	0.450005 (0.38%)	vsse:15339
#5	108.99	1.23 (1.13%)	1.09 (1.00%)	0.989998 (0.91%)	vsse:15138
#6	108.96	1.2 (1.10%)	1.06 (0.97%)	0.959999 (0.88%)	vsse:14964
#7	108.94	1.23 (1.13%)	1.09 (1.00%)	0.990005 (0.91%)	vsse:15136
#8	108.93	1.18 (1.08%)	1.03 (0.95%)	0.93 (0.85%)	vsse:14790
...	...	...	...	...	...

The log file shows that the tool runs a sub-task for each layer contribution:

```
Master creates tasks.
Task for layer "m3" is created.
Task for layer "m2" is created.
Task for layer "m1" is created.
Created 3 task(s) Waiting for Job to finish ... ..
```

When all jobs are completed, a *.sens file is created in the output folder.

See Also

- [layer0ohm](#)
- [layer_res](#)
- [layer_res_nworker](#)

Generating Output Reports

When static resistance check is complete, the tool generates the *.rout file which reports the resistances from pads to pins, sorted in decreasing order, pin coordinates (if available) and name shapes that have been created. You can also choose to save the output report in the form of heat map in CCDB format, which can be loaded into EMIR Viewer in the Custom Compiler or PrimeWave Design Environment tool.

You can generate additional reports by post-processing the calculation results, by following the steps described in the [Post-Processing Static Resistance Calculation Results](#) section.

Example

Here is a sample command file used for static resistance check:

```
file DSPF.SPF  
net vsse
```

Following is the output ASCII report:

```
;; Synopsys Inc.
;; RA-SPRES linux64 Linux 3.10.0 64Bit Version U-2023.03
;; Tracking No - RA-SPRES 2023.07.4 8031322
;; Copyright (C) 1998 - 2023. All rights reserved.
;;
;;

;;net: vsse
;;sortby=R
;;xmin=0.092 ymin=2.415 xmax=91.585 ymax=63.108
;;rmin=0
```

Count	R	W/L	(x:y)	layer	hname	name
#1	47.85	0.13	(67.982:27.273)	psd	xi0/xi0/mp7:d	
#2	42.368	0.075	(67.936:17.736)	psd	xi0/xi0/mp4:d	
#3	40.496	0	(49.885:51.626)	psub	xi0/xi1/dd15:pos	
#4	40.495	0	(49.885:51.626)	psub	xi0/dd10:pos	
#5	39.994	0	(49.885:5.103)	psub	xi0/dd18:pos	
#6	39.994	0	(49.716:2.99)	psub	xi0/xi0/dd0:pos	
#7	39.994	0	(65.817:5.807)	psub	xi0/xi0/dd12:pos	

See Also

-

Post-Processing Static Resistance Calculation Results

You can run post processing to filter information from a previous static resistance calculation with the `-rp` command line option and with or without the `-o` option. If you omit the `-o` option, the tool overwrites the existing output files.

Because the tool reads the original Tcl file during post processing, all commands not intended for violation map generation (for example, `net`, `file`, and so on) are ignored. You can use the `.sptcl` file as well as the original Tcl file for additional post processing commands.

Run the following steps to perform post processing on static resistance checking results:

1. Perform the initial static resistance check. For example:

```
xa -r test.tcl -o outdir/result
```

The following output files are in the outdir directory:

- result-rmin0.01.rlog
- result-rmin0.01-vdd.rout
- result-rmin0.01-vdd.gds
- result-rmin0.01-vdd.tech
- result-rmin0.01-vdd.lmap
- result-rmin0.01-vdd.drf
- **result-rmin0.01-vdd.npf**
- **result-rmin0.01-vdd.sprtcl**
- **result-rmin0.01-vdd.sprdb**

You can use the files in **bold** for post processing static resistance checking results.

2. Before you run post processing, edit the result-rmin0.01-vdd.sprtcl file to filter results from the initial run. For example, you can add `gdsthresh -upbnd 5` to the result-rmin0.01-vdd.sprtcl file to filter resistors in the violation map (VM) according to their upward boundary.
3. Run static resistance check post processing. For example:

```
xa -rp calc_results [-o pps_results]
```

where

calc_results is the path and prefix for resistors calculation results and *pps_results* is the path and prefix for post-processing results.

```
xa -rp outdir/result-rmin0.01-vdd -o outdir/rebin
```

This command runs static resistance check post processing using results obtained for the vdd net and writes the new output files (including violation map) into the outdir directory with the prefix rebin:

- rebin.rplog
- rebin.gds
- rebin.tech

- `rebin.lmap`
- `rebin.drf`

The `rebin.rplog` file contains the post-processing log information.

Limitations

- Static resistance check post processing only regenerates violation maps in the following formats: GDSII, PNG and RVE; it does not regenerate the `.rout` file.
- Static resistance check post processing only takes into account the Tcl commands that control the output of violation maps. All other commands are ignored. Following are the supported commands:
 - `gds`
 - `gdslayer0`
 - `gdslayers`
 - `gdslabels`
 - `gdsthresh`
 - `gdsviewer`
 - `outfiltres`
 - `outlayers`
 - `png`
 - `pngthrsh`
 - `rve`

Loading Static Resistance Checking Results to Layout Editors

You can import the output data from static resistance checking to the following layout editors:

- Laker
- Synopsys Custom Compiler
- Cadence Virtuoso

Viewing Violation Maps in Laker Layout Editor

When you use the `gdsviewer laker` command, static resistance calculation writes the following files that the Laker layout editor uses to display violation maps:

- GDSII file
- Technology file (default file name is `laker.tf`)
- Property map file (default file name is `laker.pmap`)
- Display file (default file name is `laker.dsp`). This file is loaded automatically in the layout editor.

To load a violation map, do the following steps.

1. In the Laker home page, choose File > Input > Stream.

This command opens the Input Stream dialog box.

2. In the Input File Name field, enter the name of the GDSII file generated from static resistance calculation.
3. In the Library Name field, enter any library name.
4. In the Technology File dialog box, enter the name of the technology file that is generated from static resistance checking in the ASCII field.
5. On the Property tab, enter the property map file name (*.pmap) in the Prop. Map File field.
6. Click OK.

The violation map displays in the Laker canvas for viewing and debugging errors.

Viewing Violation Maps in Custom Compiler Layout Editor

For instructions on how to load GDSII data into the Custom Compiler layout editor, see the *Custom Compiler Platform User Guide*.

Viewing Violation Maps in Cadence Virtuoso Layout Editor

To load a violation map in Cadence Virtuoso layout editor, do the following steps:

1. In the Cadence Virtuoso tool, choose File > Import > Stream from the CIW menu. The dialog box that opens includes the following information:
 - Run Directory: The working directory containing the .gds, .tech, and xa.drf files.
 - Input File: <fileName>.gds.
 - Top Cell: <fileName> if specified in rlayout.
 - Library Name: The library into which <fileName>.gds is loading.
 - ASCII Technology File Name: <netName>.tech
2. If the xa.drf file has not been loaded during the current session, load the xa.drf file by entering the following syntax in the CIW input command area:

```
drLoadDrf("<working directory>/prefix.drf", nil)
```
3. Open the following information:
 - view: layout
 - cell: <fileName>
 - library: The library into which <fileName>.ragds is loaded.

Commands for Static Resistance Check

The PrimeSim EMIR tool provides the following commands for static resistance checking.

- [addnetpin](#)
- [addnetpinxy](#)
- [addresistor](#)
- [ccdb](#)
- [ccdbcols](#)
- [changeI2P](#)
- [chkmissstrap](#)
- [chkmissstrapboxsize](#)
- [chkmissstraplayers](#)

Chapter 3: Static Resistance Check

Commands for Static Resistance Check

- [chkmissvia](#)
- [chkmissviaboxsize](#)
- [chkmissvialayers](#)
- [file](#)
- [gds](#)
- [gdslabels](#)
- [gdslayer0](#)
- [gdslayers](#)
- [gdsthresh](#)
- [gdsviewer](#)
- [ipin](#)
- [itflayernames](#)
- [layer0ohm](#)
- [layerfactor](#)
- [layer_res](#)
- [layer_res_nworker](#)
- [mergefinger](#)
- [modifyresistor](#)
- [mt](#)
- [net](#)
- [netdeletepad](#)
- [netinclude](#)
- [node2node](#)
- [node2node_option](#)
- [op2ipin](#)
- [outfiltres](#)
- [outlayers](#)

- [png](#)
- [report](#)
- [rmax_thresh](#)
- [routcols](#)
- [rmin](#)
- [rve](#)
- [subnode](#)
- [sortby](#)
- [temp](#)
- [tmpdir](#)

addnetpin

This command is used to specify a temporary net pin.

```
addnetpin <net_name> <node_name>
```

where <net_name> is the power net name that adds a pin, and <node_name> is the name of the instance pin or subnode.

Example:

```
addnetpin vdd vdd:21  
addnetpin vdd XI0|XI10|MP1:S
```

Restrictions:

Only one <node_name> can be defined in each command.

addnetpinxy

This command adds a pin by x- and y-coordinates.

```
addnetpinxy <net name> x=<coord > y=<coord> layer=<layer name|number>
```

or

```
addnetpinxy -spfunits <unit> -pinxydist <dist>
```

where <coord> defines the coordinates, specified in meters, or as number with unit letter.
<unit> specifies units used for geometry values. Use space as a separator between

parameters. Layer number corresponds to the number in the layer map table from the DSPF file.

```
addnetpinxy -spfunits 1e-6 -pinxydist 1u
addnetpinxy vdd x=411.16u y=10.93u layer=alumi
addnetpinxy vdd x=0.2u y=21.4u layer=15
```

addresistor

This command is part of the what-if capability to add or insert a new resistor and specifies where the new resistor will be added.

```
addresistor net_name new_resistor_name node1 node2 value \
[[-layer] layer]] [-w|-a value]
```

For example:

```
addresistor vdd newresvdd vdd:7 vdd:76 100 -layer metall1 -a 10
```

The above example adds a new 10 ohm via resistor, named newresvdd, between node vdd:7 and node vdd:76 in the vdd RC network.

ccdb

The `ccdb` command controls the generation of `.cc.db` file during static resistance checking.

```
ccdb [ 0 | 1 ] [ -mag value ]
```

0

Disables the generation of the output `.cc.db` file. This is the default.

1

Enables the generation of the output `.cc.db` file.

-mag

Specifies the magnification value.

ccdbcols

The `ccdbcols` command adds table columns and specifies the sequence of the columns in the output `.cc.db` file. During static resistance checking, you can add a shape table to the output `.cc.db` file.

Not all tables and all columns are controllable. Some columns, such as ids or coordinates, are not in the list of column names. The list of column names is preceded by a flag for which content it is applied.

Syntax

```
ccdbcols sh dataname
```

where *sh* adds a shape table with any of the following columns you specify:

Column Name	Description
resname	Resistor name
resistance	Resistor value
layer	Resistor original layer name
width	Resistor width
area	Resistor area for vias only.
length	Resistor length
nodes	Resistor node names

changel2P

This command converts instance pins to net pins.

```
changeI2P [-pinname {<pname>}] [-pintype {<pptype>}] \  
[-net {<nname>}] [-excludenet {<nname>}]
```

You can convert pins by:

- Pin names
- Pin types
- Net names

You can use the asterisk character (*) to specify name matching patterns.

chkmissstrap

This command flags a missing metal strap when the resistance difference between two nodes within metal layers is larger than a specified threshold and the resistance network can be improved by adding a metal strap connecting the nodes.

```
chkmissstrap <layer_list> {-d <distance>} { -r <resistance_threshold> \
  {-ub <ub_value> | -bins <b10> <b9> ... <b1>}
```

`layer_list`

List of metal layer names, which can contain one or more layer names.

`distance`

Area vicinity radius. Unit for distance is meter.

`resistance_threshold`

User-defined threshold.

`ub_value`

Upper boundary threshold for the resistance difference. When specified, the tool puts violations ($|R1 - R2| > DR$) with values larger than `<ub_value>` into the highest bin (blinking red). All the other ones are evenly distributed into the bins below `<ub_value>`.

`b10 b9 ... b1`

Lower boundary of bin values range. There must be nine values starting from the most violated bin to the least violated bin.

First, `chkmissstrap` calculates resistance from a pad to all nodes. Within the specified layers it checks all the node pairs within the user-defined vicinity radius. Then it compares resistance between the node pairs. Violations occurs when both condition 1 and condition 2 are valid:

- Condition 1: $|R1 - R2| > DR$, where DR is user-specified resistance difference threshold.
- Condition 2: $D < DD$, where d is distance between two nodes defined as $(\sqrt{dx+dy})^2$ and DD is the user-specified distance threshold.

Each node pair is reported in the .ascii file in addition to a shape in the GDSII or CCDB file. The shape is a rectangle with two points at the two nodes (it can be non-orthogonal). The rectangle width is user-defined. Finally, it distributes the highlights/shapes into the bins according to the value of $|R1 - R2|$.

Example:

```
chkmissstrap m1 -d 100u -r 200 -ub 400
```

The above example checks missing metal straps between nodes in layer m1. Missing straps with the resistance difference larger than 400 Ohms are placed in the highest bin. All others violations are evenly distributed among the rest 9 bins (from 400 down to 0).

chkmissstrapboxsize

The command generates rectangular shapes (boxes) connecting two nodes with reported violations.

```
chkmissstrapboxsize <width_value>
```

width_value

Specifies a width of the rectangle (box shape) that is generated between two points (nodes) of every violation. The default is 1e-6 (1 micron).

For the output, the tool generates rectangular shapes (boxes) connecting two nodes with reported violations. Those boxes are placed into the same output file as other GDSII/CCDB shapes. The file name is:

```
<prefix>-rmin <value>-<netname>.gds  
<prefix>-rmin <value>-<netname>.cc.db
```

where

<prefix>: Output prefix

<value>: Rmin value

<netname>: Name of the net

You can specify layer numbers used for missing straps shapes in the [chkmissstraplayers](#) command.

chkmissstraplayers

This command specifies layer numbers used for missing straps shapes.

```
chkmissstraplayers \  
  {<layer_i> <layer_j> | <layer_i>,<layer_j> | <layer_i> - <layer_j>}
```

layer_i

Layer numbers used for i-th bin.

where <layer_i> are layer numbers used for i-th bin. Layer numbers used for missing straps shapes can be specified in the [chkmissstraplayers](#) command.

Example:

```
chkmissstraplayers 200-209  
chkmissstraplayers 250 251 252 253 254 255 256 257 258 259
```

chkmissvia

The command flags missing vias between two metal layers. Any intersection of geometries of two specified layers with a resistance value difference within the threshold ranges you specify is flagged as a violation. If the resistance difference is higher than the upper boundary, the violation is in blinking red.

```
chkmissvia <layer1> <layer2> {-ub <ub_value> | -bins <b10> <b9> ... <b1>}
```

layer1

Name of the metal layer lower in a multi-layered structure.

layer2

Name of the metal layer higher in a multi-layered structure.

ub_value

Upper boundary threshold for resistance difference between two metal layers at the intersection.

b10 b9 ... b1

Lower boundary of bin values range. There must be 9 values starting from the most violated bin to the least violated bin. The bin threshold you specify starts from a greater resistance (lower bound) and less than or equal to another resistance (upper bound).

Missed vias have the following name:

```
<prefix>-rmin<value>-<netname>mvias.gds  
<prefix>-rmin<value>-<netname>.cc.db
```

where:

<prefix>: Output prefix

<value>: Rmin value

<netname>: Name of the net

You specify the layer numbers for missed vias using the [chkmissvialayers](#) command.

chkmissviaboxsize

The command generates square shapes (boxes) reported as missed vias.

```
chkmissviaboxsize <width_value>
```

Where `<width_value>` specifies a width of the square shapes. The default is 1e-6 (1 micron).

chkmissvialayers

The command specifies the via layers to be check with the `chkmissvia` command.

```
chkmissvialayers \
    {<layer_i> <layer_j> | <layer_i>,<layer_j> | <layer_i> - <layer_j>}
```

`layer_i`

Layer numbers used for i-th bin.

The following example shows how you can specify 10 bin layers:

```
chkmissvialayers 200-209 (this could be used as default value if the
    chkmissvialayers command is not specified).
chkmissvialayers 250 251 252 253 254 255 256 257 258 259
```

For the following Tcl file example:

```
file test.spf
net vdd
chkmissvia m1 m2 -ub 50
chkmissvialayers 200-209
```

The tool only analyzes the vdd net; it initializes the calculation of resistances from the vdd pin to all nodes and finds missed vias between the `m1` and `m2` layers. Missed vias with a resistance difference more than 50 Ohms are placed into the 10th bin. All others are evenly distributed among the rest of the nine bins.

file

The command specifies the name of the DSPF file containing a power net description `<DSPF_file_name>` file.

```
file gnd.dspf
```

gds

The command specifies GDS output in GDSII format.

```
gds <box_width> [-spfunits <val>] [-mag <m_val>] \
    [-texttype <val>] [-resvalue]
```

Every pin is output as a square of `<box_width>`. The `-spfunits` option is optional and specifies geometry units within the SPF or DSPF file. The default value for units is 1e-6.

If this option is skipped, the tool will process all coordinates as if they are in microns. Optional parameter `-mag` specifies magnification coefficient for geometry in SPF/DSPF file. Negative `<m_val>` means that coordinates will be divided by `<m_val>`. Positive `<m_val>` multiplies coordinates.

If you specify a `-texttype` value, the specified value is used for GDSII output. If you specify `-resvalue`, the tool also outputs the following as properties of the shapes associated with resistor geometries:

- The higher resistance of the two resistor terminals
- The layer name
- The subnode or device terminal name
- The resistor name
- The resistor value
- The resistor width

gdslabels

The command specifies the text labels generated and put into the *.gds file.

```
gdslabels [all] [pads] [maxpin]
```

All pad names are specified as *|P in DSPF files. The pin name specifies maximum resistance. The `[all]` option specifies text labels for all pads and pins.

gdslayer0

In the GDS output, pins with the smallest resistance are output as `<layer_num>` in `gdslayer0`.

```
gdslayer0 <layer_num>
```

The `<layer_num>` default is 100.

gdslayers

The command defines the output GDSII layer numbers for 10 violation bins. The default layer numbers are: 100-109. For example:

```
gdslayers 210-219
```

The above example defines violation map GDSII layer numbers to be 210 to 219. Layer 219 stores the most violated resistors.

gdsthresh

The `gdsthresh` command is an optional command used to control the distribution of resistors within levels of a violation map. If this command is skipped, all resistors are regularly distributed among 10 levels.

```
gdsthresh {[-lobnd <l_val>] [-upbnd <u_val>]} | {[-nproc <p_val_10> \
    <p_val_9> <p_val_8>...<p_val_1>]}
```

`gdsthresh` has two forms as follows:

1. In its first form, `gdsthresh` specifies the threshold for the lowest and highest violation levels. Resistors between these thresholds will be distributed at regular intervals among 8 other levels.
 - `-lobnd`: Specifies lower boundary. All resistors with value lower than `<l_val>` will be placed into the lowest violated level (dark green color).
 - `-upbnd`: Specifies upper boundary. All resistors with value higher than `<u_val>` will be placed into the most violated level (blinking red color).
2. In its second form, `gdsthresh` specifies the percentage distribution of resistors among violation levels as follows:
 - `-nproc`: Specifies the percentage of all resistors that must be presented within the particular level, starting from most violated (blinking red color). All levels without specification (skipped value `<p_val_i>`) will be regularly distributed.

gdsviewer

The command selects the viewer that the tool uses to generate technology or display resource files.

```
gdsviewer [virtuoso|calibre|laker|ccompiler]
```

- `virtuoso` specifies the Cadence® Virtuoso® layout editor (default), which requires a technology file, DRF file, and layer mapping file.
- `calibre` specifies the Mentor® Calibre® DESIGNrev™ layout editor, which requires a technology file, DRF file, and layer mapping file.

- `laker` specifies the Laker layout editor, which requires a technology file, property mapping file, and display file.
- `ccompiler` specifies the Custom Compiler™ layout editor, which requires a technology file, DRF file, and layer mapping file.

ipin

The command explicitly specifies instance pins for resistance checking. By default, resistance is calculated for every instance pin. Using `ipin` reduces the number of resistance calculations required by specifying the list of instance pins.

```
ipin <pin_pattern>
```

Example:

```
ipin vdd:f46 vdd:f8? vdd: f97*
```

itflayernames

```
itflayernames yes|no
```

In the SPF file, the StarRC tool can create two columns: one column with the LVS layer names and one column with the ITF layer names. By default, the PrimeSim EMIR tool uses the LVS layer names.

`yes`

Specifies to use the ITF layer names.

`no`

Specifies to use LVS layer names. This is the default.

Example:

The following example shows an SPF file with LVS layer names (`M3, M2, M1,..`) and ITF layer names (`metal3, metal2, metal1,...`):

```
*LAYER_MAP
```

```
*0 SUBSTRATE
*1 M3 ITF=metal3
*2 M2 ITF=metal2
*3 M1 ITF=metal1
*4 POLY ITF=poly
*5 nsd ITF=nsd
*6 psd ITF=psd
```

layer0ohm

Resistors with indicated layer names and/or numbers will be shorted if resistor layer information is available in the DSPF file. Layer names in `layer0ohm` should be cross-referenced to layer numbers in `layer_map` section of the DSPF file.

```
layer0ohm <layer_nameornum1> [<layer_nameornum2> ...]
```

Example:

```
layer0ohm 5 mtl cont
```

layerfactor

Resistor values belonging to an indicated layer in `layerfactor` are scaled by a factor.

```
layerfactor <layer_nameornum> <scale_factor>
```

Example:

```
layerfactor mtl 0.1  
layerfactor 5 100
```

layer_res

Estimates sensitivity or equivalent resistance for the specified layers.

```
layer_res -sens|-equiv layer1 [layer2 layer3 ...]
```

The command uses Common Distributed Processing Library (CDPL) for implementing internal loops to estimate sensitivity or equivalent resistance for the specified layers. CDPL enables distributed parallel processes, where each process runs the `layer0ohm` command to zero the corresponding layer resistivity based on the `layer_res` command setting and then estimates effective resistance for each run.

To specify the number of CDPL workers, use the `layer_res_nworker` command.

Syntax Definitions

`-sens | -equiv`

Chooses to calculate layer sensitivity or equivalent resistance.

- `-sens`: When specified, the `layer0ohm` command
 - Sets values for all resistors in a layer to 0.
 - Computes and saves effective resistances.
 - Repeats the previous two steps for all the specified layers.
- `-equiv`: When specified, the `layer0ohm` command
 - Sets values for all resistors in all layers to 0, except those on the specified layers.
 - Computes and saves effective resistances.
 - Repeats the previous two steps for all the specified layers.

```
layer1 [layer2 layer3 ...]
```

Specifies the name of the layers for sensitivity and equivalent resistance estimation.

Outputs

The results are collected in a text file `<output_prefix>-<net_name>.sens` or `<output_prefix>-<net_name>.equiv`. Each file includes layer mapping `lvl:layer_name` and columns with original pad-to-pin effective resistance and `dR=R_orig-R_layer` or Requiv resistance.

Example

When the following command is specified in the SPRES Tcl file:

```
layer_res -sens ME1 VI1 ME2 VI2 ME3
```

It means that sensitivity analysis will run on the 5 layers (ME1, VI1, ME2, VI2 and ME3). When analysis is completed, the following result file `*.sens` is generated in the output directory.

Following is an excerpt of the output .sens result file:

```
Layer resistance sensitivity (Ohm). dR(lvl) = R_orig - R_layer0ohm (dR/R_orig %)
```

Layer mapping:

lvl	layer_name
139	m1
141	m2
144	m3

Count	R_orig	dR(139)	dR(141)	dR(144)	node_name
#1	119.09	10.52 (8.83%)	0.789993 (0.66%)	0.449997 (0.38%)	vsse:15346
#2	119.06	10.49 (8.81%)	0.790001 (0.66%)	0.449997 (0.38%)	vsse:15335
#3	118.84	10.53 (8.86%)	0.789993 (0.66%)	0.449997 (0.38%)	vsse:15344
#4	118.8	10.49 (8.83%)	0.790001 (0.66%)	0.450005 (0.38%)	vsse:15339
#5	108.99	1.23 (1.13%)	1.09 (1.00%)	0.989998 (0.91%)	vsse:15138
#6	108.96	1.2 (1.10%)	1.06 (0.97%)	0.959999 (0.88%)	vsse:14964
#7	108.94	1.23 (1.13%)	1.09 (1.00%)	0.990005 (0.91%)	vsse:15136
#8	108.93	1.18 (1.08%)	1.03 (0.95%)	0.93 (0.85%)	vsse:14790
...	...	...	...	...	...

See Also

- [layer_res_nworker](#)
- [Calculating Layer Resistance Sensitivity and Layer Equivalent Resistance](#)

layer_res_nworker

Specifies the number of CDPL workers to define how many processes are run in parallel for sensitivity or equivalent resistance calculation. Default value is 2.

Note:

Each process requires a static resistance check (SPRES) license.

```
layer_res_nworker n
```

where *n* is the number of the CDPL workers. The default is 2.

To enable CDPL, you need to specify hosts information in a CDPL hosts file and then load this configuration file with the `-rhosts` command-line command. For example:

```
xa -r r.cmd -rhosts hosts_file -o result/res
```

An Example of Hosts File:

```
# HOSTS FILE
# Each valid host definition occupies an entire line.
# Lines with '#' as the first character are comment lines.
# Format of each line:
```

```
# enabled| hostname| #slots| tmpDir| mode| command
# Each field is terminated by "|" except the last field.
# A space MUST be present for an empty field.
# enabled: 1 means the host is enabled. Anything else indicates it
#           is disabled
# hostname: This field is ignored for SGE & LSF entries
# slots: the number of worker slots on this host (rsh and ssh),
# or farm (lsf and sge); -1 indicates unlimited
# tmpDir: The (optional) temporary work directory
# mode: RSH, SSH, LSF or SGE
# command: The remote command
#
#####
#1 | fsei313 | 4 | /remote/cdpl/tmp | RSH | rsh
#####
|localhost| 4 | /remote/cdpl/tmp | SSH | ssh
```

If no hosts file is provided, CDPL will run processes on the local server.

Only pad-to-pin resistivity is estimated when the [layer_res](#) command is specified in the SPRES command file. The number of CDPL runs is equal to the number of layers specified with the [layer_res](#) command.

See Also

- [layer_res](#)
- [Calculating Layer Resistance Sensitivity and Layer Equivalent Resistance](#)

mergefinger

The command merges instance pins for finger devices.

```
mergefinger finger_suffix
```

For example:

```
mergefinger @
```

There are basic pin names and finger pin names. The structure of basic pin name is:

```
<base_instance_name><delimiter><terminal>
```

For example:

```
X1/MP0:SRC
```

The finger device instance pin name structure is:

```
<base_instance_name><finger_suffix><finger_index><delimiter><terminal>
```

For example, in the finger device instance pin name X1/MP0@2:SRC:

- *<base_instance_name>* is X1/MP0
- *<finger_suffix>* is @
- *<finger_index>* is 2
- *<delimiter>* is :
- *<terminal>* is SRC

This command merges instance pins for finger devices with names:

```
<base_instance_name><finger_suffix><finger_index><delimiter><terminal>
```

To the instance pin with basic names:

```
<base_instance_name><delimiter><terminal>
```

This means that finger sources are merged to base instance source, drains to drain, and bulks to bulk. The static resistance calculation report contains pad-pin resistors only for base instance pins.

modifyresistor

This command is part of the what-if capability. It lets you modify a resistor in the specified net.

```
modifyresistor <net_name> <resistor_name> <new_value>
```

For example:

```
modifyresistor vdd r3245 100
```

This command changes a resistance of resistor device, named r3245, to 100ohm in the vdd RC network.

mt

This command selects the number of processes for resistor estimation in parallel mode when the tool finds a speed advantage using parallel mode. Otherwise, parallel mode is ignored.

```
mt <number_of_processes>
```

If the input number of processes exceeds the number of system CPU cores, it is adjusted to the number of CPU cores. If `-mt` is set on the command line, for example as `xa -r r.tcl -mt 4`, it has precedence over the command in Tcl file. The default value is 1.

net

This command specifies the DSPF net name.

```
net <net_name>
```

Example:

```
net gnd
```

netdeletepad

This command permits removal of some net pads from the resistance calculation. You can use an asterisk (*) wildcard character in the pad names.

```
netdeletepad <net_name> <pad_name1> [<pad_name2> ...]
```

Example:

```
netdeletepad vdd *  
netdeletepad vdd VDD_1  
netdeletepad vdd VDD_1 VDD_5 VDD_7
```

netinclude

The command inserts lines from <file.name> to the DSPF file after the *INET <net.name> statement. It allows the inclusion of extra statements without touching the DSPF file.

```
netinclude <net_name> <file_name>
```

Example:

```
netinclude vdd vdd_added_ipins.txt
```

node2node

The `node2node` command specifies pairs of nodes for which the tool computes the effective resistance between the nodes.

```
node2node net_name node1 node2
```

`net_name`

Specifies the net name.

`node1 node2`

Specifies the net pair of nodes in the net (subnodes or instance pins). You can specify multiple pairs.

node2node_option

The `node2node_option` command controls how the results of node-to-node resistances are reported, in a log file or separate file, and whether to include all or only some larger than a specified value.

```
node2node_option [-n2n_file 0|1] \  
  [-n2n_threshold value] \  
  [-net net_name] \  
  \
```

`-n2n_file`

A value of 0 (default) outputs results to the .rlog file. A value of 1 outputs results to a separate <output>.node2node file.

`n2n_threshold`

When you specify a threshold value (default is 0), only the pairs with the resistance larger than the threshold are reported. For example:

```
node2node_option -n2n_file 1 -n2n_threshold 100
```

The above example saves pair resistances in a file, but only those that are larger than 100 Ohms.

`-net net_name`

Specifies different threshold values for different nets. For example:

```
node2node_option -n2n_file 1 -net a -n2n_threshold 200  
node2node_option -n2n_file 1 -net b -n2n_threshold 300
```

Only the pairs with the resistance larger than the threshold of 200 ohms from net a and pairs with the resistance larger than the threshold 300 ohms from net b are reported into the <output>.node2node file. For the rest of the nets, the threshold is the default value of zero, and results are only output to the .rlog file.

op2ipin

The command recognizes the observation point as an instance pin so resistance to the node can be calculated and reported. The tool now recognizes the `**|OP` node definition in the DSPF file.

This command is on by default.

```
op2ipin <yes|no>
```

outfiltres

The StarRC tool generates non-physical resistors that are used for connectivity purpose. They do not physically exist in the layout. You can use this command to filter out (or not report) non-physical resistor from the static resistance checking results. Conditions to filter are based on resistor width and resistance.

This command is on by default.

```
outfiltres {<w>@<res>}
```

Where `w` is the resistor width and `res` is the resistance.

You can specify more than one `<w>@<res>` pair within one command. Each pair must be separated by a space. You can also specify multiple `outfiltres` commands in one Tcl command file.

By default, the tool automatically filters out resistors with the following width-value pairs generated by the StarRC tool:

```
r = 0.001 Ohm and w = 10 um;  
r = 0.1 Ohm and w = 9 um;  
r = 0.01 Ohm and w = 100 um.  
r = 0.0001 Ohm and w = 10 um
```

For example:

```
outfiltres 10@0.001 9@0.1 100@0.01
```

outlayers

To use this command in `ccdb` output format, you must set the `ccdb 1` command as a prerequisite. The `outlayers` command specifies the original layers that are used to generate the violation map. You can use commas (,) or spaces () to separate layer names.

```
outlayers layer1,layer2,layer3, ...
```

For example:

```
outlayers psd,nsd,substrate
```

png

If the `png` command is not set or `<size_x>` is less than 100, the color map dimensions are automatically calculated. If coordinates are available, instance pins are displayed

as pixels with colors ranging from red (maximum resistance) to green (zero resistance). For example, setting `png 1000` generates a color map of size 1000 X - 1000 times the *aspect_ratio*. Graphic output is generated using a publicly available GD library.

Example:

```
png <size_x>
```

If `<size_x>` is set to 0, the generation of png files is disabled.

report

This command post-processes the static resistance checking results, and uses the *prefix.rout* file as input data. Arguments of this command are used as filters for input data providing a way to generate different reports, which are subsets of *prefix.rout* file data.

```
report -nr number_of_nodes
-minr res_value
-maxr res_value
-layer layer_number
-xmin coord,-ymin coord -xmax coord,-ymax coord
-pat "pattern_with_wildcards"
-patskip "pattern_with_wildcards"
-png x_size_in_pixels
-gds box_width
-gdslobnd value
-gdsupbnd value
-gdsmag value
-gdsunits value
```

```
-nr number_of_nodes
```

Outputs no more than the specified number of nodes.

```
-minr res_value
```

Outputs only nodes with a resistance greater than the specified value.

```
-maxr res_value
```

Outputs only nodes with a resistance less than the specified value.

```
-layer layer_number
```

Outputs only nodes with a layer number equal to the specified layer number (or with no layer information).

```
-xmin coord,-ymin coord -xmax coord,-ymax coord
```

Outputs nodes with coordinates inside specified area (or with no coordinates information).

`-pat "pattern_with_wildcards"`

Outputs only nodes that match the specified hierarchical name:terminal pattern.

`-patskip "pattern_with_wildcards"`

Skips the name:terminal names from the list obtained by the `-pat` argument.

`-png x_size_in_pixels`

Controls the X-size of a png image. Use `-png 0` to disable png output.

`-gds box_width`

Outputs in GDSII format. Every pin is output as a square of the specified *box_width*.

`-gdslobnd value`

Sets the lower boundary/threshold for the distribution of resistors among violation levels. All resistors with values lower than the specified value are placed into the lowest violated level (dark green color).

`-gdsupbnd value`

Sets the upper boundary/threshold for the distribution of resistors among violation levels. All resistors with values higher than the specified value are placed into the most violated level (blinking red color).

`-gdsmag value`

Specifies geometry magnification. A negative value means coordinates are divided by the specified value. A positive value multiplies coordinates.

`-gdsunits value`

Specifies geometry units within the SPF and DSPF file. The default value for units is 1e-6. If you do not specify this argument, the tool processes all coordinates as if they are in microns.

For example:

```
report -nr 11 -minr 100 -xmin 0 -layer 24 -pat "*:s"
```

rmax_thresh

This command allows you to specify the threshold value. Resistors with values higher than the threshold value are removed from the netlist while reading the SPF file.

By default, the rmax threshold is 100K Ohm.

`rmax_thresh <thresh_value_in_Ohm>`

routcols

```
routcols <column name>
```

The `routcols` command is used to control all of the columns within the *.rout file except for the first Count column, which contains each resistor number within the output table.

The `routcols` command is specified in the static resistance check input file (`resistance_command_file`).

Syntax Description

<column name> may have the following names:

Number	Name	Description
1	resval	Resistance value
2	wl	W/L ratio
3	xy	X, Y coordinates
4	gdsxy	X, Y coordinates within the GDSII file
5	name	Instance pin name
6	hname	Hierarchical instance pin name
all	—	All of the above mentioned columns

rmin

The `rmin` command specifies the minimum resistor values read from the DSPF file. Resistors with smaller values will be shorted so that there is a reduction of resistor elements with a corresponding accuracy trade-off. In the example below, any resistors smaller than 1e-9 Ohm are shorted.

```
rmin 1e-9
```

rve

```
rve -width -mag val [-spfunits geometry_units] [-topcell cell_name] \  
    [-diagtype {{poly | 2} | {edge | 1}}] [-resvalue] [-resolution <value>]
```

The command specifies the magnification coefficient for geometry in the SPF/DSPF file.

The `-mag` option is supported in both gds and rve formats. You can specify the magnification coefficient for geometry in the SPF/DSPF file. A positive *val* multiplies coordinates. If *val* is negative, the coordinates are divided by *val*. For example:

```
rve 1 -mag 20
```

`-spfunits`

Specifies the units used for SPF file geometries. The default is 1um.

`-topcell`

Specifies the name of top cell. The default is TOP_CELL.

`-diagtype`

Specifies the type of a shape generated for non-orthogonal resistors. The default is {poly | 2}.

`-resvalue`

Switches the violation map from node to resistor based.

`-resolution`

Specifies the resolution value for coordinates data in the rve report. The default is 10000.

subnode

```
subnode <subnode_pattern> [<node_pattern2> ...]
```

The command causes the resistance to be calculated for a sub-node such as vdd:53 vdd:6*. By default, resistance to sub-nodes is not calculated.

sortby

```
sortby <sortkey>
```

The command specifies the method for sorting entries in the .rout file using the following values:

Syntax Options

`r`

Default. Generates the .rout file with entries sorted by resistance from the pads.

`w/l`

Sorts by the W/L ratio of MOSFET transistors.

`rw/l`

Sorts by the product of resistance and the W/L ratio of the transistor corresponding to the source or drain pin.

temp

`temp`

The command specifies the reference temperature. The `temp` value is used to recalculate the resistance values when temperature factors (such as `TC1` and `TC2`) are specified in the SPF file.

The default is 25 Celsius, unless there are `.model` statements for resistors in DSPF file. If there are `.model` statements for resistors in the DSPF file, the reference temperature for each model is taken from these statements.

In the following example a value of 90 Celsius is used to calculate the resistance values.

```
temp 90
```

tmpdir

The command redirects pseudo-temporary files to a user-specified location. You use this command in cases when the `-o` argument generates large files that cause a lot of network traffic.

```
tmpdir <path>
```

where `path` specifies the directory path to store `.npf` files created during the static resistance calculation.

4

Power Net EMIR Analysis

This chapter provides information on how to run power net EMIR analysis using the PrimeSim XA, PrimeSim Pro and PrimeSim SPICE simulators.

This chapter contains the following topics:

- [EMIR Flow Manager With PrimeSim Pro, PrimeSim SPICE, PrimeSim HSPICE and PrimeSim XA](#)
- [EMIR Analysis With PrimeSim XA](#)

EMIR Flow Manager With PrimeSim Pro, PrimeSim SPICE, PrimeSim HSPICE and PrimeSim XA

Describes how to run the EMIR flow which unifies EMIR setting for different simulator and provides flow options for easy usage.

This part includes the following chapters:

- [Before You Start](#)
- [Simulator Agnostic Solution for EMIR Analysis](#)
- [EMIR Flow Manager Workflow](#)
- [Flow Options in EMIR Flow Manager](#)
- [Specifying Nets for Power Net EMIR Analysis](#)
- [Handling Internal Power Nets](#)
- [Viewing EMIR Analysis Results](#)
- [Adding Design or Functional Resistors](#)
- [Applying Electromigration Check on the Functional Resistors](#)

Before You Start

Before running EMIR analysis EMIR analysis with flow manager, make sure

- The license path is set up using one of the following environment variables:
 - `SNPSLMD_LICENSE_FILE`: A pointer to the Synopsys license paths
 - `LM_LICENSE_FILE`: A pointer to all license paths. If you specify both environment variables, `SNPSLMD_LICENSE_FILE` takes precedence over `LM_LICENSE_FILE`.

In C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

In Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

- The shell path is set up to include the path to the PrimeSim EMIR launch script. For example,

```
> setenv PRIMESIM_HOME emir_home_folder_path
```

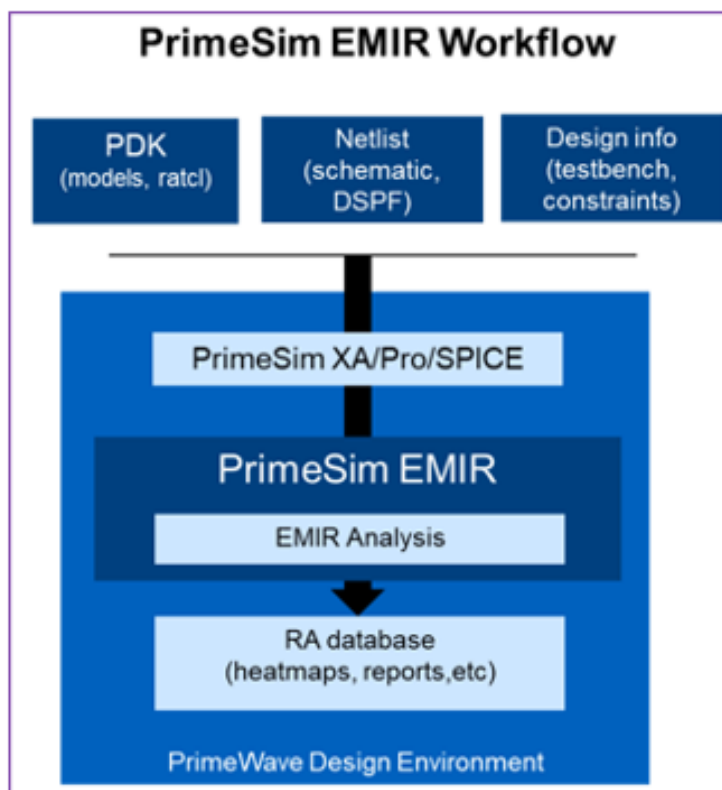
```
> setenv PRIMESIM_EMIR_HOME emir_home_folder_path
```

- All the required input files are present.

Simulator Agnostic Solution for EMIR Analysis

PrimeSim EMIR tool provides a simpler application and specific workflow to leverage same setup and collateral from post-layout simulations. The advanced features including SelfHeat, statistical EM, etc are supported across all designs styles with consistent reporting and visualization in PrimeWave Design Environment.

Figure 9 PrimeSim EMIR Workflow



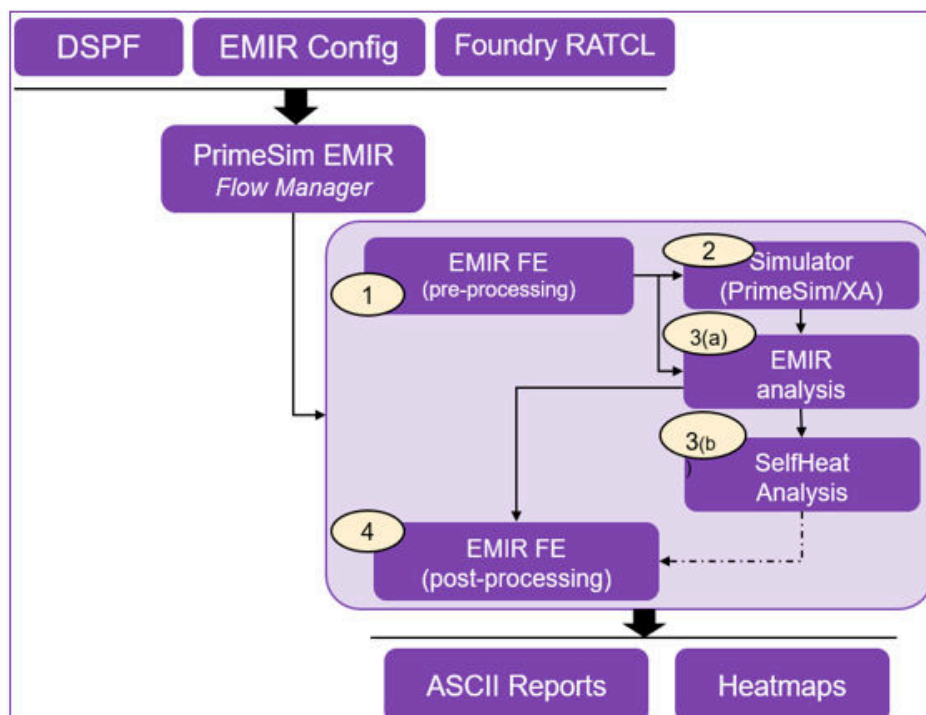
EMIR Flow Manager Workflow

In the flow, you can provide EMIR config with foundry RATCL, DSPF and basic flow settings.

The EMIR flow manager parses the configuration and invokes the flow which includes the following steps:

1. EMIR FE(front-end): pre-process the netlist to extract relevant information and to create additional settings according to selected simulator.
2. Simulation: run SPICE/Fast SPICE simulation to capture currents/voltages.
3. EMIR analysis/SelfHeat: run EMIR and SelfHeat analysis.
4. EMIR FE: post-process the analysis results to generate ASCII reports and heatmaps.

Figure 10 EMIR Flow Manager Workflow



Flow Options in EMIR Flow Manager

Three flows are supported in this EMIR flow manager which are libchk(library check), emir and fast_ir flow. The Libchk flow targets library EM analysis with spice level accuracy. The Emir flow is a traditional two phase flow which targets AMS IP and embedded memories. The Fast_ir flow is based on PrimeSim Pro one phase flow which targets big memory such as DRAM/FLASH.

The design style of each flow option and supported simulators are as follows:

Table 3 *Workflow centric analysis*

Flow	Option	Description	Target	Spice Simulator
Fast IR	-flow fast_ir	Perform fast IR analysis with reduced power grid and report only IR on devices. Apply reduction approaches on the fly for appropriate simulators.(Target blocks: DRAM/Flash memories)	DRAM	PrimeSim Pro
EMIR	-flow emir	Dynamic EMIR, self heat and statistical EM analysis across all PrimeSim simulators (Target blocks: AMS IP, embedded memories)	Analog/Memory IP	PrimeSim Pro(default) PrimeSim SPICE PrimeSim XA
Library EMIR	-flow libchk	High accuracy simulations for library cells	Logic Library	PrimeSim SPICE(default) PrimeSim HSPICE
Static IR	-flow static_ir	Static EMIR analysis. Vector-less and constant device current distributed on power network to calculate EMIR in quick turnaround time	All	N/A

Running EMIR analysis – library check flow

The library check flow supports the PrimeSim SPICE and PrimeSim HSPICE simulators for library EM simulation and analysis such as standard cells and so on. The tool first back-annotates the post-layout parasitic and device files with the pre-layout design netlist, and speeds up the simulation with its unique parallel technology. With the simulation

results (including resistor currents and parasitic data), PrimeSim EMIR calculates current violations and reports EM results in ASCII and CCDB formats.

To run library check analysis flow, perform the following steps:

Table 4 Steps for Library Check Analysis Flow

Step number	Step	Command
1	Prepare simulation data, including vector file, model file, back-annotation flow, and so on. Prepare the RATCL file in which you specify EM rules or other EMIR commands for EMIR analysis.	
2	Config EMIR_config file to specify testbench with command options, simulator and simulator binary(if not set, the tool looks for simulator binary from environment variable \$path).	<pre>set_flow_option -ra_simulator_input testbench set_flow_option -ra_simulator primesim_spice or primesim_hspice set_flow_option -ra_simulator_binary simulator_binary</pre>
3	Specify RATCL file in EMIR config	<pre>set_ra_option -ratcl foundry_ratcl</pre>
4	Specify time step and analysis window	<pre>set_ra_option -ratau time_step -twindow start stop</pre>
5	Enable SelfHeat analysis if required	<pre>set_ra_option -she 1</pre>
6	Add user specific RA setting (ASCII,CCDB overrides, OA view, and so on)	<pre>set_ra_option -ratcl user_ra_option</pre>
7	Launch primesim_emir with EMIR Config which contains all ra setting	<pre>primesim_emir -flow libchk -c emir_config -o prefix</pre>
8	Review the EMIR analysis	See EMIR Output Files and Generating Violation Maps. results.

The following is an example of EMIR config for libchk flow:

```
set_flow_option -ra_simulator_input "top.spi -mt 4"

set_flow_option -ra_simulator_binary /global/apps/primesim_2023.12/bin/
primesim
```

```
Set_flow_option -ra_simulator primesim_spice

set_ra_option -ratau 1ps -ratcl libchk_default.ratcl -twindow 1n 1.5n
```

Running EMIR analysis – emir flow

The emir flow supports the PrimeSim SPICE, PrimeSim Pro and PrimeSim XA simulators for IR drop Electromigration (IREM) simulation and analysis such as AMS IP. The tool first back-annotates the post-layout parasitic and device files with the pre-layout design netlist, and speeds up the simulation with its unique parallel technology. With the simulation results (including resistor currents, node voltages and parasitic data), PrimeSim EMIR calculates voltage/current violations and reports IREM results in ASCII and CCDB formats.

To run emir analysis flow, perform the following steps:

Table 5 Steps for EMIR Analysis Flow

Step number	Step	Command
1	Prepare simulation data, including vector file, model file, back-annotation flow, and so on. Prepare the RATCL file in which you specify EM rules or other EMIR commands for EMIR analysis.	
2	Config EMIR_config file to specify testbench with command options, simulator and simulator binary(if not set, the tool looks for simulator binary from environment variable \$path).	<pre>set_flow_option -ra_simulator_input testbench set_flow_option -ra_simulator primesim_spice or primesim_pro or primesim_xa set_flow_option -ra_simulator_binary simulator_binary</pre>
3	Specify net, net type, and reference voltage for analysis	<pre>set_ra_net_type-pwnet list_of_power_ground_net</pre>
4	Specify RATCL file in EMIR config	<pre>set_ra_option -ratcl foundry_ratcl</pre>
5	Specify time step and analysis window	<pre>set_ra_option -ratau time_step -twindow start stop</pre>

Table 5 Steps for EMIR Analysis Flow (Continued)

Step number	Step	Command
6	Enable SelfHeat analysis if required	<code>set_ra_option -she 1</code>
7	Add user specific RA setting (ASCII,CCDB overrides, OA view, and so on)	<code>set_ra_option -ratcl user_ra_option</code>
8	Launch primesim_emir with EMIR Config which contains all ra setting	<code>primesim_emir -flow emir -c emir_config -o prefix</code>
9	Review the EMIR analysis	See EMIR Output Files and Generating Violation Maps. results.

The following is an example of EMIR config for emir flow:

```
set_flow_option -ra_simulator_input "top.spi -mt 4"

set_flow_option -ra_simulator_binary /global/apps/primesim_2023.12/bin/
primesim

Set_flow_option -ra_simulator primesim_pro

set_ra_net_type -pwnet vp vphreg gd

set_ra_pwnet_option -net vp -vref 1.2

set_ra_pwnet_option -net vphreg -vref 2.5

set_ra_pwnet_option -net gd -vref 0

set_ra_option -ratau lps -ratcl emir_default.ratcl -twindow 1n 1.5n
```

Running EMIR analysis – fast_ir flow

The `fast_ir` flow supports the PrimeSim Pro simulator for IR simulation and analysis such as DRAM/FLASH. The tool first back-annotates the post-layout parasitic and device files with the pre-layout design netlist, and perform fast IR analysis with reduced power grid and report only IR on devices. With the simulation results, PrimeSim EMIR calculates device pin voltage violations and reports IR results in ASCII and CCDB formats.

To run emir analysis flow, perform the following steps:

Table 6 Steps for Fast_IR Analysis Flow

Step number	Step	Command
1	Prepare simulation data, including vector file, model file, back-annotation flow, and so on. Prepare the RATCL file in which you specify EM rules or other EMIR commands for EMIR analysis.	
2	Config EMIR_config file to specify testbench with command options, simulator and simulator binary(if not set, the tool looks for simulator binary from environment variable \$path).	<pre>set_flow_option -ra_simulator_input testbench set_flow_option -ra_simulator primesim_pro set_flow_option -ra_simulator_binary simulator_binary</pre>
3	Specify net, net type, and reference voltage for analysis	<pre>set_ra_net_type -pwnet list_of_power_ground_nets set_ra_pwnet_option -net net_name -vref ref_volt</pre>
4	Specify time step and analysis window	<pre>set_ra_option -ratau time_step -twindow start stop</pre>
5	Add user specific RA setting (ASCII,CCDB overrides, OA view, and so on)	<pre>set_ra_option -ratcl user_ra_option</pre>
6	Launch primesim_emir with EMIR Config which contains all ra setting	<pre>primesim_emir -flow fast_ir -c emir_config -o prefix</pre>
7	Review the EMIR analysis	See EMIR Output Files and Generating Violation Maps. results.

The following is an example of EMIR config for emir flow:

```
set_flow_option -ra_simulator_input "top.spi -mt 4"
set_flow_option
  -ra_simulator_binary /global/apps/primesim_2023.12/bin/primesim
Set_flow_option -ra_simulator primesim_pro
set_ra_net_type -pwnet vp vphreg gd
set_ra_pwnet_option -net vp -vref 1.2
set_ra_pwnet_option -net vphreg -vref 2.5
set_ra_pwnet_option -net gd -vref 0
set_ra_option -ratau lps -ratcl fast_ir_default.ratcl -twindow 1n 1.5n
```

Running EMIR analysis – static_ir flow

Static IR drop is the voltage drop that occurs when a constant current passes through a power network with varying resistance and the circuit is in a steady state (when the inputs are not switching).

To run static_ir analysis flow, perform the following steps:

Step number	Step	Command
1	Prepare simulation netlist, including model file, back-annotation flow, and so on. Prepare the RATCL file in which you specify EM rules or other EMIR commands for EMIR analysis.	
2	Config EMIR_config file to specify testbench. If testbench is for primesim, need to specify set_flow_option -ra_simulator primesim to identify primesim syntax	set_flow_option -ra_simulator_input testbench
		set_flow_option -ra_simulator primesim or primesim_xa(default)
3	Specify net, net type, and reference voltage for analysis	set_ra_net_type-pwnet list_of_power_ground_ net
4	Specify RATCL file in EMIR config	set_ra_option -ratcl foundry_ratcl
5	Specify DC current for each PG domain	set_ra_pwnet_option -net net -vref ref_voltage -static_current dc_current
6	Add user specific RA setting (ASCII,CCDB overrides, OA view, and so on)	set_ra_option -ratcl user_ra_option

Step number	Step	Command
7	Launch primesim_emir with EMIR Config which contains all ra setting	primesim_emir -flow static_ir -c emir_config -o prefix
8	Review the EMIR analysis	See EMIR Output Files and Generating Violation Maps. results.

The following is an example of EMIR config for static_ir flow:

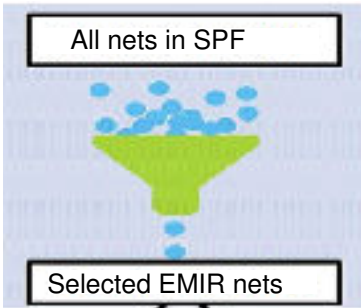
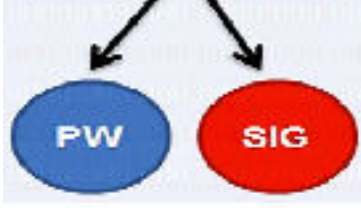
```
set_flow_option -ra_simulator_input "top.spi"
set_ra_net_type -pwnet vp vphreg gd
set_ra_pwnet_option -net vp -vref 1.2 -static_current 1
set_ra_pwnet_option -net vphreg -vref 2.5 -static_current 2
set_ra_pwnet_option -net gd -vref 0 -static_current 3
set_ra_option -ratcl emir_default.ratcl
```

Specifying Nets for Power Net EMIR Analysis

To specify which nets to analyze in the power net EMIR flow, run the `set_ra_net` command to select EMIR nets and then run the `set_ra_net_type` command to define net types for the selected EMIR nets.

Figure 11 illustrates the flow of selecting nets and defining net types for the selected nets.

Figure 11 *Selecting EMIR Nets*

Concept		Command
Selects nets to include in the EMIR analysis		<pre>set_ra_net -selectnet list_of_nets -skipnet list_of_nets</pre>
Defines the net types (PW or SIG) for the selected nets		<pre>set_ra_net_type -pwnet list_of_nets -signet list_of_nets</pre>

Setting EMIR Nets

When selecting EMIR nets, you can specify net names explicitly or use a wildcard character. By default, the tool selects all nets in the circuit for EMIR analysis. The net name is based on the back-annotation net name, not the schematic name. You can also apply multiple `set_ra_net` commands. If the net names are matched by both the `-selectnet` and `-skipnet` arguments, `-skipnet` has a higher priority. The EMIR flow always honors your setup when you specify an explicit net name.

For more information about these commands, see the command pages in *PrimeSim™ XA Command Reference*. [Table 7](#) shows the net selection rules.

Table 7 *Net Selection Rules*

Rule	EMIR Command
The name and hierarchical delimiter are based on the back-annotation net name.	<code>set_ra_net -selectnet x1/x2/vss</code>

Table 7 Net Selection Rules (Continued)

Rule	EMIR Command
Wildcard matching (*) is allowed for the list of nets. Separate net names by spaces.	<pre>set_ra_net -selectnet *</pre> Selects all nets for EMIR analysis. This is the default behavior. <pre>set_ra_net -selectnet vdd vss vint*</pre> Selects nets vdd and vss, and all the nets that match vint* for EMIR analysis.
Use multiple <code>set_ra_net</code> commands.	<pre>set_ra_option -pwra 1 set_ra_net -selectnet vdd set_ra_net -selectnet vss set_ra_net -selectnet vpp</pre> Processes nets vdd, vss and vpp in power net EMIR analysis.
An explicit name takes higher priority over wildcard matching for the same net.	<pre>set_ra_option -pwra 1 set_ra_net -selectnet vdd vss set_ra_net -skipnet *</pre> Processes nets vdd and vss in power net EMIR analysis.
The <code>-skipnet</code> argument has higher priority than <code>-selectnet</code> when matching the same net (both explicit name or wildcard).	<pre>set_ra_option -pwra 1 set_ra_net -selectnet vdd vss set_ra_net -skipnet vss</pre> In this example, the net vdd is processed in the power net EMIR analysis. The net vss is skipped because vss is matched by explicit name matching and <code>-skipnet</code> has a higher priority over the <code>-selectnet</code> argument. <pre>set_ra_option -sigra 1 set_ra_net -selectnet * set_ra_net -skipnet wl* bit*</pre> In the above example, all signals nets are processed in the signal net EM analysis. Net names beginning with "wl" and "bit" are skipped. The <code>-skipnet</code> option has precedence over the <code>-selectnet</code> option.
All net that are not selected by the <code>set_ra_net -selectnet</code> command are ignored in the EMIR analysis.	<pre>set_ra_option -pwra 1 set_ra_net -selectnet vdd</pre> In this example, net vdd is selected for power net EMIR analysis, all other nets that are not specified by the <code>set_ra_net -selectnet</code> command are ignored in the power net EMIR analysis.

Defining Net Types

After you select which nets to include in the EMIR analysis with the `set_ra_net` command, run the `set_ra_net_type` command to define the net type of the selected EMIR nets as PWNET (power net) or SIGNET (signal net).

You can apply multiple `set_ra_net_type` commands in the same analysis run. Only nets that are defined as PWNET (power net) or SIGNET (signal net) are selected for EMIR analysis.

When the `set_ra_net_type` command is not used, the PrimeSim EMIR tool uses a default power net ID method to identify power nets (including both external and internal power nets) and uses those power nets for power net EMIR simulation if the `set_ra_option -pwra` command is set to 1. When you specify one or more `set_ra_net_type` commands, the PrimeSim EMIR tool disables the power net ID, so you can specify whether the nets are power nets or signal nets. [Table 8](#) shows the rules for defining net types for the selected EMIR nets.

Table 8 Rules for Defining Power or Signal Net Type

Rule	EMIR Command
The name and hierarchical delimiter are based on back-annotation net name.	<code>set_ra_net_type -pwnet x1/x2/vss</code>
Wildcard matching (*) is allowed for the list of nets. Separate multiple net names by spaces.	<code>set_ra_net -pwra 1</code> <code>set_ra_net -pwnet vdd* vss*</code> Selects all vdd and vss nets for power net EMIR analysis. <code>set_ra_option -sigra 1</code> <code>set_ra_net_type -signet w1* b1*</code> Selects w1* and b1* nets for signal net EM analysis.
You can use multiple <code>set_ra_net_type</code> commands.	<code>set_ra_option -pwra 1</code> <code>set_ra_net_type -pwnet vdd</code> <code>set_ra_net_type -pwnet vss</code> <code>set_ra_net_type -pwnet vpp</code> Processes nets vdd, vss and vpp in power net EMIR analysis.
An explicit name takes higher priority over wildcard matching for the same net.	<code>set_ra_option -pwra 1 -sigra 1</code> <code>set_ra_net_type -pwnet vdd_*</code> <code>set_ra_net_type -signet vdd_int</code> Processes vdd_int net for signal net EM analysis.

Table 8 Rules for Defining Power or Signal Net Type (Continued)

Rule	EMIR Command
The <code>-pwnet</code> argument has higher priority than <code>-signet</code> .	<pre>set_ra_option -pwra 1 -sigra 1 set_ra_net_type -pwnet vdd_int set_ra_net_type -signet vdd_int</pre> In this example, the <code>vdd_int</code> net is processed in power net EMIR analysis.
All nets that are not selected by either <code>set_ra_net_type -pwnet</code> or <code>set_ra_net_type -signet</code> are ignored in the EMIR analysis.	<pre>set_ra_option -pwra 1 -sigra 1 set_ra_net_type -pwnet vdd set_ra_net_type -signet w1</pre> Selects the <code>vdd</code> power net for power net EMIR analysis and the <code>w1</code> signal net for signal net EM analysis. All other nets are ignored for EMIR analysis.

Handling Internal Power Nets

By default, the PrimeSim EMIR flow automatically handles the internal power supply as long as the internal power supply is defined by the following command:

```
set_ra_net_type -pwnet internal_supply_name
```

For every internal supply name selected, the PrimeSim EMIR tool automatically identifies the net name, the source external supply, the reference voltage, and its driver instance pin name. The driver pins are selected from the instance pin of the SPF net. It is assumed that net name is a net that was identified as an internal power net (automatically or manually).

During automatic identification, the PrimeSim EMIR tool identifies the drivers of the power net by tracing the devices connected from the power net to a constant `vsrc` node. The voltage value of the node is then used for the `-vref` value.

To avoid ambiguity when tracing may lead to two different constant `vsrc` nodes, it is recommended that you specify a source power net node, so the false drivers (leading to a wrong constant `vsrc` node) are not identified by a PrimeSim EMIR tracing algorithm.

During EMIR analysis, the PrimeSim EMIR tool generates the `output_name.pw_driver` output file that contains all the information about internal supply for debugging purposes. You can use the information in this output file to manually override the detection rule to ensure proper setup in cases the tool mishandles the internal supply. Note that any manual specification discards all the results of the automatic detection in that area, including net type, `vref` or drivers.

To manually specify the internal supply nets, use the `set_ra_pwnet_option` and `set_ra_pwnet_driver` command. The reason why these two command are separated is because, for a given internal supply net, there could be many driver instance pins.

To make the choice more flexible, the PrimeSim EMIR tool allows wildcard (*) pattern matching when defining the driver instance pin name. However, for defining the reference voltage value and the source voltage, there should only be one value and therefore wildcard usage is not allowed.

In the following example, the internal supply net xi0/vss has source supply as vss and reference voltage of 0 volt. This internal supply net has two driver instance pins.

```
set_ra_pwnet_option -net XI0/VSS -src vss -vref 0.0
set_ra_pwnet_driver -net XI0/VSS -driver_ipname \
  XI0/XDRIVER/LEFT_B/MXN2@2:D
set_ra_pwnet_driver -net XI0/VSS -driver_ipname XI0/XDRIVER/LEFT_B/MXN2:D
```

Selecting Time Windows for EMIR Analysis

In PrimeSim EMIR analysis, you have the flexibility to define a particular time window for the current collection (RASIM file) to reduce the RASIM file size. You can also specify more than one time window. To specify a time window, use the `set_ra_option` command:

The following example generates an RASIM file between t=10ns to t=20ns; and t=90ns to t=100ns.

```
set_ra_option -pwra 1 -ratcl ra.tcl -twindow 10n 20n 90n 100n
```

Setting RATAU Values for Current Resolution in EMIR Analysis

In PrimeSim EMIR analysis, a fixed time step is used for calculate IR drop and electromigration violations. The default for the fixed time step is 10ps. If a more detailed time step is needed for high precision simulation, select a smaller ratau value for simulation.

The following example selects 5ps as the time step for the EMIR simulation:

```
set_ra_option -pwra 1 -ratcl ra.tcl -ratau 5p
```

Enabling EMIR File Splitting

For some EMIR analyses, large power nets result in a very large RASIM file. To help viewing the RASIM results more efficiently, you can split the RASIM file into multiple files.

Here is the syntax:

```
set_ra_option -pwra 1 -ratcl ra.tcl [-waveform_split size]
```

The default waveform file split size is 2G (the default unit is 1M). The following example sets the waveform file size to 4G (4000M) for splitting.

```
set_ra_option -pwra 1 -ratcl ra.tcl -waveform_split 4000
```

Redirecting Temporary EMIR Files

PrimeSim EMIR analysis lets you redirect pseudo temporary files to a user-specified location. Use the `-tmpdir` command when the `-o` argument generates large files that cause a lot of network traffic, or when the `-o` location does not have sufficient disk space.

The following example redirects the storage of the temporary EMIR files to the `/tmp` directory.

```
set_ra_option -pwra 1 -ratcl ra.tcl -tmpdir /tmp
```

For more information about the commands used in this section, see *PrimeSim™ XA Command Reference*.

Using Variable Time Steps

PrimeSim EMIR supports `vts` and `vtsopt` RATCL commands to enable and apply the variable time step technique for IR drop analysis during the analysis stage (Phase II). Using variable time steps provides benefits on runtime with comparable accuracy when switching activity is low. By default, PrimeSim EMIR uses constant time steps when analyzing IR drops during the analysis (Phase II) stage.

When a variable time step is specified with the `vtsopt` command, the power net matrix is stamped and factorized for each step. The factorized matrices are available during simulation. The default variable time steps are: `ratau`, `2*ratau`, `4*ratau`.

Simulation accuracy is controlled by the `dv` tolerance value specified with the `-dv` option of the `vtsopt` command. The tool selects a larger (or smaller) step if the simulated voltage difference is smaller (or larger) than the `-dv` value. For example, `dv=10mV`.

At the end of simulation at phase II, the tool reports the step information in the `*.ralog` file of each power net, in the following format:

```
Total number of steps: n/N (ns1% ns2% ... - nr%)
```

Where:

- `n`: Number of actual time steps during simulation (including `presim` time)
- `N`: Number of steps if simulated with a constant step of `ratau` (including `presim` time)
- `ns1%`, `ns2%`, ...: Percentage of steps for step1, step2, ... of actual time steps
- `nr%`: Percentage of rejected steps

For example, if `ratau=2ps`, phase II simulation window=`1ns-5ns`, `presimtime=1ns`, `steps=1 2 4`, the total number of steps will be reported as:

```
Total number of steps: 1356/2500 (35.6% 40.8% 23.6% - 8.9%)
```

RATCL File Example

```
vt_s 1  
vtsopt -dv 0.05 -steps 1 3 9
```

In this example, the `vt_s` command is set to 1 to enable variable time step technique. The `dv` tolerance is 50mV and the steps are `1*ratau`, `3*ratau`, `9*ratau`.

Viewing EMIR Analysis Results

When EMIR analysis is finished, you can examine the output analysis results or find violations in the violation maps by importing the output data from EMIR analysis to a layout editor, such as Custom Compiler.

PrimeSim EMIR provides various analysis reports to analyze IR drop results and find possible violations in a circuit. For example, when IR drop analysis is finished, you can write `Vmin`, `Vmax`, `Vavg`, `dVmax`, or `eflow` data to the output report file. Least resistive path (LRP) can be calculated and displayed for finding the root cause of high IR drops. Reporting effective resistance from any node to power net's pads is also supported.

See [EMIR Output Reports and Maps](#) for more information.

Adding Design or Functional Resistors

This section covers how to enable design resistor EM rules to work with the existing EMIR flow methodology. It describes how to make the existing EMIR flow infrastructure work to support design resistors. These design or functional resistors are either two-terminal or three-terminal components and is represented by either a regular model or macro model.

Examples

Following is an example of a two-terminal functional resistor represented by a regular model named `rhim` with width of 1 μm and resistance of 1 mohm.

```
rtc net2 net5 rhim r=1e-3 w=1e-6
```

Here is an example of a three-terminal functional resistor represented by a macro model named `rhim_m` with width of 2 μm and resistance of 2 mohm.

```
xrpo pd pad vddq rhim_m r=2e-3 wr=2e-6
```

The concept is to create a virtual signal net and place all design resistors under the virtual net to collect current through the design resistors. This approach can take advantage of the existing phase II EMIR flow.

The EMIR flow parses all design resistors through a single virtual SPF net to allow collection of currents on one end of the resistor to check against EM rules. The virtual SPF net has the following definition:

- It is container to store all design resistors.
- It is treated it as signal net to parse through signal net EM flow.
- Resistors are simulated in Phase II and checked for EM similar to other resistors in the EMIR flow.
- The virtual SPF net is a "shadow copy" and does not impact original connectivity and simulation results.

Applying Electromigration Check on the Functional Resistors

Use the `set_ra_functional_resistor` command to create a virtual SPF net and specify model name for the design resistors to be analyzed. Optionally, you can specify parameter names for resistor width and length using the `-map_w` and `-map_l` options.

Examples

In the following example, the PrimeSim EMIR identifies functional or design resistors with model named `rhim` and checks currents through resistors of this type.

```
set_ra_functional_resistor -res_model rhim
```

In the following example, the functional or design resistors with model named `rhim` are identified and then the tool checks the currents through these resistors. The netlist uses `wr` to represent width and `lr` to represent length for the resistor.

```
set_ra_functional_resistor -res_model rhim_m -map_w wr -map_l lr
```

After the analysis, PrimeSim EMIR generates the output files in the formats such as ASCII, RAGDS, and CCDB for the analyzed functional or design resistors.

Reports	Example
ASCII report: <output_prefix>-fr_vnet_<analysis_type>.ascii	rundir1/xara1-fr_vnet_iavg.ascii
RAGDS (violation map): <output_prefix>-fr_vnet_<analysis_type>.ragds	rundir1/xara1-fr_vnet_iavg.ragds
CCDB (violation map): <output_prefix>-fr_vnet_<analysis_type>.cc.db	rundir1/xara1-fr_vnet_iavg.cc.db

For more information, see [EMIR Output Files](#).

EMIR Analysis With PrimeSim XA

Describes how to run EMIR analysis in PrimeSim XA.

In PrimeSim XA, the EMIR tool supports the static resistance check calculation, dynamic power net EMIR analysis and signal net EM analysis capabilities.

This part describes how to use the EMIR analysis tool in PrimeSim XA to check power and ground net resistance, capture resistor current of the power network, perform electromigration and EMIR analysis, and save the analysis results to output reports and violation maps.

This part includes the following chapters:

- [Before You Start](#)
- [Introducing the Power Net EMIR Analysis](#)
- [Power Net EMIR Analysis Flow](#)
- [Defining Net Pins by Specifying XY Coordinates](#)
- [Performing Power Net EMIR Analysis - Simulation Stage](#)
- [Performing Power Net EMIR Analysis - Analysis Stage](#)
- [Advanced Electromigration and IR Drop Analysis](#)

Before You Start

Before you start to run EMIR analysis in PrimeSim XA, make sure

- The license paths is set up using one of the following environment variables:
 - `SNPSLMD_LICENSE_FILE`: A pointer to the Synopsys license paths
 - `LM_LICENSE_FILE`: A pointer to all license paths. If you specify both environment variables, `SNPSLMD_LICENSE_FILE` takes precedence over `LM_LICENSE_FILE`.

In C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

In Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

- The shell path is set up to include the path to the PrimeSim EMIR launch script. For example,

```
> setenv XA_HOME XA_home_folder_path  
> setenv PRIMESIM_EMIR_HOME primesim_emir_home_folder_path
```

- All the required input files are present

Introducing the Power Net EMIR Analysis

Power nodes in simulated circuits are considered to have constant voltage. In reality, resistive networks between the power node pad and the devices always exist. When devices draw current from the power source, the voltage or IR drop appears on the device terminals connected to power nodes. If significant, this IR drop may affect the performance of the circuit, such as the delay value.

The IR drop value depends on the value of the resistive path from the power pad to the device connection. The current flowing through each power net RC network resistor may cause electromigration (EM) problems in the circuit if the current density is greater than a specified threshold.

Encrypted PrimeSim HSPICE netlists with unencrypted SPF files are supported for back-annotation in the EMIR flow.

The tool provides the power net EMIR analysis to perform IR drop and electromigration analysis on the power network. The tool

- Uses a DSPF file representation of the power net to calculate all the pad-to-pin resistances. The number of resistors in the power nets of modern circuits makes it impossible to simulate the circuit with back-annotated power nets, so RC reduction on power nets must be applied.
- Accepts power net RC networks by back-annotating the resistors and capacitors from a DSPF file.
- Simulates the circuit with the reduced power net RC network added to the circuit devices, providing dynamic IR drop on circuit behavior.
- Calculates the maximum IR drop at each connection between the power net and circuit devices.
- Calculates the maximum, average, or RMS current density value through each power net resistor and compares it with the threshold for electromigration analysis.
- Displays the violation map highlighting locations where IR drop or current density exceeds the threshold.

Power Net EMIR Analysis Flow

Figure 12 illustrates the power net EMIR flow consists of the following phases:

- **Phase I simulation (Simulation Stage)**, in which a simulation is performed to capture the currents of devices connected to the power nets. The current information is stored in the .rasim file for use in the proceeding phase II simulation.

For details, see the [Performing Power Net EMIR Analysis - Simulation Stage](#) section.

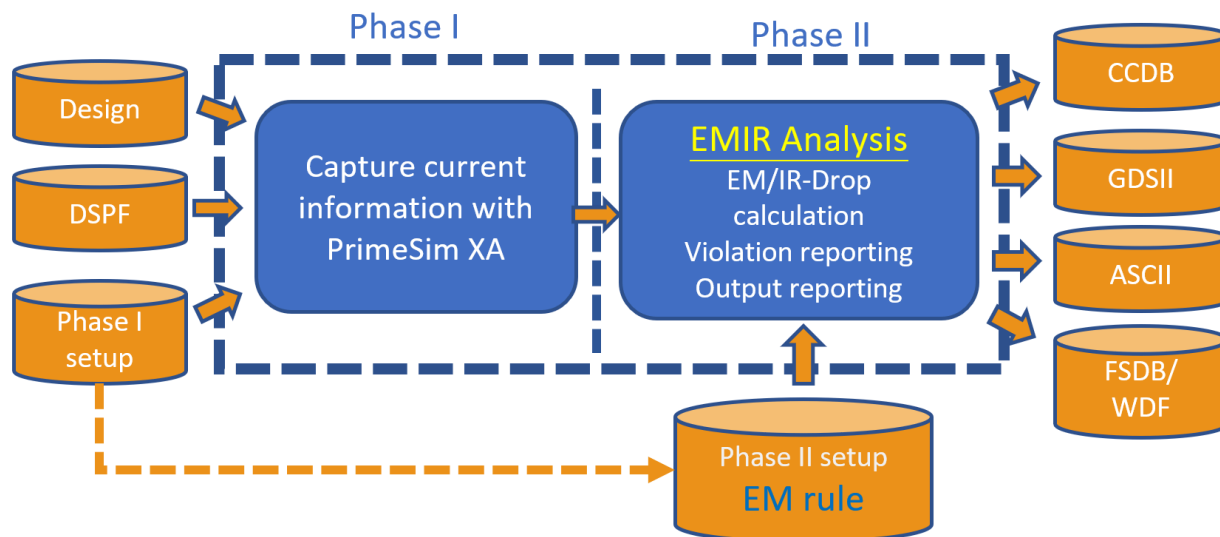
- **Phase II simulation (Analysis Stage)**, in which the power nets (which may include internal supply nets if these internal supply nets are defined as power nets) are simulated and the stored currents are injected into the power nets and the IR drop and resistor current values are calculated.

In the EMIR flow, back-annotation is a prerequisite for analyzing IR drop and electromigration violations on a power network in a circuit. However, the EMIR analysis is independent of the back-annotation setup, which allows you to specify which EMIR nets to analyze for better performance and more accurate analysis results.

For details, see the [Performing Power Net EMIR Analysis - Analysis Stage](#) section.

When simulation is complete, PrimeSim EMIR provides violation reports and violation maps for EM and voltage drop violation checking. For details, see the [EMIR Output Files](#) and [Generating Violation Maps](#) sections.

Figure 12 Power Net EMIR Analysis Flow



Defining Net Pins by Specifying XY Coordinates

You can define "*"P" net pins by specifying the XY coordinates during phase I and phase II in the power net EMIR analysis flow. This capability is also available for static resistance checking.

To define net pins by specifying X/Y coordinates during power net EMIR analysis:

- In phase I simulation, use the `load_ba_file -add_netpin_by_xy_file` command.
- In phase II simulation, use the `addnetpinxy` command in the RATCL file.

To include the `addnetpinxy` command in the RATCL file and the TCL file used for static resistance calculation, use the following syntax:

```
addnetpinxy=<net> X=<xcoord> Y=<ycoord> layer=<mlayer> pinxydist=<xydist>
```

where

`net`

Specifies the net name.

`X=<xcoord>`

Specifies the X coordinate (in microns).

`Y=<ycoord>`

Specifies the Y coordinate (in microns).

`layer=<mlayer>`

Specifies the metal layer name.

`pinxydist=<xydist>`

Specifies that only nodes that are within the xydist distance from the user-specified XY coordinates are searched. By default, the xydist distance is 1u.

The following formula is used to calculate the distance.

```
sqrt (dx^2 + dy^2)
```

where

$dX = X_n - X_u$; $dY = Y_n - Y_u$

- X_n and Y_n : Node coordinates
- X_u and Y_u : User-specified coordinates

Performing Power Net EMIR Analysis - Simulation Stage

To perform power net EMIR analysis flow, do the following steps:

Table 9 Steps for Power Net EMIR Analysis Flow

Step	Command
1	Prepare simulation data, including vector file, model file, back-annotation flow, and so on. Prepare the RATCL file in which you specify EM rules or other EMIR commands for EMIR analysis. <pre>set_ra_option -pwra 1 -ratcl list_of_ratcl_file</pre> See Preparing RATCL Files .
2	Enable a power net EMIR simulation. <pre>set_ra_option -pwra</pre> See Enabling Power Net EMIR Flow .
3	Specify nets and the net types for EMIR analysis. <pre>set_ra_net and set_ra_net_type</pre> Example: <pre>set_ra_net_type -pwnet list_of_powernet_name</pre> See Specifying Nets for Power Net EMIR Analysis .
4	Launch the PrimeSim EMIR simulation with the configuration file that contains the EMIR commands. <pre>xa top.sp -c cmd -o output/xa_emir</pre>
5	Review the EMIR analysis results. See EMIR Output Files and Generating Violation Maps .

Enabling Power Net EMIR Flow

To enable power net EMIR flow, run the `set_ra_option -pwra` command. For example,

```
set_ra_option -pwra 1 -ratcl ratcl_file
```

You can specify more than one .ratcl file with the `-ratcl` command argument.

```
set_ra_option -pwra 1 -ratcl ratcl_file1 ratcl_file2
```

By default, the PrimeSim EMIR tool continues to run the analysis stage right after simulation is complete. To discontinue the process after simulation is complete, set the `-rap2auto` command to 0. By default, the `-rap2auto` command is set to 1.

```
set_ra_option -pwra 1 -ratcl ra.tcl -rap2auto 0
```

When the manual simulation run is completed, the tool provides the run script that you can add to manually launch the analysis stage at the end of the simulation log file. For example,

```
Peak Virtual Memory = 409 M
Grand Total Time = 2 sec
Total Wall Time = 3 sec
Total number of comparison errors = 0
Use XA wrapper to manually execute phase 2 of EMIR analysis:
/tools/install/xa_2013.03-SP2/ bin/xa -ra output/xa_ra-vdd
output/xa_ra-gnd
Source Files:
-----
| name                               | type |
|-----+-----|
| output/xa_ra-gnd.dcmmap           | dcmmap |
| output/xa_ra-vdd.dcmmap           | dcmmap |
|-----+-----|
XA done (Thu Oct 17 10:54:27 2013 )
```

Preparing RATCL Files

To run power net EMIR analysis, you first need to prepare an RATCL file.

The RATCL file provides the Tcl interface to create EM rules for calculating RMS, as well as average and peak current threshold values that can then be used to verify the individual rules. Usually one average, one RMS and one peak current rule is defined per metal layer and one average current rule per via layer. Layer mapping is performed at the beginning of the RATCL file.

Add other EMIR commands to the RATCL file as necessary. When ready, use the `-ratcl` option of the `set_ra_option` command to run power net EMIR analysis with the specified RATCL file.

For example,

```
set_ra_option -pwra 1 -ratcl test.ratcl
```

For a complete list of the commands, see the [Commands for EMIR Analysis Post-processing](#) section.

For a detailed description about the `set_ra_option` command options, see the command page in *PrimeSim XA Command Reference*.

Performing Power Net EMIR Analysis - Analysis Stage

To start the phase II simulation in the power net EMIR flow, run the `xa -ra` command in the following syntax:

```
xa -ra output_prefix-power net name1 ... \
    output_prefix-power net nameN
```

For example, to start the phase II simulation for the VDD net only, run the following command:

```
xa -ra test-vdd
```

The tool reads the .ratcl file for the corresponding power net and performs the specified analysis types. The tool reads the .ratcl file and executes the commands specified in the .ratcl file in which you specify commands to set analysis types, set waveform viewer, control the content of the output files, and so on. See [Table 10](#) for a list of commonly used commands for phase II simulation. For a full list of commands that are supported in the phase II simulation, see [Commands for EMIR Analysis Post-processing](#).

During the phase II simulation, the currents stored from the phase I simulation are injected into the entire power net. For each time interval equal to the RATAU value, the tool recalculates and updates the following information:

- Voltages at every power net node
- Currents through each power net resistor

Table 10 Commonly Used Commands in Phase II Simulation

Command	Description
<code>ra</code>	Specifies the analysis types to perform. The supported analysis types are: power net EMIR analysis, signal net EM analysis, and IR drop analysis with the Vmax, Vmin, and Vavg output values.
<code>raviewer</code>	Sets a waveform viewer to display the output waveforms
<code>raformat</code>	Specifies the output layout format. The supported formats are: <code>all</code> , <code>gds2</code> (default), <code>ccdb</code> , <code>ascii</code> (default) and <code>rve</code> .
<code>ralayout</code>	Specifies the types of analysis results to write to the output files, named <prefix>_<analysisName>.<ext> .
<code>ralayers</code>	Sets the layer numbers of the output violation map.
<code>vref</code>	Specifies a voltage reference value for internal power supplies.
<code>outformat</code>	Specifies the output waveform format.

Table 10 Commonly Used Commands in Phase II Simulation (Continued)

Command	Description
<code>asciicols</code> <code>asciicolsort</code> <code>ccdbcols</code> <code>ccdbmerge</code> <code>gdsprops</code>	Controls content for the output file.

When EMIR analysis is complete, the tool generates the following outputs:

- `test-vdd>.ralog`: Contains VDD log information
- `test-vdd.radb`: Contains VDD net results
- `test-vdd_vmax.ragds`: Contains VMAX analysis violation map of the VDD net in GDSII format

Using User-Defined Current Waveforms for What-If Analysis

Phase II simulation uses pin voltage and current waveforms saved in the RASIM file as input data. These waveforms are generated from phase I simulation as results of full circuit simulation, so running phase I simulation might result in significant run time. In cases where you want to rerun phase II simulation on different corners for what-if purposes, use the `replace_waveform` command to replace waveforms from the RASIM file with the user-defined waveforms. The command is applicable for all nets, including external power nets, internal power nets and signal nets.

To specify user-defined waveforms, add the `replace_waveform` command in the phase II Tcl file in the following syntax:

```
replace_waveform net_name pin_name \
    -dc dc_value | -pwl t1 val1 t2 val2 [t3 val3 ...]
```

To specify the type of waveform to replace, use either the `-dc` or `-pwl` option.

- `-dc`: Specifies a waveform with a constant value.
- `-pwl`: Specifies a PWL waveform.

Example

The following example replaces waveform of the pin `m1tp0:src` in the `net_a` net with the user-defined waveform on timing points (0.000000E+00,0.000000E+00) and (1.000000E-11, 0.000000E+00).

```
replace_waveform net_a m1tp0:src \
    -pwl 0.000000E+00 0.000000E+00 1.000000E-11 0.000000E+00
```

Enabling Multicore Processing

The phase II simulation in the PrimeSim EMIR power net EMIR flow supports multicore processing to improve performance on power network simulation. When multicore processing is enabled, the PrimeSim EMIR tool uses the multicore partitioning and simulation algorithm to execute multiple tasks in parallel.

To run multicore simulation and specify the number of threads used per process, use the `-mt` command option. For example,

```
xa -ra net_name -mt number_of_threads
```

Note:

The `-mt` command-line option must be the last in the command line, and takes precedence over the `mt` command if both are specified.

Advanced Electromigration and IR Drop Analysis

This chapter describes the features in the advanced PrimeSim EMIR flow.

- [Power Net and Signal Net RASIM Reuse Flow](#)
- [Command-Based What-If Analysis](#)
- [Adding a Net Pin by XY Coordinates](#)
- [Adding Design or Functional Resistors](#)
- [EMIR Pipeline Processing](#)

Power Net and Signal Net RASIM Reuse Flow

Running electromigration and IR drop (EMIR) analysis flow often requires iterations after an initial simulation. Use these iterative analysis results to debug layout problems. When layout issues are identified and corrected, a new DSPF file is generated and used for the next iteration of simulation. The process of changing the layout, generating new DSPF files, and iterating simulations might continue until all layout issues are fixed.

Transient analysis can take a significant amount of time in these simulations. To save time in the EMIR verification cycle, the RASIM reuse flow lets you reuse the transient analysis results from the first EMIR simulation and skip transient analysis for subsequent EMIR simulations. [Figure 13](#) shows an overview of the flow.

Figure 13 RASIM Reuse Flow

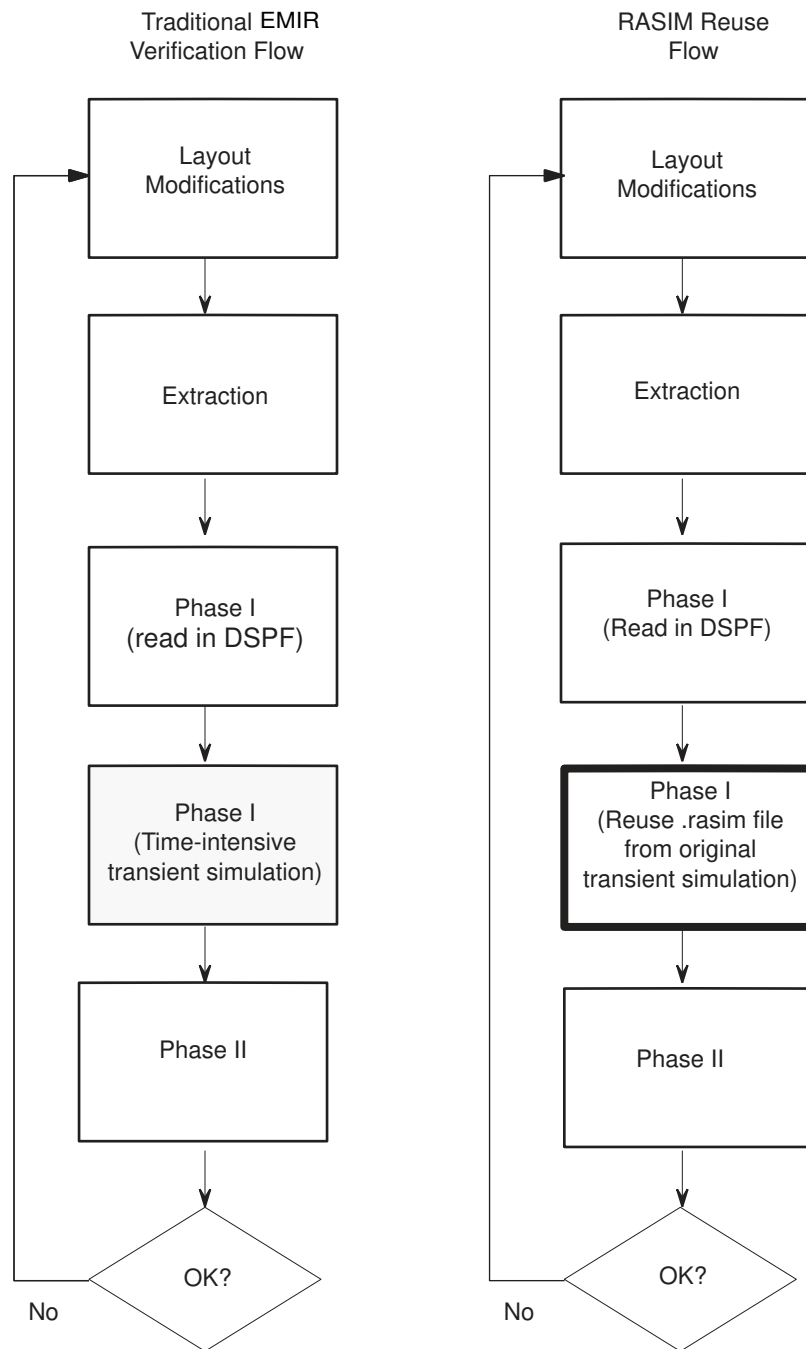
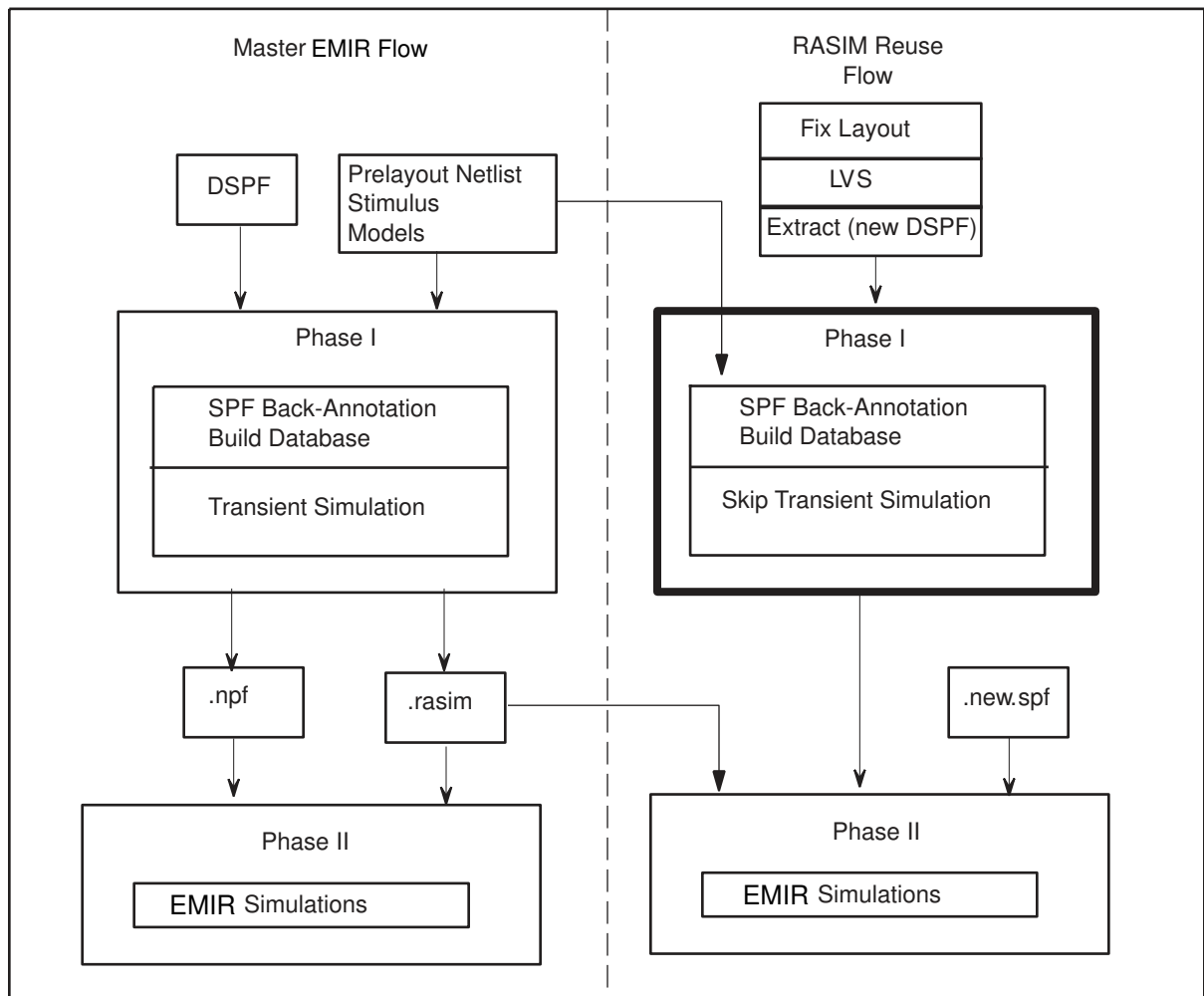


Figure 14 shows a more detailed view of the RASIM reuse flow.

Figure 14 Detailed View of RASIM Reuse Flow



RASIM Reuse Flow Example

The following simulation example assumes that there are one master run, including a fast-fast corner (test_ff.sp) and an RASIM reuse run on the slow-slow corner (test_ss.sp):

```
xa test_ff.sp -c cmd -o orig/test_ff
```

The cmd file contains the following commands:

```
set_ra_option -pwra 1 -sigra 1
set_ra_net_type -pwnet vdd vss
load_ba_file -file original.spf
```

To set up the RASIM reuse phase I simulation, add the following commands to the cmd file:

```
set_ra_option -pwra 1 -sigra 1
set_ra_net_type -pwnet vdd vss
#load_ba_file -file original.spf $replace the original.sp with #new.spf
load_ba_file -file new.spf
set_ra_reuse -rasim orig/test_ff -inst_pin_tol 0.01
```

This cmd file sets up the RASIM reuse flow with an instance pin tolerance of 1%. To run the simulation by reusing the .rasim file from the ./orig directory, specify the following command:

```
xa test_ss.sp -c cmd -o reuse/test
```

This simulation produces the following phase I outputs:

- *.npf for the new DSPF netlist database for phase II simulation
- .pinmap ...
- reuse/test-vdd.ratcl, reuse/test-vss.ratcl, reuse/test-@.ratcl

The contents of the reuse/test-vdd.ratcl file are:

```
rasim orig/test_ff-vdd.rasim -instpintol 0.01
```

If you specify the *.rasim file correctly, the PrimeSim EMIR tool compares the *.pinmap files between orig/test_ff*pinmap and reuse/test*pinmap, and reports discrepancies for the instance pin names. The file contains a list of instance pin names that are successfully back-annotated.

The PrimeSim EMIR tool reports a discrepancy of the pin lists to the *.pin_mismatch file and errors out if the discrepancy is greater than 1%. For this example, the default -instpintol value is 0%. To use a different set of *.rasim files in the phase II simulation, you can modify the reuse/xa-vdd.ratcl file or the EMIR Tcl file defined by the RATCL command and specify the new *.rasim file path and tolerance. For example:

```
rasim orig2/test_ss-vdd.rasim -instpintol 0.05
```

Command-Based What-If Analysis

The *what-if* analysis feature lets you analyze and make changes to an existing SPF file before doing actual changes in the layout. In the typical RASIM flow, you work with the violation map in some layout editor. After you identify some weak spots in the violation map, you typically come up with some specific actions for how to modify the layout to fix violations. In terms of the violation map, these actions might be changing the resistors that

were extracted from the actual (original) layout and provided in the SPF file. Some of the typical layout modifications that can be mapped to the resistor-based modifications are:

- Changing the metal wire width by changing the resistor value of the existing resistor
- Adding a metal wire by adding a resistor between existing nodes in the net definition
- Adding vias to the existing vias or via array by adding a via resistor in parallel with an existing via resistor
- Adding or removing vias to/from the existing vias or via array by changing the value of the existing via resistor

While this limited set does not cover all the possible layout modifications, it might still be useful in quickly experimenting with the possible impact of those changes by doing a what-if analysis of the original layout by modifying the extracted SPF file.

This feature lets you specify which resistors to modify at the time of reading the SPF file for back-annotation at phase I of EMIR flow. As a result, the PrimeSim EMIR tool reads in the modified SPF file and uses it as if it were extracted from the modified original layout. So you can make changes without going through the full layout/extraction/simulation flow, which shortens the turnaround time in the entire post-layout verification flow.

After you check the impact of the planned layout changes in the EMIR flow (possibly going through more than one iteration), you can reproduce the verified changes in the actual layout. As a final verification check, you might decide to re-extract the final layout and put it through the EMIR flow for one final check. [Figure 15](#) shows an overview of the what-if EMIR flow.

Figure 15 What-If EMIR Flow

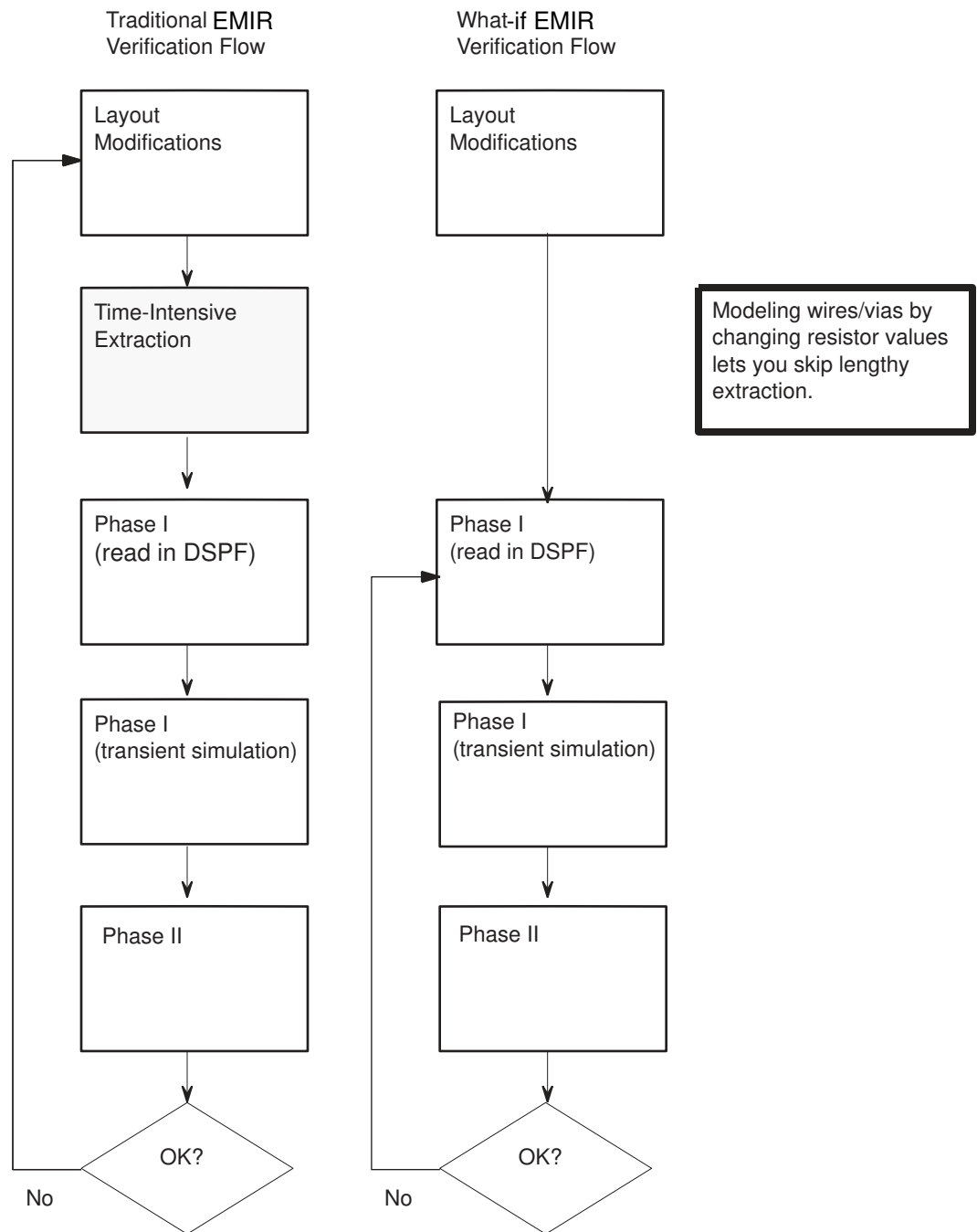
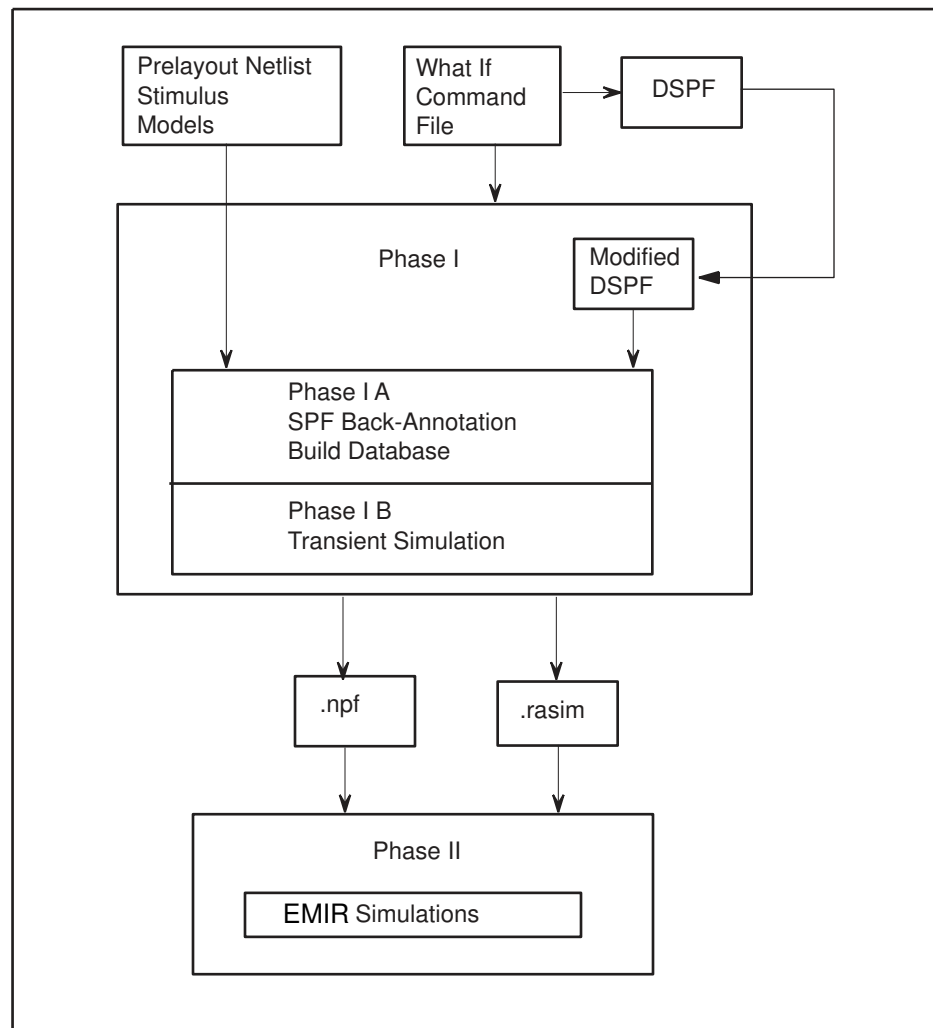


Figure 16 shows a more detailed view of the flow.

Figure 16 Detailed View of the What-If EMIR Flow



What-If EMIR Flow Example

The `load_ba_file -what_if` command controls the modifications that can be done to the SPF file. Only one command file can be apply to the SPF file. For more information about the `load_ba_file` command, see the *PrimeSim™ XA Command Reference*.

The following example uses a `cmd_file` file to modify an existing SPF file.

```
load_ba_file -file spf_file -what_if cmd_file
```

The cmd_file file contains the following commands:

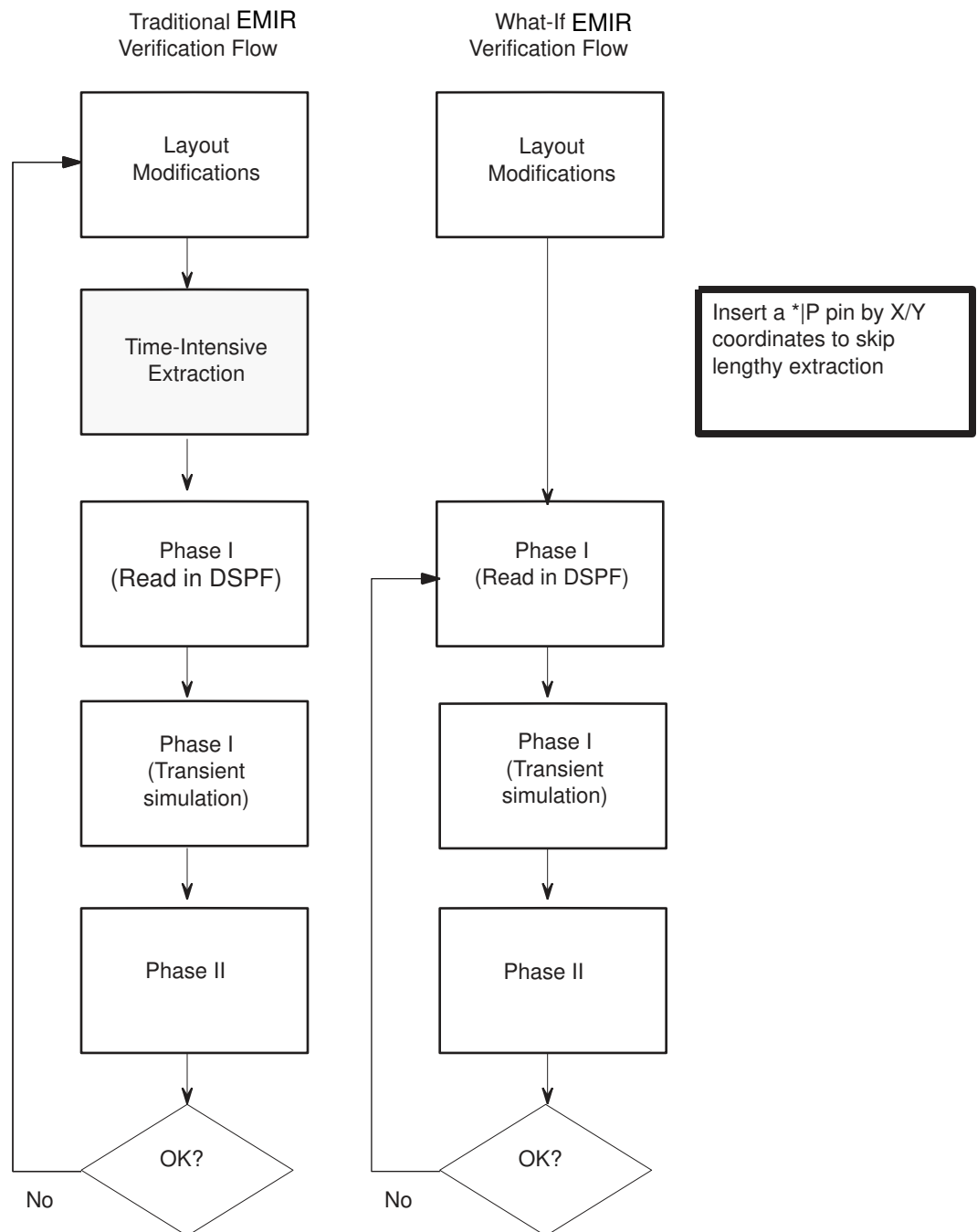
```
modifyresistor vdd r138 100  
modifyresistor vdd r136 200  
addresistor vss vss:10 vss:12 350 metal2  
addgcap vss vss:15 20e-15
```

The above example changes the value for two resistors in the VDD net by adding a new resistor in the VSS net between existing sub-nodes of the net. It also adds a ground capacitor in the VSS sub-node.

Adding a Net Pin by XY Coordinates

To modify an existing SPF file for what-if purposes before doing actual changes in the layout, add a net pin by XY coordinates to an existing SPF file that does not contain net pin coordinates information. [Figure 17](#) shows the flow to perform this analysis.

Figure 17 Adding a Net Pin by XY Coordinates



Adding a Pin by XY Coordinates Example

You use the `load_ba_file-add_netpin_by_xy_file` command to add a pin by XY coordinates.

This command instructs the PrimeSim EMIR tool to insert a "*"P" net pin by finding the closest location to the specified XY coordinates on the defined metal layer. It uses the following formula to calculate the distance:

$$d = \text{sqrt}(dX^2 + dY^2)$$

where

- $dX = X_n - X_u$; $dY = Y_n - Y_u$
- X_n and Y_n : Node coordinates
- X_u and Y_u : User-specified coordinates

The following example uses a `cmd_file` to modify an existing SPF file.

```
load_ba_file -file spf_file -add_netpin_by_xy_file xy_file
```

where `xy_file` contains:

```
net_name=vdd x=495 y=45 layer=m1 dist=1
```

This example looks for a subnode at (495u 45u) with a radius of 1u. A subnode found in this range is converted to a pad.

EMIR Pipeline Processing

The EMIR pipeline processing feature improves runtime performance by allowing power and signal net analysis to run in phases I and II in parallel. This means that phase II can start while phase I is still running.

Running Phase I Pipeline Process

Pipeline processing of power and signal nets in phase I is controlled by the `-pp`, `-pp_config`, and `-pp_nets` arguments in the `set_ra_option` command. The `-pp_nets` argument specifies which power and signal nets go to EMIR processing in phases I. The pipeline processing for selected nets does not need to wait until phase I completes, decreasing the total processing time for power/signal net analysis. [Figure 18](#) shows an example of how phase I pipeline processing works. After the front-end processing and the set up time in transient analysis completes in phase I, pipeline processing of power/signal nets starts. The time of EMIR analysis is marked as phase II time.

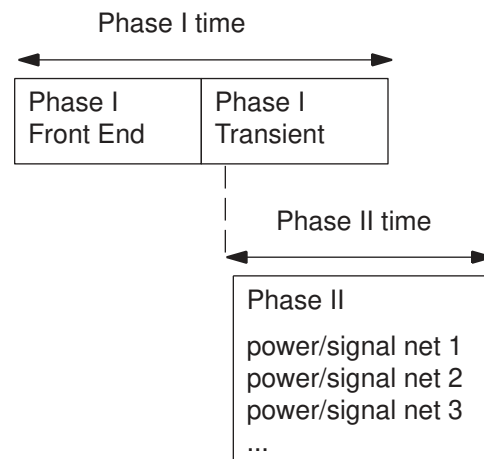
Running Phase II Distributed Processing

Distributed processing of power/signal net analysis in phase II is used to start EMIR analysis in parallel. To specify the number of phase II power/signal net analysis processes and the CDPL hosts configuration file, use the `-dp` and `-dpconfig` arguments in the

`set_ra_option` command. You can also use the `-radp` and `-radp_config` command line options.

Figure 18 shows an example of how phase I pipeline processing and phase II distributed processing works. After the front end processing and set up time in transient analysis completes in phase I, distributed processing of power/signal nets starts in phase II.

Figure 18 Phase I Pipeline and Phase II Distributed Processing Example



Note:

The `twin` command does not work in the EMIR pipeline processing feature.

For more details about the EMIR pipeline processing feature, see the Examples section in the `set_ra_option` command description.

5

Signal Net EM Analysis

Describes the Phase I and Phase II control parameters used in signal net EM analysis.

This chapter includes the following topics:

- [Introducing Signal EM Analysis](#)
- [Running Phase I Simulation](#)
- [Running Phase II Simulation](#)
- [Defining Net Pins by Specifying XY Coordinates](#)
- [Running Power Net EMIR and Signal Net EM Together](#)
- [Examining Output Files](#)

Introducing Signal EM Analysis

The steps of performing signal net EM analysis are similar to the power net EMIR analysis described in the [Power Net EMIR Analysis](#) section. The analysis consists of the following phases:

- **Phase I Simulation (Simulation Stage):** The tool simulates a circuit with back-annotated signal nets and saves currents flowing through the net ports.
- **Phase II Simulation (Analysis Stage):** The saved currents are injected into the non-reduced signal nets and the current through each individual resistor of the net is computed. Only electromigration (EM) analysis is performed for signal nets so that the current density for each resistor is calculated, which then may be displayed in the form of a violation map in a GDSII file. The spots where the current density exceeds the specified threshold value are highlighted. If you use the `set_ra_net_type` command, only the nets or net pattern specified in this command will be considered for EM analysis. For more information about the `set_ra_net_type` command, see the *PrimeSim XA Command Reference*.

An encrypted PrimeSim HSPICE netlist is supported with unencrypted SPF files for back-annotation in the signal net EM analysis flow.

Analyzing EM effects on signal nets is dependent on the input vector stimulus required to switch or activate these signal nets. If a less-than-complete vector is supplied, not all signal nets may be simulated.

Running Phase I Simulation

Phase I simulation consists of the following steps.

- [Setting Up the Phase I Simulation in Signal EM Analysis](#)
- [Running Pre-Phase I Net Filtering](#)
- [Running Phase I and Phase II Simulations in the Signal EM Flow](#)
- [Selecting Nets for Signal EM Analysis](#)
- [Selecting a Time Window for Signal EM Analysis](#)
- [Setting an *ratcl* Value for Current Resolution in Signal EM Analysis](#)
- [Enabling RASIM File Splitting](#)
- [Redirecting Temporary EMIR Files](#)
- [Using Variable Time Steps](#)

Setting Up the Phase I Simulation in Signal EM Analysis

To set up the Phase I simulation in the signal net EM flow, do the following steps:

1. Prepare your simulation data (vector/model file/BA flow) to enable a signal net simulation.
2. Insert or create the EMIR global command in your existing command file. For example:

```
set_ra_option -sigra 1 -ratcl ratcl_file  
set_ra_net_type -signet signalnet_name
```

Here are some basic rules for these commands.

- You can specify more than one RATCL file with the `-ratcl` command argument. For example:

```
set_ra_option -sigra 1 -ratcl ratcl_file1 ratacl_file2
```

- The tool has the flexibility to allow you to specify multiple `set_ra_net_type` commands for signal nets, or provide one or more signal nets in a single `set_ra_net_type` command. For example:

```
set_ra_net_type -signet *analog* *wl* xtop/xio* x1/x2/vss  
               xtop.xblock.int_vdd*
```

or

```
set_ra_net_type -signet *analog*  
set_ra_net_type -signet *wl*  
set_ra_net_type -signet xtop/xio*  
set_ra_net_type -signet x1/x2/vss  
set_ra_net_type -signet xtop.xblock.int_vdd
```

The default is to use all the signal nets if you do not specify `set_ra_net_type`.

For more information about these commands, see the command pages in *PrimeSim™ XA Command Reference*.

3. Launch the PrimeSim EMIR simulation with the configuration file that contains the EMIR commands. For example:

```
xa top.sp -c cmd -o output/xa_ra
```

4. Review the analysis results.

Running Pre-Phase I Net Filtering

A restricting limitation for signal net EM runs can be the creation of the `.rasim` files that consume a large amount of disk space in phase I simulation. To address this limitation, a pre-phase I run is introduced, when signal net EM filters only signal nets without saving the device currents in `.rasim` files.

This pre-phase I signal net EM run produces a list of nets, filtered according to the specified [iavmin](#) value. The device currents are saved only for the filtered nets. This approach allows for much a smaller disk space footprint for phase I signal net EM runs.

The signal net EM flow with the `set_ra_option -sigra_filter_run 1` option is as follows.

1. Phase I runs with `set_ra_option -sigra_filter_run 1`. It filters the nets to keep only the nets that have a current value greater than the `lavmin` value.

A file with a `-@.ranets` extension is created. This file contains the list of filtered nets preceded with the `set_ra_net` command.

2. The entire signal net EM flow (phase I and phase II) is restarted with the filtered nets using the `-c` command line option.

Here is an example of a flow using signal net EM filter run.

```
xa netlist.sp -c xa_cmd_file_w_filter -o results/ra
```

The `xa_cmd_file_w_filter` file contains a `set_ra_option` command with the `-sigra_filter_run 1` option:

```
set_ra_option -pwra 1 -sigra 1 -sigra_filter_run 1 ...
```

After this run, the following results are available:

- The `results/ra-@.ranets` file with a signal net list. For example:

```
# list of signal nets with average current above 10uA
set_ra_net -selectnet net069
set_ra_net -selectnet xr/aa
set_ra_net -selectnet net99
set_ra_net -selectnet net144
set_ra_net -selectnet net13
set_ra_net -selectnet net160
```

- The power nets data to be used in phase II runs. At this time, you can run phase II simulation for power nets either manually or automatically (if you specified the `-rap2auto 1` option in `set_ra_option`). Now, you can launch the signal net EM job for filtered signal nets as:

```
xa netlist.sp -c xa_cmd_file -c results/ra-@.ranets -o results/ \
  ra_sigra
```

The `xa_cmd_file` file contains the `set_ra_option` command without the `-sigra_filter_run` option:

```
set_ra_option -pwra 1 -sigra 1 ...
```

After this run, the signal net EM results are available (and power net EMIR results if power net EMIR analysis was also specified).

Note:

You need to be aware of your usage of the `set_ra_net` command as it impacts the results:

- All the nets in `set_ra_net` commands are used for signal net EM analysis.
- If `set_ra_net` commands are used in a signal net EM filter run, then only those nets are analyzed for filtering.
- When no `set_ra_net` command is set (default), all the nets are considered in signal net EM.

Running Phase I and Phase II Simulations in the Signal EM Flow

In the EMIR flow, phase II simulation is run automatically after the completion of the phase I simulation. If you want to stop the simulation after the completion of the phase I simulation, use the following command:

```
set_ra_option -sigra 1 -ratcl ra.tcl -rap2auto 0
```

By default, `-rap2auto` is set to 1, so phase II simulation is run automatically following the phase I simulation. After the completion of the manual phase I run, at the end of the simulation log file, you see the run script that you can add to manually launch the phase II simulation. For example:

```
Grand Total Time = 1sec
Total Wall Time = 2sec
Total number of comparison errors = 0
Use PrimeSim wrapper to manually execute phase 2 of EMIR analysis:
/global/xa_<version>/ bin/xa -ra XA/xa-@
Source Files:
-----
| name                      | type |
|-----+-----|
| XA/xa-@.dcmmap           | dcmmap |
|-----+-----|
XA done: see "XA/xa.log" for details
```

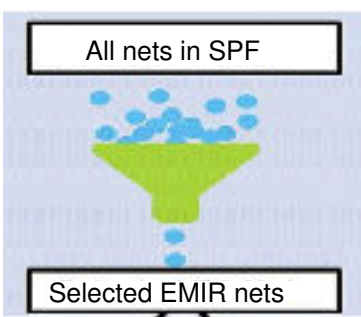
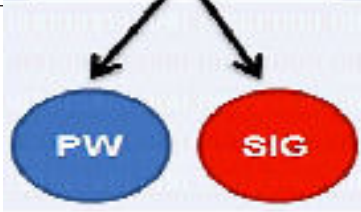
Selecting Nets for Signal EM Analysis

There are two ways to select nets for EMIR analysis, as shown in [Figure 19](#). You use the `set_ra_net` command to select the nets that go into EMIR analysis. By default the tool selects all nets for EMIR analysis.

When selecting EMIR nets, you can specify both explicit name or use a wildcard character. The net name is based on BA net name, not the schematic name. You can also apply multiple `set_ra_net` commands. If the net names are matched by both the `-selectnet`

and `-skipnet` arguments, `-skipnet` has a higher priority. The EMIR flow always honors your setup when you specify an explicit net name.

Figure 19 Selecting Nets for EMIR Analysis

Concept		Command
Selects nets to include in the EMIR analysis		<pre>set_ra_net -selectnet list_of_nets -skipnet list_of_nets</pre>
Defines the net types (PW or SIG) for the selected nets		<pre>set_ra_net_type -pwnet list_of_nets -signet list_of_nets</pre>

For more information about these commands, see the command pages in *PrimeSim™ XA Command Reference*. [Table 11](#) shows the net selection rules.

Table 11 Net Selection Rules

Rule	EMIR Command
The name and hierarchical delimiter are based on BA net name.	<code>set_ra_net -selectnet x1/x2/vss</code>
Wildcard matching (*) is allowed for the list of nets (the list is separated by one or more space).	<pre>set_ra_net -selectnet *</pre> Selects all nets for EMIR analysis. This is the default behavior. <pre>set_ra_net -selectnet vdd vss vint*</pre> Selects nets vdd, vss, and all nets match vint* for EMIR analysis.

Table 11 Net Selection Rules (Continued)

Rule	EMIR Command
Explicit name takes higher priority over wildcard matching for the same net.	<pre>set_ra_option -sigra 1 set_ra_net -selectnet vdd vss set_ra_net -skipnet *</pre> Processes nets vdd and vss are in signal net EM analysis.
The <code>-skipnet</code> argument has higher priority than <code>-selectnet</code> when matching the same net (both explicit name or wildcard).	<pre>set_ra_option -sigra 1 set_ra_net -selectnet vdd vss set_ra_net -skipnet vss</pre> In this example, all the nets that match w1* and bit* are skipped for EMIR analysis. All other nets are processed. <pre>set_ra_option -sigra 1 set_ra_net -selectnet * set_ra_net -skipnet w1* bit*</pre>
All net that are not selected by the <code>set_ra_net -selectnet</code> command are ignored in the EMIR analysis.	<pre>set_ra_option -sigra 1 set_ra_net -selectnet vdd</pre> In this example, net vdd is selected for signal net EM analysis, all other nets that are not specified by the <code>set_ra_net -selectnet</code> command are ignored in the signal net EM analysis.

There are only two types of nets that can go into EMIR analysis: signal nets (specified by `-signet`) and power nets. When selecting the types of nets, if the same nets are selected, the net is selected as power net. The net name is based on EMIR net name, not schematic name. You can also apply multiple `set_ra_net_type` commands. Only nets that are selected by `-signet` are selected for EMIR analysis; running `set_ra_option -sigra 1` triggers signal net EM analysis. For more information about the `set_ra_net_type` command, see the command page in *PrimeSim™ XA Command Reference*.

When the `set_ra_net_type` command is not used, the tool uses a default signal net ID method to identify signal nets and uses those signal nets for signal net EM simulation if `set_ra_option -sigra 1` is set. When you specify one or more `set_ra_net_type` commands, the tool disables the signal net ID and allows you to control which nets are considered to be signal nets. [Table 12](#) shows the rules for selecting signal nets.

Table 12 Signal Net Selection Rules

Rule	EMIR Command
Selects the <code>x1/x2/w1[0]</code> BA net for signal net EM analysis.	<code>set_ra_net_type -signet x1/x2/w1[0]</code>
Wildcard matching (*) is allowed for the list of nets (the list is separated by one or more space).	<code>set_ra_option -sigra 1</code> <code>set_ra_net_type -signet w1* b1*</code> Selects <code>w1*</code> and <code>b1*</code> nets for signal net EM analysis.
You can use multiple <code>set_ra_net</code> commands.	<code>set_ra_option -sigra 1</code> <code>set_ra_net -selectnet w1</code> <code>set_ra_net -selectnet b1</code> <code>set_ra_net -selectnet clk</code> Processes nets <code>w1</code> , <code>b1</code> and <code>clk</code> in signal net EM analysis.
The <code>-skipnet</code> argument has higher priority than <code>-selectnet</code> when matching the same net (both explicit name or wildcard).	<code>set_ra_option -sigra 1</code> <code>set_ra_net -selectnet w1 b1</code> <code>set_ra_net -skipnet b1</code> In this example, net <code>w1</code> is processed in the signal net EM analysis. Net <code>b1</code> is skipped because <code>b1</code> is matched by explicit name and <code>-skipnet</code> has priority over the <code>selectnet</code> argument.
The <code>-pwnet</code> argument has higher priority over <code>-signet</code> .	<code>set_ra_option -pwra 1</code> <code>set_ra_net_type -pwnet vdd_int</code> <code>set_ra_net_type -signet vdd_int</code> In this example, the <code>vdd_int</code> net is processed in power net EMIR analysis.

Selecting a Time Window for Signal EM Analysis

In the EMIR analysis, you have the flexibility to define a particular time window for the current collection (RASIM file) to reduce the RASIM file size. You can also specify more than one time window. To specify a time window, use the `set_ra_option` command:

```
set_ra_option -sigra 1 -ratcl ra.tcl \
  [-twindow tstart tstop {tstart tstop}[tstart]]
```

The following example generates an RASIM file between `t=10ns` to `t=20ns` and `t=90ns` to `t=100ns`:

```
set_ra_option -sigra 1 -ratcl ra.tcl -twindow 10n 20n 90n 100n
```

Setting an ratau Value for Current Resolution in Signal EM Analysis

In the EMIR analysis, a fixed time step is used for IR and electromigration simulation. By default, the fixed time step is set to 10ps. If a more detail time step is needed for high precision simulation, you can select a smaller ratau value. For example:

```
set_ra_option -sigra 1 -ratcl ra.tcl [-ratau ratau_value]
```

The following example selects 5ps as the time step for the EMIR simulation.

```
set_ra_option -sigra 1 -ratcl ra.tcl -ratau 5p
```

The next example selects 5ps as the time step for the EMIR simulation.

```
set_ra_option -sigra 1 -ratcl ra.tcl -ratau 5p
```

Enabling RASIM File Splitting

For some EMIR analyses, large signal nets result in a very large RASIM file. To help viewing the RASIM results more efficiently, you can split the RASIM file into multiple files. For example:

```
set_ra_option -sigra 1 -ratcl ra.tcl [-waveform_split size]
```

The default waveform file split size is 2G, in unit of bytes. The following example selects 4G as the waveform file size for splitting.

```
set_ra_option -sigra 1 -ratcl ra.tcl -waveform_split 4000000000
```

Note:

Because the tool compresses the split waveform file, the size of the split waveform file might become smaller after splitting.

Redirecting Temporary EMIR Files

You can redirect pseudo temporary files to a user-specified location in the EMIR analysis flow. You use this command when the `-o` argument generates large files that cause a lot of network traffic, or when the `-o` location does not have sufficient disk space. For example:

```
set_ra_option -sigra 1 -ratcl ra.tcl -tmpdir /tmp
```

Redirects the storage of the temporary EMIR analysis files to the `/tmp` directory.

For more information about the `set_ra_option` command, see the *PrimeSim™ XA Command Reference*.

Using Variable Time Steps

PrimeSim EMIR supports `vt`s and `vtsopt`RATCL commands to enable and apply the variable time step technique for IR drop analysis during the analysis stage (Phase II). Using variable time steps provides benefits on runtime with comparable accuracy when switching activity is low. By default, PrimeSim EMIR uses constant time steps when analyzing IR drops during the analysis (Phase II) stage.

When a variable time step is specified with the `vtsopt` command, the power net matrix is stamped and factorized for each step. The factorized matrices are available during simulation. The default variable time steps are: `rata`u, `2*rata`u, `4*rata`u.

Simulation accuracy is controlled by the `dv` tolerance value specified with the `-dv` option of the `vtsopt` command. The tool selects a larger (or smaller) step if the simulated voltage difference is smaller (or larger) than the `-dv` value. For example, `dv=10mV`.

At the end of simulation at phase II, the tool reports the step information in the `*.ralog` file of each power net, in the following format:

```
Total number of steps: n/N (ns1% ns2% ... - nr%)
```

Where:

- `n`: Number of actual time steps during simulation (including `presim` time)
- `N`: Number of steps if simulated with a constant step of `rata`u (including `presim` time)
- `ns1%`, `ns2%`, ...: Percentage of steps for step1, step2, ... of actual time steps
- `nr%`: Percentage of rejected steps

For example, if `rata`u=2ps, phase II simulation window=1ns-5ns, `presim`time=1ns, `steps=1 2 4`, the total number of steps will be reported as:

```
Total number of steps: 1356/2500 (35.6% 40.8% 23.6% - 8.9%)
```

RATCL File Example

```
vt 1 vtsopt -dv 0.05 -steps 1 3 9
```

In this example, the `vt`s command is set to 1 to enable variable time step technique. The `dv` tolerance is 50mV and the steps are `1*rata`u, `3*rata`u, `9*rata`u.

Running Phase II Simulation

To start Phase II, use the following syntax:

```
xa -ra <output file name>-@
```

For example:

```
xa -ra test-@
```

The tool reads the .ratcl file and executes the commands specified in the .ratcl file in which you specify commands to set analysis types, set waveform viewer, control the content of the output files, and so on. See [Table 13](#) for a list of the commonly used commands for phase II simulation. For a full list of commands that are supported in the phase II simulation, see [Commands for EMIR Analysis Post-processing](#).

During Phase II simulation, the currents stored from Phase I simulation are injected into the non-reduced signal nets. For each time interval equal to the RATAU value, the currents through each signal net resistor are recalculated and updated.

Table 13 Commonly Used Commands in Phase II Simulation

Command	Description
<code>ra</code>	Specifies the analysis types to perform. The supported analysis types are: power net EMIR analysis, signal net EM analysis, and IR drop analysis with the Vmax, Vmin, and Vavg output values.
<code>raviewer</code>	Sets a waveform viewer to display the output waveforms
<code>raformat</code>	Specifies the output layout format. The supported formats are: <code>all</code> , <code>gds2</code> (default), <code>ccdb</code> , <code>ascii</code> (default) and <code>rve</code> .
<code>ralayout</code>	Specifies the types of analysis results to write to the output files, named <code><prefix>_<analysisName>.<ext></code> .
<code>ralayers</code>	Sets the layer numbers of the output violation map.
<code>outformat</code>	Specifies the output waveform format.
<code>vref</code>	Specifies a voltage reference value for internal power supplies.
<code>asciicols</code> <code>asciicolsort</code> <code>ccdbccols</code> <code>ccdbmerge</code> <code>gdsprops</code>	Controls content for the output file.

Defining Net Pins by Specifying XY Coordinates

As in power net EMIR analysis, you can define "*"P" net pins by specifying XY coordinates during phase I and phase II simulations in signal net EM analysis.

For a detailed description about defining net pins, see the [Defining Net Pins by Specifying XY Coordinates](#) section in the Power Net EMIR Analysis chapter.

Running Power Net EMIR and Signal Net EM Together

In PrimeSim EMIR analysis, the tool allows you to specify both the power net and signal net EM analyses in a single run. You need to ensure the `-pwra` and `-sigra` arguments in the `set_ra_option` command are set to 1. For example:

```
set_ra_option -pwra 1 -sigra 1 -ratcl ra.tcl
```

The following example runs the nets `vdd` and `gnd` for power net EMIR flow and runs everything else through the signal net EM flow.

```
set_ra_option -pwra 1 -sigra 1 -ratcl ra.tcl  
set_ra_net_type -pwnet vdd gnd
```

If a given net is specified both as `-signet` and `-pwnet`, the `-pwnet` has the higher priority, and that net is treated as a power net. In the following example, the `vdd_internal` net is treated as power net.

```
set_ra_option -pwra 1 -sigra 1 -ratcl ra.tcl  
set_ra_net_type -pwnet vdd gnd vdd_internal  
set_ra_net_type -signet vdd_internal
```

Examining Output Files

The FSDB, OUT, WDF, and tr0 waveforms are in various formats; however, FSDB is the default format. One waveform format is always output and the format type is specified by the `set_waveform_option` command. The file name has the following syntax:

```
<prefix>-@.fsdb
```

In the FSDB, OUT, and WDF, and tr0 waveforms, `<prefix>` is `xa` or the name as specified on the command line after `-o`. The waveform file contains at least two signals for each signal net, stored as follows:

- `lpin_tot (net)`: max (sum_of_pin_currents_into_the_net, sum_of_pin_currents_out_of_the_net)
- `lpin_max (net)`: Maximum pin current injected to the net

6

Static IR Drop Analysis

This section describes how to run a static IR drop analysis with the PrimeSim EMIR simulator.

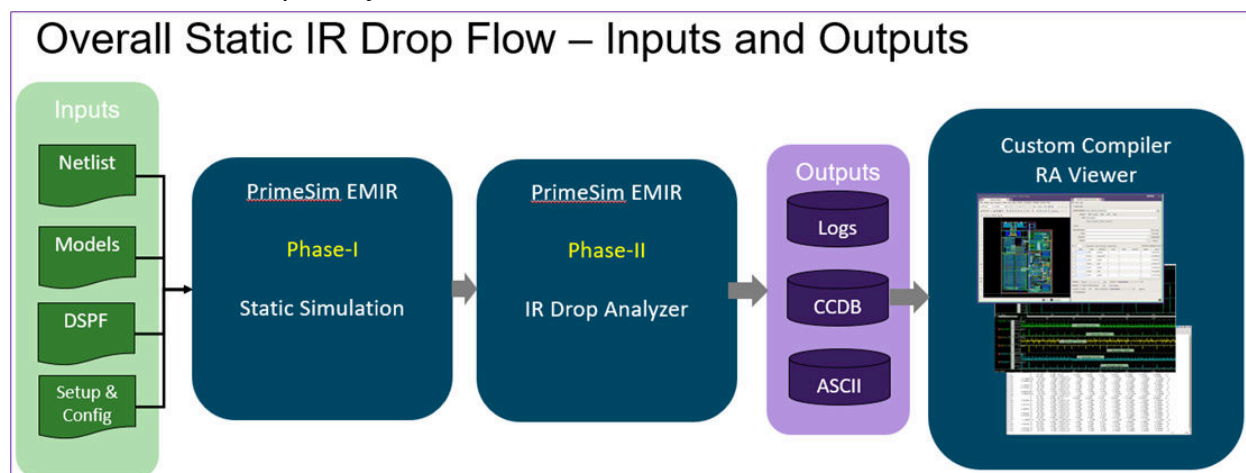
Overview of the Static IR Drop Analysis Flow

Static IR drop is the voltage drop that occurs when constant current draws through the power network with varying resistance. This IR drop occurs when the circuit is in the steady state (that is, when inputs are not switching), and is dependent on the RC elements of the power grid that connects the power supply to the respective standard cells. Static IR drop flow does not depend on the stimuli and hence it is called “vector less” flow.

In PrimeSim EMIR, the difference between static IR drop flow and dynamic IR drop flow is that during dynamic flow, the device currents get injected only at the specific simulation time (based on the user-provided stimuli), whereas during static IR drop flow the tool calculates the worst-case current drawn from all the devices at the same time. Therefore, there is no transient simulation for static IR drop flow.

Figure 20 illustrates the static IR drop analysis flow:

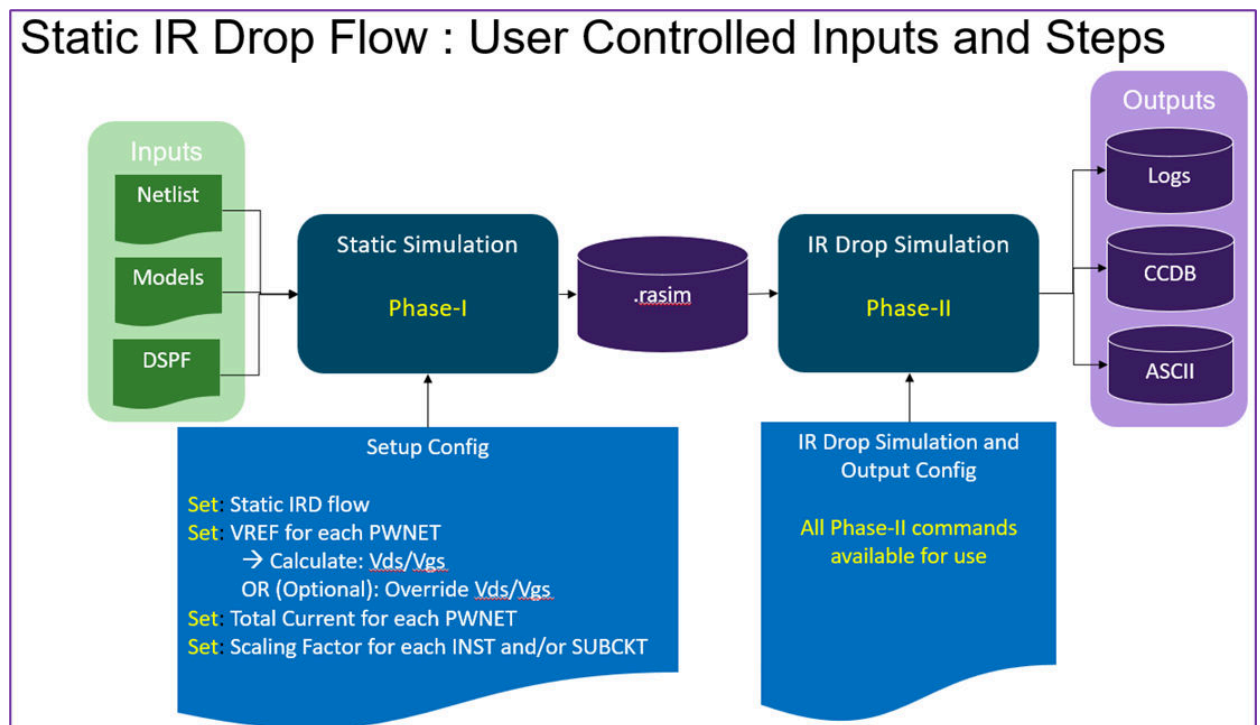
Figure 20 Static IR Drop Analysis Flow



- Static Simulation (Phase I) : Calculates and saves the current for each MOS device connected to the power net by a drain or source terminal. To calculate static device current, PrimeSim EMIR utilizes the size of the device (device parameters) and device model. Constant currents for each MOS device connected to power nets are calculated and then used in Phase II.
- IR Drop Analysis (Phase II): Uses the currents saved from Phase I for RC simulation.

Figure 21 shows the static IR drop analysis flow with user-controlled inputs and the outputs that are generated when analysis is complete.

Figure 21 Static IR Drop Analysis Flow With User-Controlled Inputs



In the static IR drop analysis flow with user-specified inputs, PrimeSim EMIR first pre-characterizes each MOS device using the device size explicit biasing conditions, as follows:

$$V_{ds} = 0.5 * V_{ref}$$

$$V_{gs} = 0.5 * V_{ref}$$

Where V_{ref} is the reference voltage and V_{ds}/V_{gs} can be overridden by user inputs. PrimeSim EMIR performs automatic power net detection and reference voltage identification to derive V_{ref} .

For example, if PrimeSim EMIR detects a VDD power net which connects to a 0.8V supply, bias conditions are decided by the following equation:

$$V_{ds} = V_{gs} = 0.5 * 0.8V = 0.4V$$

If V_{ref} is zero (GND net), PrimeSim EMIR uses the maximum V_{ref} :

$$V_{ds} = V_{gs} = 0.5 * \max(V_{ref})$$

Running Static IR Drop Analysis

This topic describes how to run a static IR drop analysis in PrimeSim EMIR.

1. Before running a static IR drop analysis, you need to specify necessary settings using the EMIR setup commands. These setup commands can be specified at the `-c` command line option.

Provide simulation netlist in which dspf file and model file are included by

```
set_flow_option -ra_simulator_input <netlist>
```

Specify PrimeSim as simulator to recognize the syntax in netlist properly if `.option primesim_spf` is used

```
set_flow_option -ra_simulator primesim
```

Following are the most commonly used commands and options for static IR drop analysis. For a list of EMIR commands supported in the PrimeSim EMIR tool, see [EMIR Commands](#).

- `set_ra_option -static_pwra 1`
- `set_ra_pwnet_option -vref`

Specify a user-defined VREF value as reference voltage for the specified net in V. The VREF value is reported in the `.ralog` file generated by the tool.

In the following example, the VREF value for the net VDD is set to 1.5V, and $V_{ds} = V_{gs} = 0.5 * 1.5V = 0.75V$.

```
set_ra_pwnet_option -net vdd -vref 1.5
```

```

RAWF                : 0
RDNCAUTO            : 0
RDNCRALAYOUTAUTO    : 1
RAVIEWER            : ccompiler
REDIW               : 1
                   : .1@16
REDV                : 0
RARVE               : -diagtype poly
SOLVER              : slap
TEMP                : 27
VREF                : 1.5
USEBBOXWIDTH        : 1

```

Content of .ralog file.

- `set_ra_pwnet_option -net vdd -static_vds -static_vgs`

Specify user-defined Vds and Vgs values for the specified net. This overrides the default Vgs and Vds of the device, which was calculated on the basis of $V_{gs}=V_{ds}=0.5 \text{ (Vref)}$. The tool uses the user-defined Vgs and Vds values for device current calculations.

The following example sets $V_{ds} = 1.2\text{V}$ and $V_{gs} = 1.4\text{V}$.

```
set_ra_pwnet_option -net vdd -static_vds 1.2 -static_vgs 1.4
```

- `set_ra_pwnet_option -static_current`

Define the total current for each power net in A. Instead of using device models for static current calculation, you can set a total static current constraint for a net by using the `-static_current` option of the `set_ra_pwnet_option` command. The device current is scaled by the expression:

$$IMOS = (I_{static}/I_{total}) IMOS_{orig}$$

Where $IMOS_{orig}$ is calculated using device models, Vds, and Vgs of the MOS device. I_{total} is the total sum of the original MOS currents while $-I_{static}$ is the `current_value` from the `-static_current` option.

The tool first compares the total current and the user-provided current constraint, and then scales the device currents so that the total current is equal to the user specified constraint.

When current is normalized by the total net current, only the current value is changed; no direction is changed. The tool issues an error message when the specified current is negative.

```
set_ra_pwnet_option -net vdd -static_current 3
```

```
◦ set_ra_static_scaling_factor <factor_value> [-inst <inst_name>]
  [-subckt <subckt_name> <subckt_name1> ...]
```

or

```
set_ra_static_scaling_factor <factor_value> [-net <net_name> -inst
<inst_name>> <inst_name1>> ...]
```

Define a scaling factor. A scaling factor is used to adjust device current in specific instances or subcircuits, or specific instances for a particular net. Wildcard characters (*) are supported to specify an instance name, but cannot specify a subcircuit name.

- Instance- or Subcircuit-based: The device currents in specific instances or subcircuits are scaled per user-defined scaling value.
- Net-specific instance-based. The device currents in specific instances for a particular net are scaled per user-defined scaling value. Note that it is mandatory to follow the order as `-net <netname>` and then `-inst <inst_name>`.

When both scaling factor and total power net's current constraint are provided, scaling factor is applied first when calculating $I_{MOSorig}$ and I_{total} .

The following example scales device currents by 100 for the devices with instance names which match the symbol `m*` inside the subcircuit `nand3b`.

```
set_ra_static_scaling_factor 100 -subckt nand3b -inst m*
```

```
◦ set_ra_option -static_use_ic 1
```

When set to ON, the tool uses the MOS terminal voltages specified in an `.ic` file for current calculation, not the default `vds=vgs=0.5*vref`.

For this to work correctly, all four terminals of a MOS device should have their `.ic` voltage values specified. If any of them is missing, the usual default will be applied to all four terminals.

The tool also reports tables in the log file for MOS terminals that were found or missed in the whole process. It is also recommended to specify `-static_use_ic 1` with `cnet` back-annotation. Or a warning will be printed in the log file. For example,

```
Warning: When "set_ra_option -static_use_ic 1" command is given,
more reliable static RA results are expected with
cnetback-annotation. Consider using command "load_ba_file ... -cnet
*".
```

Following is an example of the configuration file:

```
set_flow_option -ra_simulator_input "top.spi"
set_ra_option -ratcl for_xa.tcl
```

```
#set_ra_pwnet_option -net vdd -vref 1.5
#set_ra_pwnet_option -net vdd -static_vds 1.2 -static_vgs 1.4
#set_ra_pwnet_option -net vdd -static_current 3
set_ra_static_scaling_factor 100
-subckt nand3b -inst m* set_ra_option -static_use_ic 1
```

2. Launch the PrimeSim EMIR simulation with the configuration file which contains the EMIR commands you specify at Step 1. For example:

```
primesim_emir -flow static_ir -c emir_config.cfg -o test/
```

Specifying User-Defined Pin Waveforms

Use the `replace_waveform` command to replace the pin waveform obtained from the RASIM file with a user-defined one in the RATCL file. Use this command when you want to run the Phase II simulation with a new waveform value without rerunning the Phase I simulation. The command is applicable for all nets: external power nets, internal power nets, and signal nets.

Here is the syntax of the command:

```
replace_waveform net_name pin_name -dc current_value
```

Viewing Static IR Drop Analysis Results

Similar to dynamic EMIR analysis, the tool writes the analysis results to the output files. You can either examine the output files or visualize the results in the Custom Compiler tool.

To do this, load the output .ccdb file by choosing **File > Load EMIR Results** in the **PrimeSim EMIR** dialog box in a layout editor GUI.

7

Running the EMIR Application

This chapter describes how to invoke the EMIR application in the PrimeWave Design Environment or Custom Compiler tools to perform static resistance check and dynamic EMIR analysis to analyze and examine IR drop violations in your design.

Getting Started

This chapter provides information on how you get started with EMIR application.

Setting Path

Before you start to run EMIR application in the graphical user interface, make sure

- The DISPLAY environment variable is set to the name of your UNIX or Linux system display. The EMIR application uses the DISPLAY environment variable to determine the display location.
- The license path is set up using one of the following environment variables:
 - SNPSLMD_LICENSE_FILE: A pointer to the Synopsys license paths
 - LM_LICENSE_FILE: A pointer to all license paths. If you specify both environment variables, SNPSLMD_LICENSE_FILE takes precedence over LM_LICENSE_FILE.

In C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

In Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

- The shell path is set up to include the path for the following tools to the launch script:
 - Custom Compiler or PrimeWave Design Environment for Reliability Analysis
 - Custom Infrastructure
 - PrimeSim XA simulator
 - PrimeSim EMIR

For example,

```
> setenv PATH <path_to_layout_editor_install_dir>/bin:$PATH  
> setenv PATH <path_to_PrimeSim_EMIR_install_dir>/bin:$PATH  
> setenv PATH <path_to_XA_install_dir>/bin:$PATH  
> setenv PATH <path_to_Custom_Infrastructure_install_dir>/bin:$PATH
```

- All the required input files are present

Opening the PrimeSim EMIR GUI

The PrimeSim EMIR GUI provides menu commands for setting up and running Library EMIR analysis, Dynamic EMIR analysis, Fast IR analysis, Static IR analysis, and static resistance check (SPRES), browsing analysis results, and invoking what-if analysis and least resistance path checking tools. The look and feel of the PrimeSim EMIR GUI is consistent with the look and feel of the Custom Compiler GUI.

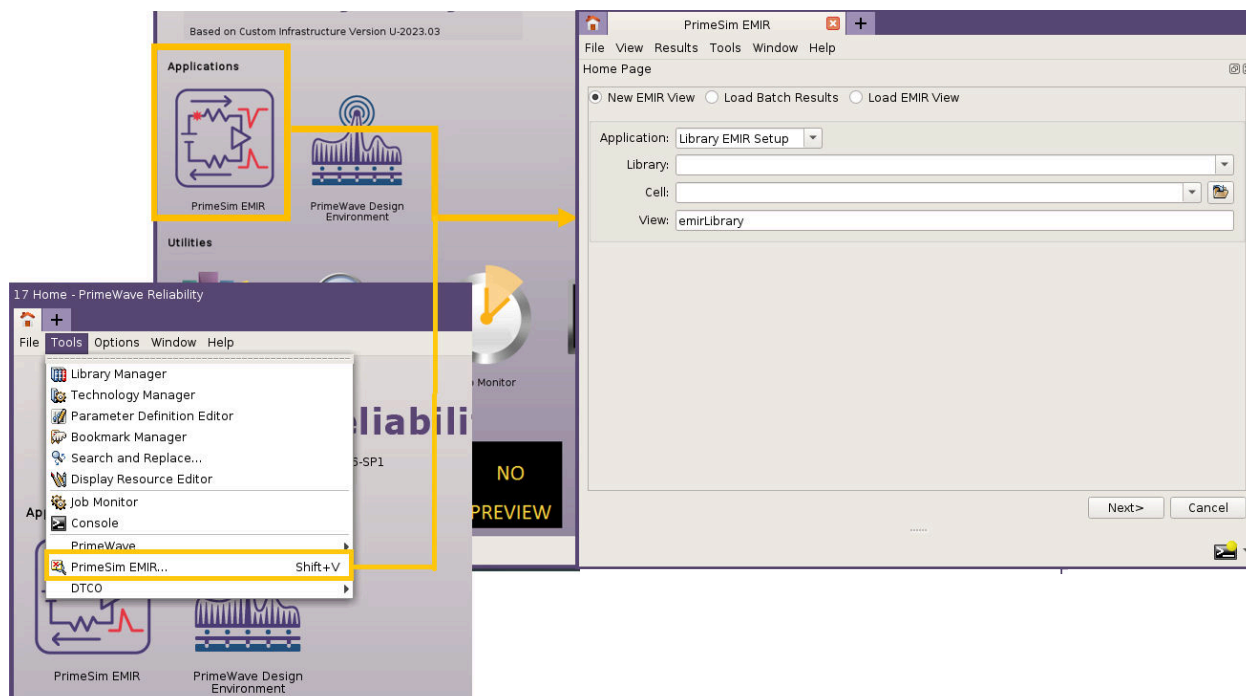
The PrimeSim EMIR GUI lets you select and view the EMIR and resistance check results and overlay the results on the design layout for targeted evaluation and debugging. This integrated GUI also enables you to overlay the results on the design layout. For example, you can check net resistance for pads to pin and pads to internal instance pins, or select, and zoom or pan to view individual violations. In the accompanying data table, you can change bin settings to control the range of violation values shown in each bin, enabling a more targeted evaluation.

Note:

The content of the data table depends on the results and other settings you choose.

To open the PrimeSim EMIR GUI,

- In PrimeWave Design Environment or Custom Compiler, choose **Tools > PrimeSim EMIR** or click the **PrimeSim EMIR** icon.



The **PrimeSim EMIR** window opens.

In the PrimeSim EMIR GUI, you can perform the following tasks:

- Create, load and save EMIR Views.

Running the PrimeSim EMIR application requires EMIR views for retrieving and storing EMIR-related data in the Library EMIR, Dynamic EMIR, Fast IR, Static IR, and static resistance check (SPRES) analysis flow.

- Load EMIR results from the batch mode run

If you have completed EMIR analysis using the PrimeSim EMIR tool in the batch mode, you can import the generated EMIR results, and evaluate them by overlaying the results on the master design layout in the GUI environment.

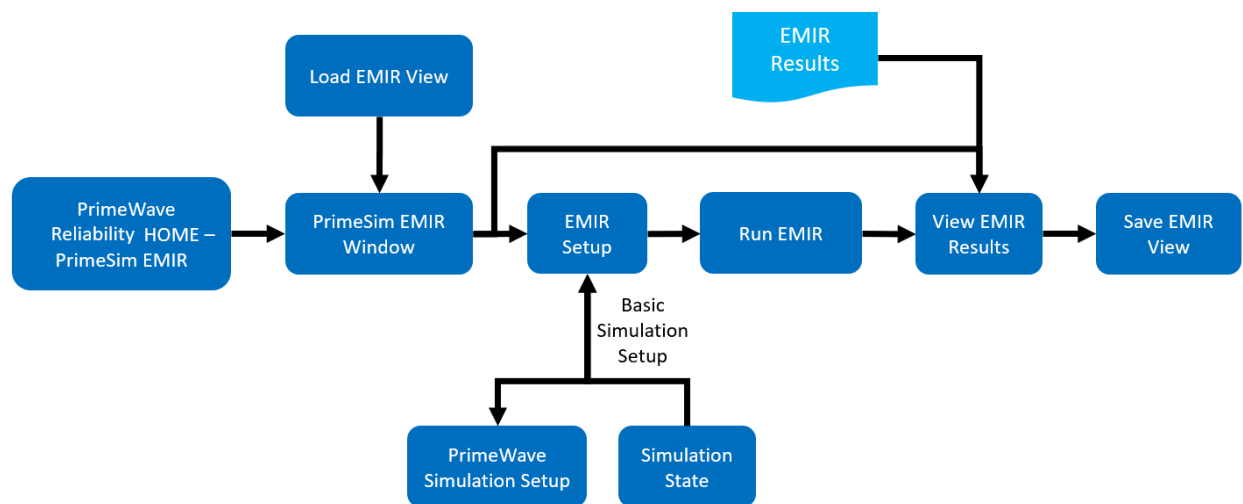
- Specify settings and perform the dynamic EMIR analysis or static resistance check run
- View analysis results and check for pins with violations

After you load the analysis data in the EMIR application, you can select the data you want to view in the data table. You can choose to redistribute the bin values to control the set of results shown in each bin to enable a more focused evaluation.

EMIR Application Workflow

The following figure describes the PrimeSim EMIR workflow in the graphical user interface, such as Custom Compiler or PrimeWave Design Environment for Reliability Analysis.

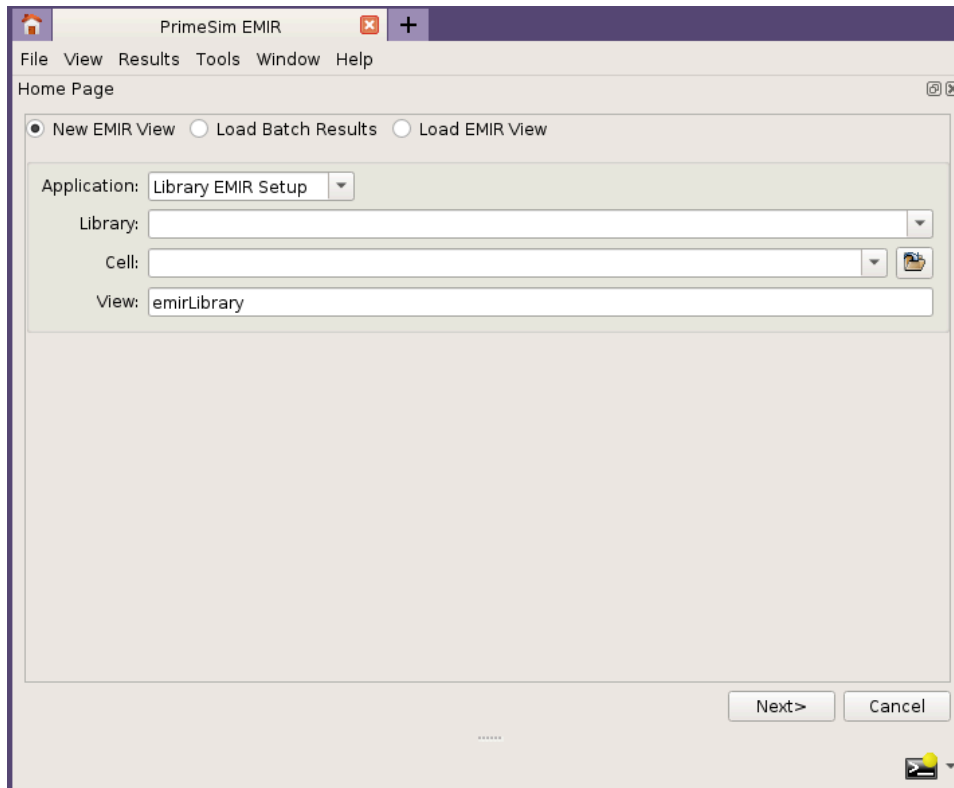
EMIR Analysis and Static Resistance Check Workflow



PrimeSim EMIR Home Page

The Home Page is a widget that provides a quick access to perform the following tasks:

- **New EMIR View:** Create a new view for an application and click **Next** to setup and run the application.
- **Load Batch Results:** Reuse any saved analysis data in the current run and load the results. Click **Next** to view the results in the Results Viewer.
- **Load EMIR View:** Load a saved EMIR view data and click **Next** to view the results in the Results Viewer.

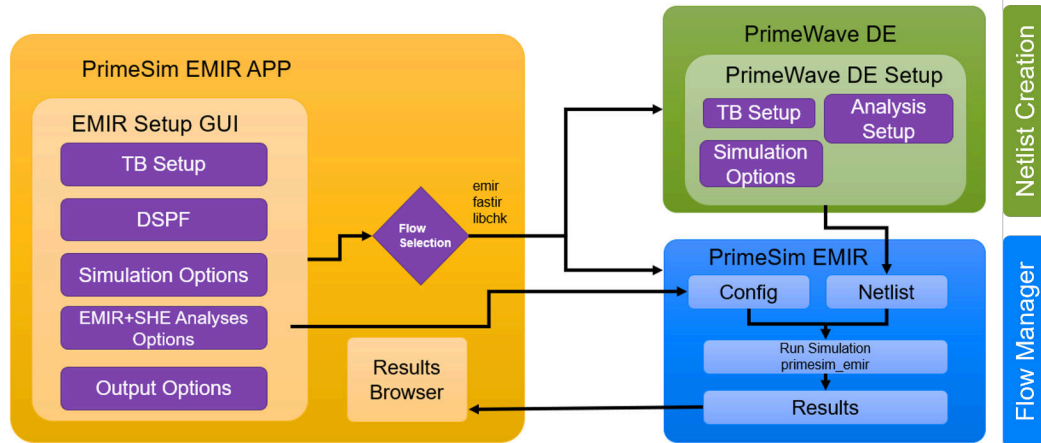


Setting up PrimeSim EMIR Applications for Analysis

After you have created an EMIR view for an application, you must setup the analysis option.

The PrimeSim EMIR app setup GUI allows you to do the following:

- Flow selection
- Testbench setup (Schematic or Netlist based)
- Analysis setup
- Flow steps:
 - Netlist creation
 - Run simulation and EMIR through Flow Manager
- Automatically load results in Results Viewer on completion of simulation



Use the following command to invoke PrimeSim EMIR:

```
primesim_emir -flow libchk|emir|fast_ir -c <config_file> -o
<output_prefix>
```

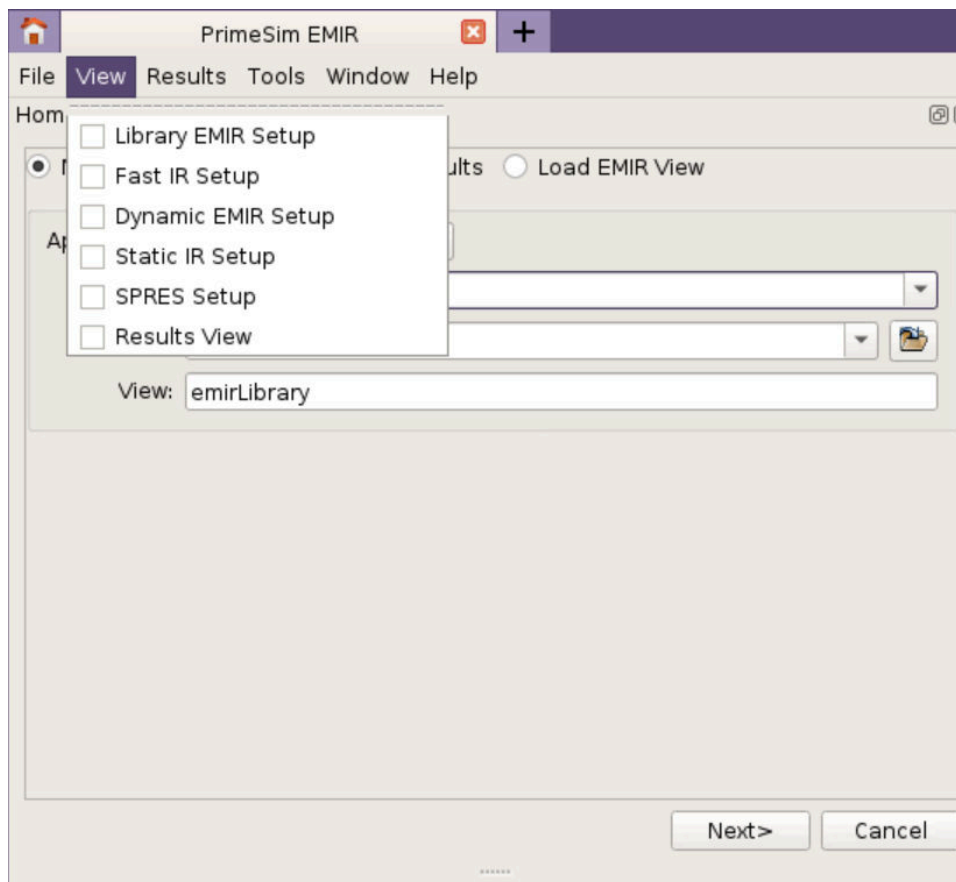
Most of the settings in the setup have corresponding commands, arguments, and options. This topic provides brief summaries of the commands, arguments, and options. For detailed descriptions, see the EMIR Commands topic in PrimeSim EMIR Reference Manual.

The setup also uses key PrimeWave Design Environment settings to run the simulations. For detailed descriptions of the PrimeWave Design Environment settings, see the Setting Up and Invoking the PrimeWave Design Environment topic in PrimeWave Design Environment user documentation.

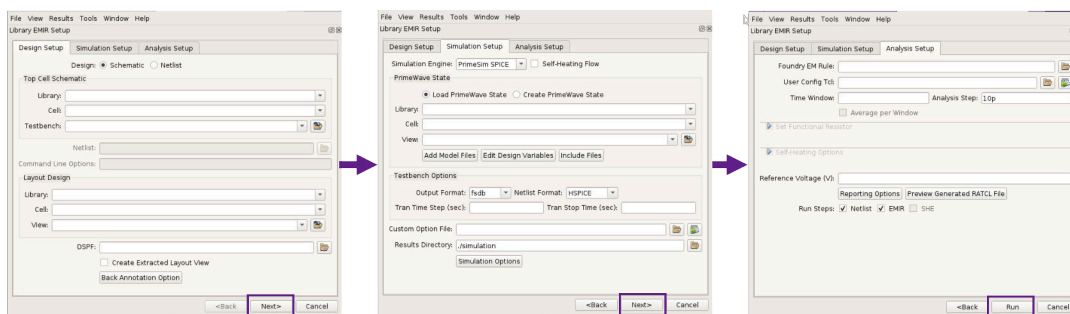
PrimeSim EMIR Setup GUI

To select a PrimeSim EMIR setup, click **View** and select the analysis setup. This opens the associated setup GUI.

1. Click **View** and select a setup. This displays the **Design Setup** tab.



2. In **Design Setup** tab, fill out the necessary options and click **Next**.
This displays the **Simulation Setup** tab.
3. In the **Simulation Setup** tab, fill out the necessary options and click **Next**.
This displays the **Analysis Setup** tab.
4. In the **Analysis Setup** tab, fill out the necessary options and click **Run**.



The results of the analysis is automatically loaded in Results Viewer.

Note:

The Static IR and SPRES setups do not have the **Simulation Setup** tab. So, clicking **Next** in the **Design Setup** tab takes you to the **Analysis Setup** tab.

Running EMIR Simulations

When you are ready to run the simulation, in the EMIR setup GUI, click **Run** in the **Analysis Setup** tab. The simulation starts and the Job Monitor appears so that you can view the progress. When the simulation finishes, the Job Monitor shows the job completion status.

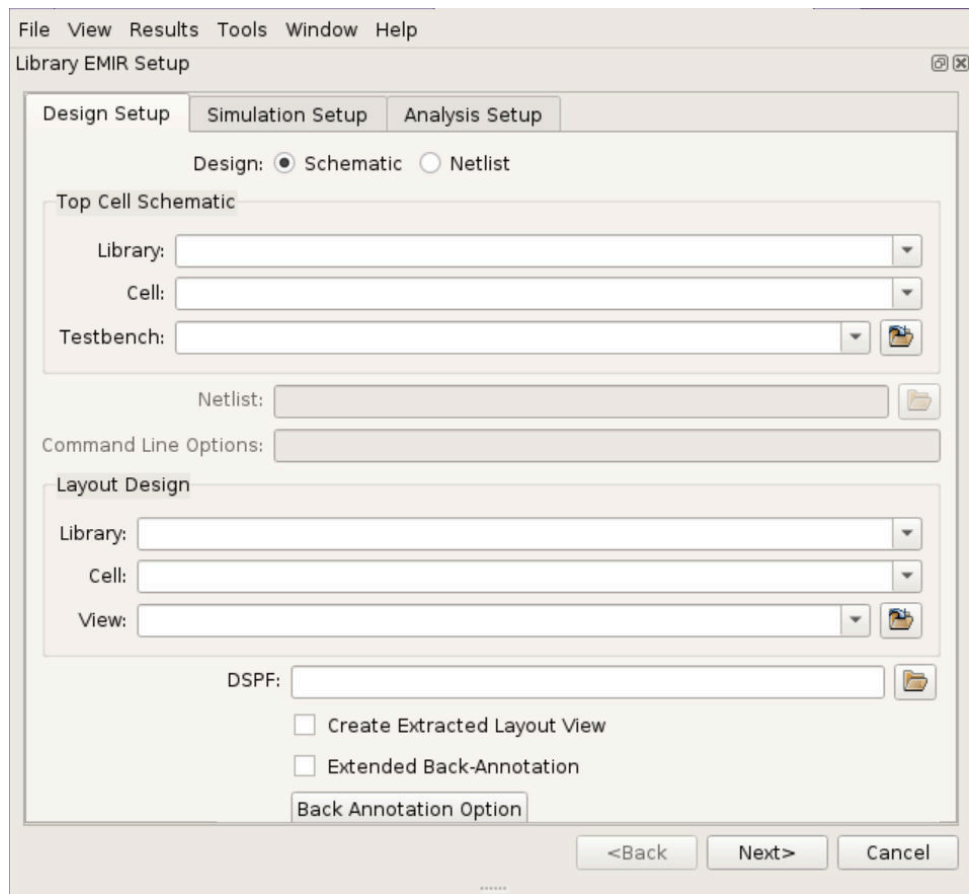
Library EMIR Setup

The **Library EMIR Setup** consists of setup options, which are presented in the following tabs:

- [Design Setup Tab](#)
- [Simulation Setup Tab](#)
- [Analysis Setup Tab](#)

Design Setup Tab

The following figure shows the **Design Setup** tab for Library EMIR.

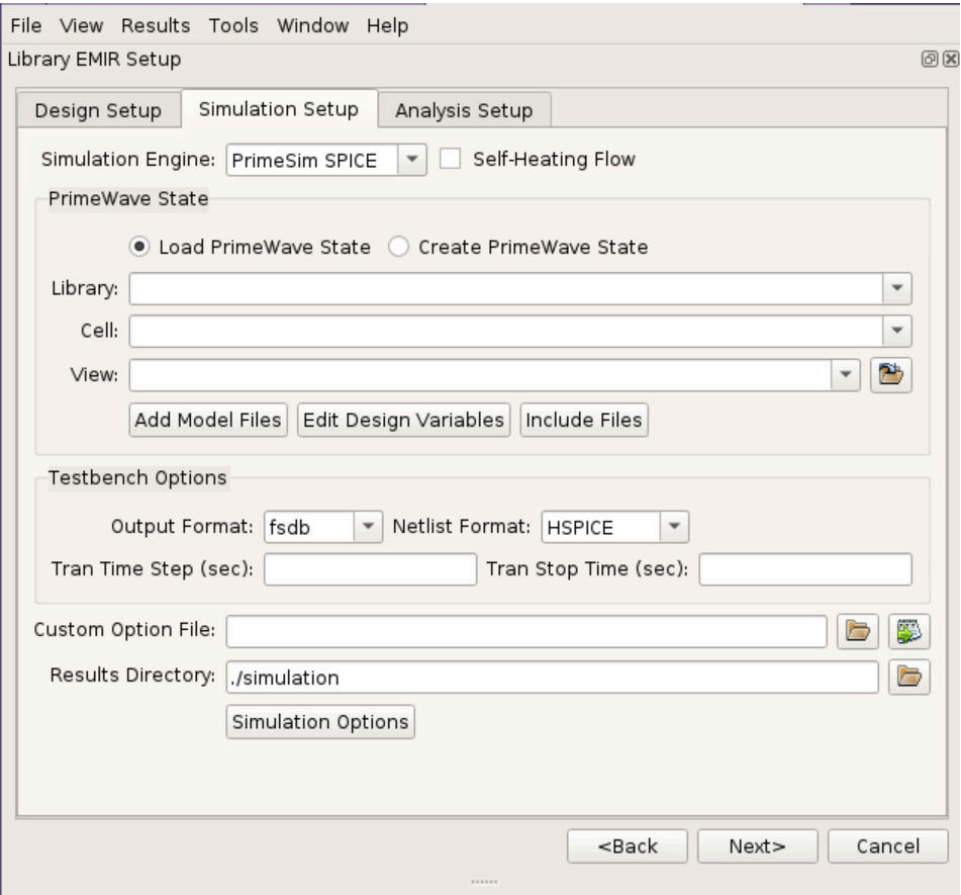


Option	Description
Design: Schematic	• Schematic: The default flow for Library EMIR • Netlist: ◦ Disables Schematic flow and enables Netlist and Command Line Options fields in Layout Design setup. ◦ Disables Simulation Engine selection– Supports PrimeSim XA only ◦ Disables PrimeWave State The default flow for Library EMIR.
Design: Netlist	Library EMIR– Netlist based flow. • Disables Schematic flow and enables Netlist and Command Line Options fields in Layout Design setup. • Disables Simulation Engine selection– Supports PrimeSim XA only • Disables PrimeWave State
Top Cell Schematic	Library, Cell, View: The library, cell, and EMIR view information for the EMIR view.
Layout Design	Library, Cell, View: The library, cell, and EMIR view information for the EMIR view.

Option	Description
DSPF	Specify the DSPF file and enable Extended Back-Annotation option.
Create Extracted Layout View	Select to create an extracted layout. Creates OA layout view shape information from DSPF file. The extracted layout is saved in the flow manager results directory in LCV format –It can be added to library manager view.
Extended Back-Annotation	Select to run the extended back-annotation flow. For more details, see Running Extended Back-Annotation Flows
Back Annotation Option	Opens the Parasitic Back Annotation dialog box. For more information, see Parasitic Back Annotation Dialog Box .

Simulation Setup Tab

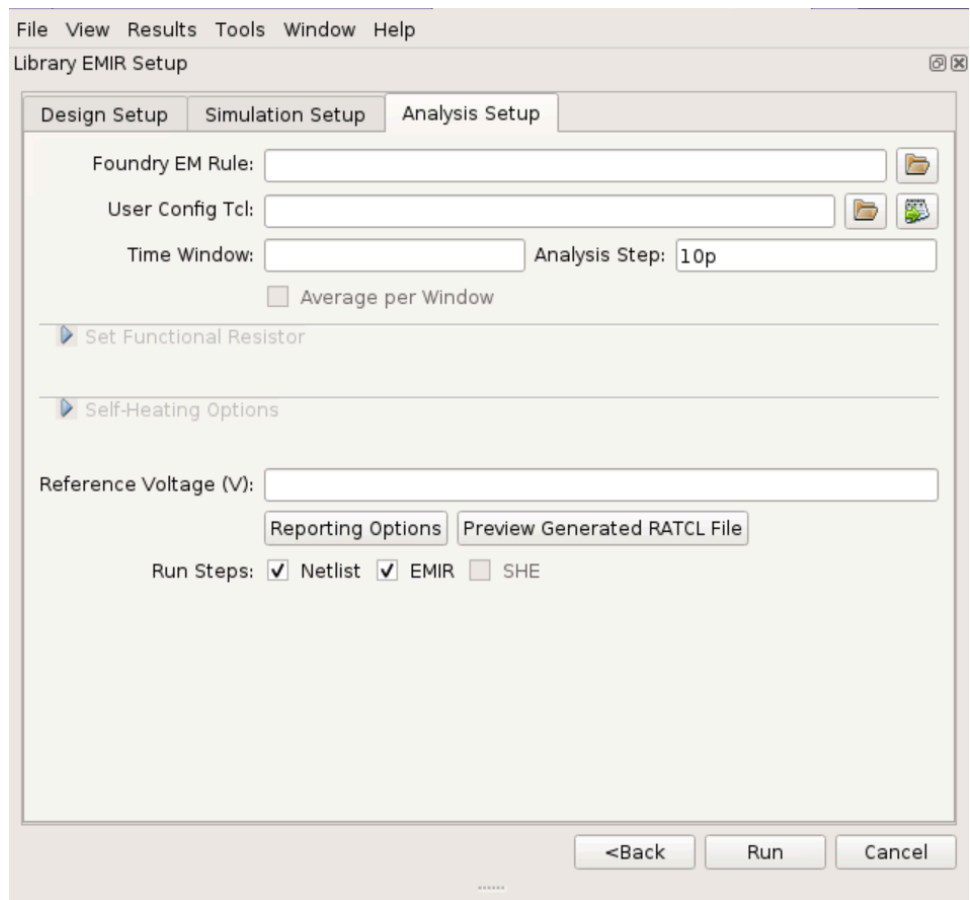
The following figure shows the **Simulation Setup** tab for Library EMIR.



Option	Description
Simulation Engine	Select the simulator. Default: PrimeSim SPICE
Self-Heating Flow	Select to enable self-heating flow. Note: Enabled only through specific Process Node preference. Reach out Synopsys support to get relevant details.
PrimeWave State	The library, cell, and view information for the PrimeWave state view. - Load PrimeWave State: Select to load an existing PrimeWave State. - Create PrimeWave State: Select to create a new PrimeWave state that you want to save.
Add Model Files	The model files to use in the simulation. Model files contain calculations that predict design behaviors under different conditions. Click to open the Model Files dialog box.
Edit Design Variables	Parameters, such as MOSFET widths, that you can alter to assess the impact on the design. Click to open the Edit Design Variables dialog box.
Include Files	Additional files you want to process during the simulation. Click to open the Include Files dialog box.
Testbench Options	- Output Format: One of fsdb, wdf (default), or psf. This option is disabled for Netlist-based flow. - Netlist Format: Either HSPICE (default) or Third-party. - Tran Time Step: The fixed time step used for EM/IR simulation. The option is equivalent to <code>rataui</code> in the simulator tools. Use this option to define a non-default time step value. This option is disabled for Netlist-based flow. - Tran Stop Time: Stop time. This option is disabled for Netlist-based flow.
Custom Option File	: Specifies the text file containing additional commands and options you want to run during simulation and analysis. : (Applicable only if you specify a Custom Option File) Opens your system-default text editor to let you view the content of the custom option file.
Results Directory	The directory where you want to save the simulation results. See also Saving Simulation Results in PrimeWave Design Environment user documentation.
Simulation Options	Click to open the Simulation Options dialog box.

Analysis Setup Tab

Analysis Setup options are set in this tab. Click **Run** to run the analysis.



Option	Description
Foundry EM Rule	Add the foundry electromigration rule file.
User Config Tcl	Additional Tcl configuration file.
Time Window	The time window during which current data is collected.
Analysis Step	Fixed time step used to calculate IR drop and electromigration violations. Default: 10ps.
Reference Voltage	Use to calculate the IR drop percentage.
Reporting Options	Opens the Reporting Options dialog box. For more information, see Reporting Option Dialog Box
Preview Generated RATCL File	An RATCL file provides the Tcl interface to specify analysis types, set waveform viewer, control the content of the output files, and so on. An RATCL file is automatically generated when an EMIR analysis run is complete. To examine the content of the RATCL file, click Preview Generated RATCL File .

Option	Description
Run Steps	Selects the steps to run. • Netlist: Netlisting the schematic design. If the design source is a netlist, this option has no effect. • EMIR: Simulation and EMIR step through Flow Manager. Automatically loads the Netlist File and Commands File from the simulation netlist directory. • SHE: Run self heating analysis steps.

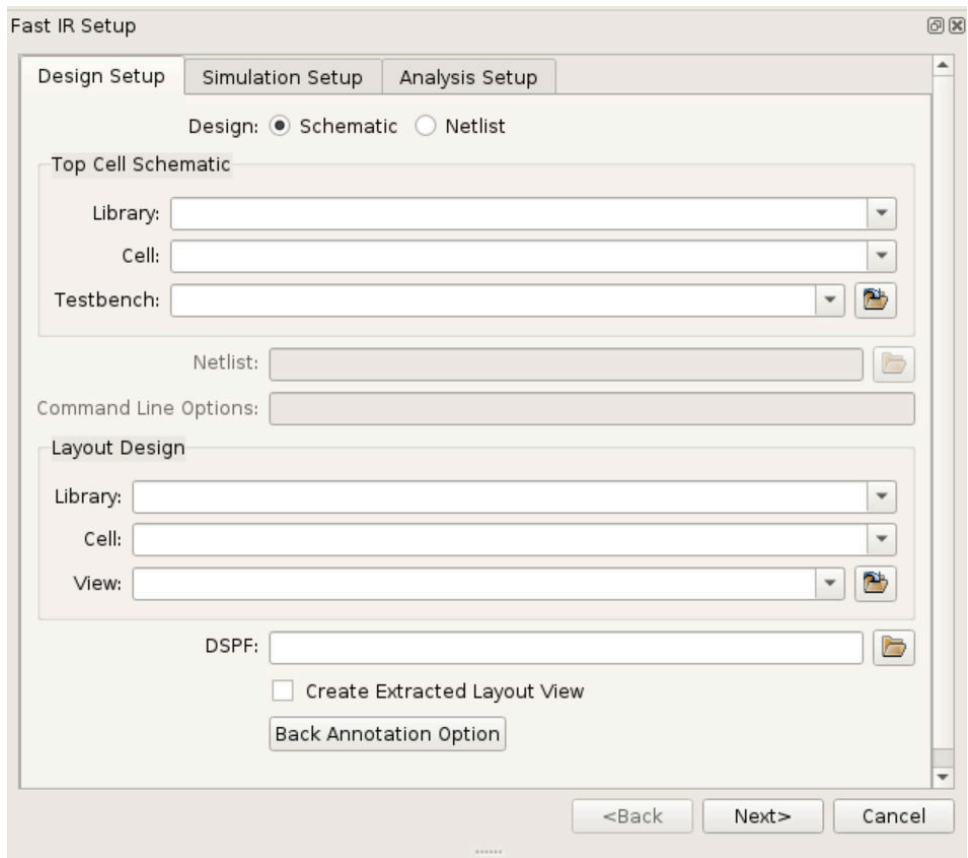
Fast IR Setup

The **Fast IR Setup** consists of setup options, which are presented in the following tabs:

- [Design Setup Tab](#)
- [Simulation Setup Tab](#)
- [Analysis Setup Tab](#)

Design Setup Tab

The following figure shows the **Design Setup** tab for Fast IR.

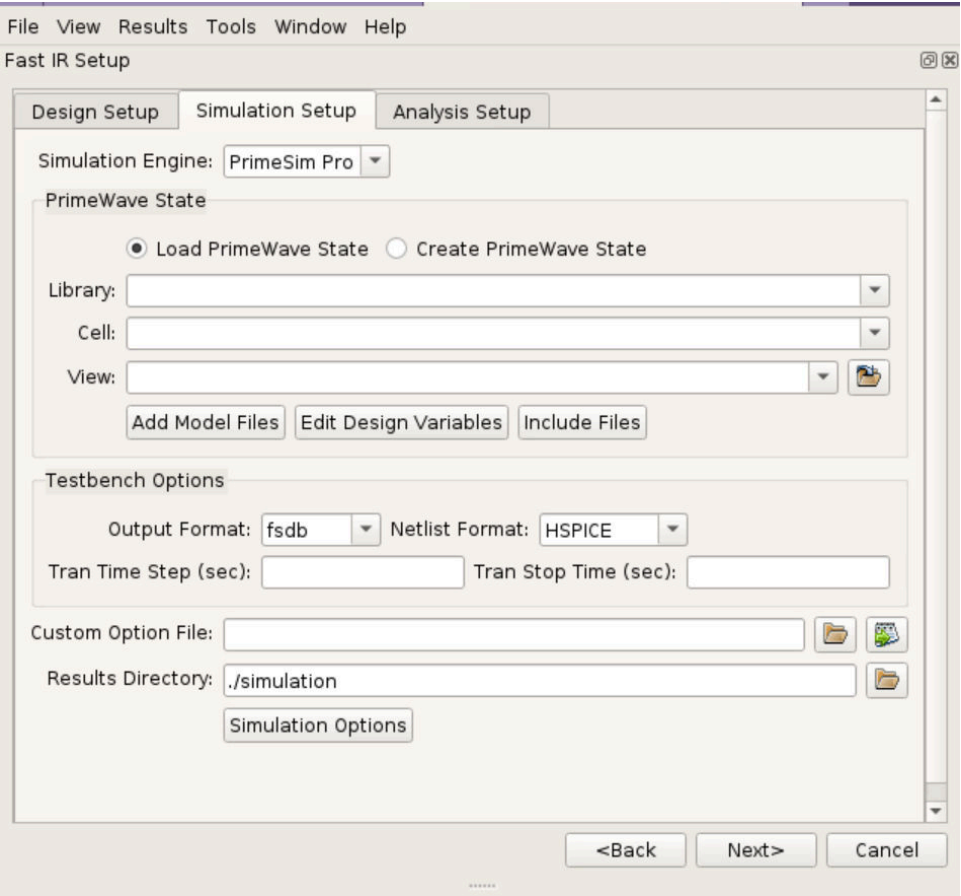


Option	Description
Design: Schematic	• Schematic: The default flow for Library EMIR • Netlist: ◦ Disables Schematic flow and enables Netlist and Command Line Options fields in Layout Design setup. ◦ Disables Simulation Engine selection– Supports PrimeSim XA only ◦ Disables PrimeWave State The default flow for Library EMIR.
Design: Netlist	Library EMIR– Netlist based flow. • Disables Schematic flow and enables Netlist and Command Line Options fields in Layout Design setup. • Disables Simulation Engine selection– Supports PrimeSim XA only • Disables PrimeWave State
Top Cell Schematic	Library, Cell, View: The library, cell, and EMIR view information for the EMIR view.
Layout Design	Library, Cell, View: The library, cell, and EMIR view information for the EMIR view.
DSPF	Specify the DSPF file and enable Extended Back-Annotation option.

Option	Description
Create Extracted Layout View	Select to create an extracted layout. Creates OA layout view shape information from DSPF file. The extracted layout is saved in the flow manager results directory in LCV format –It can be added to library manager view.
Back Annotation Option	Opens the Parasitic Back Annotation dialog box. For more information, see Parasitic Back Annotation Dialog Box .

Simulation Setup Tab

The following figure shows the **Simulation Setup** tab for Fast IR.

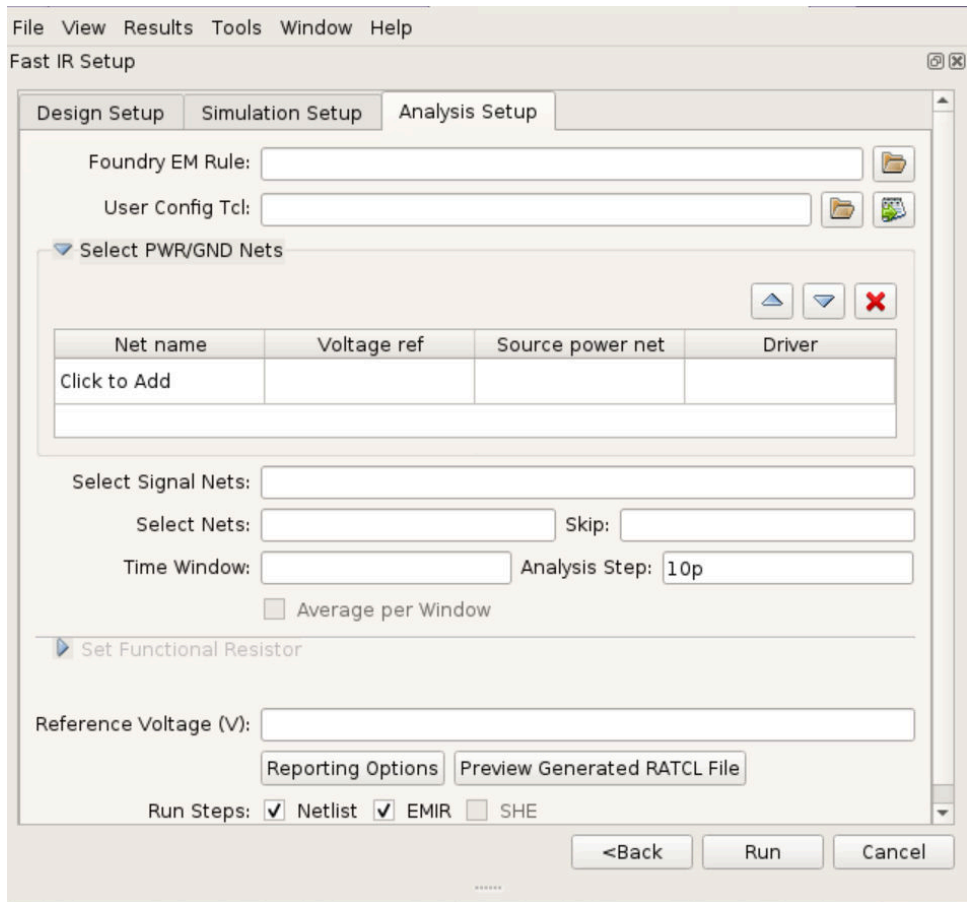


Option	Description
Simulation Engine	Select the simulator. Default: PrimeSim SPICE

Option	Description
PrimeWave State	The library, cell, and view information for the PrimeWave state view. • Load PrimeWave State: Select to load an existing PrimeWave State. • Create PrimeWave State: Select to create a new PrimeWave state that you want to save.
Add Model Files	The model files to use in the simulation. Model files contain calculations that predict design behaviors under different conditions. Click to open the Model Files dialog box.
Edit Design Variables	Parameters, such as MOSFET widths, that you can alter to assess the impact on the design. Click to open the Edit Design Variables dialog box.
Include Files	Additional files you want to process during the simulation. Click to open the Include Files dialog box.
Testbench Options	• Output Format: One of fsdb, wdf (default), or psf. This option is disabled for Netlist-based flow. • Netlist Format: Either HSPICE (default) or Third-party. • Tran Time Step: The fixed time step used for EM/IR simulation. The option is equivalent to <code>ratau</code> in the simulator tools. Use this option to define a non-default time step value. This option is disabled for Netlist-based flow. • Tran Stop Time: Stop time. This option is disabled for Netlist-based flow.
Custom Option File	• : Specifies the text file containing additional commands and options you want to run during simulation and analysis. • : (Applicable only if you specify a Custom Option File) Opens your system-default text editor to let you view the content of the custom option file.
Results Directory	The directory where you want to save the simulation results. See also Saving Simulation Results in PrimeWave Design Environment user documentation.
Simulation Options	Click to open the Simulation Options dialog box.

Analysis Setup Tab

The following figure shows the **Analysis Setup** tab for Fast IR.



Option	Description
Foundry EM Rule	Add the foundry electromigration rule file.
User Config Tcl	Additional the Tcl configuration file.
Time Window	Reliability analysis simulation time window.
Analysis Step	A fixed time step used for calculate IR drop and electromigration violations. Default: 10ps.
Reference Voltage	Use to calculate the IR drop percentage.
Reporting Options	Opens the Reporting Options dialog box. For more information, see Reporting Option Dialog Box
Preview Generated RATCL File	An RATCL file provides the Tcl interface to specify analysis types, set waveform viewer, control the content of the output files, and so on. An RATCL file is automatically generated when an EMIR analysis run is complete. To examine the content of the RATCL file, click Preview Generated RATCL File .

Option	Description
Run Steps	Selects the steps to run. • Netlist: Netlisting the schematic design. If the design source is a netlist, this option has no effect. • EMIR: Simulation and EMIR step through Flow Manager. Automatically loads the Netlist File and Commands File from the simulation netlist directory. • SHE: Run self heating analysis steps.

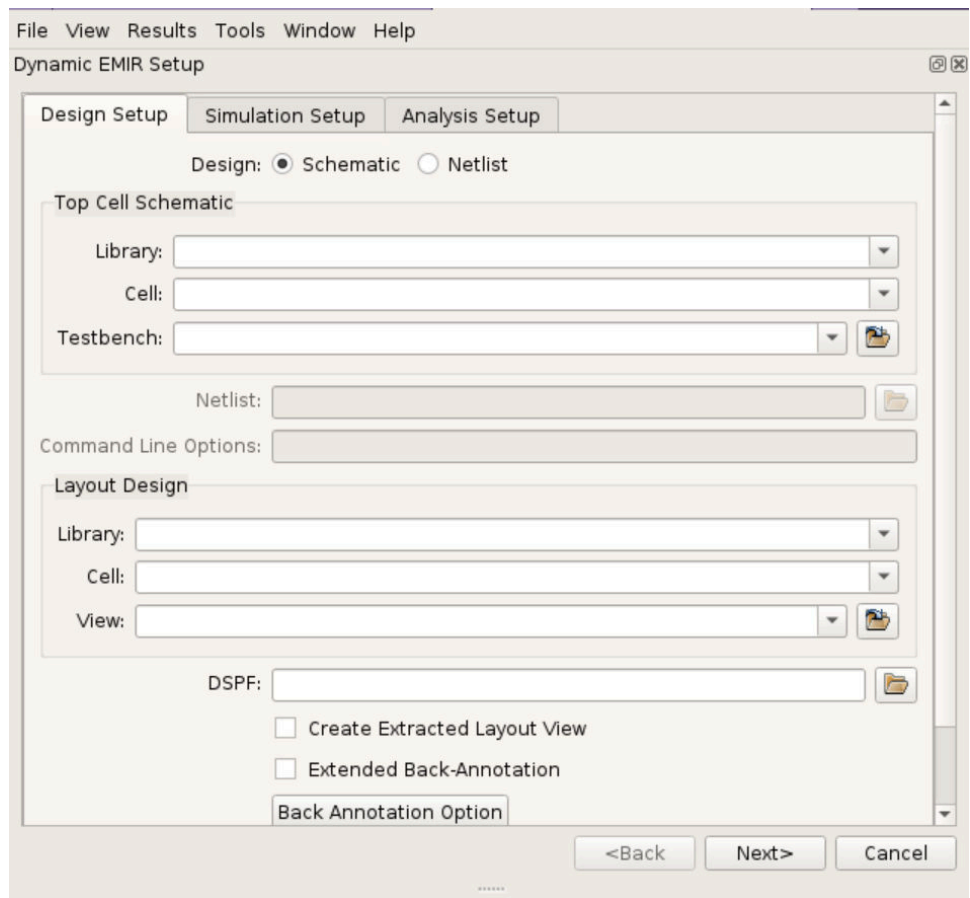
Dynamic EMIR Setup

The **Dynamic EMIR Setup** consists of setup options, which are presented in the following tabs:

- [Design Setup Tab](#)
- [Simulation Setup Tab](#)
- [Analysis Setup Tab](#)
- [Reporting Option Dialog Box](#)

Design Setup Tab

The following figure shows the **Design Setup** tab for Dynamic EMIR.

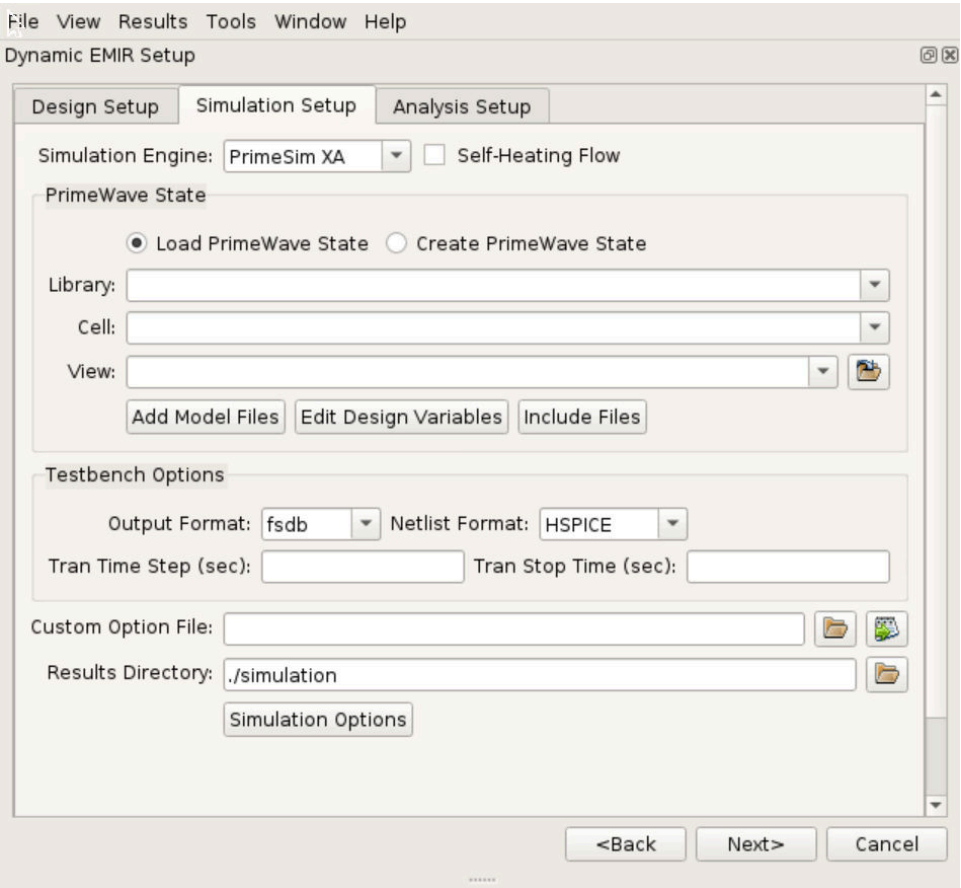


Option	Description
Design: Schematic	• Schematic: The default flow for Dynamic EMIR • Netlist: ◦ Disables Schematic flow and enables Netlist and Command Line Options fields in Layout Design setup. ◦ Disables Simulation Engine selection– Supports PrimeSim XA only ◦ Disables PrimeWave State The default flow for Library EMIR.
Design: Netlist	Library EMIR– Netlist based flow. • Disables Schematic flow and enables Netlist and Command Line Options fields in Layout Design setup. • Disables Simulation Engine selection– Supports PrimeSim XA only • Disables PrimeWave State
Top Cell Schematic	Library, Cell, View: The library, cell, and EMIR view information for the EMIR view.
Layout Design	Library, Cell, View: The library, cell, and EMIR view information for the EMIR view.

Option	Description
DSPF	Specify the DSPF file and enable Extended Back-Annotation option.
Create Extracted Layout View	Select to create an extracted layout. Creates OA layout view shape information from DSPF file. The extracted layout is saved in the flow manager results directory in LCV format –It can be added to library manager view.
Extended Back-Annotation	Select to run the extended back-annotation flow. For more details, see Running Extended Back-Annotation Flows
Back Annotation Option	Opens the Parasitic Back Annotation dialog box. For more information, see Parasitic Back Annotation Dialog Box .

Simulation Setup Tab

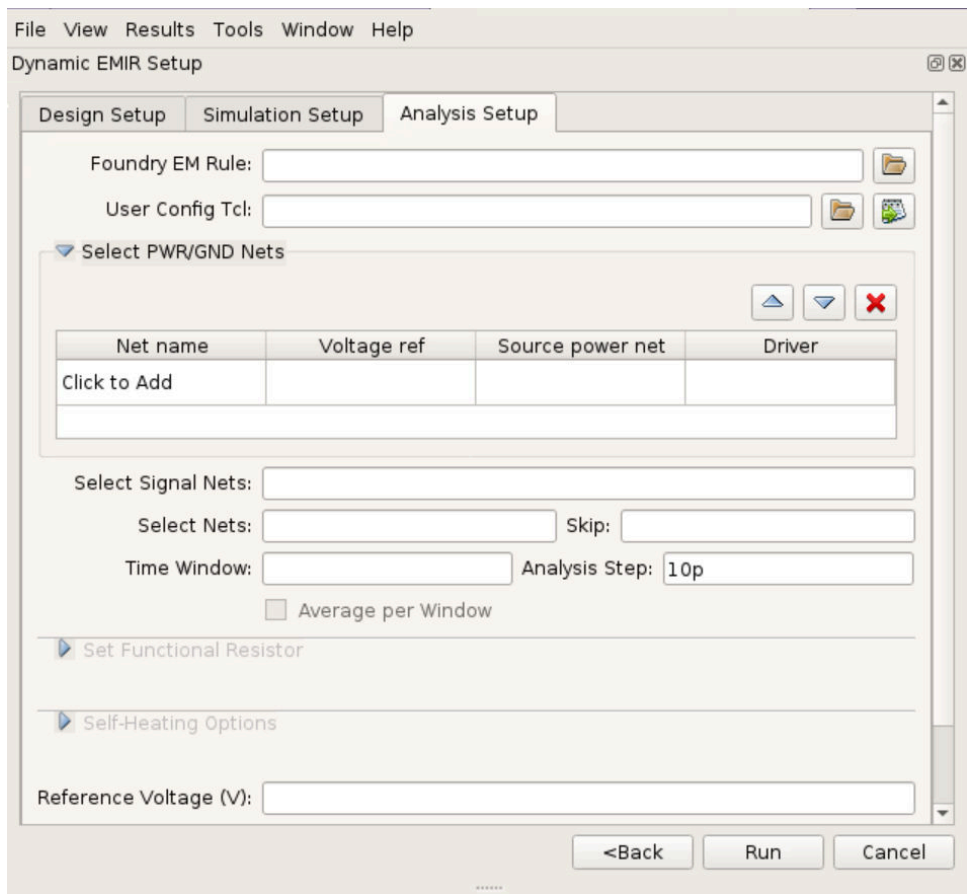
The following figure shows the **Simulation Setup** tab for Dynamic EMIR.



Option	Description
Simulation Engine	Select the simulator. Default: PrimeSim SPICE
Self-Heating Flow	Select to enable self-heating flow. Note: Enabled only through specific Process Node preference. Reach out Synopsys support to get relevant details.
PrimeWave State	The library, cell, and view information for the PrimeWave state view. • Load PrimeWave State: Select to load an existing PrimeWave State. • Create PrimeWave State: Select to create a new PrimeWave state that you want to save.
Add Model Files	The model files to use in the simulation. Model files contain calculations that predict design behaviors under different conditions. Click to open the Model Files dialog box.
Edit Design Variables	Parameters, such as MOSFET widths, that you can alter to assess the impact on the design. Click to open the Edit Design Variables dialog box.
Include Files	Additional files you want to process during the simulation. Click to open the Include Files dialog box.
Testbench Options	• Output Format: One of fsdb, wdf (default), or psf. This option is disabled for Netlist-based flow. • Netlist Format: Either HSPICE (default) or Third-party. • Tran Time Step: The fixed time step used for EM/IR simulation. The option is equivalent to <code>rataui</code> in the simulator tools. Use this option to define a non-default time step value. This option is disabled for Netlist-based flow. • Tran Stop Time: Stop time. This option is disabled for Netlist-based flow.
Custom Option File	• : Specifies the text file containing additional commands and options you want to run during simulation and analysis. • : (Applicable only if you specify a Custom Option File) Opens your system-default text editor to let you view the content of the custom option file.
Results Directory	The directory where you want to save the simulation results. See also Saving Simulation Results in PrimeWave Design Environment user documentation.
Simulation Options	Click to open the Simulation Options dialog box.

Analysis Setup Tab

The following figure shows the **Analysis Setup** tab for Dynamic EMIR.



Option	Description
Foundry EM Rule	Specify the foundry-supplied EM rule files to use for the simulation.
User Config Tcl	Additional the Tcl configuration file.
Select PWR/GND Nets	Click to add the names of the power nets and ground nets for simulation.
Select Signal Nets	The names of the signal nets for simulation.
Select Nets	Specify nets for EMIR analysis. Wildcards are supported.
Skip	Specify the nets to exclude from the EMIR analysis.
Time Window	The time window during which current data is collected.
Analysis Step	Fixed time step used to calculate IR drop and electromigration violations. Default: 10ps.

Option	Description
Average per Window	When specified, power net EMIR calculates the IAVG, IRMS and IABS currents separately for each defined simulation window, and then reports the peak value. When not specified, power net EMIR calculates the overall IAVG, IRMS and IABS currents for all of the defined simulation windows.
Reference Voltage	Use to calculate the IR drop percentage.
Reporting Options	Opens the Reporting Options dialog box. For more details, see Reporting Option Dialog Box
Preview Generated RATCL File	An RATCL file provides the Tcl interface to specify analysis types, set waveform viewer, control the content of the output files, and so on. An RATCL file is automatically generated when an EMIR analysis run is complete. To examine the content of the RATCL file, click Preview Generated RATCL File.
Run Steps	Selects the steps to run. • Netlist: Netlisting the schematic design. If the design source is a netlist, this option has no effect. • EMIR: Simulation and EMIR step through Flow Manager. Automatically loads the Netlist File and Commands File from the simulation netlist directory. • SHE: Run self heating analysis steps.

Reporting Option Dialog Box

The **ccdb Options** section consists of options that are only applicable if you selected **ccdb** as the output format in the EMIR tab.

Reporting Options <@ams-32cx240g-3744-001>

ccdb Options

ccdb Column Filter

ccdbcols em cccdbcols ird cccdbcols sh

☒ Enable all

Enabled	
current	☒
curdns	☒

☐ cccdbmerge

EMIR Results Filter

Analyses	Filter By	Value
☐ vmax	level	
☐ irms	level	
☐ iavg	level	

IR Drop Threshold:

Upper Boundary: Lower Boundary:

Varying Map Scale:

Time Duration:

Help Defaults OK Apply Cancel

The **ccdb Options** section lets you select the data columns that will appear in the EMIR analysis results database file (.cc.db). Select the columns in the following subtabs:

Subtab	Description
ccdbcols em	EM table columns
ccdbcols ird	IR drop table columns
ccdbcols sh	Shapes table columns

Select the **ccdbmerge** option to overwrite only reported analyses in the existing .cc.db file. When the **ccdbmerge** option is deselected, a new .cc.db file is created when analysis run is complete.

Select the **remove** option to remove the existing analysis data in the .cc.db file and does not overwrite them. If the .cc.db file does not exist, the tool creates a cc.db file. When the option is deselected, the tool keeps the existing analysis data in the .cc.db file and does not overwrite them.

For more information, see the PrimeSim EMIR Reference Manual.

EMIR Results Filter

In the **EMIR Results Filter** section, specify options to filter EMIR results by analysis types and filter attributes.

The analysis types shown in the list correspond to the analysis types you select in the **Analysis Setup** tab of the Dynamic EMIR Setup GUI. If **all** is selected in Analysis Types section in the EMIR tab, all the default analysis types will be shown in the list.

When an analysis type is selected, specify to filter results by: value, level or IImax/JJmax.

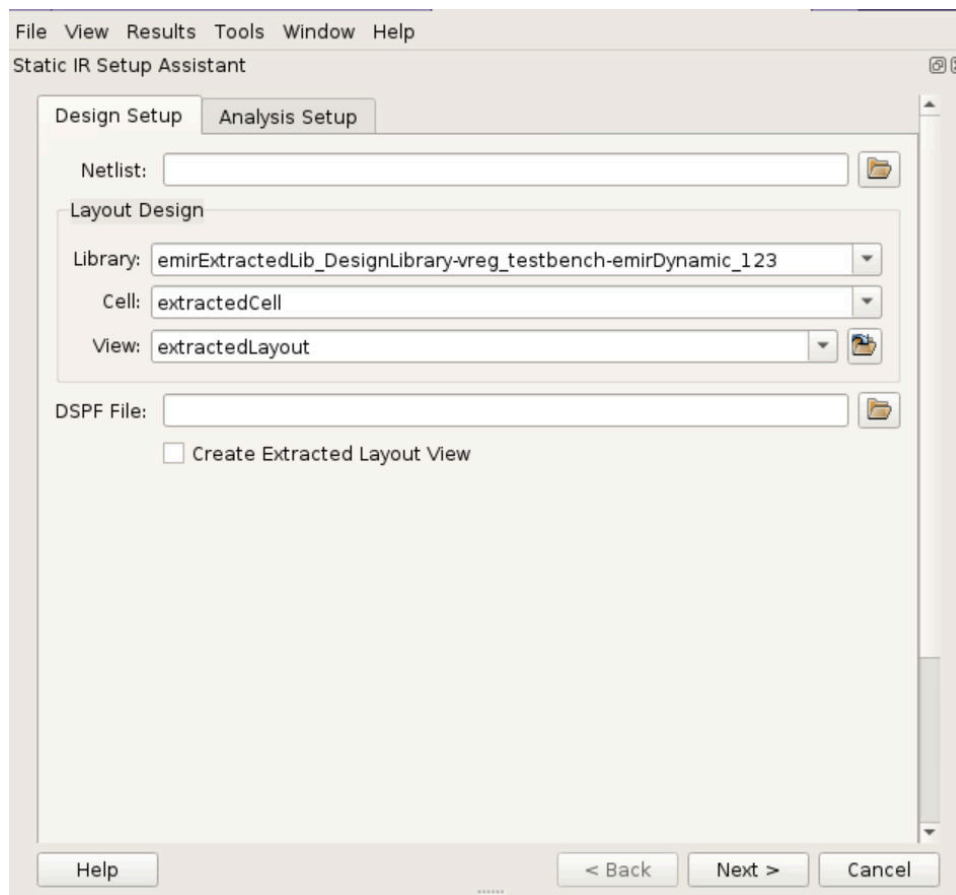
Static IR Setup

The **Static IR Setup** consists of setup options, which are presented in the following tabs:

- [Design Setup Tab](#)
- [Analysis Setup Tab](#)

Design Setup Tab

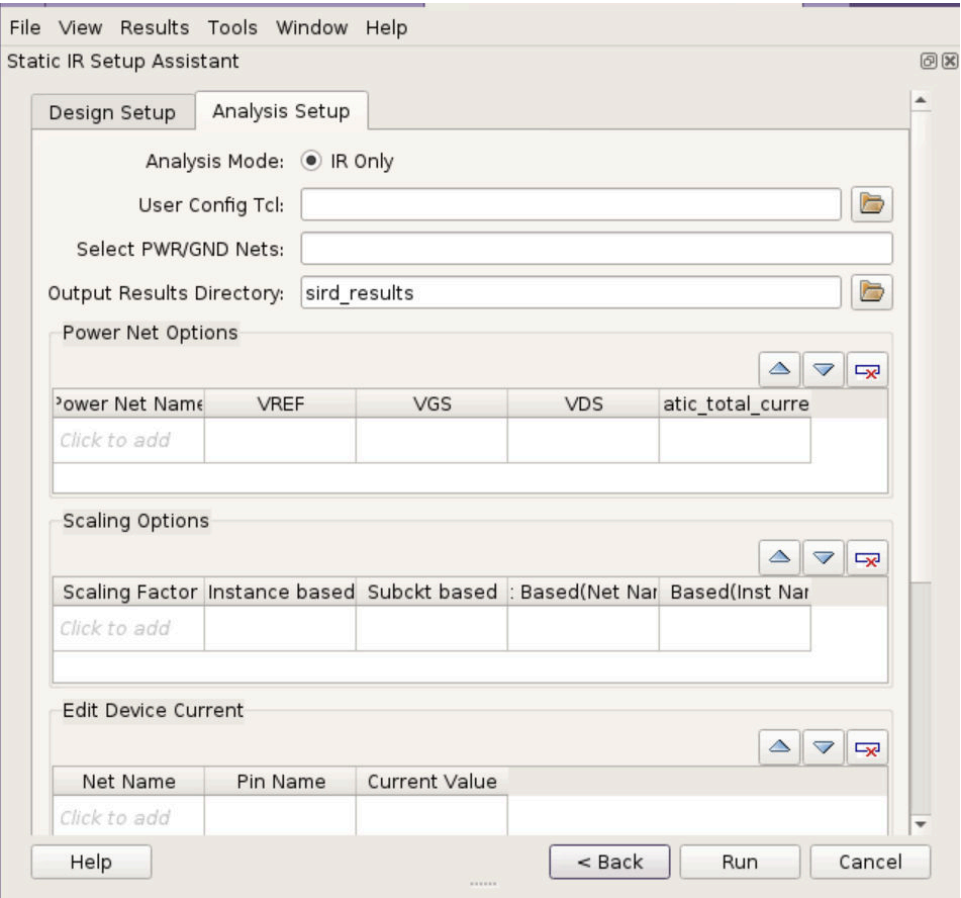
The following figure shows the **Design Setup** tab for Static IR.



Option	Description
Netlist	The pre-layout SPICE netlist that was used to run the simulation
Layout Design	The library, cell and view information for the layout design
DSPF File	Specify the DSPF file.
Create Extracted Layout View	Select to create extracted layout. Creates OA layout view shape information from DSPF file. The extracted layout is saved in the flow manager results directory in LCV format –It can be added to library manager view.

Analysis Setup Tab

The following figure shows the **Analysis Setup** tab for Static IR.



Option	Description
Analysis Mode	IR Only is the analysis mode.
User Config Tcl	Add your Tcl configuration file.
Select PWR/GND Nets	The names of the power nets and ground nets to run simulation on.
Output Results Directory	The directory where the results output files are stored.
Power Net Options	Add to override the VREF value of the power net by a user-defined value.
Scaling Options	Provide an instance or subckt based scaling factor.
Edit Device Current	
IC File	Provide .ic file to override devices voltages that are being used to calculate SIRD.
Reporting Options	Opens the Reporting Options dialog box. For more information, see Reporting Option Dialog Box
Preview Generated RATCL File	An RATCL file provides the Tcl interface to specify analysis types, set waveform viewer, control the content of the output files, and so on. An RATCL file is automatically generated when an EMIR analysis run is complete. To examine the content of the RATCL file, click Preview Generated RATCL File .

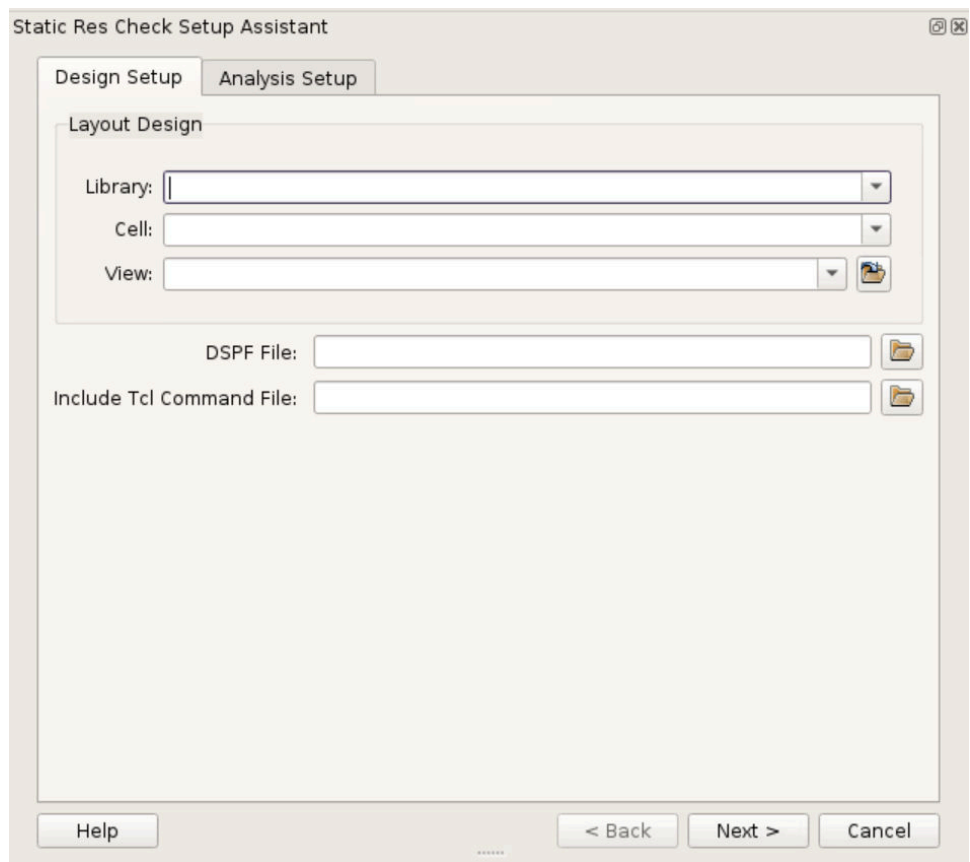
Static Res Check Setup

The **SPRES Setup** consists of setup options, which are presented in the following tabs:

- [Design Setup Tab](#)
- [Analysis Setup Tab](#)

Design Setup Tab

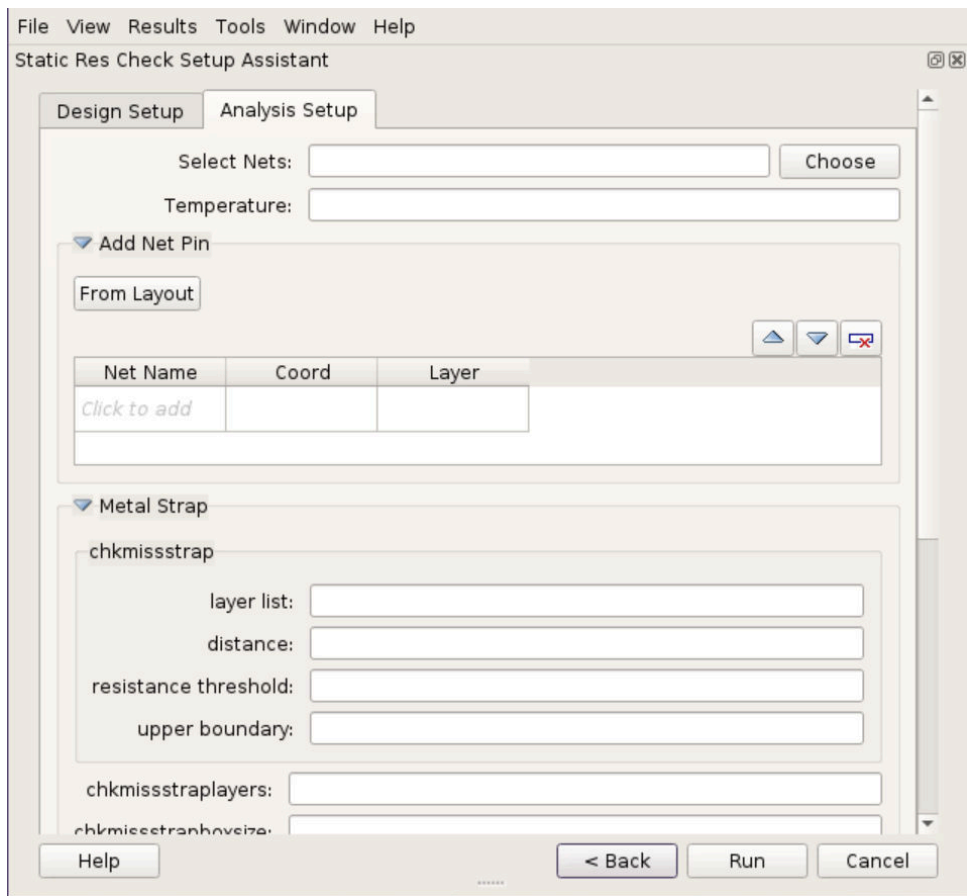
The following figure shows the **Design Setup** tab for Static Res Check.



Option	Description
Layout Design	The library, cell and view information for the layout design
DSPF File	Specify DSPF file.
Include Tcl Command File	Name and location of the Static Res Check Tcl file that you want to use for the calculation. The Tcl file contains Static Res Check commands.

Analysis Setup Tab

The following figure shows the **Analysis Setup** tab for Static Res Check.



Option	Description
Select Nets	Nets that are used in the resistance calculations.
Temperature	Reference temperature used to recalculate the resistance values when temperature factors (such as TC1 and TC2) are specified in the SPF file. This option is equivalent to the PrimeSim EMIR <code>temp</code> command.
Add Net Pin	Click From Layout , and click a pin on the layout canvas to automatically add the pin information in the table. Alternatively, you can enter the pin information manually.
Metal Strap	

Option	Description
chkmissstrap	Flags a missing metal strap when the resistance difference between two nodes within metal layers is larger than a specified threshold and the resistance network can be improved by adding a metal strap connecting the nodes. • layer list: List of metal layer names. • distance: Area vicinity radius. Unit for distance is meter. • resistance threshold: Threshold for acceptable resistance values. • upper boundary: Upper boundary threshold for the resistance difference.
chkmissstraplayers	Specifies layer numbers used for missing straps shapes.
chkmissstrapboxsize	Generates rectangular shapes (boxes) connecting two nodes with reported violations.
Via	
chkmissvia	Flags missing vias between two metal layers. • layer1: Lower metal layer. • layer2: Upper metal layer. • upper boundary: Upper boundary threshold for resistance difference between two metal layers at the intersection.
chkmissvialayers	Checks the specified via layers.
chkmissviaboxsize	Generates square shapes (boxes) to represent missing vias.
Output Results Directory	The directory where the results output files are stored.
Reporting Options	
layer0ohm	Resistors with indicated layer names and/or numbers will be shorted if resistor layer information is available in the DSPF file. Layer names in <code>layer0ohm</code> should be cross referenced to layer numbers in <code>layer_map</code> section of the DSPF file.
layerfactor	Resistor values belonging to an indicated layer in <code>layerfactor</code> are scaled by a factor.
mergefinger	Merges instance pins for finger devices.
netdeletepad	Permits removal of some net pads from the resistance calculation. You can use the * wildcard character in the pad names.
outfiltres	Filters out (or does not report) non-physical resistor from outputs. Conditions to filter are based on resistor width and resistance. The StarRC tool generates non-physical resistors that are used for connectivity purpose. They do not physically exist in the layout. This option addresses the non-physical resistors.

Option	Description
rmin	Minimum resistor values to read from the DSPF file. Resistors with smaller values will be shorted so that there is a reduction of resistor elements with a corresponding accuracy trade-off.

Running Extended Back-Annotation Flows

To run the extended back-annotation flow, select the **Extended Back-Annotation** option on the Design Setup tab in the EMIR Setup GUI for Library EMIR or Dynamic EMIR.

The extended back-annotation flow can be used instead of the traditional back-annotation to guarantee a clean back-annotation and incorporate existing RC back-annotation functions. The extended back-annotation flow requires the SPF file, which must contain subcircuit definitions with the .SUBCKT statement and the instance section.

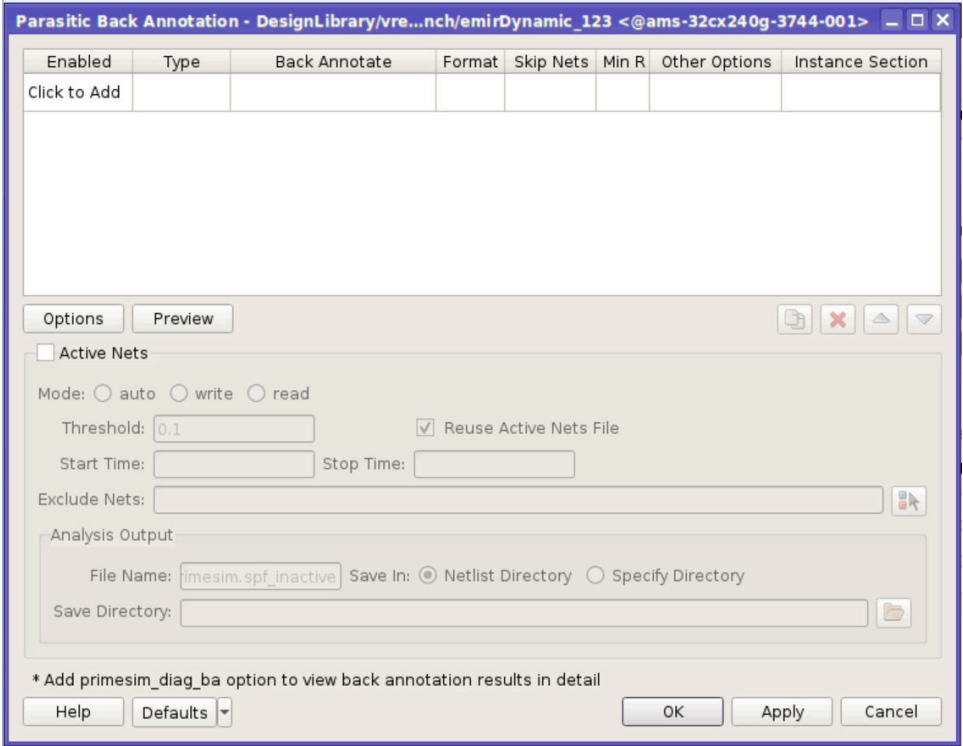
For the full description and instructions, see the PrimeSim XA User Guide and the [How To Specify an Analysis Statement in the Extended Back Annotation \(XBA\) Flow](#) application note on SolvNetPlus.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Parasitic Back Annotation Dialog Box

In the **Design Setup** tab of Library EMIR setup, Fast IR Setup, and Dynamic EMIR Setup, click the **Back Annotation Option** button to open the **Parasitic Back Annotation** dialog box.



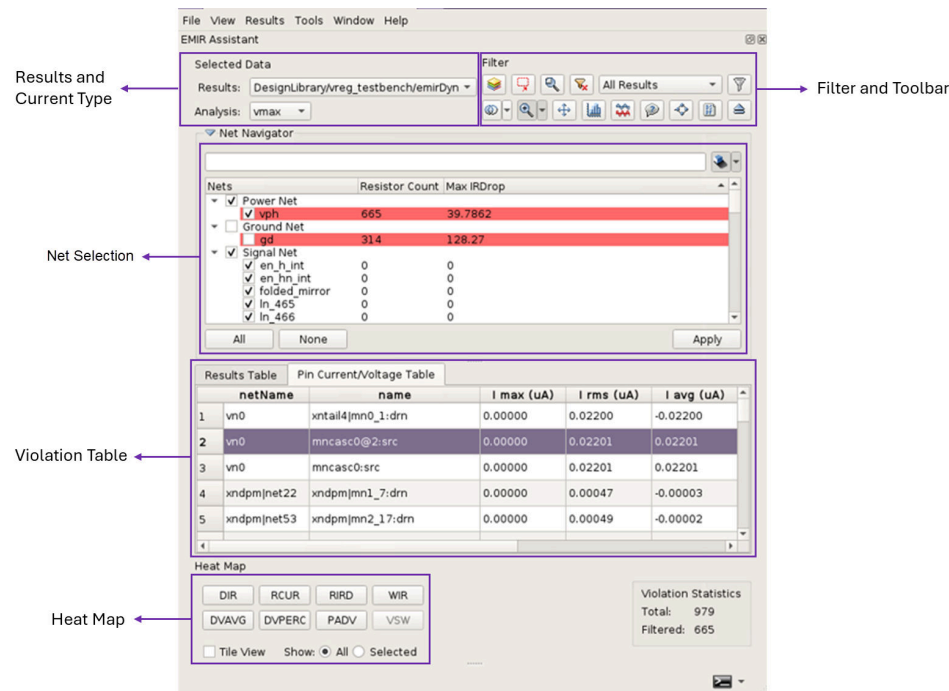
Viewing Analysis Results

When the analysis is complete, you can select which EMIR results to view by specifying options in the PrimeSim EMIR GUI. If **Run** is clicked after analysis setup, the results are automatically loaded in the Results Viewer.

To view the analysis results, click **View** and select **Results View**.

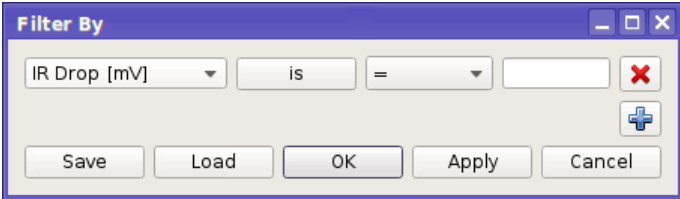
Results Viewer

The following figure shows the different areas for viewing results in PrimeSim EMIR GUI during the setup and viewing process.



Option	Description
Selected Data	
Result	Select and load EMIR Results.
Analysis	Select an analysis: • iavg: Average current • vmax: Maximum voltage
Filter	
	Filter by layer. When clicked, opens the Layer Selection dialog box. Select the layer for which you want to apply filter.
	Filter by region. To specify the region of interest on the canvas: • Type the coordinates that define the four vertices of the rectangular region.
	• Click  and draw a rectangle on the canvas.
	Filter by Viewport. Click to set the viewport as the area of interest

Option	Description
	Opacity. Click the little down arrow on this button to show the opacity slider. Use the Opacity slider to change the opacity of the colored shapes. To increase the opacity, move the slider towards 100. To decrease the opacity, move the slider towards 0.
	Zoom. Click the little down arrow on this button to show the zoom level slider. You can specify the magnification using the Zoom Level slider.
	Pan. Moves the viewport so that the violating shapes you selected are centered.
	Histogram. Click to display the Show Histogram dialog box opens. For more details see Displaying the Violation Data in a Histogram
	Plot waveform. You can plot the waveforms to display in the WaveView window. Plots resistor currents and voltages from Results Viewer and layout canvas
	What-if. Clicking this button opens the EMIR What-If Analysis dialog box opens. For more details, see Running EMIR What-If Analyses
	LRP. For more details, see Tracing Least Resistive Paths

Option	Description
	To create custom filter. To filter the data in the table, click Multi-parameter filtering . The Filter By dialog box appears. 1. In the Filter By dialog box, click  to add a filtering rule based on the value of one of the table parameters. Select the parameter in the first drop-down list, complete the rule using the other fields, and click Apply to filter the table data.  2. Repeat Step a to add more filtering rules. 3. (Optional) To delete a rule, click  against it. 4. (Optional) Click Save to save the filter configuration. You can later load the last-saved configuration by clicking Load. 5. When you are finished, click OK to filter the table data applying all the existing rules and close the dialog box. 6. (Optional) To reset the table to its original/unfiltered state, delete all the filtering rules in the Filter By dialog box and click Apply/OK.
	Toggle view. Toggles between condensed view and expanded view. In the condensed view, the Net Navigator section moves as tab in the results table. In the expanded view, the Net Navigator is a separate section.
Net Navigator	Lists the net data of Selected Data .
Results Table	Lists the violations of the nets selected in Net Navigator .
Pin Current/Voltage Table	Shows all the pin current and voltage of the selected nets. For more details, see Pin Current/Voltage Table .
Heat Map	Shows dedicated Heat Map buttons according to analysis selection. Heat Map buttons work like toggle switch to show or hide violations. • EM: Heat map button shows I/Imax violations. • WCUR: Wire Current • DCUR: Device Pin Current • WIR: IR Drop violations If the Tile View option is enabled, the heat map button shows the tile view.
Tile View	When enabled, displays the selected heatmap in tile view. For more details, see the Locating Small Shapes With High-Bin Violation Counts
Show	Click Show: All to show all the violations for the selected tile.

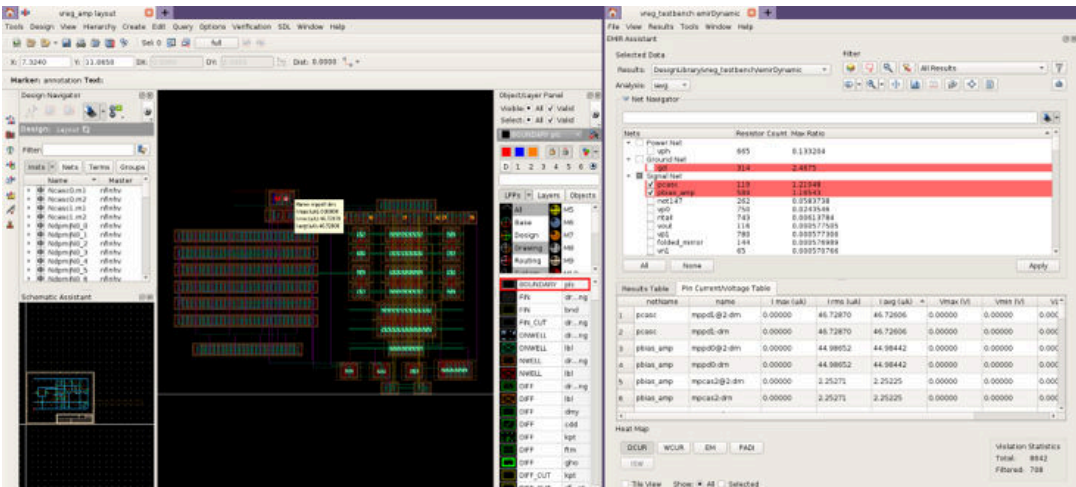
Pin Current/Voltage Table

The Pin Current/Voltage Table shows all the pin current/voltage of the selected nets.

The DCUR heat map shows the pin location according the bin color for the selected current value

Hovering mouse pointer on the heat map shows the Instance Pin hierarchy and pin current values.

Cross probe pin shapes from table to layout canvas.



Results Viewer Context-Sensitive Menu

Right-click on the canvas for the Results Viewer context-sensitive menu options.

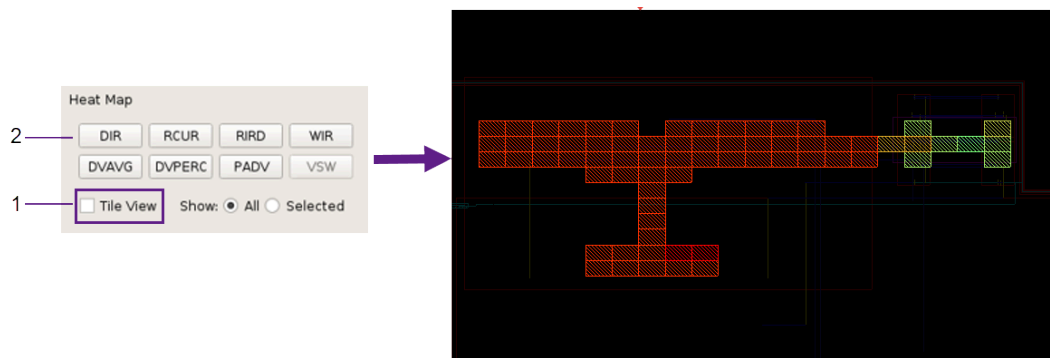
Option	Description
Turn Off Other Nets	Turns of violation for other nets.
Plot	Plots node voltage/current for the selected node.
Plot by Region	Plots node voltage/current for the shapes selected in a region.
EMIR Trace LRP	Traces Least Resistive Path from selected node to power pad.
What-if Modify Resistor	Adds selected resistors (source and destination) to What-if GUI for Add Resistor.
N2N Resistance	Adds selected resistors(Source and destination) to What-if GUI for N2N Resistance check (Supported in SPRES only).
Waive	Sets Waive flag for the selected resistor.

Locating Small Shapes With High-Bin Violation Counts

While viewing violations on the layout canvas, you have the option to show and use the Tile View to quickly locate shapes that are very small but have violation counts in the upper bins of the violation table in the Change Bin dialog box. The Tile View is a kind of a grid overlay on the layout canvas showing hotspots or grid cells/tiles with such small high-bin violation shapes.

To generate and use the **Tile View**:

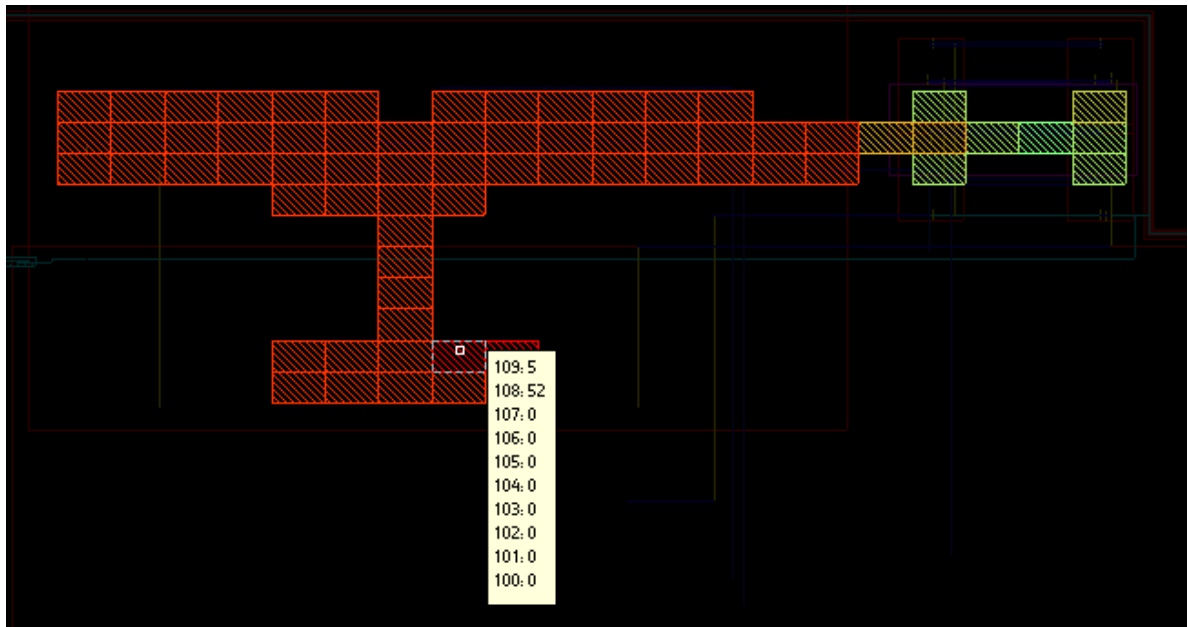
1. In the **PrimeSim EMIR Results Viewer**, enable the **Tile View** option.
2. Click a **Heat Map** button. This generates and displays the tile view on the layout canvas.



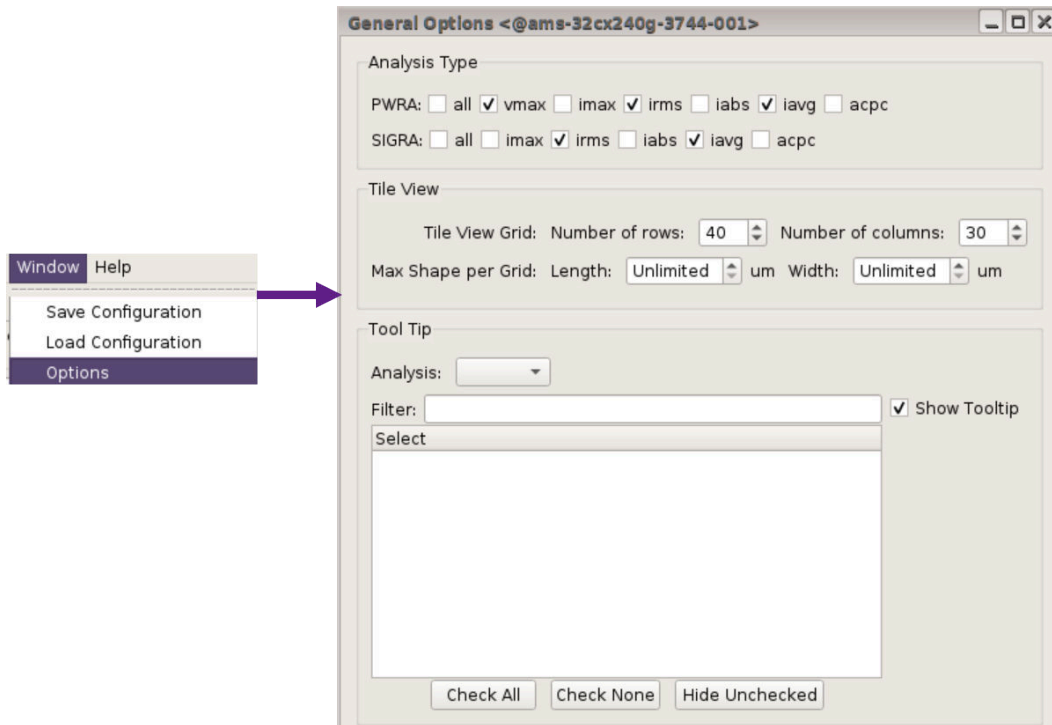
The Tile View shows the hotspot tiles in blinking red. The color of each tile depends on the count of the highest bin violation present within the tile boundary. The tile colors range from red which means violation counts in the uppermost/upper bins of

the violation table, to orange, to yellow, to green which means violation counts in the lowermost/lower bins of the violation table.

3. Move the mouse and hover over a hotspot or any other tile to display the high-bin violation count table in the tooltip for that tile.



4. Additionally, to control tile view options, choose **Window > Options** to open the **General Options** dialog box.



5. In the **Tile View** section, provide the following information

Setting	Description
Tile View Grid: Number of rows Number of columns	The number of rows and columns for the Tile View grid.
Max Shape per Grid: Length Width	The maximum resistor segment length and width to include in the Tile View grid.

Displaying the Violation Data in a Histogram

You can display the violation data in a histogram to see the distribution of the violation data. The graphical visualization can facilitate and focus your troubleshooting efforts.

To display the violation data in a histogram:

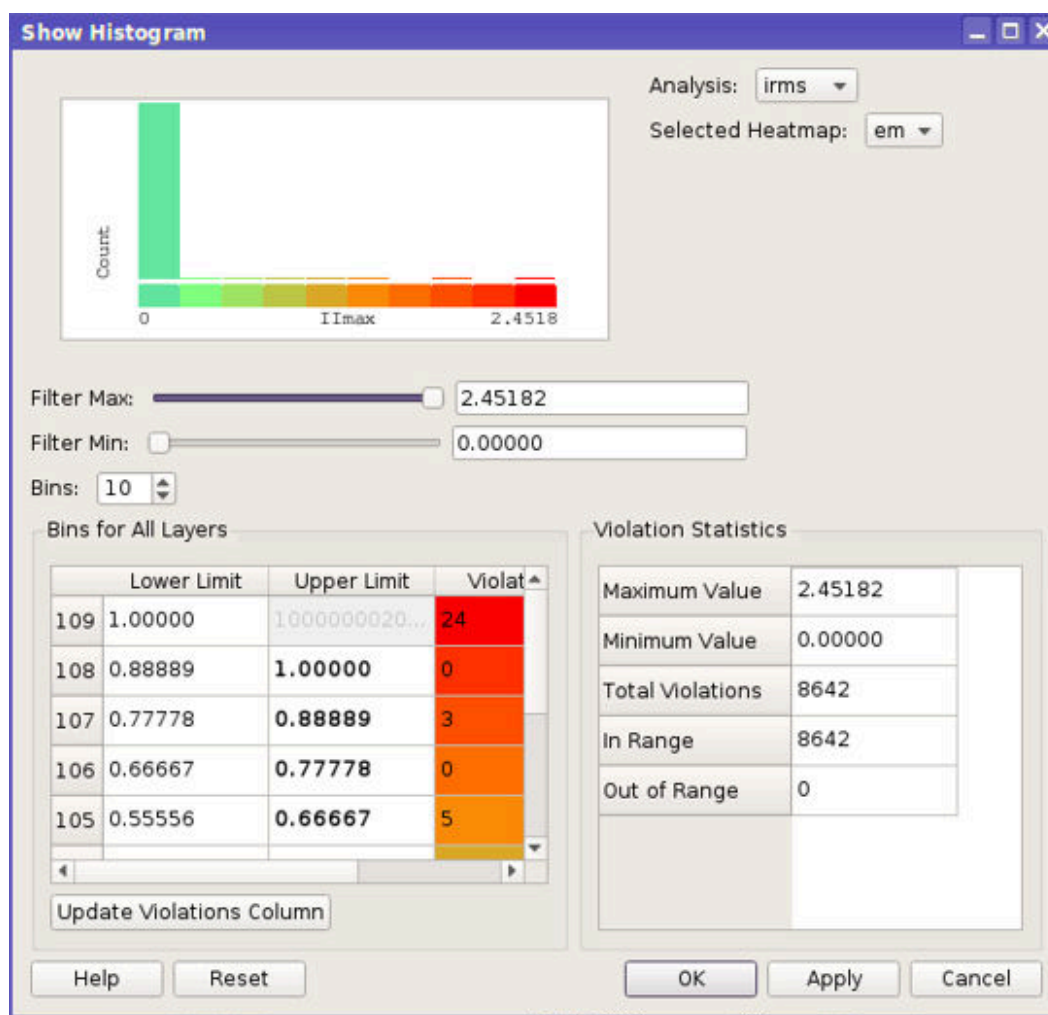


1. In the Results Viewer, click .

The **Show Histogram** dialog box opens.

Chapter 7: Running the EMIR Application

Displaying the Violation Data in a Histogram



2. Select an **Analysis**.
3. Select the corresponding heat map in **Selected Heatmap**.
4. Review the histogram and the information in the **Violation Statistics** table.
5. To increase or decrease the number of bins and redistribute the data, change the **Bin** setting.

To change the bin color assignments, modify the `PrimeSim_EMIR` display packets. See the Creating and Editing Display Packets and Packet Aliases topic in *Custom Compiler Environment User Guide*.

The maximum number of bins is set in the PrimeWave Design Environment.

6. To filter the violation data, move one or both of the following sliders:
 - **Filter Min:** Filters the **Minimum Value**. For example, the Minimum Value is 0.0. Moving the slider to 0.07 filters out violations that are below 0.07 and redistributes the remaining data across the 10 bins.
 - **Filter Max:** Filters the **Maximum Value**. For example, the Maximum Value is 1.0. Moving the slider to 0.9 filters out violations that are above 0.9 and redistributes the remaining data across the 10 bins.

As you move the sliders, the histogram updates. Violation data that are outside of the new range are shown as gray bars on the left and right sides of the histogram. The **Violation Statistics** table also updates to show the number of violations that are within and outside of the filtered range.

7. When you are finished, click **OK** or **Apply**. The dialog box refreshes and shows the re-binned violation data.

Redistributing Bin Values

The EMIR analysis data is distributed uniformly across 10 bins by default, which corresponds to the default setting in the simulation EMIR analysis tool. See the user documentation for the simulator EMIR analysis tool, such as the PrimeSim EMIR Reference Manual.

You have the option of redistributing the bin values to control the set of data in each bin. This enables a more focused evaluation.

To redistribute the bin values,

1. In the **Show Histogram** dialog box, double-click and edit the **Upper Limit** values to change the bin thresholds.
2. Click **Update Violations Column**. The number of violations are redistributed.
3. To change the bin color assignments:
 - a. Double-click anywhere in the table. The **Display Resource Editor** appears.
 - b. Modify the `PrimeSim_EMIR` display packets.
4. (Optional) Click **Reset** to revert bins and other column values to their default values.
5. When you are finished, click **OK** in the **Show Histogram** dialog box.

Saving and Loading Results Viewer Configurations

You can save the results viewer configuration and reuse them later, so you do not have to set the options every time you open the PrimeSim EMIR results viewer. The configuration data is saved to an XML file.

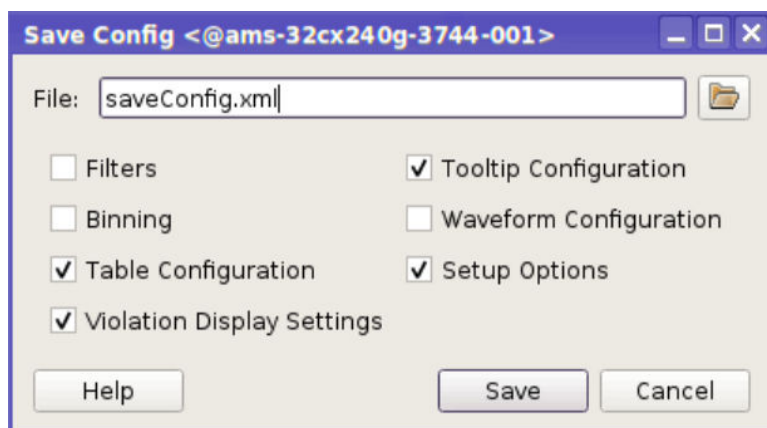
This topic describes the following:

- [Saving Results Viewer Configurations](#)
- [Loading Results Viewer Configurations](#)

Saving Results Viewer Configurations

To save the results viewer configuration:

1. In the **PrimeSim EMIR** GUI, choose **Window > Save Configuration**. The **Save Config** dialog box opens.
2. In the **Save Config** dialog log, specify the name and location to save the configuration file.



3. Select the configuration components you want to save. By default, **Table Configuration**, **Violation Display Settings**, and **Tooltip Configuration** are selected.
4. Click **Save**. The configuration (.xml) file is saved in the specified folder.

Loading Results Viewer Configurations

To load a saved results viewer configuration:

1. In the **PrimeSim EMIR** GUI, choose **Window > Load Configuration**. The **Load Config** dialog box opens.
2. In the **Load Config** dialog box, locate the configuration (.xml) file.
3. Click **Open**. The setup from the configuration (.xml) file displays in Results Viewer.

Running EMIR What-If Analyses

This topic describes the following:

- [About the EMIR What-If Analyses](#)
- [Setting Up What-If Analyses](#)
- [Saving What-If Analysis Setups](#)
- [Running and Saving What-If Analysis Results](#)
- [Loading and Viewing What-If Analysis Results](#)
- [Exporting the What-If Resistor Data](#)
- [EMIR What-If Analysis Dialog Box Reference](#)

About the EMIR What-If Analyses

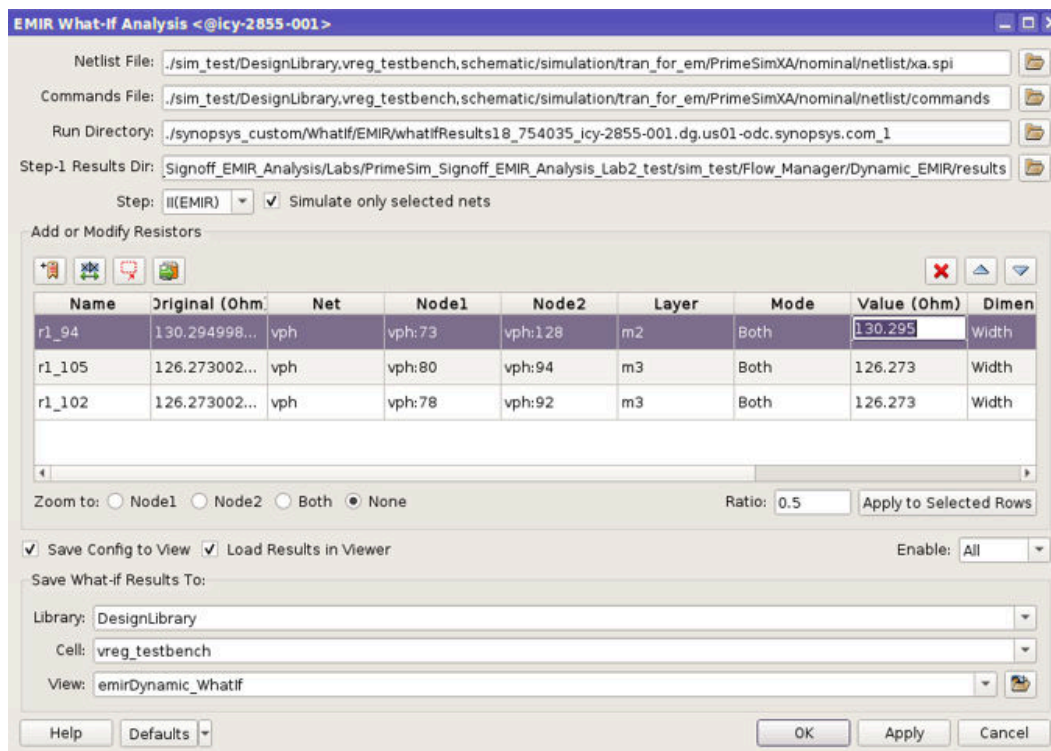
If the EMIR analysis results contain violations, you can use the EMIR What-If Analysis to quickly assess different modifications without changing the original design. During the what-if analysis, you can add new resistors and modify existing resistors (resistance values). You can view the outcome of the modifications in the results viewer. After you arrive at the optimal results, you can update the original design.

The EMIR What-If analysis is also available while you trace least resistive paths. See [Tracing Least Resistive Paths](#).

Setting Up What-If Analyses

To set up a what-if analysis:

1. Make sure the simulation results are loaded in the **PrimeSim EMIR GUI**.
2. In the **PrimeSim EMIR GUI**, choose **Tools > What-If**. The **EMIR What-If Analysis** dialog box opens.



3. (Step II, and Step I + II only) Locate and select the following:
- **Netlist File:** The pre-layout SPICE netlist that was used to run the simulation.
 - **Command File:** The simulation command script file that was used to control the simulation options.
 - **Run Directory:** The directory in which run files are stored. The default is `synopsys_custom/What-If/WhatIf_*`.

If you choose **Step II**, these two options are automatically populated from the simulation netlist directory.

4. (Step II only) In the **Step-I Results Dir** box, specify the directory of the phase-I analysis results.
5. (Step I and II) In the **Command Line Option** box, type the location and name of the PrimeSim EMIR command file.
6. Choose the analysis **Step** you want to run: **Step II (EMIR)**, or **Step I + II**.

Choosing **Step II (EMIR)**:

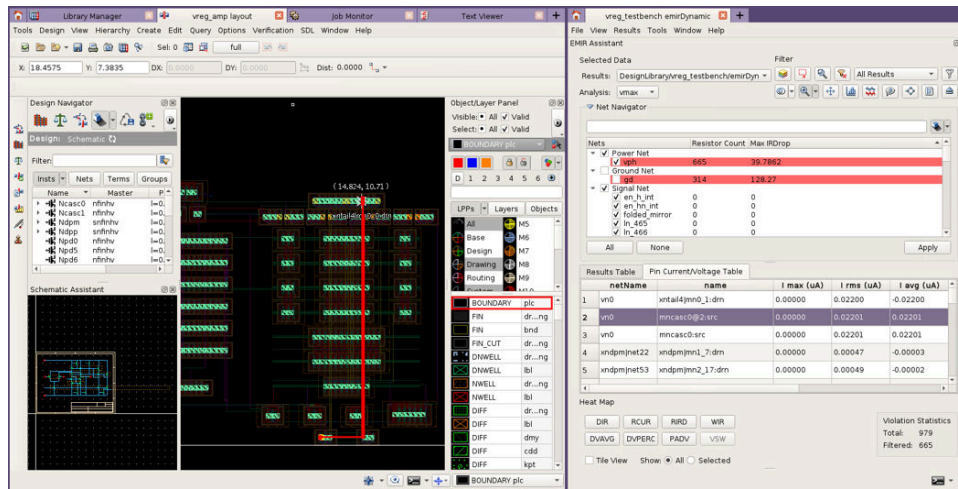
- Automatically loads the Netlist File and Commands File from the simulation netlist directory.
- Allows you to modify resistors only. During the modification, you can specify the width of the metal shapes and the area of the via shapes in the Dimension and Area/Width columns).

Choosing **Step I + II** allows you to:

- Select **Reuse Flow** (uses the last-modified DSPF file as input to the EMIR analysis).
- (Reuse Flow only) Specify the **Instance Pin Tolerance**.
- Add resistors.
- Modify resistors.

Note: Dimension and Area/Width cannot be changed when modifying resistors.

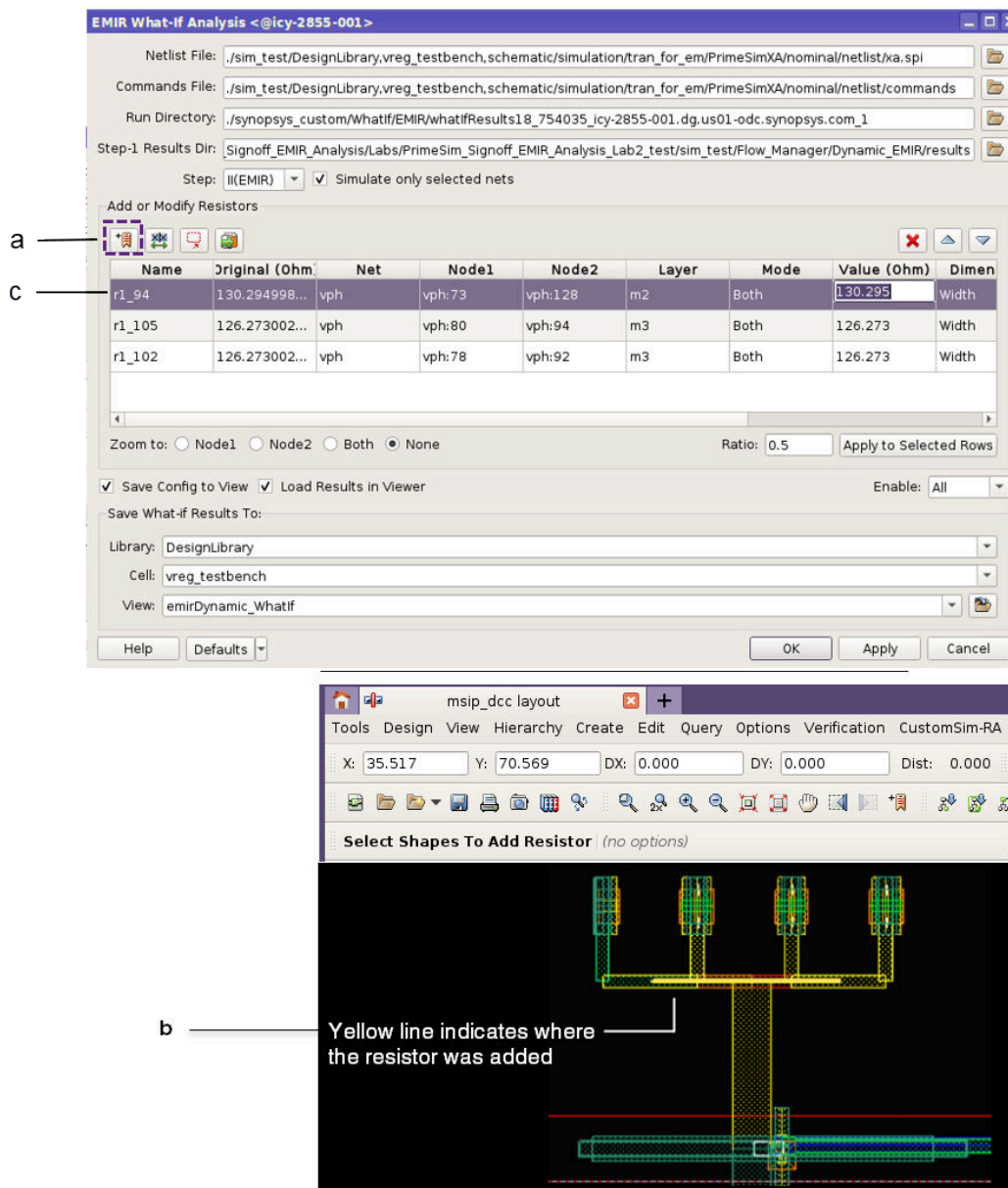
7. In the PrimeSim EMIR GUI, select a violation (a row in the data table). If you selected **On Show: Zoom**, the viewport automatically zooms in to the violation on the canvas. .



8. To add a resistor:

- In the **What-If Analysis** dialog box, click **Add Resistor** . The **Select Shapes to Add Resistor** message appears above the canvas.
- On the canvas, select two shapes (source and destination) between which a connecting path will be added.

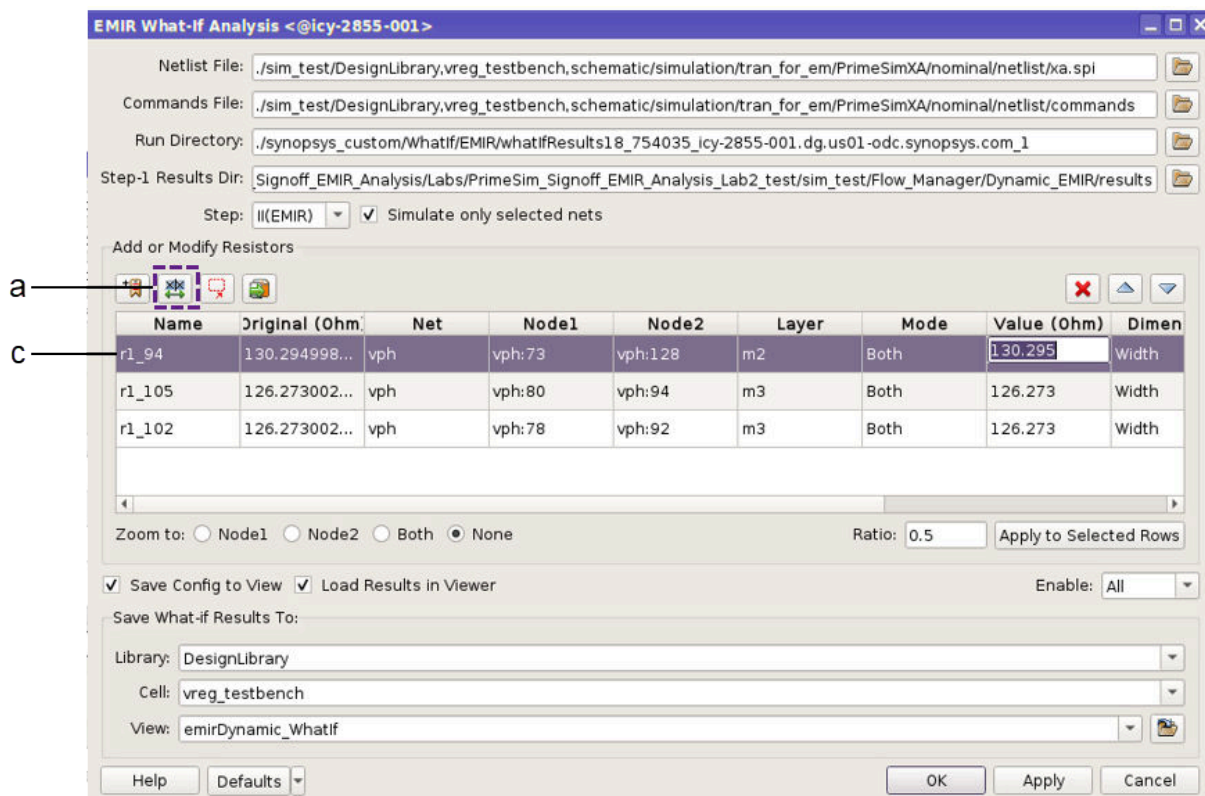
A yellow line appears on the canvas to mark the location and length of the connecting path. The path also appears as a new resistor in the **What-If Analysis** dialog box. Node1 and node 2 are the start and end points of the path, respectively.
- In the **What-If Analysis** dialog box, review the resistor information, select the resistor **Layer**, and type a new resistance **Value**.
- Repeat Step a through Step c to add more resistors.



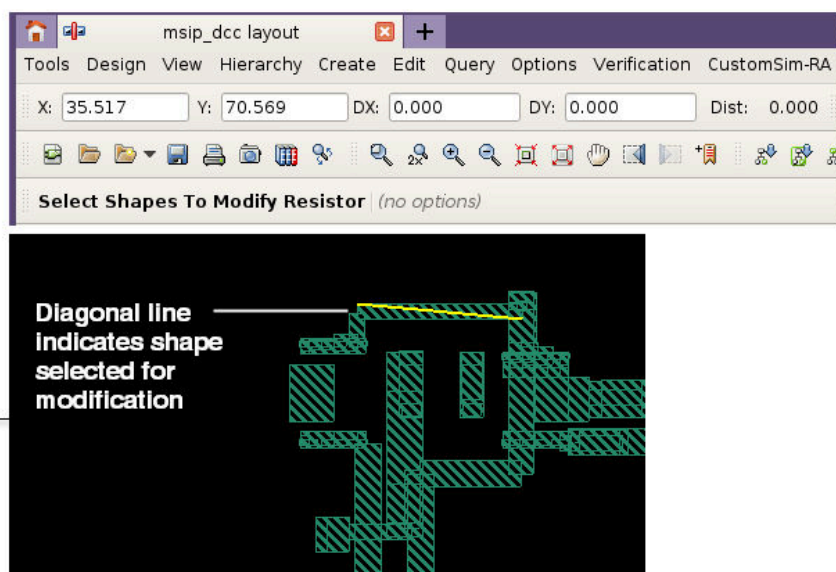
9. To modify a resistor:

- In the **What-If Analysis** dialog box, click **Modify Resistor** . The **Select Shapes to Modify Resistor** message appears above the canvas.
- On the canvas, select the shape whose resistance information you want to modify. Information about the selected resistor appears in the **What-If Analysis** dialog box. Node1 and node 2 are derived from the simulation results database.

- c. In the **What-If Analysis** dialog box, modify the resistor information. See [EMIR What-If Analysis Dialog Box Reference](#).
- d. Repeat Step a through Step c to modify more resistors.

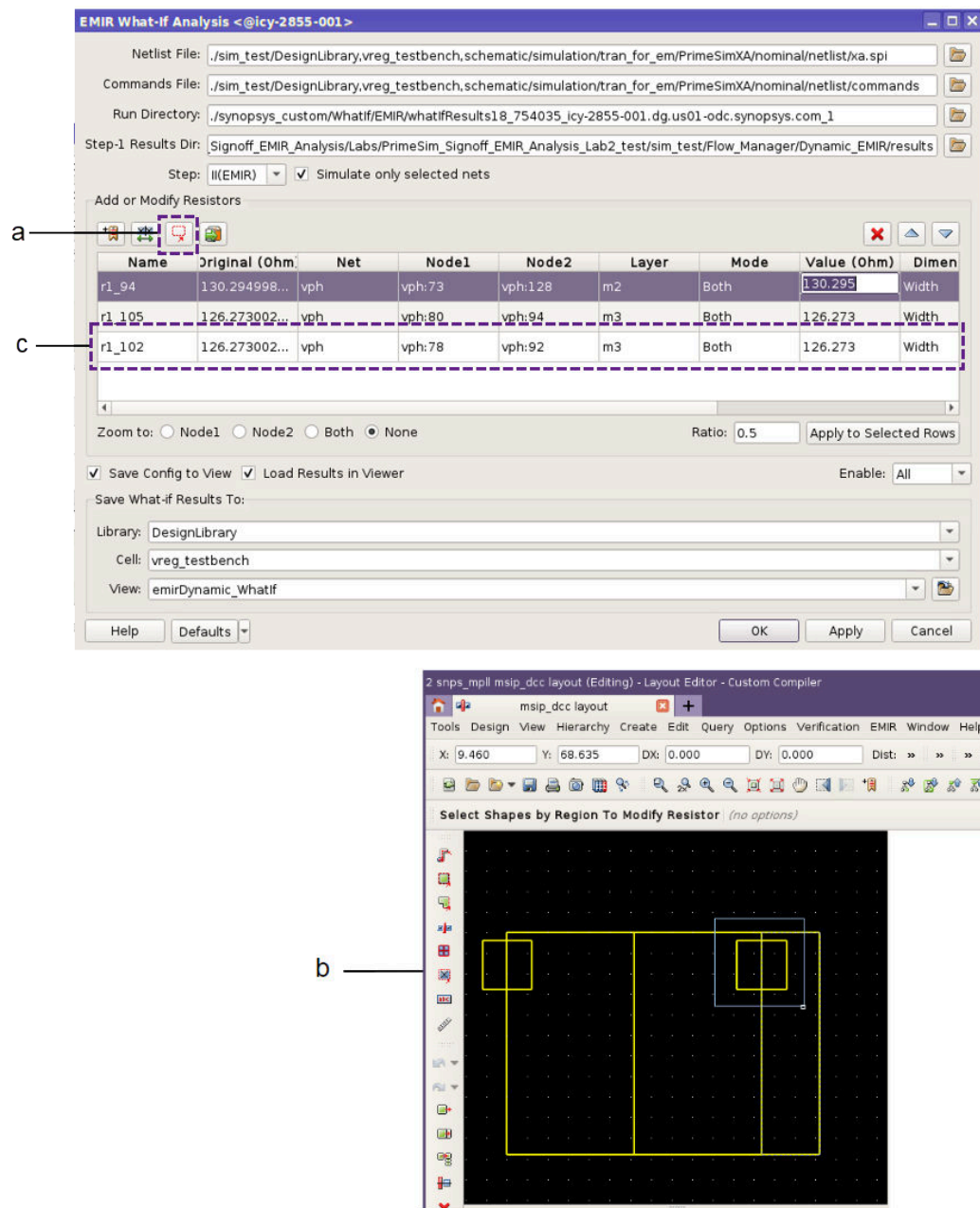


b



10. To modify multiple resistors by selecting shapes in a region:

- a. In the **What-If Analysis** dialog box, click **Modify Resistor by Region** . The **Select Shapes by Region to Modify Resistor** message appears above the canvas.
- b. On the canvas, hold down the left mouse button, drag diagonally, and release the button, to specify the region and select the shapes in the region whose resistance information you want to modify. Information about the selected resistors appears in the **What-If Analysis** dialog box. Node1 and node2 are derived from the simulation results database.



- c. In the **What-If Analysis** dialog box, modify the resistor information (see Step 12) .
See also [EMIR What-If Analysis Dialog Box Reference](#).
- d. Repeat Step a through Step c to modify more resistors.

11. (Optional) For a closer view of a selected resistor, use the **Zoom to** selection:
 - **Node1**: Fits the node 1 region within the viewport.
 - **Node2**: Fits the node 2 region within the viewport.
 - **Both**: Displays both nodes within the viewport.
 - **None**: No zooming occurs.
12. To edit a resistance value, type the value in the **Value** column and press Enter. To edit the resistance value for multiple resistors, select the resistor rows, type the value in the **Value** column of the first selected resistor row, and press Enter.

EMIR What-If Analysis <@icy-2855-001>

Netlist File: ./sim_test/DesignLibrary.vreg_testbench.schematic/simulation/tran_for_em/PrimeSimXA/nominal/netlist/xa.spi

Commands File: ./sim_test/DesignLibrary.vreg_testbench.schematic/simulation/tran_for_em/PrimeSimXA/nominal/netlist/commands

Run Directory: ./synopsys_custom/WhatIf/EMIR/whatIfResults18_754035_icy-2855-001.dg.us01-odc.synopsys.com_1

Step-1 Results Dir: Signoff_EMIR_Analysis/Labs/PrimeSim_Signoff_EMIR_Analysis_Lab2_test/sim_test/Flow_Manager/Dynamic_EMIR/results

Step: II(EMIR) ☒ Simulate only selected nets

Add or Modify Resistors

Name	Original (Ohm)	Net	Node1	Node2	Layer	Mode	Value (Ohm)	Dimen
r1_94	130.294998...	vph	vph:73	vph:128	m2	Both	130.295	Width
r1_105	126.273002...	vph	vph:80	vph:94	m3	Both	126.273	Width
r1_102	126.273002...	vph	vph:78	vph:92	m3	Both	126.273	Width

Zoom to: ☐ Node1 ☐ Node2 ☐ Both ☒ None

Ratio: 0.5

☒ Save Config to View ☒ Load Results in Viewer

Enable: All

Save What-if Results To:

Library: DesignLibrary

Cell: vreg_testbench

View: emirDynamic_Whatif

To edit the resistance value for multiple resistors using a single ratio, select the resistor rows, type the ratio value in the **Ratio** field below the table, and click **Apply to Selected Rows**.

EMIR What-if Analysis <@icy-2855-001>

Netlist File:

Commands File:

Run Directory:

Step-1 Results Dir:

Step: ☒ Simulate only selected nets

Add or Modify Resistors

Name	Original (Ohm)	Net	Node1	Node2	Layer	Mode	Value (Ohm)	Dimen
r1_94	130.294998...	vph	vph:73	vph:128	m2	Both	130.295	Width
r1_105	126.273002...	vph	vph:80	vph:94	m3	Both	126.273	Width
r1_102	126.273002...	vph	vph:78	vph:92	m3	Both	126.273	Width

Zoom to: ☐ Node1 ☐ Node2 ☐ Both ☒ None

Ratio:

☒ Save Config to View ☒ Load Results in Viewer Enable:

Save What-if Results To:

Library:

Cell:

View:

13. (Optional) To rearrange the order of the resistors in the table, select a resistor (a row in the table), and click or to move it up or down, respectively.

14. (Optional) To delete a resistor, select the resistor row in the table and click .

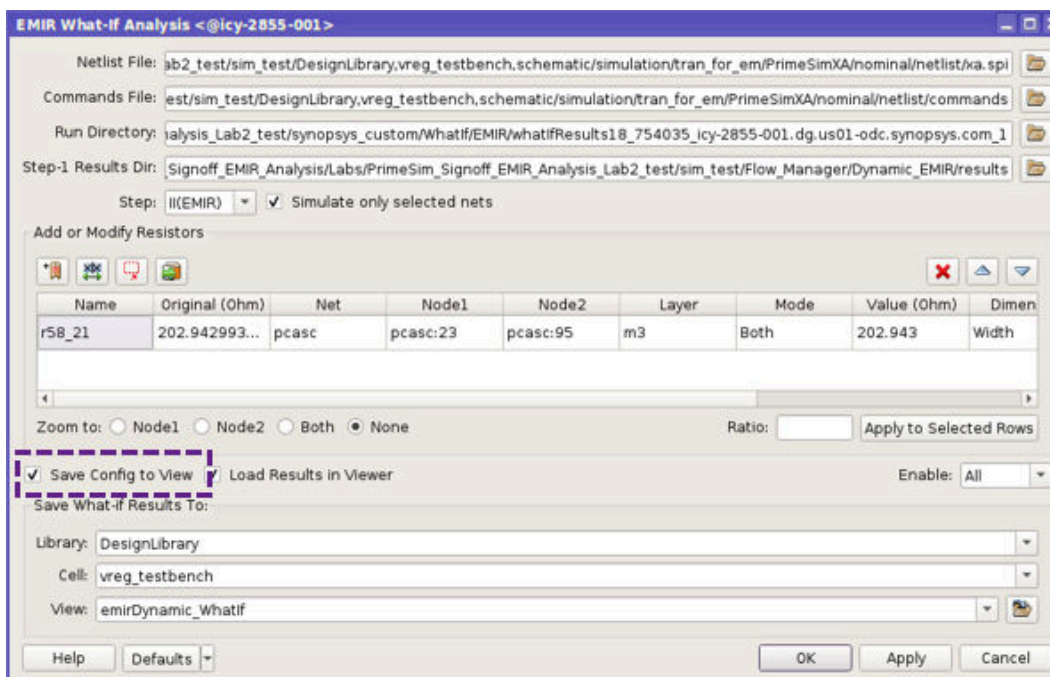
Alternatively, press Ctrl and left-click the resistor in the layout canvas, to delete the resistor.

Saving What-If Analysis Setups

You can save the what-if setup data in the EMIR view. When you re-open the design and the What-If dialog box later, the setup data will reload automatically.

To save a what-if setup:

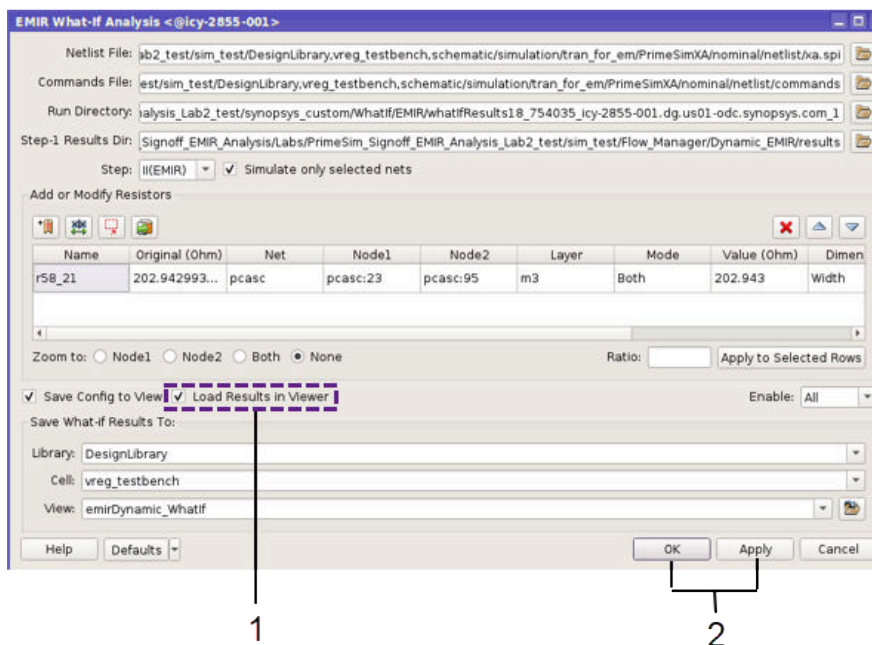
- In the **EMIR What-If Analysis** dialog box, select **Save Config to View**. The setup will be saved when you run the analysis.



Running and Saving What-If Analysis Results

To run the what-if analysis, in the **What-If Analysis** dialog box:

1. Select **Load Results in Viewer** if you want to display the what-if analysis results in the PrimeSim EMIR GUI.
2. Click **OK** or **Apply**. The analysis produces a results database file (.db).



Loading and Viewing What-If Analysis Results

If you selected **Load Results in Viewer** before you started the analysis run, the results are automatically loaded and displayed in the PrimeSim EMIR GUI.

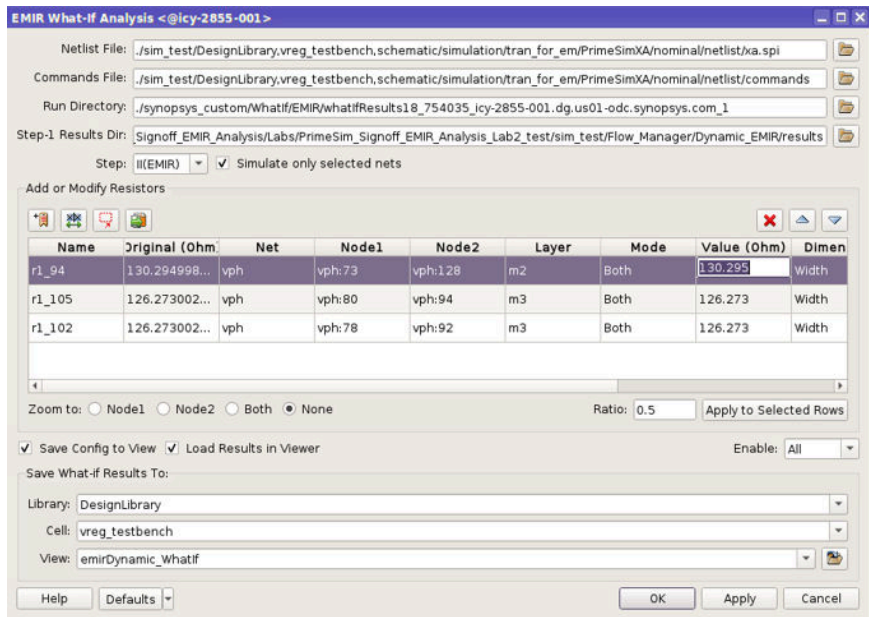
Exporting the What-If Resistor Data

To export the resistor data to a text file:

1. In the **EMIR What-If Analysis** dialog box, click **Export Table to File**  above the resistor table. The **Save What-If Data** dialog box opens.
2. Type the **File Name** and select the location where you want to store the exported file.
3. Click **Save**. The resistor data from the table are exported to a text file.

EMIR What-If Analysis Dialog Box Reference

This section provides a quick reference to the **EMIR What-If Analysis** dialog box. For instructions on setting up and running what-if analyses, see .



Setting	Description
Netlist File	The pre-layout SPICE netlist that was used to run the simulation.
Command File	The simulator command script file that was used to control the simulation options.
Run Directory	The directory in which run files are stored. The default is synopsys_custom/What-If/WhatIf_*.
Step-1 Results Dir	The directory of the phase-I analysis results. The option is available only when you select Step: II (EMIR Analysis).
Command Line Option	The location and name of the PrimeSim EMIR command file.
Step	The step (or phase) of the EMIR analysis: Step I, Step II, or Step I + II.
Reuse Flow	Uses the last-modified DSPF file as input to the EMIR analysis. If you select this option, you can also specify the Instance Pin Tolerance value. Select this option for iterative analyses.
 Add Resistor	Lets you add resistors.
 Modify Resistor	Lets you modify resistors.

Setting	Description
 Modify Resistor By Region	Lets you select shapes by region to modify resistors.
 Export Table to File	Exports the resistor data in the table.
	Deletes the selected resistor.
	Moves the selected resistor up one row in the table.
	Moves the selected resistor down one row in the table
Resistor table	See Resistor Table .
Zoom to	Zooms in to the selected area: • Node1: Zooms in to the node 1 region. • Node2: Zooms in to the node 2 region. • Both: Displays both node 1 and node 2 within the viewport. • None: Does not zoom.
Ratio	Ratio value to modify multiple selected resistors.
Enable	Applies the selected resistor data to the what-if analysis: • All: All resistor data in the table. • Add: The added resistor data. • Modify: The modified resistor data. • None: None of the resistor data. • Custom: The resistor data you selected in the table.
Overwrite Previous Results	Overwrites the existing what-if analysis data.
Save Config to View	Saves the what-if analysis setup with the design (cellview).
Load Results in Viewer	Loads the what-if analysis results when the analysis is finished.

Resistor Table

Column	Description
Name	Resistor name
Original (Ohm)	Original resistance value derived from the simulation database.
Net	Net to which the resistor belongs.

Column	Description
Node 1/2	Two points between which a new resistor is added. For existing resistors, the node data is derived from the simulation database.
Layer	Layer on which the resistor is added.
Mode	Specifies the values that can be added or modified, depending on whether you are adding or modifying resistors. When adding new resistors, choose one of the following: • New Resistance • Both (New Resistance and Dimension) When modifying existing resistors, choose one of the following: • New Resistance: Recalculates the Dimension based on the New Resistance value you specify. • New Area/Width: Recalculates the Resistance based on the New Area/Width value you specify. • Both (New Resistance and Dimension): Replaces the Resistance and Dimension values with the values you specify.
Dimension	Specifies the dimension to use: • Area: Via area (um²) • Width: Width of metal shape (um) The Dimension information is derived from the .ccdb file.
Area/Width	Value of the Area or Width, based on the Dimension you chose.
Value (Ohm)	New resistance value that overrides the Original value.
Enabled	Applies the selected resistor data to the what-if analysis.

Tracing Least Resistive Paths

This chapter describes how to trace (or highlight) paths with the least resistance between two nodes during static resistance check and EMIR analysis flows.

This section describes the following topics

- [About the Least Resistive Path Viewer](#)
- [Workflow for Least Resistive Path Tracing](#)
- [Preparing for Least Resistive Path Tracing](#)
- [Tracing Paths From the PrimeSim EMIR GUI](#)
- [Tracing Paths From the Layout Canvas](#)

- [Viewing Traced Path Details](#)
- [Deleting Path Tracings](#)
- [Exporting the Traced Path Results](#)
- [Least Resistive Path Viewer Dialog Box Reference](#)
- [Tcl Commands for Least Resistive Path Tracing](#)

The procedures in this chapter rely on the knowledge from [Viewing Analysis Results](#) chapter.

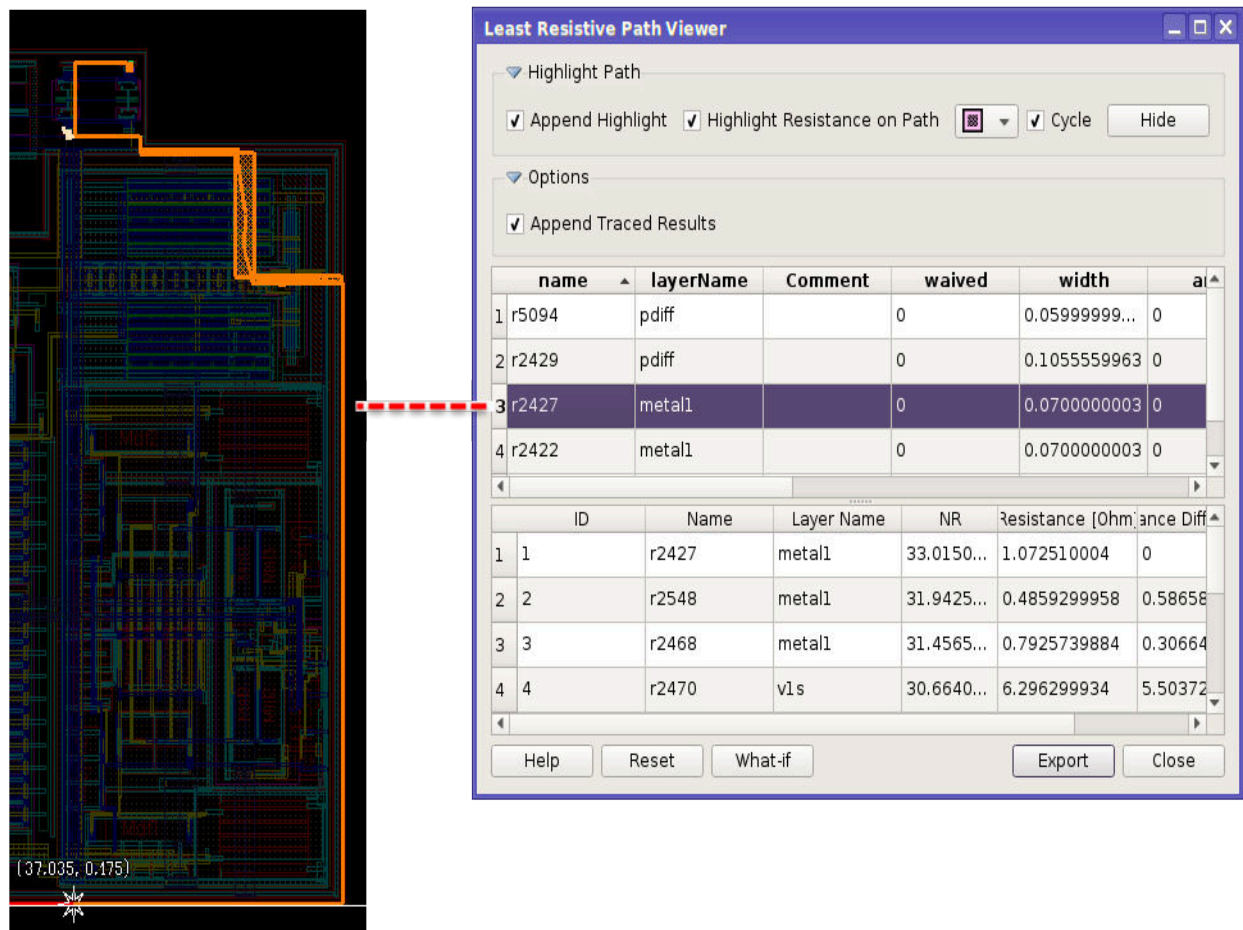
About the Least Resistive Path Viewer

You use the Least Resistive Path Viewer to trace least resistive paths. The Least Resistive Path Viewer is a feature of the PrimeSim EMIR.

You use the viewer to assess and optimize power pad placement through:

- Tracing the shortest or least resistive paths from selected devices (or nodes) to power pads.
- Examining individual path segments by layer to locate and resolve resistance bottlenecks along the traced paths.

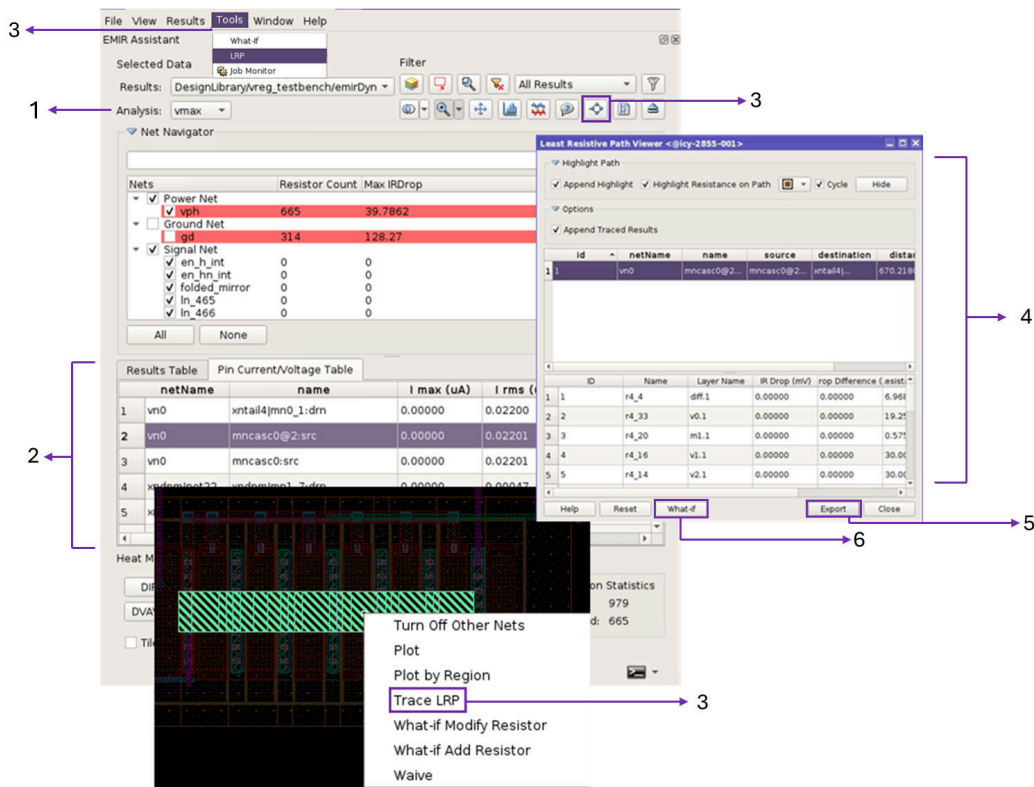
The traced path results displayed in the Least Resistive Path Viewer are linked to the layout canvas. When you select a traced path in the viewer, the path on the layout canvas becomes highlighted. While viewing the traced paths, you can add or modify resistors and run what-if analyses.



Workflow for Least Resistive Path Tracing

Before you start tracing least resistive paths, see [Preparing for Least Resistive Path Tracing](#).

After the preparations, proceed as follows from the **PrimeSim EMIR GUI**:



Step	See...
1 (EMIR only) Select Analysis: vmax . The Trace LRP button is available only if you select the vmax analysis.	Not required
2 Select one or more violations of interest.	
3 Trace the path.	Use one of the following methods: • Tracing Paths From the PrimeSim EMIR GUI • Tracing Paths From the Layout Canvas
4 View the traced paths and path segments.	Viewing Traced Path Details
5 (Optional) Export the traced path results.	Exporting the Traced Path Results
6 (Optional) Run what-if analysis.	

Preparing for Least Resistive Path Tracing

This topic describes the following:

- [Software Requirements](#)
- [NPF File](#)
- [Display Setup](#)

Software Requirements

Using the Least Resistive Path Viewer requires the PrimeSim XA tool. See the [Custom Compiler Tools Compatibility Table](#) for minimal version requirements.

NPF File

The NPF file, generated by the PrimeSim or PrimeSim XA tool, contains the net and resistor information that the Least Resistive Path Viewer needs. Ensure that the NPF file is located in the simulation directory in which the .cc.db file is stored.

The tool uses the following sequence to search for the NPF library to use to read the NPF file:

1. The path specified by the SYNOPSIS_CUSTOM_LIBSPRA_PATH environment variable
2. The path specified by the LD_LIBRARY_PATH environment variable
3. The PrimeSim simulator installation folder or if using the PrimeSim XA simulator, the PrimeSim XA installation folder

Display Setup

To increase the contrast between the selected paths and the surrounding design, turn on **Cross Shadow**  on the Design Navigator toolbar.

Tracing Paths From the PrimeSim EMIR GUI

After you select **Analysis: vmax** (EMIR only) and the violations of interest in the results viewer, you can start to trace least resistive paths to the power pad.

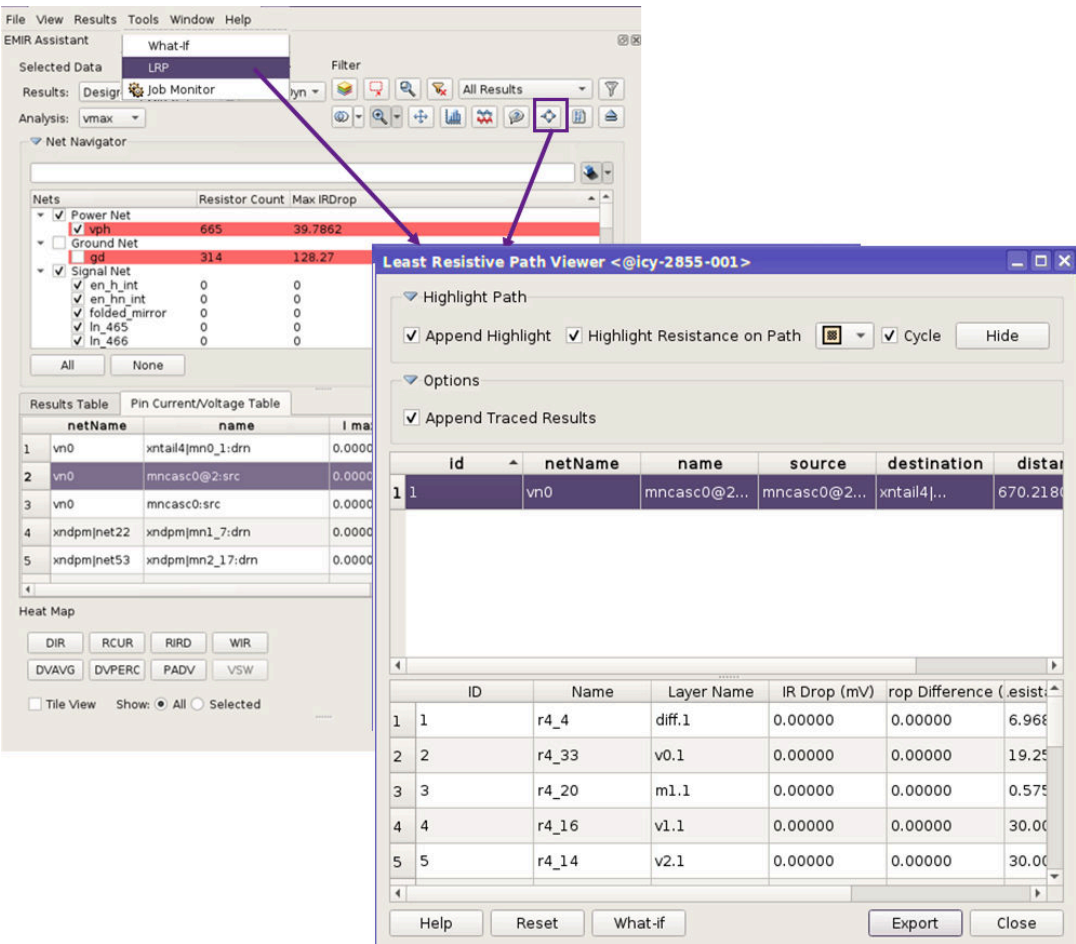
This topic describes how to start tracing paths from the PrimeSim EMIR GUI. To start tracing paths from the violation shapes on the layout canvas, see [Tracing Paths From the Layout Canvas](#).

To start tracing from the **PrimeSim EMIR** GUI:

1. Use one of the following methods:

- Click  above the violation data table.
- Choose **Tools > LRP**.

The **Least Resistive Path Viewer** dialog box opens. In the example shown, two violations were selected, so two paths are listed in the Least Resistive Path Viewer dialog box.



The screenshot shows the PrimeSim EMIR GUI with the **Least Resistive Path Viewer** dialog box open. The dialog box has a title bar that reads "Least Resistive Path Viewer <@icy-2855-001>". It contains several sections:

- Highlight Path**: Includes checkboxes for "Append Highlight" and "Highlight Resistance on Path", a color selection button, and checkboxes for "Cycle" and "Hide".
- Options**: Includes a checkbox for "Append Traced Results".
- Table**: A table with columns: ID, netName, name, source, destination, and distance. It lists two paths for selected violations.
- Bottom Section**: Includes a table with columns: ID, Name, Layer Name, IR Drop (mV), rop Difference (resist), and distance. It lists five components along the paths.
- Buttons**: "Help", "Reset", "What-if", "Export", and "Close".

The background window shows the **Net Navigator** with a tree view containing "Power Net" and "Signal Net". The "Power Net" is expanded, showing "v0" and "mncasc0@2:src". The "Signal Net" is also expanded, showing "en_h_int", "en_hn_int", "folded_mirror", "ln_465", and "ln_466". The "Results Table" at the bottom of the background window shows a table with columns: netName, name, and I max.

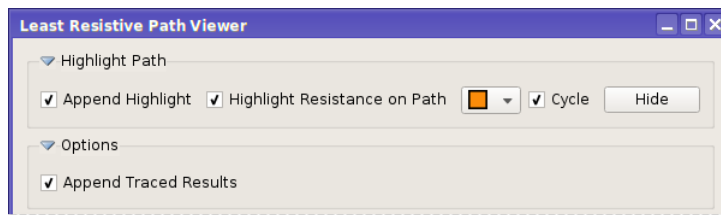
2. To highlight the traced paths, in the **Highlight Path** section:

- Select **Highlight Resistance on Path**.
- Choose the highlight color .
- Select **Cycle** to apply different color packets by cycling through the list of available colors.

To remove the highlighting, click **Hide**.

3. (Optional) To view cumulative trace results:

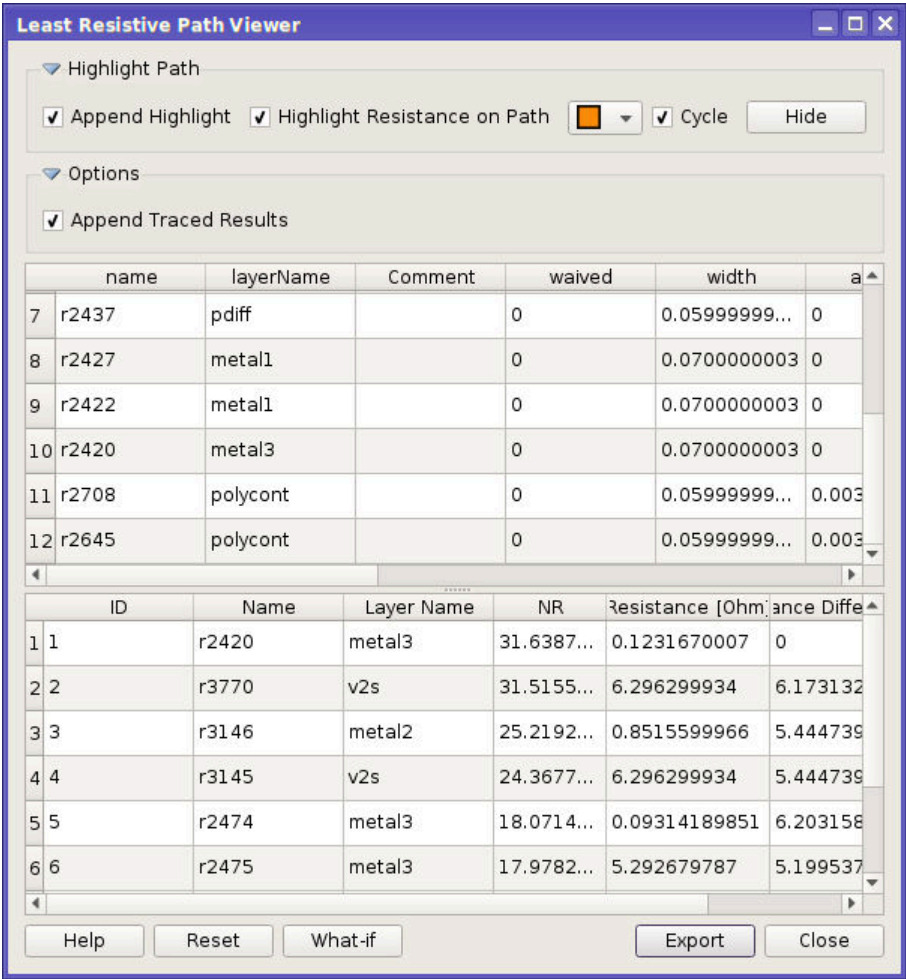
a. Select the following in the **Least Resistive Path View** dialog box:



Option	Description
Append Traced Results	Retains existing results when adding new results.
Append Highlight	Retains the highlights of the traced paths on the canvas as you trace new paths.

b. In the **PrimeSim EMIR** GUI, select other violations, and repeat steps 1 and 2 to trace additional paths.

The following example shows how new results are appended to existing results in the **Least Resistive Path Viewer** dialog box.



- 4. To view the details of the traced paths, see [Viewing Traced Path Details](#).
- 5. (Optional) To add or modify resistors and run what-if analyses, click **What-If**.

Tracing Paths From the Layout Canvas

After you select one or more violations of interest in the results viewer, you can start to trace the least resistive path to the power pad.

Chapter 7: Running the EMIR Application Tracing Least Resistive Paths

This topic describes how to trace paths from violation shapes on the layout canvas using one of the following methods. To start tracing paths from the results viewer, see [Tracing Paths From the PrimeSim EMIR GUI](#).

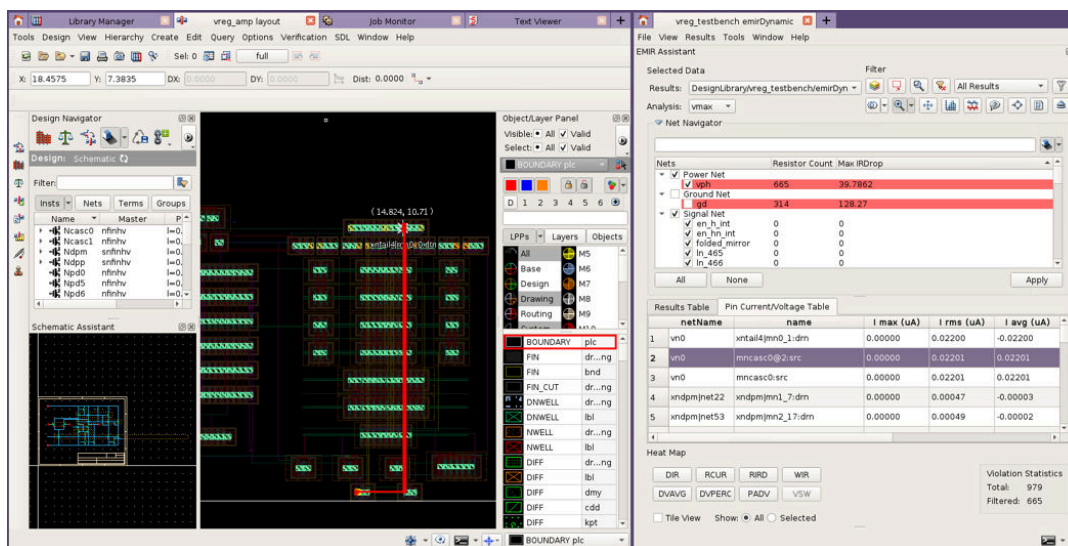
- [Before You Start](#)
- [Tracing Paths From Violation Shapes](#)
- [Tracing From Instance Pins](#)
- [Tracing Using Shortcut Menu Command](#)

Before You Start

(EMIR only) Make sure you select **Analysis: vmax** before you start to trace least resistive paths.

Tracing Paths From Violation Shapes

When you selected the violations in the results viewer data table, the layout canvas zooms in and highlights the violation shapes.

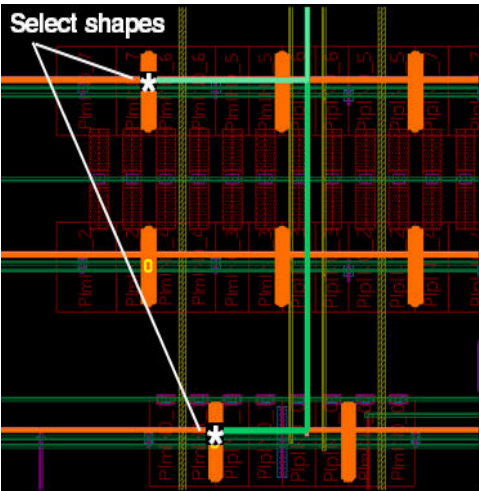


To start tracing from the shapes on the layout canvas:

1. Select one or more highlighted violation shapes on the canvas. The **Selected Violations** dialog box opens.

In the following example, two shapes were selected on the canvas, so two rows appear in the table.

Select shapes



Selected Violations

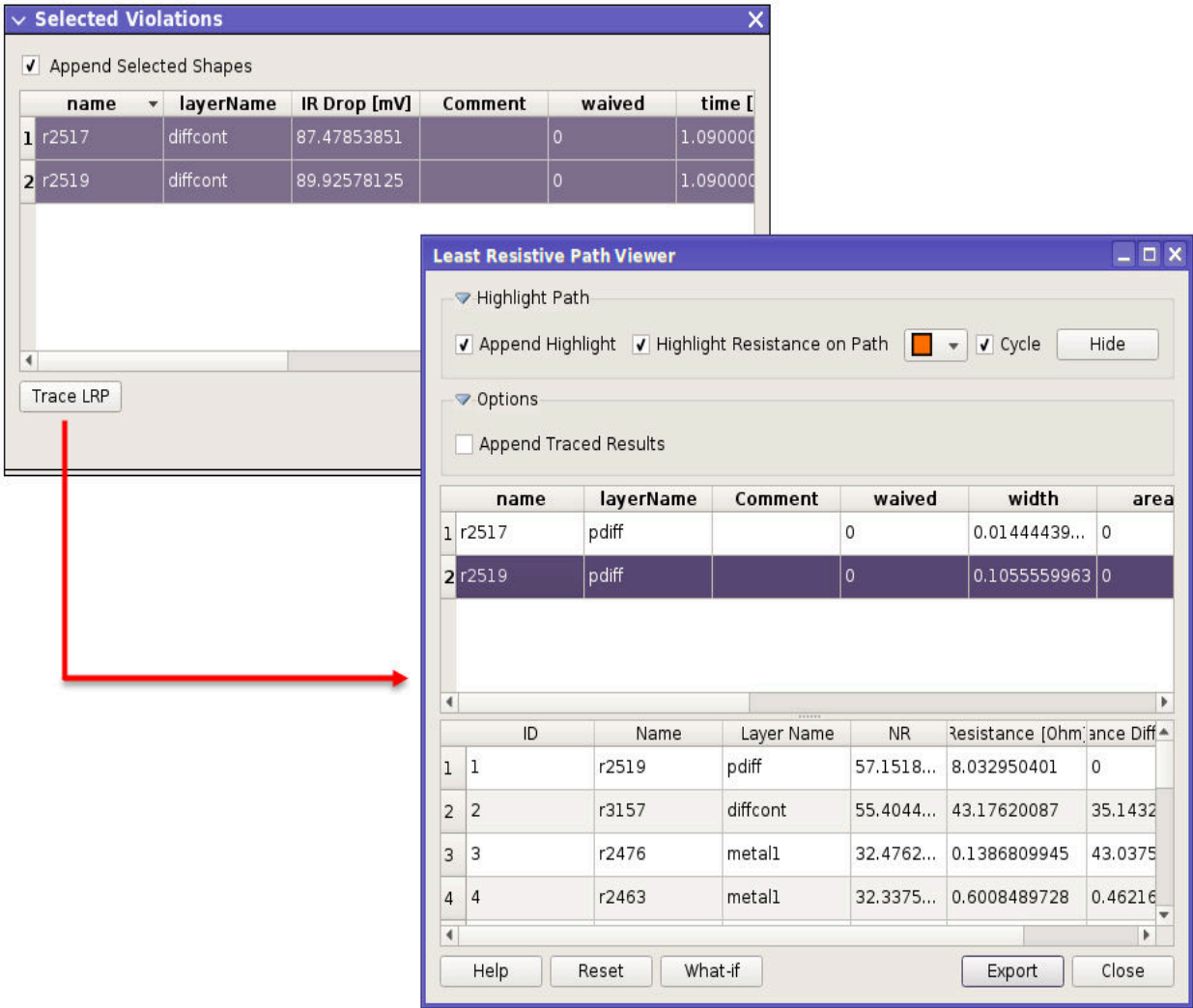
☐ Append Selected Shapes

	name	layerName	IR Drop [mV]	Comment	waived	time [s]
1	r6_50117	diffcont	87.47853851		0	1.090000
2	r6_50119	diffcont	89.92578125		0	1.090000

Trace LRP

Close

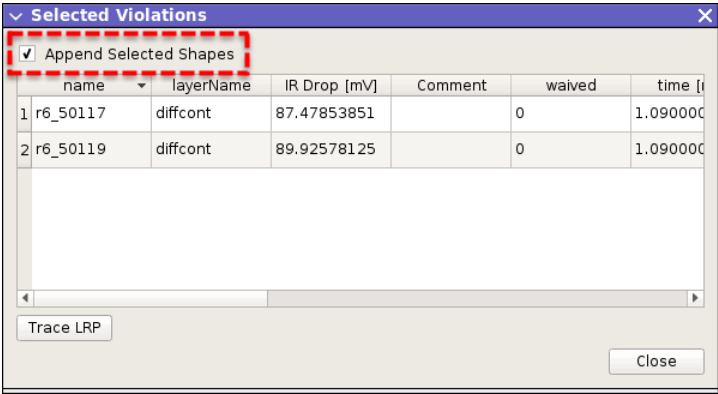
2. Select one or more shapes in the table and click **Trace LRP**. The **Least Resistive Path Viewer** dialog box opens.



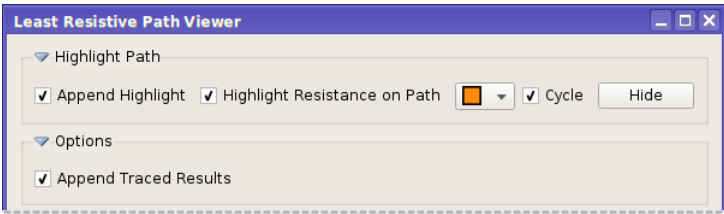
3. To highlight the traced paths, in the **Highlight Path** section:
- Select **Highlight Resistance on Path**.
 - Choose the highlight color .
 - Select **Cycle** to apply different color packets by cycling through the list of available colors.

To remove the highlighting, click **Hide**.

4. (Optional) To view cumulative trace results:
- a. In the **Selected Violations** dialog box, select **Append Selected Shapes**.



- b. In the **Least Resistive Path View** dialog box, select the following:



Option	Description
Append Traced Results	Retains existing results when adding new results.

Option	Description
Append Highlight	Retains the highlights of the traced paths on the canvas as you trace new paths.

- c. The following example shows how new results are appended to existing results in the **Selected Violations** dialog box and the **Least Resistive Path Viewer** dialog box.

In the results viewer, select other violations, and repeat steps 1 and 2 to trace the paths.

The image shows two dialog boxes from the EMIR application. The 'Selected Violations' dialog on the left has a table with columns: name, layerName, IR Drop [mV], Comment, waived, and time. It lists violations r2514 through r2519. A 'Trace LRP' button is at the bottom. A red arrow points from this button to the 'Least Resistive Path Viewer' dialog on the right. This dialog has a 'Highlight Path' section with checkboxes for 'Append Highlight', 'Highlight Resistance on Path', 'Cycle', and 'Hide'. Below is an 'Options' section with 'Append Traced Results'. The main part of the dialog contains a table with columns: name, layerName, Comment, waived, width, and area. It lists violations r2517, r2519, r2421, and r2422. At the bottom is another table with columns: ID, Name, Layer Name, NR, Resistance [Ohm], and Resistance Diff. It lists four traced paths. Buttons for 'Help', 'Reset', 'What-if', 'Export', and 'Close' are at the bottom.

	name	layerName	IR Drop [mV]	Comment	waived	time
1	r2514	pdiff	97.57331848		0	1.0900
2	r2515	pdiff	108.4144669		0	1.25
3	r2516	pdiff	112.6823807			
4	r2517	pdiff	87.47853851			
5	r2519	pdiff	89.92578125			

	name	layerName	Comment	waived	width	area
1	r2517	pdiff		0	0.01444439...	0
2	r2519	pdiff		0	0.1055559963	0
3	r2421	metal1		0	0.0700000003	0
4	r2422	metal1		0	0.0700000003	0

	ID	Name	Layer Name	NR	Resistance [Ohm]	Resistance Diff
1	1	r2519	pdiff	57.1518...	8.032950401	0
2	2	r3157	diffcont	55.4044...	43.17620087	35.1432
3	3	r2476	metal1	32.4762...	0.1386809945	43.0375
4	4	r2463	metal1	32.3375...	0.6008489728	0.46216

5. To view the details of the traced paths, see [Viewing Traced Path Details](#).

Tracing From Instance Pins

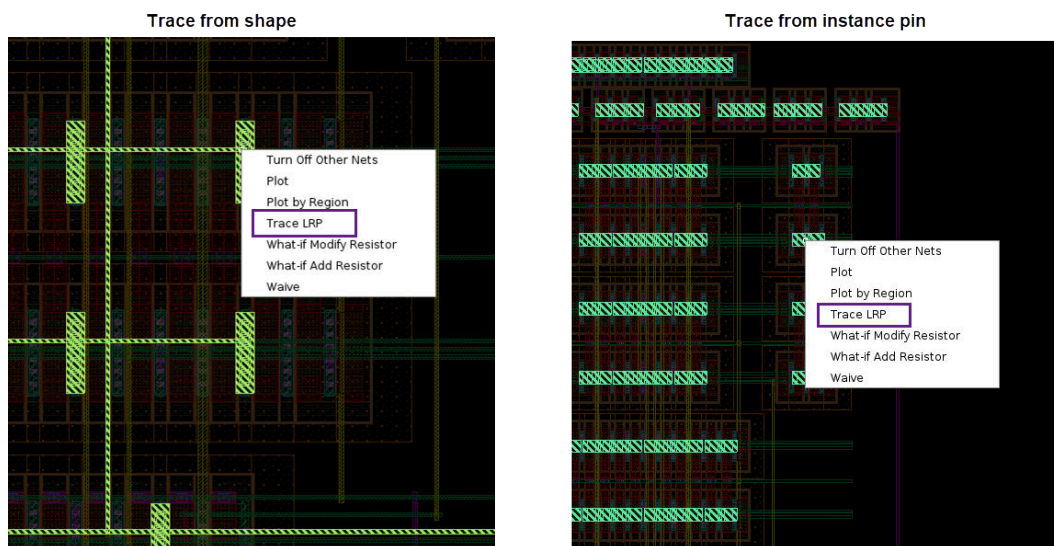
To trace least resistive paths between instance pins:

1. In the PrimeSim EMIR GUI, display the instance pins on the layout canvas.
2. On the layout canvas, zoom in to the instance pins.
3. Select the instance pins.

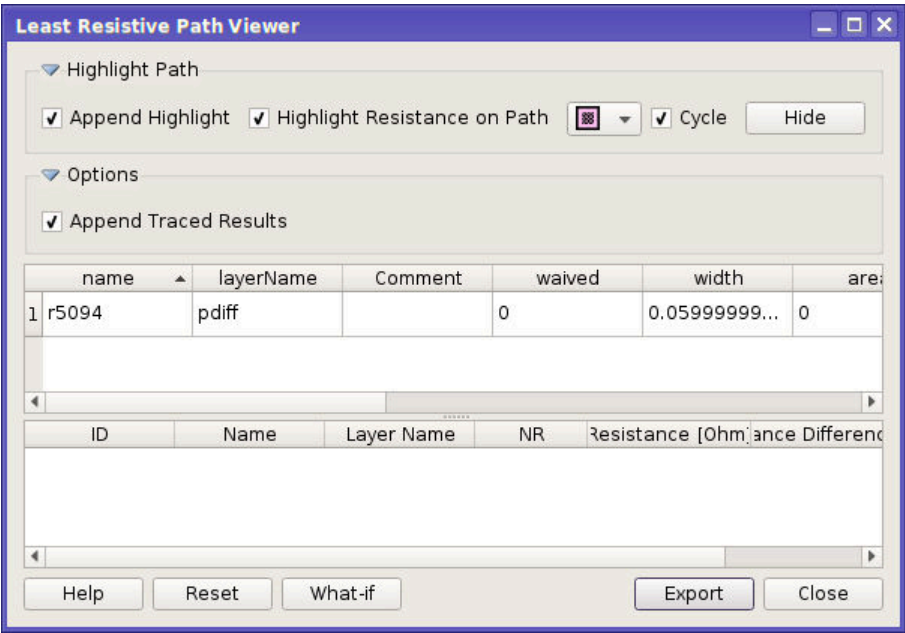
Tracing Using Shortcut Menu Command

To use the shortcut menu directly from the layout canvas:

1. On the layout canvas, right-click the violation shape or instance pin, and choose **Trace LRP**.



The **Least Resistive Path View** dialog box appears.

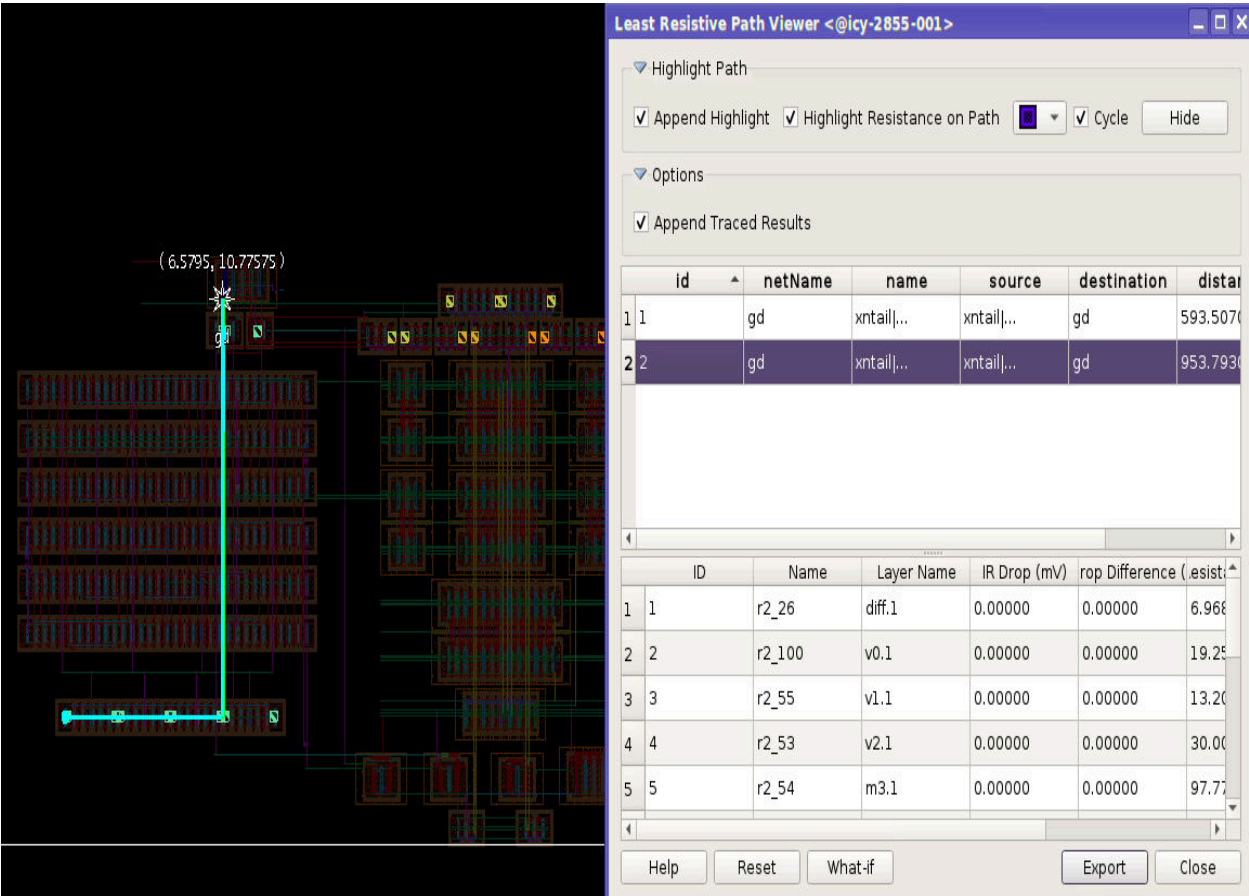


- 2. To view the details of the traced paths, see [Viewing Traced Path Details](#).
- 3. (Optional) To add or modify resistors and run what-if analyses, click **What-If**.

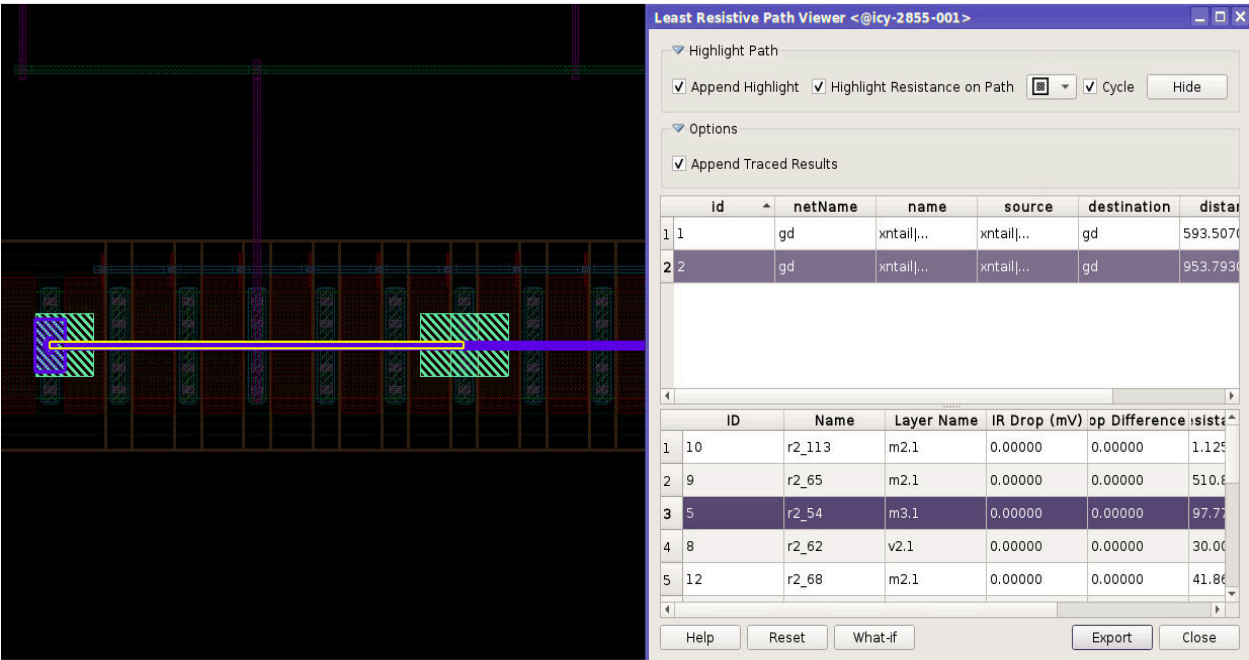
Viewing Traced Path Details

To view a traced path, in the **Least Resistive Path Viewer** dialog box:

1. Select a path from the full path table. The path segment table refreshes and lists the path segments. In addition, the viewport zooms in and displays the selected path.



2. In the path segment table, select a path segment. The layout canvas zooms in to and highlights the selected segment.



3. In the path segment table, scroll to the **Resistance Difference** column, which shows the difference in resistance between adjacent segments. You can sort the table to view the highest resistance difference in the traced segments.

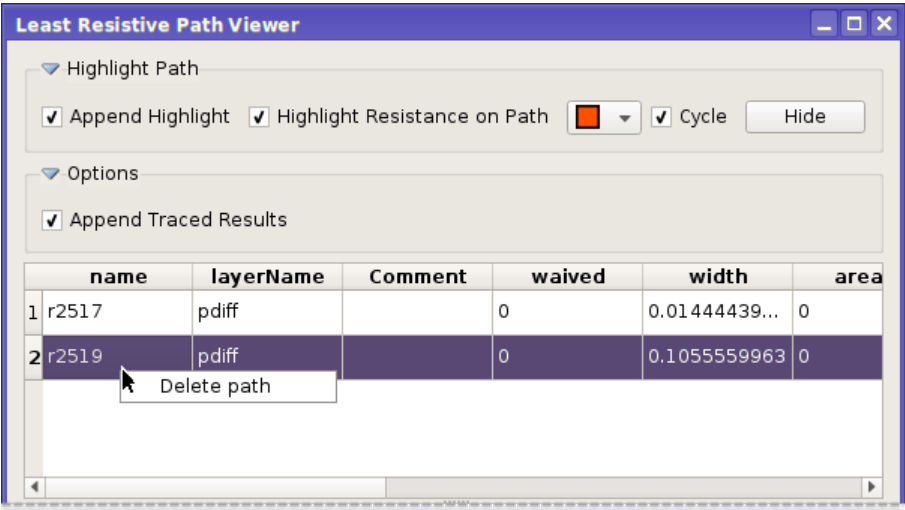
	ID	Name	Layer Name	IR Drop [mV]	Resistance [Ohm]	Resistance Difference [Ohm]
1	2	r6_48839	metal1	87.50390625	1.569314003	41.60692501
2	3	r6_48840	metal1	87.41589355	0.03831598908	1.530998014
3	45	r6_60515	metal1	86.33599091	0.6447917819	0.5558492914
4	46	r6_60514	metal1	86.07439423	0.3747644126	0.2700273693
5	381	r6_61518	metal1	48.06845856	0.0553490296	0.2236100659
6	63	r6_60563	metal1	84.21276093	0.2789322138	0.2058785409

4. (Optional) To add or modify resistors and run what-if analyses, click **What-If**. See .

Deleting Path Tracings

To delete a path tracing from the Least Resistive Path Viewer:

- In the **Least Resistive Path Viewer** dialog box, right-click the path, and choose **Delete Path**.

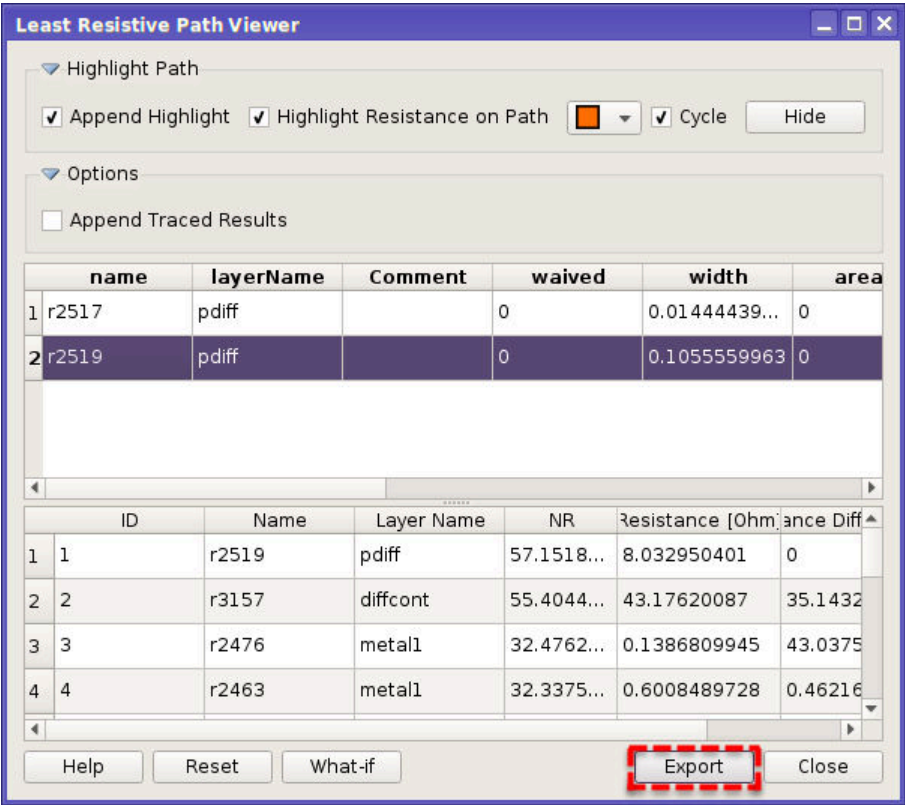


Exporting the Traced Path Results

You can export the traced path results to a CSV file for review and downstream processing.

To export the traced path results:

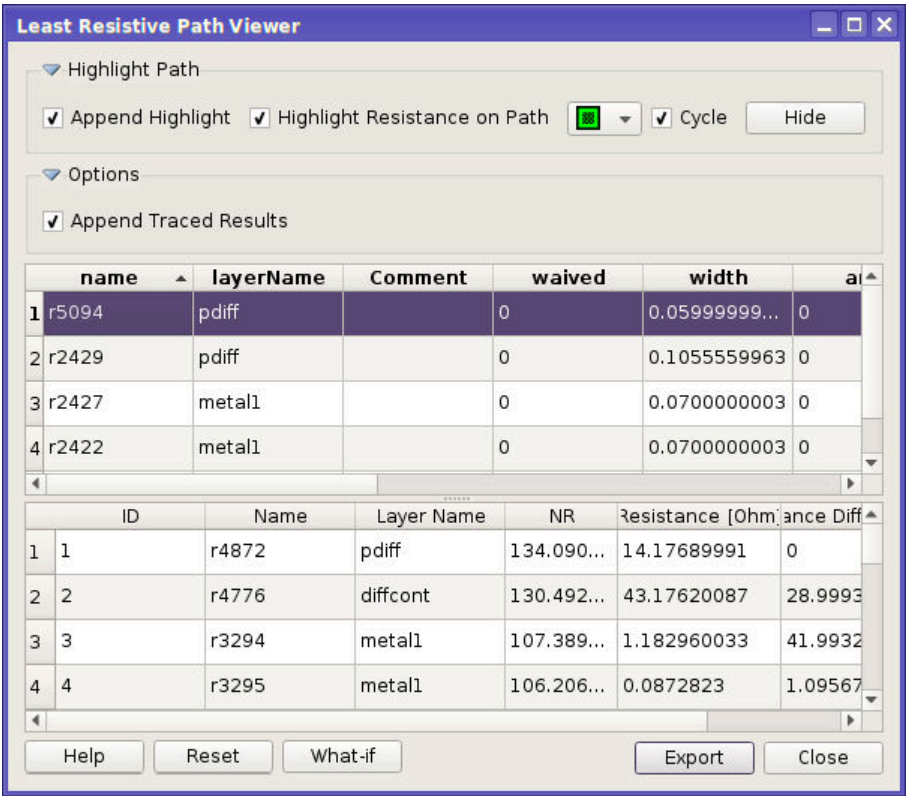
- In the **Least Resistive Path Viewer** dialog box, click **Export**.



Least Resistive Path Viewer Dialog Box Reference

This topic provides a quick reference of the Least Resistive Path Viewer dialog box. For instructions on how to trace least resistive paths, see [Workflow for Least Resistive Path Tracing](#).

The **Least Resistive Path Viewer** dialog box consists of four sections:



Highlight Path

Option	Description
Append Highlight	Retains the highlights of the traced paths on the canvas as you trace new paths. The option is selected by default.
Highlight Resistance on Path	Turns on traced path highlighting. The option is selected by default.
	Applies the chosen color to the traced paths.
Cycle	Applies different color packets by cycling through the list of colors. The option is selected by default.

Options

Option	Description
Append Traced Results	Specifies whether to retain existing results while you trace new paths for multiple traced result viewing. The option is selected by default.

Full Path Table

Table	Description
Full path table	Lists full paths from the trace. Because Append Traced Results is selected by default, previously traced paths are listed in the table when you open the dialog box.

Path Segment Table

Table	Description
Path segment table	Lists path segments of the selected full path.

Buttons

Button	Description
Reset	Resets the dialog box to default values.
What-If	Displays the What-If Analysis dialog box.
Export	Exports the traced path results to a CSV file.

Tcl Commands for Least Resistive Path Tracing

You can use the following Tcl commands for least resistive path tracing:

- `cs::traceShortestPath`
- `cs::showShortestPathOverlays`
- `cs::loadNpfForShortestPath`

For information about using Tcl commands and writing scripts, see the [Custom Compiler Tcl Reference Manual](#).

8

EMIR Output Reports and Maps

Describes how to generate the EMIR output reports and violation maps.

This chapter includes the following chapters:

- [EMIR Output Files](#)
- [Generating Violation Maps](#)

EMIR Output Files

When EMIR analysis is complete, the tool generates output files based on the command settings you specify in the .ratcl file. The support output file types are described in the following topics:

- [Violation Map in ASCII Format](#)
- [Display Resource File \(DRF\)](#)
- [CCDB File](#)
- [FSDB, OUT or WDF Waveform File](#)
- [RADB Simulation Result File](#)
- [RALOG Log File](#)

Violation Map in ASCII Format

You use the `raformat ascii` command to generate the ASCII violation map file when EMIR analysis is complete. The ASCII file name may be composed of prefix, net name, analysis type, and extension, like

```
<prefix>-<net name>_<analysis_type>.ascii
```

where <prefix> is the name specified in the command line after `-o`. If `-o` is used in `ralayout`, the file name is as follows:

```
<prefix>_<analysis_type>.ascii
```

Violation Data in the Output ASCII File

The ASCII file is in the form of a text table and its content is dependent on the analysis type. In the ASCII file, the violation results are displayed in the table columns. To control which result columns are shown in the ASCII output file, use the `asciicols` command. To control the sequence of the columns, use the `asciicolsort` command.

For IR drop analysis, the *.ascii file might contain the following columns:

- Resistor name
- Voltage drop value
- Original layer name and number
- X coordinate
- Y coordinate
- Node name
- Time when IR drop occurs
- Actual voltage value at dV
- Level of violation
- Voltage drop value in percentage
- Maximum, minimum, or average voltage value
- Average voltage drop value
- Time of the maximum and minimum voltage occurrences
- Voltage drop value in percentage of the base voltage value
- Electron flow direction

For EM analysis, the *.ascii file might have the following columns:

- Resistor name
- Current through resistor
- Current density through resistor
- Ratio: current density over threshold
- Original layer name and number
- X coordinate
- Y coordinate

- Resistor width in the unit of um
- Resistor length in the unit of um
- Time of maximum current occurrence
- Level of violation
- Slack or threshold - current density
- Area: Calculated for VIA's or layers with a specified height using `layerh`
- New Area: Recommended area for resistor in the unit of um2
- Wire length
- Electron flow direction (IAVG analysis only)
- Length and width of the wire connected to the VIA node
- Net name of the signal nets
- Pulse width of a periodic signal
- Pulse to period ratio
- Number of vias
- APW: The highest average, RMS or peak current time window. This is used to print the time window where the highest value of average, RMS or peak current is observed when the `swin` command is used with `-apw` option.

Example 1 An Example of Violation Map in ASCII Format

The following example runs EMIR analysis with the test.ratcl file and generates an ASCII violation map.

```
xa -ra test.ratcl ...
```

In the test.ratcl file:

```
... ...
raformat ascii
asciicols em current layer x y i_imax width length area netName apw
... ...
```

Output ASCII Violation File:

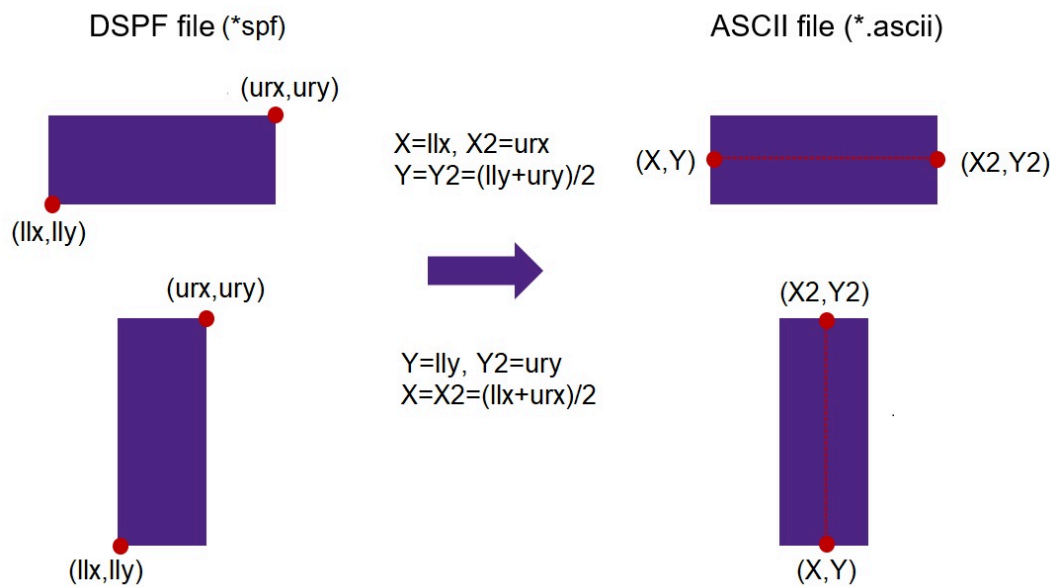
```
Reg. I,      I,      I/Imax Layer  X      Y      Width  Length  Area  EM  APW
      current threshold                (um)   (um)   (um)   (um)   (um*um) (um) (ns)
      (uA)      (uA)
Processing net: vdd
```

```
r38  5723.75  9536.38  0.600  m2  -558.52000  66.680000  2  26  2  69.2  2:5
r39  5723.75  9536.38  0.600  m2  -558.52000  66.680000  2  26  2  69.2  2:5
```

Coordinates in the ASCII File

In the output ASCII violation file, the coordinates in the (X,Y) and (X2,Y2) columns are the start and end points of the geometry center line. These coordinates are shifted based on width and direction.

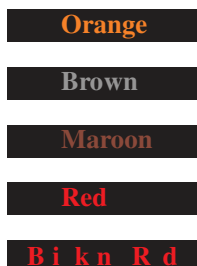
The following figure illustrates how the bounding box coordinates (llx, lly, urx and ury) are translated to the X/X2 and Y/Y2 coordinates in the ASCII violation file for a horizontal resistor (\$dir=0) and a vertical resistor (\$dir=1):



Display Resource File (DRF)

The DRF Display Resources file is always generated if GDSII is requested for a net. The DRF file name is always *output_prefix-net_name.drf*. The DRF file defines ten colors for violation map layers as follows:

- Forest Green
- Green
- Lime
- Yellow
- Gold



The smallest IR drop or EM corresponds to the color Forest Green and the values greater than the threshold correspond to the color Blinking Red.

CCDB File

The PrimeSim EMIR tool supports the Custom Compiler database format (.cc.db) to view EMIR analyses results in the Custom Compiler EMIR Results Browser (EMIR Viewer). Custom Compiler EMIR Viewer provides a graphical GUI environment for displaying electromigration and voltage drop violation maps for layout debugging. For a detailed description about how to display CCDB files in Custom Compiler EMIR Viewer, see the “Power and Signal Net EMIR Analysis” topics in *Custom Compiler Electrical Checking and Reporting User Guide*.

This section provides information about how to generate CCDB files, specify columns to include in the generated CCDB file and merge multiple CCDB files, as described in the following topics:

Generating CCDB File

Use the `raformat ccdb` command to save power net EMIR analysis data to an output CCDB file.

The `raformat` command specifies the format of the output EMIR data during power net EMIR flow. The supported formats are: CCDB and ASCII. You can choose to generate multiple formats at a time. For example, the following command generates .ascii files and .cc.db files when power net EMIR analysis is finished:

```
raformat ascii ccdb
```

There is only one .cc.db file per power and signal net. The following example shows the output files when the following analyses are run: `imax`, `iabs`, `iavg`, `irms`, and `vmax`.

The CCDB output files are:

- `xa-gd.cc.db`
- `xa-@.cc.db`

For a description about the command options, see `raformat` in the [Commands for EMIR Analysis Post-processing](#) section.

Specifying Table Columns for CCDB File

Use the `ccdbcols` command to specify table columns to include in an output `.cc.db` file, as well as the sequence of columns. You also need to specify a table type to include: *ird* for voltage drop data, *em* for electromigration data, and *sh* for shapes data.

Example 2

```
ccdbcols ird nodeName dv time
ccdbcols em current threshold i_imax j_jmax time
ccdbcols sh resname resistance layer
```

The above examples write the following information in the output CCDB file:

- Node names
- The dv and time values for IR Drop analysis
- Current, threshold, i_max, j_jmax, and time values for EM analysis
- Resistance name, resistance value, and layer name in the shape columns

For a description about the command options, see `ccdbcols` in the [Commands for EMIR Analysis Post-processing](#) section.

Merging CCDB File

When re-running RA analysis, you might want to merge the results with those that are generated in the previous run. Use the `ccdbmerge` command with the `raformat ccdb` command to determine whether to overwrite or save EMIR analysis results.

The syntax is

```
ccdbmerge [ 0|1 ] [ -remove [0|1] ]
```

Example 3

```
ccdbmerge 1 -remove 0
```

The above example writes the current analysis results to the `.cc.db` file of the previous run without overwriting the existing analyses data in the `.cc.db` file.

For a description about the command options, see `ccdbmerge` in the [Commands for EMIR Analysis Post-processing](#) section.

FSDB, OUT or WDF Waveform File

The waveform file is generated in various formats: FSDB, OUT and WDF. The default is FSDB. To specify the format of the waveform file, use the `set_waveform_option` command. The file name is composed of the net name, analysis type, and extension, like:

```
<prefix>-<net name>.fsdb
```

For the FSDB, OUT or WDF waveform file, *prefix* is *xa* or the name that is specified in the command line after `-o`. Each waveform file contains at least 3 signals stored as follows:

- `dVmax(vdd)`: Worst IR drop value reported at given time
- `Ipin_tot(vdd)`: Total current injected from all pins to the power net
- `Ipin_max(vdd)`: Maximum pin current injected to the power net

To specify which nodes and resistors to include in the waveform files, use the following TCL commands:

- `printv`
- `printi`
- `printipad`
- `printipin`
- `printvmode`

RADB Simulation Result File

The RADB simulation result file is always created and consists of voltage and current information. It is a binary file name composed of the prefix, net name, and extension.

```
<prefix>-<net name>.radb
```

RALOG Log File

The RALOG log file is always created and is named as:

```
prefix-net name.ralog
```

In the RALOG file, *prefix* is either *xa* or the name that is specified in the command line after `-o`. This file contains the following information:

- Notification of program step completion
- TCL commands read from *.ratcl files and processed by the program into internal structures
- A simulation results table for each power net simulation time step, containing the following is injected into the power net:
 - Worst IR drop (dVmax)
 - Node number
 - Maximum pin current (Imax)
 - Pin number
 - Total current (Itot) is the sum of all the pin currents at that time step
- Additional statistics including: worst cases, total resistance, currents through PADs, etc.
- Information about files generated as a result of simulation. This information is presented in *.ralog only if ralayout was specified in *.ratcl file, otherwise they are located in the *.ltlog file.

Generating Violation Maps

Visualization marks different values of IR drop or EM current density with different colors producing a violation map. More precisely, a GDSII file is generated containing geometry elements, such as rectangles or tracks, with different colors ranging from Forest Green for minor violations to Blinking Red for values exceeding a specified threshold.

The color map illustrates the distribution of IR drops and current densities within a maximum of 10 ranges determined by the `redv` and `rediw` commands. The color map, violation levels, and threshold ranges for 0.45V are shown below.

Forest Green	Level 1	$ dV < 0.05$
Green	Level 2	$0.05 \leq dV < 0.1$
Lime	Level 3	$0.1 \leq dV < 0.15$
Yellow	Level 4	$0.15 \leq dV < 0.2$
Gold	Level 5	$0.2 \leq dV < 0.25$

Orange	Level 6	$0.25 \leq dV < 0.3$
Brown	Level 7	$0.3 \leq dV < 0.35$
Maroon	Level 8	$0.35 \leq dV < 0.4$
Red	Level 9	$0.4 \leq dV < 0.45$
B i k n R d	Level 10	$0.45 \leq dV $

Note:

Color mapping is defined in the xa.drf file in which you can change the colors to display violations.

Shapes correspond to an appropriate resistor from the DSPF file. The coordinates of geometry elements are taken from the pin and sub-node statements and/or physical back-annotation portion of resistor statements in the DSPF file. Element color is determined by the maximum values of voltage drop on resistor's nodes or current density through the corresponding resistor.

This section includes the following topics:

- [Generating Violation Maps](#)
- [Generating Multiple GDSII Files With One Command](#)
- [Generating GDSII for All Analyses](#)
- [User-Specified Layer Numbers](#)
- [Layer Filtering](#)
- [Names Inserted Into Geometry](#)
- [Generating Layout Formats](#)
- [Automatically Generating Violation Maps](#)

Generating Violation Maps

Assuming that `ra vmax irms` is specified in the .ratcl command file and the maximum IR drop (vmax) with RMS current density (irms) had been calculated and stored into .radb files during the power net analysis phase, a violation map showing maximum IR drop (vmax) can be generated using the following syntax:

```
xa -ralayout vmax test-vdd
```

The resulting test-vdd_vmax.cc.db file is created. To change the name of the output file, use the following syntax:

```
xa -ralayout vmax test-vdd -o vdd
```

The layout is generated in the vdd_vmax.cc.db file.

Generating Multiple GDSII Files With One Command

It is possible to generate several GDSII files using one `ralayout` command, where the names or numbers of the required analysis are specified as follows:

```
xa -ralayout vmax imax irms test-vdd
```

Generating GDSII for All Analyses

To generate GDSII for all analyses, use the following syntax:

```
xa -ralayout all test-vdd -o vdd
```

User-Specified Layer Numbers

Layer numbers may be user-specified when displaying violation maps to eliminate map element conflicts with elements from the original layout.

Note:

Do not specify more than 10 layers.

Layer numbers may be specified using `ralayers` in the `.ratcl` file as shown in the following syntax:

```
ralayers 51,53,57-60
```

This syntax distributes analysis results among six layers from 51 to 60 as follows:

51

Layer 51 represents the lowest values

53

Layer 53 represents the next higher values

57-60

Layer-60 represents values greater than the threshold specified in `redv` or `rediw`

Layer Filtering

You can specify the original layers that should be placed into violation map using the `outlayers` command in the `.ratcl` file as shown in the following example:

```
outlayers 10,3,6
```

This command generates a violation map containing only the original layers numbered 10, 3, and 6. By default, power net EMIR flow uses all layers.

Names Inserted Into Geometry

Power net EMIR may add some names into geometry such as:

- Labels for pad points
- Pin names
- Node names specified in `printi` and `printv` within `*.ratcl` file

The `gdslabels` command specifies which labels to be generated. It generates pad point(s) and node name labels specified in `printv` as shown in the following syntax:

```
gdslabels pad print
```

Generating Layout Formats

The `raformat` command is used to specify output formats for violation map display. Format selections are `gds2`, `ascii`, and `rve`.

```
raformat gds2 ascii
```

This command generates layouts in both GDSII and ASCII formats.

Automatically Generating Violation Maps

Violation maps can be automatically generated after power net analysis using the following syntax in the `.ratcl` file:

```
ralayout analysis [-o <prefix>]
```

In this syntax, *analysis* is the required analysis names or numbers (e.g., `vmax`, `imax`, etc.) separated by spaces. Set to `ALL` for all analysis.

9

EMIR Commands

Describes commands for the pre-processing and post-processing stage of power net EMIR and signal EM analysis.

This part includes the following chapters:

- [Commands for EMIR Pre-processing](#)
- [Commands for EMIR Analysis Post-processing](#)
- [Commands for Signal Net EM Post-processing](#)

Commands for EMIR Pre-processing

This section describes the following commands:

- [set_ra_functional_resistor](#)
- [set_ra_net](#)
- [set_ra_net_type](#)
- [set_ra_option](#)
- [set_ra_pwnet_driver](#)
- [set_ra_pwnet_option](#)

set_ra_functional_resistor

Description

Specifies the model name for the design resistors to be processed by the electromigration and voltage drop (EMIR) analysis flow.

When you specify the model name of the resistors, the electromigration and voltage drop (EMIR)flow automatically creates a virtual SPF net to store the design resistors, generate the current through those resistors, and propagate the current information to Phase II simulation.

The following rules apply to the `set_ra_functional_resistor` command.

You can specify only one model name per command.

- You can use one or more `set_ra_functional_resistor` commands to identify design resistors with a different model name.
- PrimeSim EMIR analysis requires a back-annotation setup. For EM analysis on design resistors, the same requirement applies.

Tcl procedure requirements for functional resistors are the same as for metals and vias. The only difference is that the resistor length is accessed through the RATCL variable `hs_rlength`.

```
set_ra_functional_resistor -res_model model_name
[-layer layer_name]
[-map_w width_name_in_netlist] [-map_l length_name_in_netlist]
[-port_names port1_name port2_name]
```

Arguments

Table 14 Arguments

Argument	Description
<code>-res_model model_name</code>	Specifies either a regular model or macro model. All design resistors that match the <code>model_name</code> are processed for reliability analysis.
<code>-layer layer_name</code>	Assigns a layer name for the design resistors. The default is to use the <code>-res_modelname</code> as the layer name. All design resistors are assumed to have the same layer name.
<code>-map_w width_name_in_netlist</code>	Defines the instance parameter for the resistor width. The default name is <code>w</code> .
<code>-map_l length_name_in_netlist</code>	Defines the instance parameter for the resistor length. The default name is <code>l</code> .
<code>-port_names port1_name port2_name</code>	By default the tool automatically takes the first two functional resistor terminals/ports to calculate the correct resistor size. You can specify this argument to define specific port names that take precedence over automatic port index recognition.

Examples

Runs a signal net reliability analysis (SIGRA) simulation that includes a design resistor with the model name `rhsim_m`.

```
set_ra_option -sigra 1
set_ra_function_resistor-res_model
rhsim_m load_ba_file -file spf_file
```

Runs a SIGRA simulation that includes design resistors with model names `rhsim_m` and `rhsim_p`.

```
set_ra_option -sigra 1
set_ra_functional_resistor-res_model
rhsim_m set_ra_functional_resistor -
res_model rhsim_p load_ba_file -file spf_file
```

Runs a SIGRA simulation that includes a design resistor with model name `rhsim_m` and a particular layer name, `design_res`, instead of the default layer name.

```
set_ra_option -sigra 1
set_ra_function_resistor -res_model rhsim_m -layer
design_res load_ba_file -file spf_file
```

Runs a SIGRA simulation that includes a design resistor with the model name `rhsim_m` and `rhsim_p`. The netlist uses `wp` and `wn` to represent width for the corresponding resistors and `ln` and `lp` to represent length.

```
set_ra_option -sigra 1
set_ra_function_resistor -res_model rhsim_m -map_w wn -
map_l ln set_ra_function_resistor -res_model rhsim_p -
map_w wp -map_l lp load_ba_file -file spf_file
```

set_ra_net

Specifies the nets for electromigration and voltage drop (EMIR) analysis flow.

Syntax

```
set_ra_net [-selectnet net_names] [-skipnet net_names]
```

Arguments

Table 15 Arguments

Argument	Description
<code>-selectnet net_names</code>	Provides a list of net names to be analyzed in EMIR flow. You must specify the full hierarchical BA name, not the schematic one. The default is to consider all back-annotated nets for EMIR flow. When selecting nets for EMIR analysis, you can specify both explicit names or use wildcard characters. The net name must be based on the back-annotation net name, not the schematic name.
<code>-skipnet net_names</code>	Provides a list of EMIR net names to be skipped for the EMIR flow. Wildcard name matching is allowed. If net names are matched by both the <code>-selectnet</code> and <code>-skipnet</code> options, <code>-skipnet</code> has a higher priority. By default no nets are skipped.

Examples

Selects only the net that matches `vdd`.

```
set_ra_net -selectnet vdd*
```

Selects all nets, but skips `vdd` and `gnd` nets.

```
set_ra_net -selectnet * -skipnet vdd gnd
```

set_ra_net_type

Description

Specifies the net types for power net reliability analysis (PWRA) and signal net reliability analysis (SIGRA) during PrimeSim electromigration and voltage drop (EMIR) flow.

The `set_ra_net_type` command selects the type of net that goes into the EMIR flow. There are only two types of nets that can go into EMIR flow: power nets or signal nets. If a net is selected as both power and signal, then the power selection takes precedence.

When you do not specify a `set_ra_net_type` command, the PrimeSim XA tool uses internal ID methods to identify net types. When you specify one or more `set_ra_net_type` commands, the automatic identification is disabled, allowing you to control net types manually.

You can specify multiple `set_ra_net_type` commands. Only nets that are selected by `-pwnet` or `-signet` are selected for the EMIR; that is, the `set_ra_option -pwra 1` command triggers power net reliability analysis (PWRA) and the `set_ra_option-sigra 1` command triggers signal net reliability analysis (SIGRA)).

Syntax

```
set_ra_net_type [-pwnet net_names]
                [-signet net_names]
```

Arguments

Table 16 Arguments

Argument	Description
<code>-pwnet net_names</code>	Provides a list of power net names to be analyzed by the EMIR flow. You must specify full hierarchical BA net names, not the schematic ones. Wildcard name matching is supported. By default the net type is determined automatically in the PrimeSim XA tool. When selecting PWRA nets, you can specify both explicit names or use wildcard characters. The net name must be based on the back-annotation net name, not the schematic name.
<code>-signet net_names</code>	Provides a list of signal net names to be analyzed by the EMIR flow. You must specify full hierarchical BA net names, not the schematic ones. Wildcard name matching is supported. By default the net type is determined automatically in the PrimeSim XA tool. When selecting SIGRA nets, you can specify both explicit names or use wildcard characters. The net name must be based on the back-annotation net name, not the schematic name.

Examples

Defines `vdd` and `vddhd` as power nets.

```
set_ra_net_type-pwnet vdd vddhd
```

set_ra_option

Description

Activates PrimeSim electromigration and voltage drop (EMIR) flow and sets the options for phase I and phase II simulations.

```
set_ra_option [-pwra enable_value] [-sigra enable_value]
[-waveform_split size]
[-rap2auto enable_value] [-ratau ratau_value]
[-ratcl ratcl_name1 ratcl_name2 ...]
[-use_def_ratcl enable_value]
[-expand_def_ratcl enable_value]
[-twindow time_points]
[-presimtime time]
[-tmpdir directory_name] [-rmin value]
[-sigra_filter_run 0|1] [-bulk2c enable_value] [-pp processes]
[-pp_config cdpl_config_file] [-pp_nets net_names]
[-pp_transient fractionfraction|0|1] [-dp processes]
[-dpconfig cdpl_config_file] [-substrate2c enable_value]
[-substrate2c_itf_layer_name list_of_layers] [-substrate2c_layer_name
list_of_layers]
[-select_subckt subckt_name]
[-select_inst instance_name]
```

Arguments

Table 17 Arguments

Argument	Description
-pwra enable_value	Activates the PrimeSim power net EMIR analysis flow. The default is 0.
-sigra enable_value	Activates the PrimeSim signal net EMIR analysis flow. The default is 0.
-waveform_split size	Sets the split size for the .rasim file. The default split size is 2G, in unit of bytes. If the .rasim file size exceeds this threshold, then additional .rasim files are created with names .rasim#, such as .rasim1, .rasim2 and so on. Note: Because the tool compresses the split waveform file, the size of the split waveform file might become smaller after splitting.
-rap2auto enable_value	Launches the phase II simulation run automatically. The default is 1. When you set this option to 0, the phase I simulation stops after the .rasim file is generated.

Table 17 Arguments (Continued)

Argument	Description
<code>-ratau ratau_value</code>	Sets the RATAU (simulation step size) value for Phase II simulation. The default is 10ps.
<code>-ratcl ratcl_name1 ratcl_name2 ...</code>	Reads the specified <code>ra_tcl</code> file with instructions for phase II simulation. You can specify multiple <code>ra_tcl</code> files.
<code>-use_def_ratcl enable_value</code>	Uses default <code>ratcl</code> file in EMIR analysis. The default is 0. When it is set, tool will automatically include specific files "primesim_emir_{flow}.ratcl" with the order of priority in several locations. Order of priority from the lowest to the highest is 1.Install directory (\$PRIMESIM_EMIR_HOME/etc/def_ratcl/primesim_emir_{flow}.ratcl) 2.Project ENV directory (\$PRIMESIM_EMIR_PROJ_HOME/primesim_emir_{flow}.ratcl) 3.User home directory (\$user/primesim_emir_{flow}.ratcl) 4.Current working directory (./primesim_emir_{flow}.ratcl) Priority: 1 st working directory 2 nd User Home directory 3 rd Project ENV directory 4 th Install directory. Project ENV Directory is user defined environment variable.
<code>-expand_def_ratcl enable_value</code>	The default is 1. All commands are copied into target file. When it is 0, all commands are copied into target file as comments.The "source" command is used to execute the file.
<code>-twindow time_points</code>	Specifies to generate <code>.rasim</code> file data only within timewindows defined by: <code>-twindow tstart tstop {tstart tstop} [tstart]</code> By default, the tool uses <code>tstart=0ns</code> and <code>tstop=end_of_transient</code> as the window. When you use the <code>-twindow</code> option, if you want a similar window to be used during phase II of reliability analysis, you must use the <code>swin</code> command in the <code>.ratcl</code> file. For more information on the <code>swin</code> command, see the PrimeSim™ EMIR Reference Manual.

Table 17 Arguments (Continued)

Argument	Description
<code>-presimtime time</code>	Sets a pre-simulation time for phase II simulation to start earlier than time windows that are specified by the <code>-twindow</code> option. This allows current values on parasitic network to settle down before the actual simulation. Default is 1 ns. Example: <pre>-twindow500n 600n -presimtime 10n</pre> In the example, phase II simulation starts at 490 ns (that is, 10 ns earlier than 500 ns) and uses currents that are saved between 490 ns to 500 ns for simulation which starts at 500 ns.
<code>-tmpdir directory_name</code>	Specifies the directory name to store temporary files. The default is the current directory.
<code>-rmin value</code>	Keeps resistors larger than the specified value in phase II simulation. The unit is Ohm. The default is 0.
<code>-sigra_filter_run0 1</code>	If set to 1, runs a pre-phase I signal net EMIR run when the tool only executes filtering of signal nets without saving the device currents in <code>.rasim</code> files. This run produces a filtered list of nets to be analyzed, based on the <code>iavmin</code> command. The device currents are saved only for those nets. The default is 0. For more information about this command, see the Running Pre-Phase I Net Filtering section in the PrimeSim EMIR Reference Manual.
<code>-bulk2c enable_value</code>	When <code>-bulk2c</code> is set to 0 (default), bulk currents are collected for Phase II simulation. When set to 1, bulk currents are not collected for Phase II.
<code>-pp processes</code>	Specifies the number of processes to run for power/signal net analysis in phase I. For details, see the EMIR Pipeline Processing section in the PrimeSim EMIR Reference Manual.
<code>-pp_config cdpl_config_file</code>	Specifies the CDPL hosts configuration file for power/signal net analysis in phase 1. For details, see the EMIR Pipeline Processing section in the PrimeSim EMIR Reference Manual. See the Common DP Library User Guide for the <code>cdpl_config_file</code> syntax.
<code>-pp_nets net_names</code>	Specifies the power/signal net names to process. You cannot use wildcards in the names. For details, see the EMIR Pipeline Processing section in the PrimeSim EMIR Reference Manual.

Table 17 Arguments (Continued)

Argument	Description
<code>-pp_transient_fraction fraction 0 1</code>	Specifies when to split the <code>rasim</code> files for EMIR pipeline processing. The <code>rasim</code> files are saved when the fraction of the transient analysis is finished. You should set this argument should in the [0,1] range. It is used in pipeline processing to eliminate waiting until the ending of transient analysis in phase 1.
<code>-dp_processes</code>	Specifies the number of processes to run for power/signalnet EMIR analysis in parallel in phase II. For details, see the EMIR Pipeline Processing section in the PrimeSimEMIR Reference Manual.
<code>-dpconfig cdpl_config_file</code>	Specifies the CDPL hosts configuration file for power/signal net analysis in phase II. For details, see the EMIR Pipeline Processing section in the PrimeSimEMIR Reference Manual. See Common DPLibrary User Guide for the <code>cdpl_config_file</code> syntax.
<code>-substrate2c enable_value</code>	When set to 1, <code>.rasim</code> currents are not collected from devices in the layers that are specified by the <code>-substrate2c_itf_layer_name</code> or <code>-substrate2c_layer_name</code> option. The capacitance of these devices are always reported in the <code>.dtdcap</code> file.
<code>-substrate2c_itf_layer_name list_of_layers</code>	Specifies the list of substrate ITF layers whose device currents are ignored. Use this option when the <code>-substrate2c</code> option is enabled. The <code>-substrate2c_itf_layer_name</code> and <code>-substrate2c_layer_name</code> options are mutually exclusive. You should specify either one based on whatever layer name is known or more convenient to specify.
<code>-substrate2c_layer_name list_of_layers</code>	Specifies the list of substrate layers whose device currents are ignored. Use this option when the <code>-substrate2c</code> option is enabled. The <code>-substrate2c_itf_layer_name</code> and <code>-substrate2c_layer_name</code> options are mutually exclusive. You should specify either one based on whatever layer name is known or more convenient to specify.

Table 17 Arguments (Continued)

Argument	Description
<code>-select_subckt subckt_name</code> <code>-select_inst instance_name</code>	Specifies which subckt or instance is analyzed. Simulation setup involves multiple SPF subcircuits or instances. To analyze nets from only one instance of all those instances/subcircuits. Either <code>-select_subckt</code> or <code>-select_inst</code> options are allowed in one setup, EMIR displays error if both are specified. If <code>-select_inst</code> is specified, EMIR only analyzes the instance from the command. If <code>-select_subckt</code> is specified, EMIR checks how many instances of the subcircuit are instantiated: • Only one instance - EMIR analyzes instance. • More than one instance - EMIR displays an error.

Examples

The following example runs the power net EMIR analysis flow.

```
set_ra_option-pwra 1 -ratcl ratcl_file
```

The following example runs both power net and signal net EMIR analysis flows.

```
set_ra_option-pwra 1 -sigra 1 -ratcl ratcl_file
```

The following example runs both power net and signal net EMIR analysis flows and stops after phase I completes.

```
set_ra_option-pwra 1 -sigra 1 -ratcl ratcl_file -rap2auto 0
```

The following example collects `.rasim` data(currents and voltages) in the time interval 5-10ns and from 20ns until the end of transient simulation.

```
set_ra_option-twindow 5n 10n 20n
```

In the following example a maximum four pipeline process run concurrently for the `vdd`, `vddmi`, `vddhd`, and `vss` nets.

```
set_ra_option-pwra 1 -ratcl ra.tcl -pp 4 -pp_nets vdd vddmi vddhd vss
```

In the following example, only the `vdd` and `vss` nets are selected for power net reliability analysis. As a result, there is not any pipeline processing on the `vddmi` net.

```
set_ra_option-pwra 1 -ratcl ra.tcl -pp 3 -pp_nets vdd vss  
vddmi set_ra_net -selectnet vdd vss
```

In the following example there are at most four nets running concurrently in the phase II simulation.

```
set_ra_option-pwra 1 -ratcl ra.tcl -dp 4
```

The following example specifies the `-pp` and `-dp` arguments. As a result, there are a maximum of three pipeline process running concurrently in phase 1 and three distributed processes running in phase II.

```
set_ra_option-pwra 1 -ratcl ra.tcl -pp 3 -pp_nets vdd vddmi vss -dp 3
```

In the following example the `vss` net is skipped because `-skipnet` has priority over the `-selectnet` argument. As a result, only the `vdd` and `vddmi` nets are selected for power net reliability analysis. Even if `-pp` is set to four, there are at most two pipeline processes in phase 1.

```
set_ra_option-pwra 1 -ratcl ra.tcl -pp 4 -pp_nets vdd vss vddmi  
vddai set_ra_net -selectnet vdd vss vddmi set_ra_option-pwra 1 -ratcl  
ra.tcl -pp 4 -pp_nets vdd vss vddmi vddai  
set_ra_net -selectnet vdd vss vddmi set_ra_net-skipnet vss
```

In the following example, `.rasim` currents is not collected from devices in the layer tap. The capacitance values of these devices is always reported in the `.dtpcap` file.

```
set_ra_option -substrate2c 1 -substrate2c_layer_name tap
```

In the following example, subcircuit SUB1 is instantiated twice and there are two instances X1 and X2 under top hierarchy XTOP. EMIR uses SPF for selected subcircuit (SUB1) and produce or simulate results for the only selected instance(X1) of that subcircuit. EMIR file names do not change.

```
set_ra_option -select_inst XTOP.X1
```

In the following example, EMIR displays error because SUB1 has two instances.

```
set_ra_option -select_subckt SUB1
```

set_ra_pwnet_driver

Description

Provides the driver pin information for the internal power net.

By default, driver instance pins are found automatically by the PrimeSim XA tool. An output file `.pw_driver` is created that contains information about internal power nets and their drivers. If you specify the `set_ra_pwnet_driver` command, automatic detection is turned off for this net. You can specify multiple `set_ra_pwnet_driver` commands for the same net.

If the net has net pins (*|P), you should delete those pins if you want only the specified driversto be used. A solution to delete the unwanted pins is to use the `-delete_netpin` option in the `load_ba_file` command.

Syntax

```
set_ra_pwnet_driver -net net_name
-driver_ipname instance_pin_names
[-exclude_netpins 0|1]
```

Arguments

Table 18 Arguments

Argument	Description
-net net_name	Provides the back-annotated net name of an internal power net. You can specify only one net name per <code>set_ra_pwnet_driver</code> command. Note:Wildcard characters are not supported.
-driver_ipname instance_pin_names	Provides driver instance pins for the specified internal power net. Wildcard characters are supported.
-exclude_netpins0 1	Specify one of the following values: • 0(default) where net pins are considered as drivers. • 1 specifies to convert all the net pins into loads instead of drivers.

Examples

Defines the driver pin of the `vss` net.

```
set_ra_pwnet_driver-net vss -driver_ipname xx/xx/clk2@2:D
```

Defines the driver pins of the `vreg` net using wildcards.

```
set_ra_pwnet_driver-net vreg -driver_ipname X_op_reg.Xdrv<*>.MNout*"
```

set_ra_pwnet_option

Description

Allows you to define the reference voltage and source external power supply node for the internal power net.

By default the PrimeSim XA tool automatically identifies internal power nodes and selects appropriate reference voltages and source nodes for them. An output file `.pw_driver` is created that contains information about internal power nets and their drivers.

You can override the automatic identification of `vref` and `source_pwnet_node` with the `set_ra_pwnet_option` command. If you specify `source_pwnet_node`, the automatic driver ID selects only those drivers that connect to the specified node.

Syntax

```
set_ra_pwnet_option -net net_name
[-vref vref]
[-src source_pwnet_node]
```

Arguments

Table 19 Arguments

Argument	Description
<code>-net net_name</code>	Provides the back-annotated net name of an internal power net. You can specify only one net name per <code>set_ra_pwnet_option</code> command. Note: Wildcard characters are not supported.
<code>-vref vref</code>	Specifies the reference voltage.
<code>-src source_pwnet_node</code>	Specifies the source power net node (its netlist name) that is supplying power into the internal power net. This is a hierarchical node name starting from the top level of the netlist. Wildcard characters are not supported.

Examples

Defines the source node and reference voltage of the `vdd_int` net.

```
set_ra_pwnet_option-net vdd_int -src vdd -vref 0.9
```

Commands for EMIR Analysis Post-processing

This section describes the following commands:

- [addnetpinxy](#)
- [alterpad](#)
- [analysisname](#)
- [asciicols](#)

Chapter 9: EMIR Commands

Commands for EMIR Analysis Post-processing

- [asciicolsort](#)
- [asciidigits](#)
- [asciimapi](#)
- [asciimapvmax](#)
- [avgrelaxf](#)
- [ccdbcols](#)
- [ccdbmerge](#)
- [coordunit](#)
- [dtemp](#)
- [emiroptions](#)
- [emldalg](#)
- [emldincdang](#)
- [emthreshproc](#), [emldlayers](#)
- [emwidthflt](#)
- [gdsdatatype](#)
- [gdsdefrw](#)
- [gdsfilechsymb](#)
- [gdslabels](#)
- [gdsmag](#)
- [gdsmapi](#)
- [gdsmapvmax](#)
- [gdsoutmode](#)
- [genspfview](#)
- [gdsprops](#)
- [itflayernames](#)
- [jjmaxlog](#)
- [layerea](#)

Chapter 9: EMIR Commands

Commands for EMIR Analysis Post-processing

- [layerh](#)
- [layermap](#)
- [netfrequency](#)
- [nettype](#)
- [outformat](#)
- [outfiltres](#)
- [outlayers](#)
- [postnets](#)
- [powernet](#)
- [presimtime](#)
- [printi](#)
- [printipad](#)
- [printipin](#)
- [printpwrpinv](#)
- [printv](#)
- [printvmode](#)
- [pwnetdriver](#)
- [pwrtype](#)
- [ra](#)
- [raformat](#)
- [ralayers](#)
- [ralayout](#)
- [rarve](#)
- [raviewer](#)
- [rawf](#)
- [redi](#)
- [redia](#)

Chapter 9: EMIR Commands

Commands for EMIR Analysis Post-processing

- [rediac](#)
- [redimaxa](#), [redirmsa](#), [rediabsa](#), [rediavga](#)
- [redimaxw](#), [redirmsw](#), [rediabsw](#), [rediavgw](#)
- [rediw](#)
- [redj](#)
- [redv](#)
- [redtrms](#)
- [replace_waveform](#)
- [reportirdrop](#)
- [rvemapi](#)
- [rvemapvmax](#)
- [rveprops](#)
- [signetascii](#)
- [simnets](#)
- [skiplayers](#)
- [source](#)
- [spresenable](#)
- [swin](#)
- [tau](#)
- [temp](#)
- [tmpdir](#)
- [tstart](#)
- [tstop](#)
- [twin](#)
- [usecursign](#)
- [usew4siw](#)
- [vref](#)

- [vts](#)
- [vtsopt](#)

addnetpinxy

Adds a pad based on the XY coordinate. The `x=` and `y=` keywords must be in lowercase.

```
addnetpinxy <net> x=<xcoord> y=<ycoord> layer=<mlayer> pinxydist=<xydist>
```

For example:

```
addnetpinxy VDD x=495u y=45u layer=M1 pinxydist=1u
```

Looks for a subnode at (495u 45u) with a radius of 1u. A subnode found in this range is converted to a pad.

alterpad

```
alterpad <move|add> dspf_pad dspf_net
```

`alterpad` specifies the new pad connections during second phase simulation. It is used for “what-if” analysis.

Syntax Definitions

`<move|add>`

Specifies whether you want to move an existing pad to another location or add a pad.

`dspf_pad`

Specifies the original/new pad name

`dspf_net`

Specifies the node you want to move to the original/new pad.

To move an existing pad in a Phase II simulation, `dspf_pad` must be a pad, and `dspf_net` must not be a pad. For example:

```
P (VDD B 0 -516.05 46.95) //$llx=-516 $lly=43.5 $urx=-515.95
  $ury=50.5 $lvl=14
*|S (VDD:276 -310.5 47.5 // $llx=-310.5 $lly=43.5 *$urx=-308.5
  $ury=50.5 $lvl=2)
```

To delete VDD as net pin and make VDD:276 a pad, use the following command:

```
alterpad move x1.vdd x1.VDD:276
```

To add a pad in a Phase II simulation, `dspf_pad` must be a pad, and `dspf_net` must not be a pad. For example, to make x1.VDD:276 a pad, specify:

```
alterpad add x1.vdd x1.VDD:276
```

```
alterpad move vdd vdd:4
```

The PrimeSim EMIR tool performs move of the original pad vdd to the node vdd:4.

```
alterpad add vdd xa1/mp:s
```

The PrimeSim EMIR tool adds a new pad to the instance pin xa1/mp:s.

analysisname

```
analysisname <new_name>@<default_name>
```

Use this command to write a different analysis name than the default in the output Custom Compiler database.

Examples

The following example writes out the average analysis result in the name "dcem" rather than "iavg" in the Custom Compiler database.

```
analysisname dcem@iavg
```

asciicols

```
asciicols < ird | em > {<column name>}
```

The `asciicols` command adds a column and specifies which columns, and in what sequence, should be listed in an ASCII file. The first parameter specifies the type of analysis: `ird` (IR drop) and `em` (electromigration).

```
ird | em
```

Specifies the table type to add to the ASCII file.

- `ird`: Adds a table for voltage drop analysis results with any of the following columns. By default, the data in the table is sorted by the dV value in descending order.

Column Name	Description
nodename	Node name
dV	Voltage drop value
layer	Original layer name/number
x	X coordinate.
y	Y coordinate.
x2	X coordinate of the resistor's second node
y2	Y coordinate of the resistor's second node
resname	Resistor name
time	Time of the IR drop occurrence
volt	Actual voltage value at dV
level	Level of violation
vmax	Maximum voltage value
vmin	Minimum voltage value
vavg	Average voltage value
dVavg	Average voltage drop value
time_vmax	Time of the maximum voltage occurrence
time_vmin	Time of the minimum voltage occurrence
dVperc	dV (IR drop) value in percentage of the base voltage value. The EMIR analyzer calculates and outputs the dV (IR drop) value in percentage of the base voltage value, using the following equation: $(dV_value / base_voltage_value) * 100$ If <code>base_voltage_value</code> is not specified with the <code>vref -base</code> command, the tool uses <code>reference_voltage_value</code> instead.

Column Name	Description
eflow	Electron flow direction. When specified, current direction for each resistor connected to the reported node is written to the output IR drop reports (IAVG analysis only). For vias layer, the <code>eflow</code> column contains U or D for direction up or down. For metal layers, the <code>eflow</code> column contains N, S, E or W for direction north, south, east or west. To report the e-flow information, the average current flowing through all resistors needs to be calculated during IR drop analysis. If IAVG is not specified during IR drop (VMAX) analysis but the <code>eflow</code> (or <code>all</code>) attribute is specified, the tool issues a warning message and does not report the eflow information in the IR drop result files. A valid e-flow reporting requires a proper layer order specified by the <code>emldlayers</code> command. Otherwise, a warning message will be issued. The syntax is: <pre>emldlayers {-m -v} {<layer name>}</pre> List the layer names from low to high in the structure. For example: <pre>emldlayers -m m1 m2 m3 m4</pre> or <pre>emldlayers -v v1 v2 v3</pre>
reff	Effective resistance values for nodes for power or ground nets.

- `em`: Adds a table for electromigration analysis results with any of the following columns.

Column Name	Description
resname	Resistor name
current	Current through resistor in the unit of uA
curdns	Current density through the resistor in the unit of uA/um. It is recommended not to include the <code>curdns</code> column when taking the Tcl-procedure approach.
j_jmax	Ratio J/Jmax. Jmax is a user-specified threshold for the layer, or the maximum J value calculated if the threshold was not set. It is recommended not to include the <code>j_jmax</code> column when taking the Tcl-procedure approach.
layer	Original layer name/number
x	X coordinate

Column Name	Description
y	Y coordinate
x2	X coordinate of the resistor's second node
y2	Y coordinate of the resistor's second node
width	Resistor width in the unit of um
length	Resistor length in the unit of um
area	Resistor area in the unit of um2
time	Time when the maximum current occurred (IMAX analysis only)
level	Level of violation
emlen	Wire length in the unit of um
eflow	Electron flow direction (IAVG analysis only). See the <code>eflow</code> description in the <code>ird</code> table above for more information.
threshold	Threshold for current through resistor
i_imax	Same as I/Imax. Imax is calculated by user-specified Tcl procedure.
area_new	Recommended area for the resistor in the unit of um2
emwirelen1	Length of wire connected to the first VIA node
emwirelen2	Length of wire connected to the second VIA node
emwirewidth1	Width of wire connected to the first VIA node
emwirewidth2	Width of wire connected to the second VIA node
netname	Net name of the signal nets
pulse_width	Pulse width of a periodic signal
signal_period	Period of a periodic signal
dratio	Pulse to period ratio
nvias	Number of vias
apw	The highest average, RMS or peak current time window. This is used to print the time window where the highest value of average, RMS or peak current is observed when the <code>swin</code> command is used with <code>-apw</code> option.

asciicolsort

```
asciicolsort <ird|em> [<[+]|-><name>|<[+]|-><numb>]
```

The `asciicolsort` command specifies the number and sequence of columns used as sorting keys for sorting ASCII files. Columns used for sorting specifications must be specified in `asciicols`. Column names and numbers are the same as in `asciicols`. The sign controls the type of sorting as follows:

- + (plus): Sorts in ascending order. This is the default if the sign is not specified.
- - (minus): Sorts in descending order.

Example

The following syntax generates and sorts an ASCII file for IR drop analysis. Time is sorted in descending order and voltage drop (dv) in ascending order.

```
asciicols ird dv time layer nodename  
asciicolsort ird -dv time
```

The output from the above syntax appears as follows:

IRDrop (mV)	Time (ns)	Layer	Node	name
2.254	2.9ns		6	vdd:385
2.253	0.2ns	3	vdd:161	
2.253	0.2ns	3	vdd:162	
2.253	2.9ns	3	vdd:182	
2.218	2.9ns	6	vdd:387	

asciidigits

```
asciidigits <num_of_digits_after_decimal_point>
```

The `asciidigits` command sets the number of digits after the decimal point for all the columns to be printed in the ASCII output file.

Example:

The following example sets the ASCII file precision (number of digits after the decimal point) to 5.

```
asciidigits 5
```

asciimapi

```
asciimapi {max | avg | rms | abs} [-pwra|-sigra] [-l] \  
  [<default_val>] <val_1>@<layer_1> <val_2>@<layer_2> | \  
  {-jjmax | -iimax} <val>}
```

The `asciimapi` command specifies the filter for the number of resistors to be stored in the ASCII output file as a result of any of the following EM analyses: `imax`, `irms`, `iavg`, or `iabs`. This command supports two types of filtering: by value of current/current density value or by level of violation. For a Tcl-procedure approach, The `asciimapi` command specifies the filtering by current value. For the previous approach with a user-defined threshold value, the `asciimapi` command specifies the filtering by current density. To request filtering by level of violation, the `-l` parameter must precede all other parameters. In this case, `<default_val>` and all other `<val_i>` must be integers that specify the number of mostly violated levels to be output into ASCII file. In case of filtering by value, `<default_val>` and all `<val_i>` must be in uA/um and specify the minimum value of the resistor current density to be output into the ASCII file.

Each layer can have its own “`<val>@<layer>`” threshold specification, where `<layer>` is layer name or number. If `<default_val>` is specified, layers without individual specifications are treated as if they have this `<default_val>` value. If `<default_val>` is not specified and a layer does not have its own specification, all resistors belonging to this layer are listed in the output file. If the `asciimapi` command is skipped, power net EMIR analysis stores all resistors.

The `-jjmax` or `-iimax` options specify filtration of output according to value of ratio `J/Jmax` or `I/Imax`. So only violations with ratios larger than `<val>` are output to the ASCII file. You can use the `-jjmax` or `-iimax` options only if the threshold is specified by the `redj`, `redi` commands or by Tcl procedures. For example:

```
asciimapiavg -iimax 0.9
```

This command filter out all violations with `I/Imax` less than 0.9.

`-pwra`

Executes Tcl for power net analysis only. Default is Tcl execution for both power and signal analyses.

`-sigra`

Executes Tcl for signal net analysis only. Default is Tcl execution for both power and signal analyses.

asciimapvmax

```
asciimapvmax [-l] <val>
```

The `asciimapvmax` command specifies the filter for the number of nodes to be stored in the ASCII output file resulting from IR drop analysis (vmax). This command supports two types of filtering: by IR drop value or by level of violation. To request filtering by level of violation, the `-l` parameter must precede `<val>`. In this case, `<val>` must be an integer specifying the number of mostly violated levels to be output into the ASCII file. In case of filtering by value, `<val>` must be in volts specifying minimum value of the node's IR drop to be output into the ASCII file. If this command is skipped, power net EMIR analysis stores all nodes.

avgrelaxf

```
avgrelaxf <val> {<layer name> | <layer pattern>}
```

The `avgrelaxf` command specifies the relaxation coefficients for *avg* threshold values. You can use the coefficient when your design requirement is different from the DRM manual to address the length of life cycle. The default coefficient value is 1.

For example, if the DRM provides currents limits for a ten year lifetime, and the design requires seven years, the relaxation coefficient can be calculated to increase the current limits:

```
avgrelaxf 1.125 ap ru  
avgrelaxf 1.032 r2_m
```

Different layers (material properties) may require different coefficient values.

Note:

The `avgrelaxf` command does not adjust the *avg* threshold values associated with `redia` and `rediw` commands, but does with user-written procedures.

Syntax

`val`

Specifies the coefficient value.

`layer name | layer pattern`

Specifies the layer name or pattern.

ccdbcols

```
ccdbcols [em|ird|sh] {<column name>}
```

The `ccdbcols` command is a Tcl command that adds the specified columns to a `.cc.db` file, as well as the sequence of columns. The first parameter specifies the type of analysis: `ird` (IR drop), `em` (electromigration), and `sh` (shapes).

Not all tables and all columns are controllable. Some columns, such as ids or coordinates, are not in the list of column names. The list of column names is preceded by a flag for which content it is applied.

Note:

Adding IR drop and electromigration (EM) tables to the output .cc.db file with the `ccdbcols` command is not supported in the static power net resistance calculation flow.

Syntax

```
ccdbcols [ em | ird | sh ] {<column name>}
```

```
em | ird | sh
```

Specifies the table type to add to the .cc.db file.

- `em`: Adds an EM table for EM analysis results with any of the following columns:

Column Name	Description
current	Current through resistor (uA)
curdns	Current density (uA/um)
i_imax	I/Imax
j_jmax	J/Jmax
threshold	Imax from procedure or Jmax
time	Time when the maximum current occurred (IMAX analysis only)
emlen	Wire length
width_new	Recommended width for resistor
area_new	Recommended area for resistor
emwirelen1	Length of wire connected to the first VIA node
emwirelen2	Length of wire connected to the second VIA node
emwirewidth1	Width of wire connected to the first VIA node
emwirewidth2	Width of wire connected to the second VIA node
pulse_width	Pulse width of periodic signal (ACPC analysis only)
signal_period	Period of periodic signal (ACPC analysis only)

Column Name	Description
dratio	Pulse to period ratio (ACPC analysis only)
eflow	Electron flow direction. When specified, current direction for each resistor connected to the reported node is written to the output IR drop reports (IAVG analysis only). To report the e-flow information, the average current flowing through all resistors needs to be calculated during IR drop analysis. If IAVG is not specified during IR drop (VMAX) analysis but the <code>eflow</code> (or <code>all</code>) attribute is specified, the tool issues a warning message and does not report the eflow information in the IR drop result files. A valid e-flow reporting requires a proper layer order specified by the <code>emldlayers</code> command. Otherwise, a warning message will be issued. The syntax is: <pre>emldlayers {-m -v} {<layer name>}</pre> List the layer names from low to high in the structure. For example: <pre>emldlayers -m m1 m2 m3 m4</pre> or <pre>emldlayers -v v1 v2 v3</pre>
dtj	dT joule (IAVG and IRMS analyses)
dtcpl	dT coupling (IAVG and IRMS analyses)

- `ird`: Adds an IR drop table for voltage drop analysis results with any of the following columns:

Column Name	Description
dV	Maximum IR drop at one of resistor nodes
voltage	Maximum actual voltage at one of resistor nodes
time	Time when max IR drop occurred
node_name	Resistor node name with maximum IR drop
vmax	Maximum voltage value
vmin	Minimum voltage value
vavg	Average voltage value
dVavg	Average voltage drop value
time_vmax	Time when maximum voltage occurred

Column Name	Description
time_vmin	Time when minimum voltage occurred
eflow	Electron flow direction (IAVG analysis only). See the <code>elow</code> description in the <code>em</code> table for more information.
dVperc	dV (IR drop) value in percentage of the base voltage value. The EMIR analyzer calculates and outputs the dV (IR drop) value in percentage of the base voltage value, using the following equation: $(dV_value / base_voltage_value) * 100$ If <code>base_voltage_value</code> is not specified with the <code>vref -base</code> command, the tool uses <code>reference_voltage_value</code> instead.
reff	Effective resistance values for nodes for power or ground nets.

- `sh`: Adds a shape table with any of the following columns:

Column Name	Description
resname	Resistor name
resistance	Resistor value
layer	Resistor original layer name
width	Resistor width
area	Resistor area for vias only
length	Resistor length
nodes	Resistor node names

Example

The following example adds an IR drop table with the all available columns in the output CCDB file.

```
ccdbcols ird all
```

ccdbmerge

Works with the `raformat` command to determine how to overwrite or save EMIR analysis results.

Syntax

```
ccdbmerge [0|1] [-remove [0|1]]
```

0

Creates a new .cc.db file (default).

1

Overwrites only reported analyses. Existing analyses stay in the .cc.db file, or are removed depending on the `-remove` option setting.

```
-remove 0|1
```

- 0: Keeps existing analyses in the .cc.db file and does not overwrite them (default).
- 1: Removes existing analyses in the .cc.db file and does not overwrite them, If the .cc.db file does not exist, it is created.

coordunit

The `coordunit` command is used to define the proper unit scale in generating IR drop and EM violation map (VM) in the GDSII format.

```
coordunit [val]
```

By default, power net EMIR analysis treats the process geometry as if it is in 1e-9m. If `coordunit` is not specified, the output geometry might have an incorrect scale. The default `coordunit` value is 1e-6.

Note:

Do not use a SPICE-formatted scaling number, such as 1n, in the `val` field.

dtemp

```
dtemp value
```

Specifies the delta T value used for the RMS threshold calculation. Its value is represented in the `hs_dtemp` variable. The default value if you do not use this command is 0.

In the following example, delta T is set to 5C.

```
dtemp 5
```

emiroptions

```
emiroptions -ascii2csv <1|0> -keep_ascii <1|0>
```

```
emiroptions -ascii2json <1|0> -keep_ascii <1|0> [-ascii2json_format  
<array|hash>]
```

This command defines machine readable output files.

Command	Description
emiroptions	This option allows you to define machine readable output files.
-ascii2csv <1 0>	You can specify the CSV output to choose 1 or 0. If it is set to 1, then PrimeSim EMIR generates CSV output, if it is set to 0, PrimeSim does not generate CSV output. If it is not set, 0 is set as a default.
-ascii2json <1 0>	You can specify the JSON output to choose 1 or 0. If it is set to 1, PrimeSim EMIR generates JSON output, if it is set to 0, PrimeSim does not generate CSV output. If it is not set, 0 is set as a default.
-keep_ascii <1 0>	You can specify the ASCII output to choose 1 or 0. 1 is default PrimeSim EMIR keeps ASCII output, if it is set to 0, PrimeSim removes the ASCII output.
[-ascii2json_for mat <array hash>]	The default is array, if you do not set this option, PrimeSim sets this as an array. You can specify the format to choose hash or array. This option is only available with -ascii2json. If -ascii2json is not set to 1, this option is not valid.

Examples

Example 1 - ASCII and CSV output.

```
emiroptions -ascii2csv 1 -keep_ascii 1
```

PrimeSim EMIR generates output in ASCII and CSV format files.

Example 2 - CSV output only

```
emiroptions -ascii2csv 1 -keep_ascii 0
```

PrimeSim EMIR generates output only in CSV files.

Example 3 - ASCII, CSV, and JSON array format output.

```
emiroptions -ascii2csv 1 -ascii2json 1 -keep_ascii 1
```

PrimeSim EMIR generates output in CSV and JSON array formats and ASCII file.

Example 4 - ASCII, CSV, and JSON hash format.

```
emiroptions -ascii2csv 1 -ascii2json 1 -keep_ascii 1 -ascii2json_format hash
```

PrimeSim EMIR generates output in CSV and JSON hash formats and ASCII file.

Example 5 - ASCII, CSV, and “-ascii2json 0” with “-ascii2json_format hash.

```
emiroptions -ascii2csv 1 -ascii2json 0 -keep_ascii 1 -ascii2json_format hash
```

PrimeSim EMIR generates output in CSV and ASCII file. PrimeSim EMIR shows warning about -ascii2json_format with -ascii2json 0.

Example 6 - ASCII, JSON array format and -ascii2csv 0.

```
emiroptions -ascii2csv 0 -ascii2json 1 -keep_ascii 1 -ascii2json_format hash
```

PrimeSim EMIR generates output in JSON hash format and ASCII file. No CSV file because “-ascii2csv 0” provided.

emldalg

```
emldalg longpath | allsegments | heuristic
```

Specifies the algorithm to use for the wire length calculation. Wire length is calculated based on the longest path between external nodes of the wire. This length value is often referred to as electromigration length (EM length).

- **longpath** specifies that wire length (EM Length) is calculated based on the longest path between any net pins/instance pins/dangling nodes of the wire. This is the default.
- **allsegments** specifies that wire length (EM Length) is calculated based on the sum of all connected wire segments between any net pins/instance pins/dangling nodes of the wire.
- **heuristic** specifies that wire length (EM Length) is calculated based on the longest path between two adjacent net pins/instance pins/dangling nodes/vias of the wire.

Note:

You can use the `heuristic` option for power net analysis to improve performance compared to the default `longpath` option. However the `heuristic` algorithm is not as accurate as `longpath` and might be too aggressive for power net EM calculations and might mask real EM violations.

The algorithms are described in details in the SolvNetPlus article, [“Calculating Metal Length for Electromigration in PrimeSim EMIR Analysis”](#).

During EMIR analysis, the EM length is determined for each resistor under analysis. The resultant value is used to evaluate length related- conditions referred in the technology EM rule set. In the EM rule `$hs_length` procedures, EM length value can be accessed through variable. It does not necessarily correspond to the extracted shape size.

EM length can be output to EMIR analysis result files to provide more visibility on the calculation of maximum allowed current. This is achieved with the following command:

- `asciicols <ird|em> emlen` adds an extra column in the ASCII files with the length value.

emldincdang

You can use this command in a procedure-based EM rule definition. It controls whether to include or exclude dangling metal resistors in the wire length calculation. Default is `yes`.

```
emldincdang <yes|no>
```

emthreshproc, emldlayers

The `emthreshproc` and `emldlayers` commands provide a flexible capability to specify EM rules in signal net EM and power net EMIR to address multiple ways of calculating the threshold values. The PrimeSim EMIR tool calculates the current through the resistor and compares the current against the maximum allowed current, which is the threshold `lth` (not the current density).

The value of `lth` for each resistor is calculated during post-processing through the call to the user-defined EMIR Tcl procedure. This procedure contains the formula/data necessary to calculate the `lth` based on technology specific parameters, as well as the resistor width and/or length attributes.

The `emthreshproc` command assigns procedures to appropriate layers. The procedures are called during post-processing to calculate the `lth` for each resistor. The values from the PrimeSim EMIR tool are passed to the procedures as global parameters. The PrimeSim EMIR tool supports the following parameters:

- `hs_width` for the resistor width
- `hs_length` for the wire length, which is the EM length in the ASCII report, not the length of a single resistor
- `hs_pduration` for the pulse duration (ACPC related)
- `hs_speriod` for the signal period (ACPC related)

- `hs_temp` for the temperature captured from the `temp` or `.temp` settings (related to DC analysis).
- `hs_ipn_neg` for the negative part of the average
- `hs_ipn_pos` for the positive part of the average
- `hs_imax` for the peak current through the resistor
- `hs_irms` for the RMS current through the resistor
- `hs_iabs` for the average current magnitude through the resistor
- `hs_iavg` for the average current through the resistor
- `hs_wwidth1` for the lower wire (metal) width of the via layer
- `hs_wwidth2` for the upper wire (metal) width of the via layer
- `hs_wlength1` for the lower wire (metal) length of the via layer
- `hs_wlength2` for the upper wire (metal) length of the via layer
- `hs_netname` for the resistor net name pattern
- `hs_varray_nx` for the via array number of columns (\$n=NYxNX, maps to NX)
- `hs_varray_ny` for the via array number of rows (\$n=NYxNX, maps to NY)
- `hs_vc` for the via coverage captured from `$vc` in the DSPF
- `hs_area` for the via coverage captured from `$a` in the DSPF
- `hs_resname` for the resistor name captured from the DSPF
- `hs_siwidth` for the silicon width captured from `$si_w` in the DSPF
- `hs_pwnet` for the flag for power net
- `hs_via_count` for the number of vias in the array
- `hs_vdx` for the via x-dimension
- `hs_vdy` for the via y-dimension
- `hs_power_grid` for the flag resistor that belonged to power grid
- `hs_via_array` flag for the via resistor that belonged to the via array
- `hs_eflow_up` for the electron flow direction
- `hs_ispaired` for the flag for S-factor
- `hs_dtemp` for deltaT captured from the `dtemp` setting (related to RMS analysis).

- `hs_msg string` prints a message in the `.ralog` file and to standard output.
- `hs_warn "string"` prints a warning message in the `.ralog` file and to standard output.
- `hs_err "string"` prints an error message in the `.ralog` file and to standard output. The simulation stops and exits after the error message.

When used, the values of these parameters are passed to the procedures as global parameters. The units for these global parameters are microns. The procedures are expected to return the *lth* value in uA.

For some EM rules, it is necessary to calculate the wire length, which is defined differently for metal and via resistors. For metal resistors, the wire length is defined as a maximum path between any two nodes on the wire. For via resistors, the wire length is the maximum wire lengths of wires that connect to the given via resistor. To correctly calculate/assign the wire length to the resistors, the PrimeSim EMIR tool needs to know which layers are metal and which are via layers. You must use the `emldlayers` command to specify the layer types.

```
emthresproc analysis proc_name@layer_name [proc_name@layer_name...]  
emldlayers <-m|-v> layer_name
```

Syntax Definitions

analysis

Specifies the one of the following types of EM analysis: `imax`, `iavg`, `iabs` or `irms`.

proc_name

Name of the user-defined Tcl procedure.

layer_name

Layer name

`-m`

Option to specify metal layer names.

`-v`

Option to specify via layer names.

Example of an EMIR Tcl File

The following EMIR Tcl file contains commands that define the EM rules. Note that the procedures are described in a separate `65nm.ratcl` file.

```
source /path/to/65nm.ratcl  
emldlayers -m almi iaa ipdef -v cxx cww
```

```
emthreshproc imax t65_m1@alml t65_mx@iaa t65_my@ipdef
emthreshproc imax t65_vx@cxx t65_vy@cw
emthreshproc irms t65_irms_m1@alml t65_irms_mx1@iaa t65_irms_mx3@ipdef
```

The following example shows a Tcl file that defines procedures for EM analysis based on the 65nm rules.

```
set UB 20
set LB 5

proc t65_metals {cW cI} {
    global hs_length hs_width UB LB

    if { $hs_length >= $UB } {
        set threshold [expr $cI * ($hs_width - $cW) ]
    } elseif { $hs_length <= $LB } {
        set threshold [expr 4 * $cI * ($hs_width - $cW) ]
    } else {
        set threshold [expr ($UB / $hs_length) * $cI * ($hs_width - $cW) ]
    }
    if {$threshold > 0} {
        set threshold [expr 1000 * $threshold ]
    }
    return $res_threshold
}

proc t65_m1 {} {

    set cW 0.02
    set cI 1.8
    set threshold 0

    set threshold [t65_metals $cW $cI]
    return $threshold
}

proc t65_mx {} {

    set cW 0.02
    set cI 1.9
    set threshold 0

    set threshold [t65_metals $cW $cI]
    return $threshold
}

proc t65_my {} {

    set cW 0.03
    set cI 4.6
    set threshold 0

    set threshold [t65_metals $cW $cI]
```

```
    return $threshold
}

proc t65_mz {} {

set cW 0.03
set cI 10.0

set threshold 0

    set threshold [t65_metals $cW $cI]
    return $threshold
}

proc t65_vias {cI} {
global hs_length hs_width UB LB

    if { $hs_length >= $UB } {
        set threshold $cI
    } elseif { $hs_length <= $LB } {
        set threshold [expr 4 * $cI ]
    } else {
        set threshold [expr ($UB / $hs_length) * $cI ]
    }
    if {$threshold > 0} {
        set threshold [expr 1000 * $threshold ]
    }
    return $res_threshold
}

proc t65_vx {} {

set cI 0.3
set threshold 0

    set threshold [t65_vias $cI]
    return $threshold
}

proc t65_vy {} {

set cI 0.9
set threshold 0

    set threshold [t65_vias $cI]
    return $threshold
}

proc t65_vz {} {

set cI 4.0
set threshold 0
```

```
    set threshold [t65_vias $cI]
    return $threshold
}

#-----  Irms

set dT 5

proc t65_irms_metals {cI cW1 cW2 cW3} {
    global hs_width dT

    set W $hs_width

    set threshold [expr sqrt ($cI * $dT * pow (($W - $cW1), 2) * ($W + $cW2)
    / ($W - $cW1 + $cW3))]
    return $threshold
}

proc t65_irms_m1 {} {
    set cI 20.0
    set cW1 0.02
    set cW2 0.4
    set cW3 0.06

    set threshold [t65_irms_metals $cI $cW1 $cW2 $cW3]
    return $threshold
}

proc t65_irms_mx1 {} {
    set cI 7.0
    set cW1 0.02
    set cW2 0.5
    set cW3 0.06

    set threshold [t65_irms_metals $cI $cW1 $cW2 $cW3]
    return $threshold
}

proc t65_irms_mx2 {} {
    set cI 5.0
    set cW1 0.02
    set cW2 0.9
    set cW3 0.06

    set threshold [t65_irms_metals $cI $cW1 $cW2 $cW3]
    return $threshold
}

proc t65_irms_mx3 {} {
    set cI 3.0
    set cW1 0.02
    set cW2 1.5
    set cW3 0.06
```

```
set threshold [t65_irms_metals $cI $cW1 $cW2 $cW3]
return $threshold
}
```

emwidthflt

Filters out resistors with width smaller than user-specified threshold.

```
emwidthflt {<threshold@<layer>}
```

Syntax Definitions

threshold

Width threshold.

layer

Original layer name.

gdsdatatype

```
gdsdatatype <val>
```

gdsdatatype specifies data types for violation map layers in the GDSII file.

Syntax Definitions

<val>

Any integer between 0 and 256. The default value is 0.

gdsdefrw

```
gdsdefrw <val>
```

gdsdefrw specifies the default resistor width. The value are used for visualization purpose only so, if a resistor has no width, current density is not calculated.

gdsfilechsymb

```
gdsfilechsymb <symb_orig><symb_new>
```

The original symbol in the GDS file is replaced with the new symbol.

Syntax Definitions

`symb_orig`

The original symbol:

`symb_new`

Is replaced with the new symbol:

```
gdsfilechsymb @$
```

In the previous example @ is substituted with \$ in all file names that contain GDSII format.

Note:

There is no space between `<symb_orig>` and `<symb_new>`.

There may be several symbol pairs (`<symb_orig><symb_new>`) in one command. Symbol pairs must be separated by space.

gdslabels

```
gdslabels [all] [1|pad] [2|print] [3|pins] [-layer <number>]
```

`gdslabels` specifies what text labels are generated and put into *.ragds file. By default, no labels are generated.

Syntax Definitions

[1|pad]

All pad names are specified as *|P in DSPF files,

[2|print]

Node names are specified in `printv` command.

[3|pins]

All pin names are specified as *|I in DSPF files.

[all]

Text labels for all objects mentioned in the previous bullets.

[-layer <number>]

Specifies the layer number for label visualization, where <number> is integer between 1 and 255. Note that the layer number should not be in the range of `ralayers`.

gdsomag

```
gdsomag <[+]|-><val>
```

The `gdsomag` command specifies the magnification of shapes within VM layout. All DSPF file coordinates are handled as follows:

Syntax Definitions

+ (plus)

Sorts in ascending order.

- (minus)

Sorts in descending order.

<val> is positive

Multiplies by **<val>**.

<val> is negative

Divides by **<val>**.

gdsmapl

```
gdsmapl {<max|avg|rms|abs>} {-v|-l} [default_val] val1@layer1  
[val2@layer2]}
```

The `gdsmapl <max|avg|rms|abs>` command specifies the filter for the number of resistors stored in the GDSII file as a results of one of the following EM analyses: `imax`, `irms`, `iavg`, or `iab`.

Syntax Definitions

-v

Specifies that the filter threshold is determined by a value, current density (uA/um).

-l

Sets the filter threshold by level.

Resistors are placed into the GDSII file if its current density is greater than the filter threshold value (**-v**) or if it belongs to the level satisfied by the following condition: $(10 - \text{level}) < \text{threshold_level} (-1)$. Each original layer may have its own `val@layer` threshold specification. If `default_val` is set, all layers without their own specification are treated as if they have the `default_val` value. If `default_val` is not set, and a layer does not have its

own specification, all resistors in this layer are in the GDSII file. If `gdsmapi` is skipped, the PrimeSim EMIR tool stores all resistors.

gdsmapvmax

```
gdsmapvmax -v float_val | -l int_val
```

`gdsmapvmax` specifies the filter for the number of resistors stored in the GDSII file resulting from IR drop analysis (vmax).

Syntax Definitions

-v

Value threshold. `float_val` specifies that resistors with an IR drop lower than this value is not stored in the GDSII file.

-l

Level threshold. `int_val` specifies that the number of mostly violated levels where resistors are stored in the GDSII file.

If this command is skipped, the PrimeSim EMIR tool stores all resistors.

gdsoutmode

```
gdsoutmode [ird@<<1|v>|<0|dv>>] [em@<<1|i>|<0|i>>]
```

The `gdsoutmode` command specifies the type of information within the GDSII file.

Syntax Definitions

For IR drop:

ird@1 or ird@v

Actual voltages.

ird@0 or ird@dv

Voltage drop [default].

For electromigration analysis:

em@1 or em@i

Actual currents.

em@0 or em@iw

Current density [default].

If `gdsoutmode` is skipped, the PrimeSim EMIR tool generates a voltage drop and current density map.

If actual voltages are specified, the PrimeSim EMIR tool ignores `redv` and builds the following maps:

- A regularly distributed voltage map using maximum voltage as the threshold.
- A regularly distributed current map using maximum current as the threshold.

genspfview

```
genspfview [<-lib LIBRARY_NAME -cell CELL_NAME -view VIEW_NAME>
```

The `genspfview` command provides predefined values for `-cell`, `-view`, and `-lib` OUTPUT Prefix.

Default: `-lib <OUTPUT_Prefix>.oalib -cell EM_CELL -view EM_LAYOUT`

Syntax Definitions

`-lib <lib>`

Specifies the name of the library.

`-cell <cell>`

Specifies the name of the cell.

`-view <view>`

Specifies the name to assign to the view.

gdsprops

```
gdsprops [all] [none] [<prop_name>] [<net_name>]
```

where `<prop_name>` is the GDSII file property and `<net_name>` is the resistor net name.

The `gdsprops` command specifies the set and sequence of properties added to each shape within the GDSII file. The command supports the following properties:

Column Name	Description
resName	Resistor name
layer	Original resistor layer name or number
value	Violation value

Column Name	Description
resistance	Resistor nominal value in Ohms
time	Time when the maximum violation occurred (in nanoseconds)
j_jmax	J/Jmax current density. Jmax is a user-specified threshold for layer or maximum J calculated if the threshold was not set.
current	Current through resistor in the unit of uA
nodes	Resistor node names
group	Number of the group to which the resistor belongs. This property is used with the via grouping command from StarRC.
width	Resistor width in the unit of um
length	Resistor length in the unit of um
area	Resistor area in the unit of um2
rescur	Current through resistor when maximum IR drop occurred
resird	Maximum IR drop on the resistor
i_max - lmax	Actual current divided by threshold
width_new	Recommended width for a resistor
area_new	Recommended area for a resistor
current_neg	Negative part of Irms or lavg
current_pos	Positive part of Irms or lavg
emlen	Wire length
eflow	Electron flow direction (for both IAVG and VMAX analysis)
vmax	Maximum voltage in IR drop analysis
vmin	Minimum voltage in IR drop analysis
vavg	Average voltage in IR drop analysis
dVavg	Average voltage drop value
time_vmax	Time of the maximum voltage occurrence in nanoseconds

Column Name	Description
time_vmin	Time of the minimum voltage occurrence in nanoseconds
dVperc	dV (IR drop) value in percentage of the base voltage value. The EMIR analyzer calculates and outputs the dV (IR drop) value in percentage of the base voltage value, using the following equation: $(dV_value/base_voltage_value) * 100$ If <code>base_voltage_value</code> is not specified with the <code>vref</code> -base command, the tool uses <code>reference_voltage_value</code> instead.

itflayernames

`itflayernames yes|no`

In the SPF file, the StarRC tool can create two columns, one column with the LVS layer names and one column with the ITF layer names. By default the PrimeSim EMIR tool uses the LVS layer names.

Syntax Definitions

`yes`

Specifies to use the ITF layer names.

`no`

Specifies to use LVS layer names. This is the default.

The following example shows an SPF file with LVS layer names (`M3, M2, M1,..`) and ITF layer names (`metal3, metal2, metal1,...`):

```
*LAYER_MAP

*0 SUBSTRATE
*1 M3 ITF=metal3
*2 M2 ITF=metal2
*3 M1 ITF=metal1
*4 POLY ITF=poly
*5 nsd ITF=nsd
*6 psd ITF=psd
```

jjmaxlog

For signal net EM analysis, there can be many nets and it can be difficult to view each net to find EM violations. To create a log report (one per net) on the J/Jmax value in order address identify nets with EM violations, use the following syntax:

```
jjmaxlog [<def_val>] [<val>@imax] [<val>@iavg] [<val>@iabs] [<val>@irms]
```

All of the options are used for filtration. Therefore, only nets with J/Jmax values that are greater than <val> or <def_val> are added to the log file.

layerea

```
layerea [<def_val>] <ea1>@<layer1> [{<ea2>@<layer2>}]
```

The `layerea` command specifies the coefficient of the effective area (EA) of VIA layers. Effective area is used for current density calculation as shown in the following syntax

$$J = I / ((w * w) * ea) \text{ uA/um}^2$$

If effective area is not specified in <ea@layer>, <def_val> is used as the default value for all layers. If <def_val> is omitted, ea=1 is used. Several effective areas may be specified in one command. Several `layerea` commands may be specified in one .ratcl file. If several `layerea` commands have <def_val>, the <def_val> from the latest `layerea` command will be used as the default value.

layerh

```
layerh [<def_val>] <h1>@<layer1> [{<h2>@<layer2>}]
```

The `layerh` command specifies the height (thickness) of non-VIA layers. Height is used for current density calculation as shown in the following syntax:

$$J = I / (w * h) \text{ uA/um}^2$$

If height is not specified in <h@layer>, <def_val> is used as the default value for all layers. If <def_val> is omitted, h=1 is used. Height is displayed in um (microns). Several heights may be specified in a single command. Several `layerh` commands can be specified in a single .ratcl file. If several `layerh` commands have <def_val>, only the latest <def_val> will be used as default value.

layermap

```
layermap <numb_1>@<name_1>[<numb_i>@<name_i> ...]
```

The `layermap` command specifies the mapping between layers numbers and names. Several layer number and name pairs can be put in a single command. Several commands may be used in one file.

netfrequency

Specifies the frequency assigned to a net name or net pattern to calculate effective pulse width.

```
netfrequency {fr@net_name | fr@net_pattern}
```

Where *fr* is the frequency. The effective pulse width is calculated with the following expression:

$$P_{Weff} = P_{Wcalc} / (Fr * T_{sim})$$

where

- *PWcalc* is the pulse width calculated by power net EMIR.
- *Fr* is the frequency from `netfrequency` command.
- *TSim* is the simulation time.

Because *PWeff* should be calculated only for the integral `dratio` method, include the following command in the RATCL file:

```
dratio -method 2
```

nettype

```
nettype -pwnet <pwnet_name1> [<pw_netname2> ...]  
      [-signet <signet_name> ...]
```

Defines the net type (power net or signal net). By default, all nets are processed as signal nets.

outformat

```
outformat [fsdb|out|wdf|none]
```

Specifies the phase II waveform file format. The default is *fsdb*. For example:

```
outformat fsdb_5.8
```

or

```
outformat fsdb_5.7
```

outfiltres

```
outfiltres [-metal] w1@r1 w2@r2... [-via a1@r1 a2@r2...] -vmax [0|1] \  
[width_value@any]
```

The `outfiltres` command is used to filter out resistors of the specific width-resistance pairs and area-resistance pairs in the violation map (VM). By default, the PrimeSim EMIR tool recognizes and filters non-physical resistors generated by StarRC in the VM. For a DSPF generated by other RC extractors, you can use `outfiltres` to filter out resistors with the specific width-value and area-resistance pairs.

Syntax Definitions

`[-metal] w1@r1 w2@r2`

Defines the width-resistance pairs of metal line resistors that should be filtered out. If `-metal` is not specified, the pairs are applied to the metal line resistors.

- `w1, w2`: The resistor widths in microns.
- `r1, r2`: The resistor widths in Ohms.

`-via a1@r1 a2@r2`

Defines the area-resistance pairs of via resistors that should be filtered out.

- `a1, a2`: The resistor area in square microns
- `r1, r2`: The resistances in Ohm

`-vmax`

Accepts a value 0 or 1. If you do not specify a `-vmax` value, or a value other than 0 or 1, the simulation errors out.

- `0` considers resistor filtering for the `vmax` report.
- `1` does not consider resistor filtering when producing `vmax` report. This is the default behavior when you do not use the `outfiltres` command.

`width_value@any`

The reserved keyword `any` in place of a resistor value means that resistors with the specified width, and any value, are considered as non-physical. Resistors having a width value of 100000 are considered a non-physical resistor by default.

outlayers

```
outlayers [anType] layer_1[{{,layer_i}}] [-f]
```

The `outlayers` command specifies the original layers that are used to generate the violation map. You can specify commas (,) or spaces () to separate layer numbers or names.

Syntax Definitions

[anType]

Specifies one of the following analysis types: `vmax`, `imax`, `irms`, `iavg`, or `iabs`.

-f

Specifies that each layer must be in separate file.

If `outlayers` is skipped, power net EMIR uses all layers.

postnets

```
postnets <net_name_1> [<net_name_2> ...]
```

Specifies nets to be processed by `primesim_emir`. Default is ALL.

powernet

```
powernet [ -type {power | ground} ] \  
        [-vref <val>] [-base <val>] -nets { <net name> | <pattern> }
```

The `powernet` command defines the net type to be power or ground with voltage reference. Use this command when you want to define multiple power nets in one command, rather than using multiple commands.

When you specify the `-vref` and `-base` options with this `powernet` command, you do not need to run the `vref`, `pwrtype` and `nettype` commands to generate the voltage drop report. When both commands (that is, `vref`, `pwrtype` and `nettype`) and the corresponding options (that is, the `-vref` and `-base` options with this `powernet` command) are used, the last command overrides the previous option.

Syntax Definition

```
-type {power | ground}
```

Sets the net type to be `power` or `ground`. When not specified, PrimeSim EMIR automatically sets along with the `-vref` value. If the `-vref` value is 0 and `-type` is no specified, the net type is set to `ground`. If the `-vref` value is set to any value other than 0 and `-type` is not specified, the net type is set to `power`.

`-vref <val>`

Voltage reference. This is used as a reference to calculate the voltage drop or ground bounce value.

`-base <val>`

Specifies a value used to calculate the percentage from the base value. Value 1 is recommended, meaning 1V. This option is used for a ground net.

`-nets { <net name> | <pattern> }`

Specifies nets either by name or pattern. Wildcards are supported. Net names are required for power nets.

Example 1

```
powernet -type power -nets vdd vddq vdda vddd -vref 0.8
```

The above example sets nets VDD, VDDQ and VDDA as power net. The nets will be reported to have IR drop values calculated from the `-vref` value (0.8v).

Example 2

```
powernet -type ground -nets vss vssq vssa vssd -vref 0.0 -base 1
```

The above example sets nets VSS, VSSQ, VSSA and VSSD as ground net. The nets will be reported to have ground bounce values calculated from the `-vref` value (0.0v). The percentage (%) of ground bounce value is calculated by the `-base` value (1.0V).

Example 3

```
nettype -pwnet VSS VDD VDDH  
pwrtype ground -net VSS  
pwrtype power -net VDD  
pwrtype power -net VDDH  
vref -val 0.8925 -net vdd  
vref -val -0.1 -net vss  
vref -net vddh -val 1.26  
powernet -type power -nets vdd vddq vdda vddd -vref 0.8  
powernet -type ground -nets vss vssq vssa vssd -vref 0.0 -base 1
```

In the above example, multiple nets are defined with multiple commands.

- Power net VDD is overridden by the lastly defined vdd (defined in line 8) and has the IR drop value calculated from the `-vref` value (0.8V), rather than 0.8925 (defined in line 5).
- Power net VDDH uses 1.26V as it described with the `vref` and `nettype` commands.
- Ground net VSS is overridden by the last defined vss (line 9) and has the ground bounce value that is calculated from the `-vref` value (0.0V), rather than -0.1 (defined in line 6).

- The percentage (%) of ground bounce value is calculated by the `-base` value (1.0V).
- All the existing commands are used as is. If the `pwrtype` command is specified multiple times, the last one takes the precedence.

presimtime

```
presimtime <time>
```

The `presimtime` command in the RATCL file specifies a pre-simulation time when time windows are specified by the `swin` command. When a pre-simulation time is set, phase II simulation starts a bit earlier than the time windows given in the `swin` command, which allows current values to become stable before the actual simulation. The default pre-simulation time is 1 ns.

Example

```
swin 500n 600n  
  
presimtime 10n
```

In the above example, phase II simulation starts at 490 ns (that is, 10 ns earlier than 500 ns) and uses currents that are saved between 490 ns to 500 ns to determine a stable circuit solution. With this, when the actual EMIR time window starts at 500 ns, the currents and voltages across the parasitic network would already be settled and the EMIR results will not be impacted.

printi

```
printi <net_name>:<resistor_name> [{<net_name>:<resistor_name>}]
```

The `printi` command specifies the resistor currents inserted into wave output files. Currents are calculated at the Phase II of simulation. Several resistors can be specified in one command. Several `printi` commands can be specified in one .ratcl file. The asterisk (*) wildcard can be used for the `<resistor_name>`.

printipad

```
printipad <pad_name> [{<printipad_name>}]
```

The `printipad` command specifies the net pad currents inserted into wave output files. Pads are specified by *|P in the DSPF file. Currents are injected at Phase II simulation. Several pads may be specified in one command. Several `printipad` commands may be specified in one .ratcl file. The asterisk (*) wildcard can be used for the `<pad_name>`.

printipin

```
printipin <pin_name> [{<pin_name>}]
```

The `printipin` command specifies the net pin currents inserted into waveform output files. Pins are specified by `*|I` in the DSPF file. Currents are injected at phase II simulation. You can specify several pins in one .ratcl file. You can use the asterisk (*) wildcard character in a `<pin_name>`.

printpwrpinv

```
printpwrpinv {1|0}
```

The `printpwrpinv` command outputs power nets' pin voltage waveforms during phase 2 simulation. The output waveform follows the naming convention: `v(net_name::in_name)`.

Syntax Definitions

0 (default)

No power net pin voltage waveform is added to the output file.

1

The EMIR tool reports pin voltage waveforms for every power net, including

- Net pin (pad)
- Instance pin

printv

```
printv <node_name> [{<node_name>}]
```

The `printv` command specifies the node voltage as either IR drop or the actual value inserted into the wave output files. Voltages are calculated at Phase II simulation. Several nodes can be specified in one command. Several `printv` commands can be specified in one .ratcl file. The asterisk (*) wildcard can be used for the `<node_name>`.

printvmode

```
printvmode [1|dv] [2|v]
```

The `printvmode` command specifies the type of output by the `printv` command.

Syntax Definitions

[1|dv]

Voltage drop.

[2|v]

Actual voltage at a node.

The default is actual voltage at a node (2 | v).

pwnetdriver

```
pwnetdriver -net <net_name> -driver {<ipin_name>}
```

Provides the driver pin information for the internal power net

pwrtype

```
pwrtype <ground|power> -net <net_name>
```

Defines how the tool estimates IR drop for particular power nets.

- If pownet is type=power, then IR drop = $V_{ref} - V_{min}$
- If pownet is type=ground, then IR drop = $V_{max} - V_{ref}$

ra

The `ra` command specifies the analysis types to be performed during Phase II simulation.

```
ra [-pwra|-sigra|-vmaxminavg] [val] \  
val=[all] [vmax] [imax] [irms] [iabs] [iavg] [acpc] [pins]
```

`-pwra`

Executes the command for power net analysis only. The default is Tcl execution for both power and signal analyses. If `val` is set to `all` or not specified, the default analysis types for power nets are: `vmax`, `irms`, `acpc` and `iavg`.

`-sigra`

Executes the command for signal net analysis only. The default is Tcl execution for both power and signal analyses. If `val` is set to `all` or not specified, the default analysis types for signal nets are: `irms`, `acpc` and `iavg`.

`-vmaxminavg`

Executes the calculation and outputs Vmax, Vmin, and Vavg values into the GDSII, ASCII, RVE or CCDB formats, depending on the setting of the `raformat` command. This option applies only to the IR drop analysis (`vmax`).

`value`

Specifies the type of calculation results to write to the output file.

- 1|all: All types of values
- 2|vmax: Maximum IR drop at nodes
- 3|imax: Peak resistor current
- 4|irms: RMS resistor current
- 5|iabs: Average absolute magnitude of resistor current
- 6|iavg: Average resistor current
- 7|acpc: Peak current for each metal resistor
- 8|pins: calculate instance pin current based on analysis type [iavg|irms|acpc]

Note:

If you want to run analysis on both power and signal nets, it is suggested not to specify either the `-pwra` or `-sigra` option. If both `-pwra` and `-sigra` are specified, the PrimeSim EMIR tool stops simulation with an error message.

raformat

`raformat [all] [1|gds2] [2|ccdb] [3|ascii] [4|rve]`

The `raformat` command specifies the output layout format. The defaults are `gds2` and `ascii`. To generate outputs in `rve` format, use `raformat rve`. The `raformat rve` command generates an error database that can be loaded into the Mentor® Graphics RVE tool for visualization.

The `ccdb` option is available to process and display EMIR data efficiently in the Custom Compiler tool. This option saves disk space, compared to the `gds2` option.

When you specify the `ccdb` option, all available analysis data are saved in the `out_prefix-net_name.cc.db` file. EMIR analysis produces one `.cc.db` file for every power net and one for all signal nets.

ralayers

```
ralayers [layer_1[{{,layer_i|-layer_j}}]]
```

The `ralayers` command specifies the layer numbers of the violation map layout.

Syntax Definitions

, (comma)

Specifies a single layer number.

- (dash)

Specifies a set of layer numbers. For example, specifying “51-54” equals to “51,52,53,54”.

Note:

The EMIR flow does not support numbers larger than 2048. If you specify numbers larger than 2048 (for example, `ralayers 3000-3009`), the `ralayers` command resets the numbers to `ralayers 100-109`.

Also, be aware of that the numbers you choose in the `ralayers` command can create conflicts with the original design layer numbers. This information is important to make sure the violation maps are correctly imported to the viewer, such as the Custom Compiler tool. In the Custom Compiler tool you can find the layer numbers used in the design database by exporting the technology file and searching for the `techLayers` definitions in the `layerDefinitions` section. For example:

```
layerDefinitions(
    techLayers (
        ;( layerName  layerNum  abbreviation )  ;(  -----
    -----  ----- )  ...
        ( POLY          15          POLY      )  ( CONT          16
    CONT      )
        ( M1            17          M1        )  ( VIA12         18
    VIA12     )
        ( M2            19          M2        )  ( VIA23         20
    VIA23     )
        ( M3            21          M3        )  ...
    )
)
```

ralayout

The `ralayout` command specifies the analysis types of analysis results to write to the output files. The layout is stored in the `<prefix>_<analysisName>.<ext>` file.

Syntax

```
ralayout [-pwra|-sigra] <val> [-o <prefix>]  
val == [all] [1|vmax] [2|imax] [3|irms] [4|iabs] [5|iavg] [6|acpc]  
       [7|pins]
```

Syntax Definitions

-o prefix

Specifies part of file name.

<prefix>

User-specified name.

<analysisName>

User-specified analysis name.

<ext>

File file extension (e.g., ragds, ascii, etc.).

-pwra

Executes the Tcl command for power net analysis only. Default is Tcl execution for both power and signal analyses.

-sigra

Executes the Tcl command for signal net analysis only. Default is Tcl execution for both power and signal analyses.

If <prefix> is omitted, the tool uses <radb name> (the name of the *.radb file without the file name extension).

Note:

If you skipped the `ralayout` command, then `ralayout` reports all the analyses defined by the `ra` command.

For example, if you specify:

Examples

Example 1:

With no `ralayout` command in `.ratcl` file, power net EMIR analysis postprocesses and outputs the `vmax` and `imax` results for power nets and the `imax` result for signal nets.

```
ra -pwra 1 -sigra 1 vmax imax
```

Example 2:

When you specify the following commands in the .ratcl file, power net EMIR analysis postprocesses and outputs the `imax`, `iavg`, and `irms` results for both power and signal nets.

```
ra -pwra 1 -sigra 1 vmax imax iavg irms
ralayout imax iavg irms
```

For backward compatibility, if you specify:

```
ra -pwra 1 -sigra 1 vmax imax iavg irms
ralayout none
```

Power net EMIR analysis postprocesses and outputs none for power nets and none for signal nets.

rarve

```
rarve [-topcell <name>] [-diagtype {<2|poly> | <1|edge>}] [-defwidth <val>]
```

The `rarve` command sets several parameters used to generate RVE files.

Syntax Definitions

-topcell

Sets the name of top cell in RVE file. The default name is `TOP_CELL`. If `<name>` is the word "file", the `topcell` name is: `<prefix>-<net name>`.

-diagtype

Specifies the type of a shape which will be generated for non-orthogonal resistor. The default value is `<2|poly>`.

-defwidth

Specifies the default width for all resistors which have zero width or which have no width statement in SPF/DSPF file.

raviewer

```
raviewer [virtuoso|calibre|laker|ccompiler]
```

The `raviewer` command selects the viewer that the PrimeSim EMIR tool uses to generate technology and/or display resource files.

- `virtuoso` specifies the Cadence Virtuoso layout editor, which requires a technology file, DRF file, and layer mapping file.
- `calibre` specifies the Mentor® Calibre® DESIGNrev™ layout editor, which requires a technology file, DRF file, and layer mapping file.
- `laker` specifies the Laker layout editor, which requires a technology file, property mapping file, and display file.
- `ccompiler` (default) specifies the Custom Compiler™ layout editor, which requires a technology file, DRF file, and layer mapping file.

rawf

```
rawf 0|1
```

The `rawf` command addresses the capacity issue when phase II exits with the following message, even with enough disk space in the run directory/dedicated disk for temporary files specified with the `tmpdir` command:

```
No space left on device
Error: fwrite failed to write 2*8 bytes
```

- 0 (default) uses the current method to interpolate multiple rasim files (`*.rasim`, `*.rasim1`, ... `*rasimn`) to a single temporary file that can trigger the capacity issue.
- 1 processes multiple rasim files (`*.rasim`, `*.rasim1`, ... `*rasimn` file by file to reduce the temporary file size.

Note:

Phase II multi-threading is not supported when you set `rawf 1`.

redi

```
redi val1 val2 val3... val9
```

The `redi` command is used for electromigration analyses with Tcl procedures only. See the [redj](#) description for syntax definitions.

redia

```
redia val1@layer1 val2@layer2...
```

The `redia` command specifies the current density threshold for VIA layers. Since all layers specified by this command are treated as VIA, power net EMIR calculates the current

density through VIA resistors using their area, defined by width*width, and compares it with the threshold.

The threshold for each layer is presented in $\mu\text{A}/\mu\text{m}^2$. There is no default value for all VIA layers as in `rediw`. Each VIA layer must have its own `val@layer` construction or it is considered to be a non-VIA layer with I/W threshold. The `redia` command covers all EM analysis.

rediac

```
rediac val_1@layer_1 val_2@layer_2...
```

The `rediac` command specifies the AC operation threshold for peak current analysis.

Syntax Definitions

`val_i`

The coefficient specified in a semiconductor foundries technology.

`layer_i`

The name of an SPF/DSPF file layer.

The `Ipeak_dc` value is a current density factor that specifies the limit of Amps/micron of width. It is typically derived from the manufacturing specification of semiconductor foundries, and is often stated as a coefficient that is multiplied by the width for the appropriate layer. The current must be in Amps and the width in microns. For periodic signals, there is a peak current at which a metal line undergoes excessive Joule heating and can begin to melt. The limit for the peak current, `Ipeak`, is calculated using the following formula:

$$I_{\text{peak}} = I_{\text{peak_DC}} / \sqrt{\text{dratio}}$$

- `Ipeak_DC` = coefficient * width;
- `dratio` = `td` / `tau`
 - `td`: signal width
 - `tau`: signal period

The `<prefix>-<net name>_acpc.dratio` file is automatically generated when the `rediac` command is used. This file contains a table with the following columns:

- Resistor name
- Signal width

- Signal period
- Peak AC operating current

An imax analysis is required when `rediac` is used. The PrimeSim EMIR tool calculates the currents through all resistors and generates violation map.

redimaxa, redirmsa, rediabsa, rediavga

`redimaxa`

Peak current.

`redirmsa`

RMS current.

`rediabsa`

Average current magnitude.

`rediavga`

Average current.

All analysis-specific commands have the same syntax as `redia`.

redimaxw, redirmsw, rediabsw, rediavgw

`redimaxw (imax)`

Peak current All `redimaxw` commands have the same syntax as `rediw`. If no command is specified in the `.ratcl` file, power net EMIR uses the maximum value for each layer for each analysis.

`redirmsw (irms)`

RMS current.

`rediabsw (iabs)`

Average current magnitude.

`rediavgw (iavg)`

Average current.

rediw

```
rediw [values_for_all_layers] [[(w_1@v_1 w_2@v_2 ...)]lval_1@layer_1]  
[lval_2@layer_2] ...
```

The `rediw` command specifies the current density (I/W) threshold for non-VIA layers. Layer threshold values can be specified as follows:

- Value for all layers
- Value for a given layer

Syntax Definitions

v1

Current density value for resistors with a width within the range 0-w1.

v2

Current density value for resistors with a width within the range w1-w2.

lval

Current density value for resistors with a width greater than w2 have threshold lval.

Command parameters:

default_val

Threshold for all layers not mentioned in this command presented in uA/um.

lval_1

Threshold for specific layer presented in uA/um.

layer_1

Layer name.

w_1

Width presented in um.

v_1

Threshold for the given width presented in uA/um.

The `rediw` command sets the threshold for all types of current analysis. To set specific values for the given analysis, use the following commands: [redimaxw](#), [redirmsw](#), [rediabsw](#), [rediavgw](#).

The following example specifies the threshold values in the AVG analysis for M1:

```
redavgw 100 { (3@828)118@m1 }
```

In the above example, J, threshold for all the layers except "m1": 100.

For M1,

- For $0 < W \leq 3$, J, threshold: 828
- For $W > 3$, J, threshold: 118

```
redavgw (3@828 5@1000)118@m1
```

In the above example:

For M1,

- For $0 < W \leq 3$, J, threshold: 828
- For $3 < W \leq 5$, J, threshold: 1000
- For $W > 5$, J, threshold: 118

Note:

For correct processing, it is recommended that you do not use any space before or after the parentheses symbols ["(" and ")"] in the syntax.

redj

```
redj val1 val2 val3... val9
```

The `redj` command is used to specify the boundaries of bin thresholds from ratios of J/Jmax of resistors starting from the lowest bin up to nine values. For example:

```
redj 0.5 1 1.5 2 2.5 3 3.5 4 4.5
```

All resistors with a J/Jmax value less than 0.5 are reported in the lowest bin (gds layer# 100). Resistors with a J/Jmax value larger than 4.5 are reported in the highest violation bin (gds layer# 109). If the # of the specified value is less than 9, the PrimeSim EMIR tool recognizes the specified threshold from the lowest bin. For example:

```
redj 0.5 1 1.5
```

Resistors with J/Jmax less than 0.5 are stored in the lowest bin (100) and resistors with J/Jmax between 0.5 and 1 are stored in the next lowest bin (101). Any resistor with J/Jmax greater than 1.5 are stored in layer 102, and layers 103-109 are empty.

Note that J is current density and $J_{\max}$ is user-specified current threshold or the maximum current density calculated by the PrimeSim EMIR tool. To set $J_{\max}$, use the following commands:

- [rediw on page 267](#)
- [redia on page 264](#)

redv

The `redv` command is used to control the IR drop threshold for violation map generation.

Syntax

```
redv [-val <dV val>] [-lb val2] [-v [{usage}]] [-td time_duration]
```

The `redv` command provides options that perform the following functions:

- Specifying the IR drop upper boundary threshold

Use the `-val` option to specify the upper violation map boundary. Elements with the maximum IR drop at any terminal larger than `<dV val>` are displayed in the Blinking Red text. All other elements are distributed across the other 9 violation map layers according to their IR drop values. The `<dV val>` must be specified in volts. If the `-val` option is not specified, power net EMIR analysis uses the maximum IR drop as `<dV val>`.

Example:

Setting `redv -val 0.09` distributes all resistors across the colored layers using voltage steps equal to 10 mV. The Forest Green first layer contains resistors with the maximum IR drop in the range of 0 mV ~ 10 mV. The Green layer contains resistors with the IR drop of 10 mV ~ 20 mV, through to the other end of the scale where the Blinking Red layer contains all resistors with the IR drop greater than 90 mV.

- Specifying the IR drop lower boundary threshold

Use the `-lb val2` option to specify the lower boundary for the IR drop threshold. Nodes with IR drop lower than this value do not output into the violation map.

- Varying the map scale

Use the `-v` option to specify the variable voltage scale as a percentage of voltage belonging to a given violation level. The 100% voltage is the maximum IR drop.

Assuming there are 10 color map layers and the maximum IR drop is 100mV, the following syntax generates a layout having the voltage distribution that follows:

```
redv -v 5 5 10 20
```

Voltage Distribution

10th

Blinking Red. Top 5% most IR drop distribution.

9th

Red. The next 5% most IR drop distribution.

8th

Maroon. The next 10% most IR drop distribution.

7th

Brown. The next 20% most IR drop distribution.

From 6th bin and beyond: For the next 10% most IR drop distribution, since it is difficult to select voltage distributions that result in useful color maps, it is recommended that the resistor distribution be specified among colored layers of the violation map.

- Specifying the IR drop time duration

Use the `-td time_duration` option to specify the time duration for IR drop threshold.

With the time duration threshold, power net EMIR analysis now reports a node with an IR drop violation when the node has:

- The worst IR drop value greater than a threshold set by `redv`.
- The worst IR drop value in a period of time which is greater than a user-defined time duration.

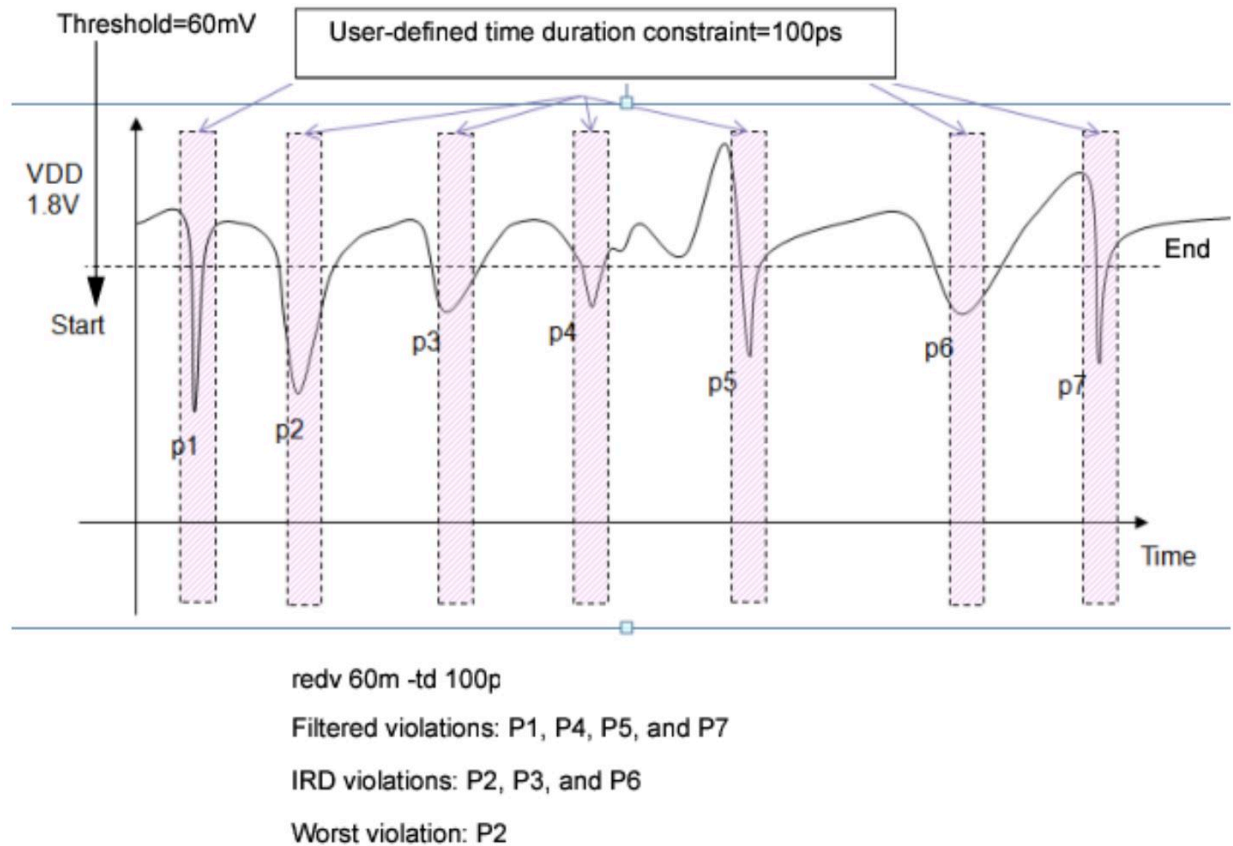
The following figure shows that:

- P1, P4, P5, and P7 are the points where the actual voltage drop durations are less than the user-defined duration threshold (100 ps). These points are filtered out (dropped, not to be reported in all EMIR outputs) and are not considered as the worst IR drop locations.
- P2, P3, and P6 are the locations where the actual IR drop values are greater than the threshold (60 mV) and the actual time periods where the IR drop is greater than

60 mV are greater than 100 ps. These locations are the candidates of the worst IR drop reporting.

- Because P2 has the absolute worst IR drop value, P2 is reported as the worst IR drop location for this node.

Figure 22 IR Drop Time Duration Example



redtrms

```
redtrms dT1@cT1@cW1@layer1 dT2@cT2@cW3@layer2...
```

The `redtrms` command specifies temperature analysis parameters using the following parameters:

Syntax Definitions

dTi

Temperature threshold for a specific layer.

cTi

Temperature coefficient for a specific layer.

cW

Width coefficient for a specific layer.

layer_i

Layer name.

replace_waveform

```
replace_waveform net_name pin_name \  
    -dc dc_value | -pwl t1 val1 t2 val2 [t3 val3 ...]
```

The `replace_waveform` command replaces pin voltage or current waveform that is saved in the .rasim file with the user-defined waveform.

Syntax Definitions

net_name

Name of the net where the specified pin resides.

pin_name

Name of the pin for which its waveform is replaced. The specified name is an instance pin name from the instance pin statement in the SPF file. All instance pin names and their waveform types (either current or voltage) are saved in the *.pimap files.

-dc dc_value

Replaces the simulated pin voltage or current waveform with a waveform with the specified constant value.

-pwl t1 val1 t2 val2 ... tN valN

Replaces the simulated pin voltage or current waveform with a user-defined PWL waveform. The time unit is second; the current unit is ampere; and the voltage unit is volt.

Example 1

```
replace_waveform vss x2/mltp0:bulk -dc 4e-06
```

Example 2

```
replace waveform vss x2/mltn0:src -pwl 0 0 1e-09 0 1.01e-09 \  
2.51334e-05 1.02e-09 -0.000279008 1.03e-09 \  
-0.000204569 1.04e-09 -2.57832e-05
```

reportirdrop

```
reportirdrop -pins_only <0|1> -w <width>
```

The `reportirdrop` command limits the amount of IR drop information to report in the output file, which facilitates a faster display of IR drop violation map. You can choose to report IR drops only at the pins of the power nets or at all nodes, including instance pins and net pins.

Syntax Definitions

```
-pins_only 0|1
```

- 0 (default): Reports IR drops at all nodes, including instance pins and net pins.
- 1: Reports IR drops at the pins of the power nets only.

```
-w <width>
```

Each pin node is represented by square shape with the specified width (in the SPF unit). The center of the square is aligned with the pin location.

rvemapi

```
rvemapi{max | avg | rms | abs} [-1] [<default_val>] <val_1>@<layer_1>  
<val_2>@<layer_2> ...
```

The `rvemapi` command specifies the filter for the number of resistors to be stored in the RVE output file as a result of any of the following EM analysis: `imax`, `irms`, `iavg`, `iabs`. This command supports two types of filtering: by level of violation or value of current density. To request filtering by level of violation, `-1` parameter must precede all other parameters. In this case, `<default_val>` and all other `<val_i>` must be integers specifying the number of most violated levels to be outputted into RVE file. In case of filtering by value `<default_val>` and all `<val_i>`, the value must be in uA/um specifying minimum value of resistor current density to be outputted into the RVE file.

Each layer can have its own `<val>@<layer>` threshold specification, where `<layer>` is layer name/number. If `<default_val>` is specified, layers without individual specifications are treated as they have this `<default_val>` value. If `<default_val>` is not specified and a layer does not have its own specification, all resistors on this layer are listed in the output file. If `rvemapi` is skipped, power net EMIR stores all resistors.

rvemapvmax

```
rvemapvmax -v float_val | -l int_val
```

The `rvemapvmax` command specifies the filter for the number of resistors stored in the RVE file resulting from IR drop analysis (vmax). When not specified, the PrimeSim EMIR tool stores all resistors in the RVE file.

Syntax Definitions

-v

Value threshold. *float_val* specifies that resistors with an IR drop lower than this value are not stored in the RVE file.

-l

Level threshold. *int_val* specifies that the number of mostly violated levels where resistors are stored in the RVE file.

rveprops

```
riveprops [all] [none] [<prop_name>]
```

where *prop_name* is the RVE property.

The `riveprops` command specifies the set and sequence of properties added to each RVE shape. To set properties, use either of the following:

Column Name	Description
resname	Resistor name
layer	Original resistor layer number or name
value	Violation value
resistance	Resistor nominal value in Ohms
time	Time of the maximum violation occurrence in nanoseconds
j_jmax	J/Jmax current density. Jmax is a user-specified threshold for layer or maximum J calculated if the threshold was not set.
current	Current that flows through the resistor
nodes	Resistor node names

Column Name	Description
group	Number of the group to which the resistor belongs. This property is used with the via grouping command from StarRC.
width	Resistor width
length	Resistor length
area	Resistor area
rescur	Current through resistor when maximum IR drop
resird	Maximum IR drop on resistor
i_max - lmax	Actual current divided by threshold
threshold	Threshold for current through a resistor
eflow	Electron flow direction (for both IAVG and VMAX analysis)
vmax	Maximum voltage in IR drop analysis
vmin	Minimum voltage in IR drop analysis
vavg	Average voltage in IR drop analysis
time_vmax	Time of the maximum voltage occurrence in nanoseconds
time_vmin	Time of the minimum voltage occurrence in nanoseconds
dvperc	dV (IR drop) value in percentage of the base voltage value. The EMIR analyzer calculates and outputs the dV (IR drop) value in percentage of the base voltage value, using the following equation: $(dV_value/base_voltage_value)*100$ If <code>base_voltage_value</code> is not specified with the <code>vref</code> -base command, the tool uses <code>reference_voltage_value</code> instead.

signetascii

`signetascii [netbynet | mixall]`

Groups or ungroups signal net EM results by the net names in the ASCII files.

simnets

```
simnets name_pattern [-skipnet <name_pattern>]
```

Filters the signal net analysis during Phase II of the signal net EM analysis.

Syntax Definitions

-skipnet *name_pattern*

Excludes the specified signal net from the post-processing stage of the signal net EM analysis.

Example

```
simnets * -skipnet dumm_port*
```

skiplayers

```
skiplayers [analysisType] layer_1[,{,layer_i}]
```

The `skiplayers` command specifies the physical design layers to exclude from GDSII or ASCII reporting. The `skiplayers` command is opposite to the `outlayers` command.

Syntax Definitions

[analysisType]

Specifies one of the following analysis types: `vmax`, `imax`, `irms`, `iavg`, or `iabs`. If no value is set, `skiplayers` applies to all analysis types

layer_1 [{,layer_i}]

Specifies the physical design layers to be excluded from output reporting

EMIR TCL may contain several `skiplayers` commands for different types of analysis. If both "`kiplayers` and `outlayers` commands are used for the same analysis, the PrimeSim EMIR tool uses the latest command in the specification.

Example

```
skiplayers poly, nwell  
skiplayers vmax via1, via2  
skiplayers iavg metal, metal2
```

In the above example, `poly` and `n-well` layers are not reported for all EMIR analysis. In addition, `via1` and `via2` layers are skipped during `vmax` analysis and `metal1` and `metal2` layers are skipped during `iavg` analysis.

source

```
source <path_ratcl_file>
```

Specifies the .ratcl file which contains EM rule information.

spresenable

```
spresenable <0|1|2>
```

The `spresenable` command enables the effective resistance calculation along with the IR drop analysis.

Syntax Definitions

[0 | 1 | 2]

- 0 - Do not calculate effective resistance (Reff) for the nodes.
 - 1 (default) - Calculates effective resistance (Reff) for the nodes if the `reff` attribute is defined with the `asciicols` or `ccdbcols` command for report effective resistance in the output result file in the ASCII or CCDB format.
 - 2 – Calculates effective resistance (Reff) for the nodes, regardless of whether the `reff` attribute is defined with the `asciicols` or `ccdbcols` command for report effective resistance in the output result file in the ASCII or CCDB format.
-

swin

```
swin <t1 t2> [<t3 t4> ...] -apw
```

The `swin` command specifies multiple tstart-tstop windows for EMIR power net analysis.

Syntax

Note:

By default, the window defined in Phase I simulation with the `set_ra_option -twindow` command is used in Phase II simulation. You do not need to redefine the Phase II window using the `swin` parameter.

You can reset the window in Phase II simulation using the `swin` command in a user-defined .ratcl file, which has a higher priority over `set_ra_option -twindow`. In that case, it is recommend to use a slightly smaller window for `swin` (with respect to the `set_ra_option -twindow` value). If you keep the exact same window, the initial conditions for the Phase II window start points

might not be correct, because capacitors are also processed at Phase II and the currents at the parasitic resistors are not yet defined. It takes some time until the solution stabilizes to a steady state.

t1 t2 [t3 t4] ...

Specifies tstart-tstop simulation windows for EMIR power net analysis. For example, if you simulate 100ns (100n) for the Phase I simulation, and you are only interested in 10n-20n and 50n-60n for the Phase II EMIR analysis, specify the following command in the .ratcl file:

```
swin 10n 20n 50n 60n
```

-apw

Stands for average per window. When specified, power net EMIR calculates the IAVG, IRMS and IABS currents separately for each defined simulation window and then reports the peak value. When the -apw option is not specified, power net EMIR calculates the overall IAVG, IRMS and IABS currents for all of the defined simulation windows.

For example, if you specify time windows without the -apw option:

```
swin t1 t2 t3 t4 ...
```

If IAVG analysis is performed, EMIR analysis uses the following formula to compute IAVG for the defined simulation windows t1-t2, t3-t4, ...

$$IAVG = \frac{1}{(t2-t1) + (t4-t3) + \dots} \left(\int_{t1}^{t2} i(t) dt + \int_{t3}^{t4} i(t) dt + \dots \right)$$

In contrast, if you specify time windows and the -apw option:

```
swin -apw t1 t2 t3 t4
```

EMIR analysis uses the following formula to compute IAVG for defined simulation windows t1-t2, t3-t4, ...

$$IAVG = PEAK \left\{ \begin{array}{cc} \frac{1}{t2-t1} & \int_{t1}^{t2} i(t) dt \\ \frac{1}{t4-t3} & \int_{t3}^{t4} i(t) dt \\ \dots & \dots \end{array} \right\}$$

Similarly, if IRMS and IABS analysis are selected, EMIR analysis uses the following formulas to calculate the corresponding currents.

$$IABS = PEAK \left\{ \begin{array}{cc} \frac{1}{t_2-t_1} & \int_{t_1}^{t_2} |i(t)| dt \\ \frac{1}{t_4-t_3} & \int_{t_3}^{t_4} |i(t)| dt \\ \dots & \dots \end{array} \right\}$$

$$IRMS = PEAK \left\{ \begin{array}{cc} \frac{1}{t_2-t_1} & \sqrt{\int_{t_1}^{t_2} i(t)^2 dt} \\ \frac{1}{t_4-t_3} & \sqrt{\int_{t_3}^{t_4} i(t)^2 dt} \\ \dots & \dots \end{array} \right\}$$

Note:

Due to the limitations of the viewers currently used, the waveform points which correspond to the end of previous window and the beginning of the new one will be connected by a straight line.

Outside the specified windows, no EMIR analysis is performed in Phase II, and the result is set to zero.

The PEAK function chooses the peak result from all `swin` windows, ignoring the sign during comparison. The IMAX and VMAX results are not affected by the `-apw` option.

Examples

The following example specifies one window, starting at 10ns.

```
swin 10ns
```

The following example specifies two windows: 5ns-15ns and 25ns-35ns. Averaging is done in each time window.

```
swin -apw 5ns 15ns 25ns 35ns
```

The following example specifies two windows: 5ns-15ns and 25ns-35ns. Averaging is done over the whole time of simulation.

```
swin 5ns 15ns 25ns 35ns
```

tau

```
tau <timestep>
```

The `tau` command permits setting a different value for the Phase II time step. In Phase I, the average currents injected from transistor terminals into the original power net are

accumulated during the time interval specified by RATAU. By default, the same value is used as the time step for the Phase II.

temp

`temp value`

The `temp` command specifies the ambient temperature. Its value is represented in the `hs_temp` variable and is used for AVG threshold calculation. If you do not use this command, the default value is 25C for an PrimeSim HSPICE netlist and 27C for Eldo® netlist.

In the following example a temperature of 90C is specified for the AVG threshold calculation.

```
temp 90
```

tmpdir

`tmpdir path`

The `tmpdir` command redirects pseudo-temporary files to a user-specified location. You use this phase II command in cases when the `-o` command line argument generates large files that cause a lot of network traffic, or when the `-o` location does not have sufficient disk space. The default is the directory specified with `-o`.

The following example redirects the storage of the temporary EMIR files to the `/tmp` directory.

```
tmpdir /tmp
```

tstart

`tstart <time>`

The `tstart` command specifies the start time for EMIR simulation. The time unit is in seconds. The default start time is 0s when you do not specify `set_ra_option -twindow`.

tstop

`tstop <time>`

The `tstop` command specifies the stop time for EMIR simulation. The time unit is in seconds. The default end time is the end of transient time when you do not specify `set_ra_option -twindow`.

twin

```
twin <w>
```

The `twin` command specifies a window of duration `<w>` for EMIR power net and signal net analyses around the time of peak current through the power source.

For example, if you simulate 100ns in the Phase I simulation and the current peaks at 35ns, then by specifying the following command in the `.ratcl` file, the Phase II EMIR analysis is automatically carried out in the window between 30ns and 40ns.

```
twin 10n
```

Note:

Typically, the time of current peak does not necessarily correspond to the time of the maximum IR drop.

usecursign

```
usecursign <0|1>
```

The `usecursign` command reports `lavg` or `lmax` values as absolute average or maximum values for resistors.

- 0 (default): Reports `lavg` and `lmax` values as absolute average or maximum values for resistors.
 - 1: Do not report `lavg` and `lmax` values as absolute average or maximum values for resistors.
-

usew4siw

```
usew4siw <0|1>
```

The `usew4siw` command overrides the value of the `$si_w` attribute and uses the value of the `$w` attribute instead.

- 0 (default): Uses `$si_w` for `hs_siwidth` when available. This is the default setting.
- 1: Uses `$w` for `hs_siwidth` even if `$si_w` is available.

vref

```
vref global_value | [-net <net_name> ] \  
    [-node <node_name> | -use_any_node | -val <value>] [-base <value>]
```

The `vref` command provides a voltage reference value for internal power supplies. If internal power net voltage is a variable value, or the `vref` value is not constant, you can specify `vref` using net pins or header voltage.

Syntax Definitions

`global_value`

Sets a constant voltage reference value for calculating worst-case IR drop.

`-net <net_name>`

Specifies the name of the net whose voltage is used as the reference voltage. Wildcard usage is supported.

`-node <node_name>`

Specifies the name of the node whose voltage is used as the reference voltage. Usually an instance pin is used as such a pin.

`-use_any_node`

When specified, EMIR analyzer uses the voltage of any instance pin as the reference voltage.

`-val <value>`

When specified, this value is used as the reference voltage.

Note:

The `-node`, `-use_any_node` and `-val` options are mutually exclusive. You can specify only one of them at a time.

`-base <value>`

Specifies the base voltage for calculation of IR drop percentage value. If this value is not set, the tool uses the power reference voltage value for calculation. If `-base` is set to 0, the following warning message is issued:

```
Warning: Cannot calculate IR drop percentages because base  
voltage is 0. Use "vref -base <value>" to specify the value.
```

To output the IR drop percentage value to the report files, use the `dvperc` option with the appropriate commands. For example, to output the IR drop percentage value to the CCDB file, use the `ccdbcols ird dvperc` command.

Example

The following example sets the internal supply reference voltage to 3.3 volts.

```
vref -val 3.3
```

In the following example, the power net EMIR picks an instance pin voltage as header voltage for all the internal power nets.

```
vref -val 1.8  
vref -net * -use_any_node
```

The following example uses an instance pin voltage as header voltage for all the nets but `vdd_int` and `vss_int`. For `vdd_int`, the node `mp:src` voltage is used, while `vref=0` is used for `vss_int`.

```
vref -net * -use_any_node  
vref -net vdd_int -node mp:src  
vref -net vss_int -val 0
```

vts

```
vts <0|1>
```

The `vts` command controls the type of time steps to use during analysis stage (phase II simulation).

Syntax Definition

0 | 1

- 0 (default): Uses constant time steps for IR drop analysis.
- 1: Uses variable time steps for IR drop analysis.

vtsopt

```
vtsopt -dv <dv> -steps <n1> <n2> ... -steps_dv <dv1> <dv2> ...
```

The `vtsopt` command is used to specify variable time step settings for IR drop analysis (phase II).

Syntax Definition

`-dv <dv>`

Sets the `dv` tolerance in unit V. Default is 0.01V.

`-steps <n1> <n2> ...`

Specifies step sizes in the `ratau` unit. For example, `1 3 9` means time steps are: `ratau`, `3*ratau`, `9*ratau`. Default is 1, 2, 4.

`-steps_dv <dv1> <dv2> ...`

Sets the `dv` tolerance for the specified step. The number of steps must match the specified `dv` values; Otherwise, an error message will be issued. Default is the `-dv` value. When both `-step_dv` and `-dv` are specified, `-step_dv` takes precedence over `-dv`.

Commands for Signal Net EM Post-processing

This chapter describes the commands that are specific for signal EM analysis:

- [avgrelaxf](#)
- [dratio](#)
- [iavmin](#)
- [itflayernames](#)
- [mt val](#)
- [net_ccaps](#)
- [outformat](#)
- [rawf](#)
- [selectsignets](#)
- [signetascii](#)
- [skipsignets](#)
- [swin](#)

avgrelaxf

`avgrelaxf <val> {<layer name> | <layer pattern>}`

The `avgrelaxf` command specifies the relaxation coefficients for `lavg` threshold values. You can use the coefficient when your design requirement is different from the DRM manual to address the length of life cycle. The default coefficient value is 1.

For example, if the DRM provides currents limits for a ten year lifetime, and the design requires seven years, the relaxation coefficient can be calculated to increase the current limits:

```
avgrelaxf 1.125 ap ru  
avgrelaxf 1.032 r2_m
```

Different layers (material properties) may require different coefficient values.

Syntax

`val`

Specifies the coefficient value.

`layer name | layer pattern`

Specifies the layer name or pattern.

dratio

Run ACPC analysis to check if adiabatic heating of the wire results in wire meltdown. This can be caused by any current, DC or AC. Usually, narrow signal nets are more susceptible to ACPC issues than power nets, because the current is flowing in both directions. Some foundries recommend this to be checked only for signal nets.

```
dratio -method <1|2> -file
```

Syntax

`method 1`

You can use this method with pre-16nm EM rules. You are required to produce three full periods in the simulation within the time window of measurement.

The *.dratio file that is generated by the `dratio -method 1` command contains the STATUS column that describes either PASS or FAIL value, where

- PASS: 3 full pluses/cycles are identified
- FAIL: 3 full pulses/cycles are NOT identified

`method 2 (default)`

You can use this method for 16nm and later. Average current is used to calculate Td (pulse duration, corresponding internal variable `$hs_pduration`) and P (pulse period, corresponding internal variable `$hs_speriod`). Also note that when you use `-method 2`, it is not required to provide three full periods during the time window of measurement. This is the default method.

However, the measured current value may change for different simulation time durations due to its dependency on average current. To improve accuracy, you can provide a specific net frequency on a net-by-net basis with the following command:

```
netfrequency <frequency1>@<net_pattern1>  
<frequency2>@<net_pattern2> . . .
```

`-file`

The `-file` option allows you to dump the resistor duty ratio and pulse the width used for acpc analysis in the `-acpc.dratio` file. By default, the dratio files are not generated.

To generate the output, specify:

```
dratio -file
```

iavmin

The command specifies the threshold value for the average current injected into a signal net. The signal net is sent to EMIR only if the average injected current in the net exceeds this threshold. This allows the nets with very small currents that do not require analysis to be filtered out.

If you do not specify a unit type, the default is Amperes. You can also use symbols such as m (milli), u (micro), n (nano), p (pico), and so on. For example:

```
iavmin 10u
```

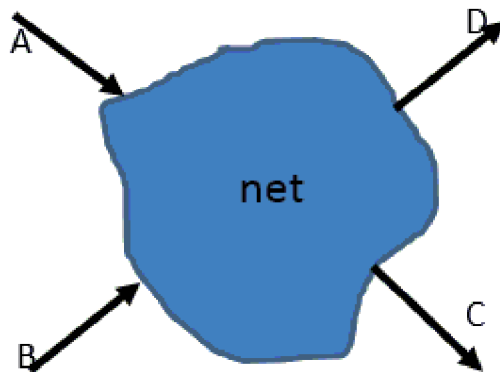
The above example specifies that `iavmin` equals to 10 micro Amperes. The default value of the threshold is 0.1uA.

Current and Average Net Current

Average net current is calculated as an average current for the waveform that is formed as follows:

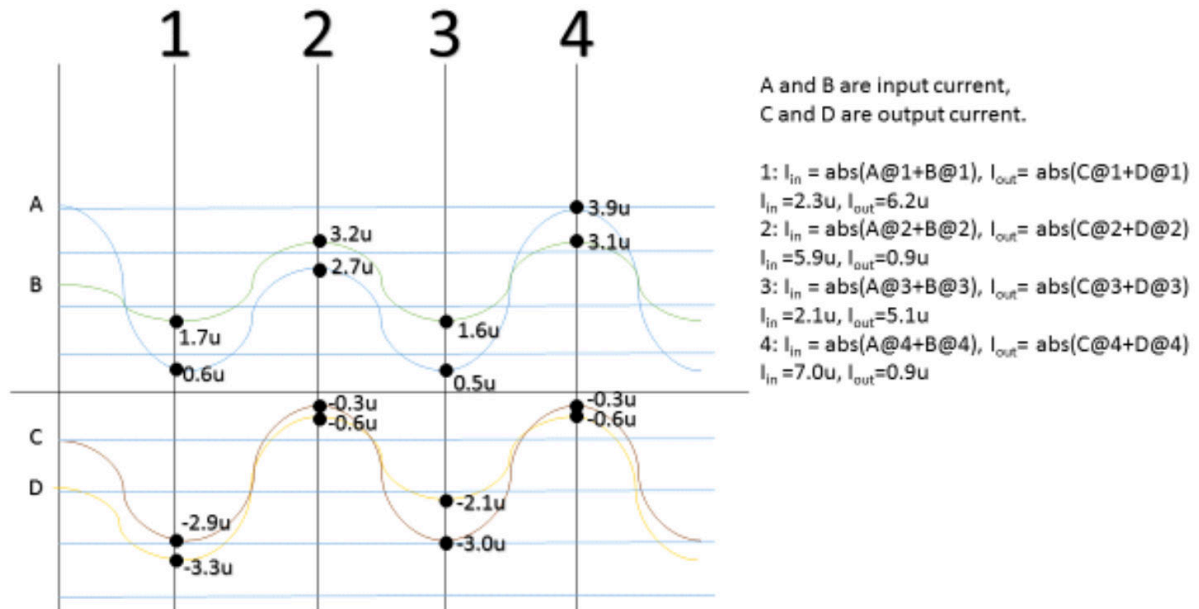
- Before the actual net simulation at phase II, the net pin currents are read from the .rasim file and analyzed;

Figure 23 Average Current Example

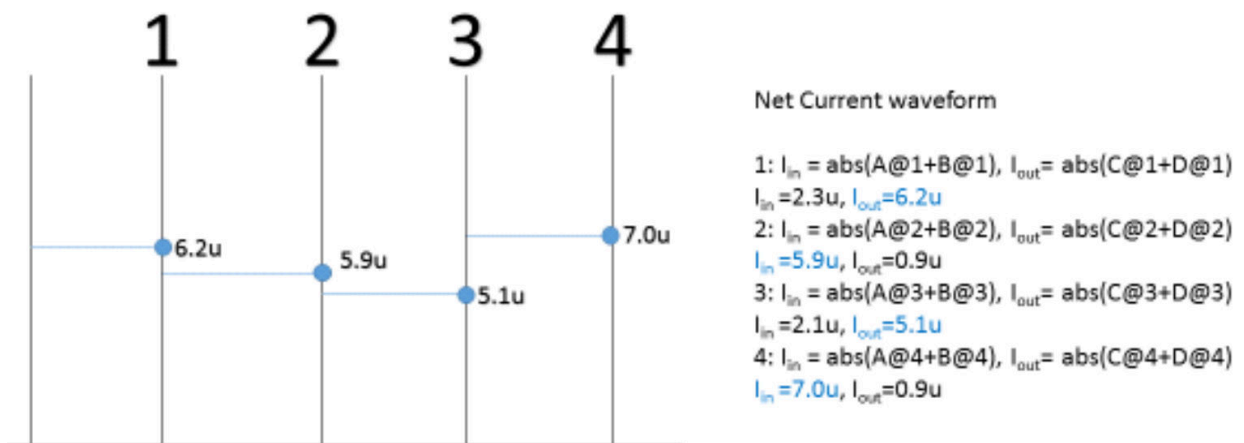


- At every time point, the device pin currents of the net are separated into the:
 - Currents flowing into the net: A and B in [Figure 23](#).
 - Currents flowing out of the net: C and D in [Figure 23](#).

- The sum of those currents is calculated as I_{in} and I_{out} ;



- The maximum (absolute) value of these two is selected $I = \max(\text{abs}(I_{in}), \text{abs}(I_{out}))$.
That value is the value of the net current for the given time point:



- That waveform represents net pin current. The average of that waveform is called net average current.

The net average value is compared against the `iavmin` parameter. The net is simulated at phase II when its average current is larger than the `iavmin` value.

In the previous example the current values at each time point were 6.2u, 5.9u, 5.1u and 7.0u. The average current is 6.05u. If the `iavmin` value is 5u while the average net current is 6.05u, the net is reported. If the `iavmin` value is 10u while the average net current is 6.05, the net is ignored.

itflayernames

```
itflayernames yes|no
```

In the SPF file, the StarRC tool can create two columns, one column with the LVS layer names and one column with the ITF layer names. By default, the PrimeSim EMIR tool uses the LVS layer names.

Syntax

`yes`

Specifies to use the ITF layer names.

`no`

Specifies to use LVS layer names. This is the default.

The following example shows an SPF file with LVS layer names (M3, M2, M1, ..) and ITF layer names (metal3, metal2, metal1, ...):

```
*LAYER_MAP*0 SUBSTRATE
*1 M3 ITF=metal3
*2 M2 ITF=metal2
*3 M1 ITF=metal1
*4 POLY ITF=poly
*5 nsd ITF=nsd
*6 psd ITF=psd
```

mt val

```
mt val
```

This command enables parallel processing of the second phase simulation of signal nets in PrimeSim EMIR signal net EM analysis to improve performance by multithreading of a large number of signal nets on multiple cores. The value is the number of cores used in signal net EM second phase simulation. It assigns and distributes selected signal nets over multiple cores for simulation according to the number of cores you specify. If this

command is not defined, the PrimeSim EMIR tool simulates using only one core. For example:

```
mt 5
```

Specifies to use five cores in the second phase of signal net EM analysis.

The `mt` command works only when signal net EM is enabled (`set_ra_option -sigra 1`).

net_ccaps

```
net_ccaps [0|1]
```

When simulating signal nets in phase II, `net_ccaps` controls how the signal net EM tool accounts for coupling capacitors between signal nets. For performance purposes, the coupling caps are split and grounded, so all nets can be simulated independently. The error introduced is expected to be small. The default is 0.

When you set `net_ccaps 1`, the coupling caps between the signal nets are taken into account. Although the results are more accurate, this setting might effect performance and increase peak memory usage.

```
net_ccaps 1
```

Accounts for coupling capacitors between signal nets.

Note:

Currently Phase II multicore simulation does not work with `net_ccaps 1`.

outformat

```
outformat [fsdb|out|wdf]
```

The command specifies the phase II waveform file format. If you do not specify this command, `fsdb` is the default waveform file format.

The following example specifies waveform output format to be FSDB version 5.7.

```
outformat fsdb_5.7
```

rawf

```
rawf 0|1
```

The command addresses the capacity issue when phase II exits with the following message, even with enough disk space in the run directory/dedicated disk for temporary files:

```
No space left on device
Error: fwrite failed to write 2*8 bytes
```

- 0 (default) uses the current method to interpolate multiple .rasim files (*.rasim, *.rasim1, ... *rasimn) to a single temporary file that can trigger the capacity issue.
- 1 processes multiple .rasim files (*.rasim, *.rasim1, ... *rasimn) file by file to reduce the temporary file size.

Note:

Phase II multi-threading is not supported when you set `rawf 1`.

selectsignets

```
selectsignets net pattern [net pattern ...]
```

The command filters the signal net analysis during Phase II of the signal net EM analysis. For example:

```
selectsignets a* in
```

This will select for analysis at Phase 2 net in and all nets which fit pattern a*.

signetascii

```
signetascii [ netbynet | mixall ]
```

The command groups or ungroups signal net EM results by net names in the ASCII files. When not specified, the signal net EM results are grouped by net names.

Syntax

`netbynet` (default)

Specifies to group signal net EM results by net names in the ASCII files. This is the default.

Inside each net group, the results are sorted by the user-specified criteria defined in the [asciicolsort](#) command.

`mixall`

Specifies to list signal net EM results in a single group in the ASCII files.

The results of all nets are sorted globally by the user-specified criteria defined in the `asciicolsort` command. Use this setting when you want to have an immediate overview of the worst-case results from all nets.

When `mixall` is used, it is recommended to add the `netname` keyword to the `asciicols` command for identifying the net name to which a given result belongs in the ASCII files. For example,

```
asciicols em resname current threshold i_imax layer x y width  
length width_new time emlen netname
```

Examples

The following example specifies to list the signal net EM results in a single group in the output ASCII reports.

```
signetascii mixall
```

In the output ASCII report:

```
r246_248354 187.361 179.49 1.044 m2 0.969 20.25 0.02 3.762 0.02088 3.842 qb[95]  
r97_1232000 187.454 179.49 1.044 m2 0.969 74.79 0.02 3.762 0.02088 3.842 qb[29]  
r254_250843 187.179 179.49 1.043 m2 0.969 22.65 0.02 3.762 0.02086 3.842 qb[91]  
r266_253582 187.191 179.49 1.043 m2 0.969 26.25 0.02 3.762 0.02086 3.842 qb[85]  
r99_1231899 187.16 179.49 1.043 m2 0.969 75.51 0.02 3.762 0.02086 3.842 qb[28]  
r105_1233853 187.257 179.49 1.043 m2 0.969 77.19 0.02 3.762 0.02086 3.842 qb[25]
```

skipsignets

```
skipsignets net pattern [net pattern ...]
```

The command allows you to skip the specified signal names from Phase II of the signal net EM analysis. You can specify a wildcard character in the signal name. For example:

```
skipsignets X1*
```

The above example skips the simulation of all signal nets under the X1 instance.

swin

```
swin <t1 t2> [<t3 t4> ...]
```

or

```
swin -apw <t1 t2> [<t3 t4> ...]
```

The command specifies multiple tstart-tstop windows for EMIR power net and signal net analysis. t1 and t2 are used to specify the first simulation window.

If you simulate 100ns (100n) for the Phase I simulation, and you are only interested in 10n-20n and 50n-60n for the Phase II EMIR analysis, you can specify the following command in the .ratcl file.

```
swin 10n 20n 50n 60n
```

Without the `-apw` option, the PrimeSim EMIR tool calculates the overall IAVG, IRMS and IABS currents for all of the defined simulation windows. In contrast, when `swin -apw` is used, the IAVG, IRMS and IABS currents are calculated separately for each defined simulation window and then the peak value is reported. For example:

```
swin t1 t2 t3 t4 ...
```

If IAVG analysis is performed, the PrimeSim EMIR tool uses the following formula to compute IAVG for the defined simulation windows t1-t2, t3-t4, ...

$$IAVG = \frac{1}{(t2-t1) + (t4-t3) + \dots} \left(\int_{t1}^{t2} i(t) dt + \int_{t3}^{t4} i(t) dt + \dots \right)$$

In contrast, if the `-apw` option is used as shown below:

```
swin -apw t1 t2 t3 t4
```

The PrimeSim EMIR tool uses the following formula to compute IAVG for defined simulation windows t1-t2, t3-t4, and so on.

$$IAVG = PEAK \left\{ \begin{array}{cc} \frac{1}{t2-t1} & \int_{t1}^{t2} i(t) dt \\ \frac{1}{t4-t3} & \int_{t3}^{t4} i(t) dt \\ \dots & \dots \end{array} \right\}$$

Similarly, if IRMS and IABS analysis are selected, PrimeSim EMIR uses following formulas

$$\text{for calculating the corresponding currents. } IABS = PEAK \left\{ \begin{array}{cc} \frac{1}{t2-t1} & \int_{t1}^{t2} |i(t)| dt \\ \frac{1}{t4-t3} & \int_{t3}^{t4} |i(t)| dt \\ \dots & \dots \end{array} \right\}$$

$$IRMS = PEAK \left\{ \begin{array}{c} \frac{1}{t_2-t_1} \sqrt{\int_{t_1}^{t_2} i(t)^2 dt} \\ \frac{1}{t_4-t_3} \sqrt{\int_{t_3}^{t_4} i(t)^2 dt} \\ \dots \quad \dots \end{array} \right\}$$

Note:

Due to the limitations of the viewers currently used, the waveform points which correspond to the end of previous window and the beginning of the new one will be connected by a straight line. However, since windows are specified by you, it should not be a problem, i.e., you know that the intervals between the windows must be ignored.

Outside the specified windows, no EMIR analysis is performed in Phase II, and the result is set to zero.

The PEAK function chooses the peak result from all swin windows, ignoring the sign during comparison. Please observe that IMAX and VMAX results are not affected by the `-apw` option.

The following command specifies one window, starting at 10ns:

```
swin 10ns
```

The following command specifies two windows, 5ns-15ns and 25ns-35ns. Also, averaging is done in each window.

```
swin -apw 5ns 15ns 25ns 35ns
```

The following command specifies two windows, 5ns-15ns and 25ns-35ns. Averaging is done over the whole time of simulation.

```
swin 5ns 15ns 25ns 35ns
```

Note:

By default, the window defined in phase I with `set_ra_option -twindow` is used in phase II. You do not need to redefine the phase II window using the `swin` parameter. However, you can reset the window in phase II using the `swin` command in a user-defined .ratcl file, which has a higher priority over `set_ra_option -twindow`. In that case, It is recommend to use a slightly smaller window for `swin` (with respect to the `set_ra_option -twindow` value). If you keep the exact same window, the initial conditions for the phase II window start points might not be correct, because capacitors are also processed at phase II and the currents at the parasitic resistors are not yet defined. It takes some time until the solution stabilizes to a steady state.

Index

Special Characters

.ratcl commands for phase II control

- alterpad 225
- asciicols 226
- asciicolsort 230
- coordunit 236
- gdsdatatype 245
- gdsdefrw 245
- gdsfilechsymb 245
- gdslayers 246
- gdsmap 247
- gdsmapi 247
- gdsmapvmax 248
- gdsoutmode 248
- gdsprops 249, 274
- jjmaxlog 252
- layerh 252
- layermap 252, 272, 273
- outlayers 254
- printi 257, 257
- printipad 257, 258
- printv 258
- printvmode 258
- ralayers 261
- ralayout 261
- rarve 263
- raviewer 263, 264, 290
- redia 264
- rediac 265
- rediw 267, 267
- redj 264, 268
- redtrms 271
- redv 236, 236, 269
- rvemapvmax 274
- skiplayers 276
- swin 277, 292
- tau 279
- tstart 280
- tstop 280

twin 281

.ratcl commands for phase II control

- asciimapi 231
- asciimapvmax 231
- layerea 252, 252
- outlayers 255
- redv 269, 269

A

- addresistor 42
- asciimapi 231
- asciimapvmx 231

D

- drLoadDrf 39

E

- EMIR Results Browser 157

G

- gdslayers 208

L

- layer_map 51
- layerea 252
- layerh 200

M

- modifyresistor 55
- Multi-analysis browser 157

N

- net resistance calculator commands
 - addnetpin 41
 - gds 47
 - gdslayer0 48
 - gdsthresh 49, 49

- ipin 50
- layer0ohm 51, 51
- layerfactor 51, 58
- net 56, 56
- netdeletepad 41, 41, 56
- netinclude 56
- png 56, 57, 59
- rmin 61
- sortby 62
- subnode 62

O

- outfiltres 254
- outlayers 208

P

- phase II output files
 - ASCII 198, 198, 199
 - DRF 201, 201
 - FSDB 204, 204
 - OUT 204, 204
 - RADB 204
 - RALOG 204, 205
 - WDF 204, 204

R

- raformat 208
- ralayers 207
- routcols 61

S

- static resistance calculation 32
- static resistance calculation commands
 - routcols 61

V

- violation map 205